

Módulo 2 - Prácticas de modelado de datos con Entity Framework Core 8

Estas prácticas continúan exactamente desde `M01/PROYECTO/1.12`. Cada punto dispone de una solución autónoma y completa en `M02/PROYECTO/2.x/AceriaData.sln`. El estado de un punto se construye sobre el punto anterior y las migraciones evolucionan el mismo esquema de `AceriaDB`.

Punto 2.1 - Convenciones de modelado en Entity Framework Core

Ejercicio guiado: inspeccionar el modelo que EF Core construye para `AceriaData` y reconocer, antes de modificar nada, qué entidades, tablas, claves, propiedades, relaciones y comportamientos ha descubierto por convención.

Contexto del proyecto: este punto continúa exactamente desde `M01/PROYECTO/1.12`. El modelo heredado ya contiene `OrdenFabricacion`, `PlanchaAcero`, `Aleacion` y `EstadoOrden`, el `AceriaDbContext`, SQL Server LocalDB y el contenedor de dependencias. En 2.1 no se rediseña el dominio ni se crea una base nueva: se observa el modelo existente para comprender qué ha deducido EF Core automáticamente. Esa lectura será la base de 2.2, donde comenzará la configuración explícita de propiedades.

Objetivos de aprendizaje

- Reconocer las convenciones que EF Core aplica al descubrir entidades.
- Identificar la tabla asociada a cada entidad.
- Comprobar cómo se detecta una clave primaria.
- Identificar claves foráneas y entidades principales.
- Interpretar la nulabilidad y las navegaciones a partir del modelo construido.
- Diferenciar lo que EF Core ha inferido por convención de lo que se configurará explícitamente en puntos posteriores.
- Resolver el `AceriaDbContext` mediante un ámbito local de DI, sin conservar un `ServiceProvider` estático.
- Verificar que una inspección de metadatos no necesita una migración nueva.

Paso 1: Abrir la solución autónoma de 2.1

Trabaja directamente sobre el estado completo de este punto:

```
cd M02/PROYECTO/2.1
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

La solución está dentro de la carpeta del punto. No es necesario abrir una solución global del repositorio ni copiar archivos desde `M1: 2.1` ya contiene el estado heredado que debe inspeccionarse.

Paso 2: Reconocer el estado heredado antes de tocar el modelo

Abre `Program.cs` y localiza las cuatro entidades y el `AceriaDbContext`. Antes de continuar, comprueba estas ideas:

- `OrdenFabricacion`, `PlanchaAcero`, `Aleacion` y `EstadoOrden` ya existen.
- `AceriaDbContext` expone los `DbSet<T>` correspondientes.
- `PlanchaAcero` contiene `OrdenId` y la navegación `Orden`.
- `OrdenFabricacion` contiene la colección `Planchas`.
- 2.1 no introduce todavía `DetalleOrden`, `CertificadoCalidad` ni `OrdenAleacion`.

El objetivo de este paso es separar claramente **observación** de **configuración**. Si se añade una relación nueva aquí, se estaría adelantando contenido de 2.4 o 2.5.

Paso 3: Resolver el DbContext con un ámbito local

Para evitar estado global innecesario, el proveedor y el ámbito se mantienen dentro de `Main`:

```
using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateScopes = true,
    ValidateOnBuild = true
});

using var scope = provider.CreateScope();
var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
```

Qué hace cada línea:

1. `BuildServiceProvider(...)` construye el contenedor después de registrar `AceriaDbContext`, repositorio y servicio.
2. `ValidateScopes = true` ayuda a detectar usos incorrectos de servicios `Scoped`.
3. `ValidateOnBuild = true` comprueba el grafo de dependencias al construir el proveedor.
4. `CreateScope()` crea el ámbito dentro del cual se resolverá el `DbContext`.
5. `GetRequiredService<AceriaDbContext>()` obtiene el contexto configurado para SQL Server LocalDB.
6. Los dos `using var` garantizan que proveedor y ámbito se liberen correctamente al finalizar.

No se necesita un campo `static IServiceProvider`. La inspección del modelo puede hacerse con el mismo patrón local que seguirá usando el curso.

| Paso 4: Enumerar entidades, tablas y claves primarias

A continuación se consulta **el modelo que EF Core ya ha construido**, no las clases mediante reflexión:

```
global::System.Console.WriteLine("--- MODELO EF CORE 2.1 ---");

foreach (var entity in context.Model.GetEntityTypes().OrderBy(e => e.ClrType.Name))
{
    var pk = entity.FindPrimaryKey();

    global::System.Console.WriteLine(
        $"Entidad: {entity.ClrType.Name} | " +
        $"Tabla: {entity.GetTableName()} | " +
        $"PK: {string.Join(", ", pk?.Properties.Select(x => x.Name) ?? Array.Empty<string>())}");
}
```

Observa especialmente:

- `context.Model.GetEntityTypes()` devuelve las entidades que forman parte del modelo EF Core.
- `GetTableName()` muestra la tabla relacional asociada.
- `FindPrimaryKey()` devuelve la clave primaria que EF Core reconoce.
- En `AceriaData`, las entidades usan `Id`, por lo que cumplen la convención de clave primaria.
- La convención general relevante es `Id` o `<NombreDelTipo>Id`; no depende del nombre del `DbSet`.

| Paso 5: Inspeccionar las claves foráneas descubiertas

El estado final de 2.1 también recorre las claves foráneas:

```
foreach (var fk in entity.GetForeignKeys())
{
    global::System.Console.WriteLine(
        $" FK: {string.Join(", ", fk.Properties.Select(x => x.Name))} -> " +
        $"{fk.PrincipalEntityType.ClrType.Name}");
}
```

En `PlanchaAcero`, EF Core puede asociar `OrdenId` con la navegación `Orden` y con `OrdenFabricacion`. El ejercicio no crea esa relación en este punto: simplemente comprueba la relación que ya existía en el modelo heredado.

Esta diferencia es importante:

- **Descubrir una relación existente** pertenece a 2.1.
- **Configurar explícitamente una relación uno-a-muchos** pertenece a 2.3.
- **Crear relaciones uno-a-uno** pertenece a 2.4.
- **Modelar muchos-a-muchos con `OrdenAleacion`** pertenece a 2.5.

Paso 6: Compilar y ejecutar la inspección

Ejecuta el estado completo:

```
dotnet build AceriaData.sln --configuration Release
dotnet run --project AceriaData.Console.csproj --configuration Release
```

La salida debe contener la cabecera:

```
--- MODELO EF CORE 2.1 ---
```

y, a continuación, una línea por entidad con su tabla y su clave primaria. Para las entidades que tengan claves foráneas aparecerán además las líneas `FK:`.

No memorices la salida: úsala para contrastar lo que EF Core ha construido con lo que esperabas a partir de las clases.

Paso 7: Interpretar las convenciones observadas

Con la salida delante, comprueba una a una las convenciones relevantes.

Entidades y tablas. Las entidades incluidas en el modelo se mapean a las tablas determinadas por el modelo relacional y los `DbSet<T>` heredados.

Claves primarias. Una propiedad `Id` cumple la convención de clave primaria. También sería válida una propiedad con el patrón `<NombreDelTipo>Id`.

Claves foráneas. Una propiedad como `OrdenId`, combinada con la navegación `Orden`, permite a EF Core reconocer la relación con `OrdenFabricacion`.

Navegaciones. La referencia `PlanchaAcero.Orden` y la colección `OrdenFabricacion.Planchas` describen los dos extremos de la relación existente.

Nulabilidad. EF Core utiliza la información del tipo CLR y la configuración de nulabilidad de C# para construir la obligatoriedad del modelo. En 2.2 se trabajará esta parte de forma explícita.

Paso 8: Confirmar que inspeccionar el modelo no crea una migración

2.1 no cambia el esquema. Compruébalo con:

```
dotnet ef migrations list
```

Debe aparecer el historial heredado de M1 y **no** una migración nueva de 2.1.

Tampoco se usa `EnsureCreated()`. El curso mantiene el esquema gobernado por Migrations; inspeccionar metadatos no justifica recrear ni sustituir el historial existente.

Paso 9: Diagnosticar los errores más frecuentes

Antes de continuar, revisa qué ocurriría en cada uno de estos casos:

Situación	Qué problema provoca	Corrección
Guardar el <code>ServiceProvider</code> en un campo estático	Introduce estado global innecesario y oculta el ciclo de vida de los servicios	Mantener <code>provider</code> y <code>scope</code> locales dentro de <code>Main</code>
Eliminar o renombrar <code>Id</code> sin configurar otra clave	EF Core deja de poder determinar la PK de la entidad	Mantener <code>Id</code> o configurar la clave explícitamente en el punto correspondiente
Ejecutar <code>EnsureCreated()</code> para “ver si funciona”	Se sale del flujo basado en Migrations	Conservar el historial y limitar 2.1 a inspeccionar el modelo
Añadir una relación muchos-a-muchos	Adelanta contenido de 2.5	No modificar el dominio en 2.1
Interpretar el nombre del <code>DbSet</code> como regla de clave primaria	Confunde dos convenciones distintas	La PK se descubre por <code>Id</code> o <code><NombreDelTipo>Id</code>

Si quieres comprobar el error de clave primaria, hazlo únicamente sobre una copia desechable del ejercicio y restaura después el estado original. El checkpoint entregado debe permanecer válido.

Paso 10: Resolver el reto de inspección avanzada

Reto: ampliar la inspección para mostrar también el `DeleteBehavior` de cada clave foránea, sin cambiar el modelo.

El propio checkpoint contiene un bloque pedagógico comentado que puede activarse para realizar la prueba:

```
global::System.Console.WriteLine("--- RETO 2.1: DELETE BEHAVIOR ---");

foreach (var entity in context.Model.GetEntityTypes().OrderBy(e => e.ClrType.Name))
{
    foreach (var fk in entity.GetForeignKeys())
    {
        global::System.Console.WriteLine(
            $"FK: {string.Join(", ", fk.Properties.Select(x => x.Name))} -> " +
            $"{fk.PrincipalEntityType.ClrType.Name} | DeleteBehavior: {fk.DeleteBehavior}");
    }
}
```

Este reto sigue perteneciendo a la inspección de metadatos: no añade tablas, columnas ni relaciones. La finalidad es aprender a leer con más detalle la relación que EF Core ya conoce.

Errores comunes

- Confundir inspeccionar el modelo con modificarlo.
- Copiar una versión antigua que conserva el proveedor de servicios en un campo estático.
- Usar `EnsureCreated()` o `EnsureDeleted()` como sustituto del historial de migraciones.
- Adelantar relaciones que corresponden a 2.4 o 2.5.
- Suponer que la clave primaria se deduce a partir del nombre del `DbSet`.
- Mirar sólo las clases y no contrastarlas con `context.Model`.

Analogía final

Antes de modificar una línea de producción, un técnico revisa los planos y recorre la instalación para saber qué máquinas existen, cómo están conectadas y qué función cumple cada unión. En 2.1 ocurre lo mismo: las clases son el diseño escrito, pero `context.Model` es el plano que EF Core ha construido realmente. Primero se aprende a leer ese plano; después, en los puntos siguientes, se empezará a modificarlo de forma consciente.

Resultado esperado

Al finalizar 2.1 debes poder abrir el estado autónomo, compilarlo y ejecutar la inspección sin alterar el esquema. La salida debe identificar las entidades del modelo, sus tablas, las claves primarias y las claves foráneas existentes. `dotnet ef migrations list` debe seguir mostrando únicamente el historial heredado y el reto debe permitir consultar `DeleteBehavior` sin introducir una migración nueva.

Conexión con el siguiente punto

Ya sabes distinguir lo que EF Core descubre automáticamente de lo que todavía no se ha configurado de forma explícita. En 2.2 partirás de este mismo modelo para trabajar con propiedades de negocio: requeridos, opcionales, longitudes máximas, precisión decimal y valores por defecto. El proyecto no se reinicia y las decisiones válidas de 2.1 se conservan.

Código acumulativo completo del estado 2.1

El siguiente archivo es el `Program.cs` real del estado ejecutable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
}

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class Aleacion
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
}

public class EstadoOrden
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Descripcion { get; set; } = string.Empty;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
}
```

```

    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
}

public interface IOordenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    int Guardar();
}

public sealed class OrdenRepositorio : IOordenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o =>
o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) =>
_context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOordenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOordenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();
        services.AddDbContext<AceriaDbContext>(options =>
            options
                .UseSqlServer(connectionString, sql =>
                    {
                        sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                        sql.CommandTimeout(60);
                    }
                )
        );
    }
}

```

```

    })
    .LogTo(global::System.Console.WriteLine, new[] {
"Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)
    .EnableDetailedErrors());

services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateScopes = true,
    ValidateOnBuild = true
});

using var scope = provider.CreateScope();
var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
global::System.Console.WriteLine("--- MODELO EF CORE 2.1 ---");
foreach (var entity in context.Model.GetEntityTypes().OrderBy(e => e.ClrType.Name))
{
    var pk = entity.FindPrimaryKey();
    global::System.Console.WriteLine($"Entidad: {entity.ClrType.Name} | Tabla:
{entity.GetTableName()} | PK: {string.Join(", ", pk?.Properties.Select(x => x.Name) ??
Array.Empty<string>())}");
    foreach (var fk in entity.GetForeignKeys())
    {
        global::System.Console.WriteLine($" FK: {string.Join(", ", fk.Properties.Select(x =>
x.Name))} -> {fk.PrincipalEntityType.ClrType.Name}");
    }
}
}
}

```

Explicación línea a línea del código acumulativo 2.1

Las referencias siguientes usan la numeración real de `M02/PROYECTO/2.1/Program.cs`. Se explican todas las líneas no vacías para que el documento sea trazable con el archivo que compila.

Línea 1: `using System.ComponentModel.DataAnnotations;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 2: `using System.ComponentModel.DataAnnotations.Schema;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 3: `using Microsoft.EntityFrameworkCore;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 4: `using Microsoft.Extensions.Configuration;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 5: `using Microsoft.Extensions.DependencyInjection;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 6: `using Microsoft.Extensions.Logging;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 8: `namespace AceriaData.ConsoleApp;` → declara el espacio de nombres del estado acumulativo.

Línea 10: `public class OrdenFabricacion` → declara la clase `OrdenFabricacion` que forma parte del estado acumulativo del ejercicio.

Línea 11: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 12: `public int Id { get; set; }` → declara la propiedad `Id` de tipo `int` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 13: `public string NumeroOrden { get; set; } = string.Empty;` → declara la propiedad `NumeroOrden` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 14: `public string Cliente { get; set; } = string.Empty;` → declara la propiedad Cliente de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 15: `public DateTime FechaCreacion { get; set; }` → declara la propiedad FechaCreacion de tipo DateTime que EF Core o la lógica de negocio utiliza en este modelo.

Línea 16: `public List<PlanchaAcero> Planchas { get; set; } = new();` → declara la propiedad Planchas de tipo List que EF Core o la lógica de negocio utiliza en este modelo.

Línea 17: `}` → cierra el bloque de código actual.

Línea 19: `public class PlanchaAcero` → declara la clase PlanchaAcero que forma parte del estado acumulativo del ejercicio.

Línea 20: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 21: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 22: `public int OrdenId { get; set; }` → declara la propiedad OrdenId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 23: `public double Espesor { get; set; }` → declara la propiedad Espesor de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 24: `public double Ancho { get; set; }` → declara la propiedad Ancho de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 25: `public double Largo { get; set; }` → declara la propiedad Largo de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 26: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.

Línea 27: `}` → cierra el bloque de código actual.

Línea 29: `public class Aleacion` → declara la clase Aleacion que forma parte del estado acumulativo del ejercicio.

Línea 30: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 31: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 32: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 33: `public double PorcentajeCarbono { get; set; }` → declara la propiedad PorcentajeCarbono de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 34: `public double PorcentajeManganeso { get; set; }` → declara la propiedad PorcentajeManganeso de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 35: `}` → cierra el bloque de código actual.

Línea 37: `public class EstadoOrden` → declara la clase EstadoOrden que forma parte del estado acumulativo del ejercicio.

Línea 38: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 39: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 40: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 41: `public string Descripcion { get; set; } = string.Empty;` → declara la propiedad Descripcion de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 42: `}` → cierra el bloque de código actual.

Línea 44: `public class AceriaDbContext : DbContext` → declara la clase AceriaDbContext que forma parte del estado acumulativo del ejercicio.

Línea 45: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 46: `public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }` → declara el constructor o método AceriaDbContext con los parámetros necesarios para este componente.

Línea 48: `public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 49: `public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 50: `public DbSet<Aleacion> Aleaciones => Set<Aleacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 51: `public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 53: `}` → cierra el bloque de código actual.

Línea 55: `public interface IOrdenRepositorio` → declara el contrato IOrdenRepositorio para desacoplar consumidores e implementación.

Línea 56: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 57: `List<OrdenFabricacion> ObtenerTodas();` → declara en el contrato la operación que debe devolver todas las órdenes visibles para el repositorio.

Línea 58: `OrdenFabricacion? ObtenerPorId(int id);` → declara en el contrato la búsqueda de una orden por su clave primaria y permite devolver null si no existe.

Línea 59: `OrdenFabricacion? ObtenerPorNumero(string numeroOrden);` → declara en el contrato la búsqueda por el identificador de negocio NumeroOrden.

Línea 60: `void Agregar(OrdenFabricacion orden);` → declara la operación que incorpora una orden al repositorio para que EF Core la siga como nueva.

Línea 61: `void Eliminar(OrdenFabricacion orden);` → declara la operación que marca una orden para eliminación física cuando el escenario lo requiere.

Línea 62: `int Guardar();` → declara la operación que confirma los cambios pendientes y devuelve el número de entradas afectadas.

Línea 63: `}` → cierra el bloque de código actual.

Línea 65: `public sealed class OrdenRepositorio : IOrdenRepositorio` → declara la clase OrdenRepositorio que forma parte del estado acumulativo del ejercicio.

Línea 66: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 67: `private readonly AceriaDbContext _context;` → declara el campo privado _context de tipo AceriaDbContext que conserva una dependencia de la clase.

Línea 68: `public OrdenRepositorio(AceriaDbContext context) => _context = context;` → declara el constructor o método OrdenRepositorio con los parámetros necesarios para este componente.

Línea 70: `public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();` → ordena el resultado de la consulta antes de materializarlo.

Línea 71: `public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);` → implementa la búsqueda por clave primaria mediante Find, aprovechando primero el ChangeTracker y después la base de datos.

Línea 72: `public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);` → recupera la primera coincidencia o null cuando no existe.

Línea 73: `public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);` → registra una entidad o relación nueva en el ChangeTracker para insertarla al guardar.

Línea 74: `public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);` → marca la entidad indicada para borrado al confirmar los cambios.

Línea 75: `public int Guardar() => _context.SaveChanges();` → persiste en SQL Server los cambios seguidos por el DbContext.

Línea 76: `}` → cierra el bloque de código actual.

Línea 78: `public interface IServicioOrdenes` → declara el contrato IServicioOrdenes para desacoplar consumidores e implementación.

Línea 79: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 80: `string ObtenerResumen(int ordenId);` → declara el servicio que construye un resumen de una orden a partir de su identificador.

Línea 81: `}` → cierra el bloque de código actual.

Línea 83: `public sealed class ServicioOrdenes : IServicioOrdenes` → declara la clase ServicioOrdenes que forma parte del estado acumulativo del ejercicio.

Línea 84: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 85: `private readonly IOrdenRepositorio _repositorio;` → declara el campo privado _repositorio de tipo IOrdenRepositorio que conserva una dependencia de la clase.

Línea 86: `private readonly AceriaDbContext _context;` → declara el campo privado _context de tipo AceriaDbContext que conserva una dependencia de la clase.

Línea 87: `public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)` → declara el constructor o método ServicioOrdenes con los parámetros necesarios para este componente.

Línea 88: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 89: `_repositorio = repositorio;` → guarda en el servicio el repositorio inyectado para reutilizarlo en sus operaciones.

Línea 90: `_context = context;` → guarda el DbContext inyectado para consultar los datos necesarios durante el servicio.

Línea 91: `}` → cierra el bloque de código actual.

Línea 93: `public string ObtenerResumen(int ordenId)` → declara el método que obtiene la orden y compone el resumen solicitado por el servicio.

Línea 94: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 95: `var orden = _repositorio.ObtenerPorId(ordenId);` → consulta la orden mediante el repositorio y conserva el resultado anulable en una variable local.

Línea 96: `if (orden is null) return $"Orden {ordenId} no encontrada";` → evalúa la condición y ejecuta el bloque asociado sólo cuando se cumple.

Línea 97: `var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);` → cuenta las filas que cumplen la consulta construida.

Línea 98: `return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";` → devuelve el resultado calculado al código que invocó el método.

Línea 99: `}` → cierra el bloque de código actual.

Línea 100: `}` → cierra el bloque de código actual.

Línea 102: `public static class Program` → declara la clase Program que forma parte del estado acumulativo del ejercicio.

Línea 103: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 104: `public static void Main()` → declara el punto de entrada ejecutable de la aplicación de consola.

Línea 105: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 106: `var configuration = new ConfigurationBuilder()` → inicia la construcción de la configuración de la aplicación.

Línea 107: `.SetBasePath(AppContext.BaseDirectory)` → establece la ruta base usada para localizar los archivos de configuración.

Línea 108: `.AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)` → incorpora appsettings.json a la configuración.

Línea 109: `.AddEnvironmentVariables()` → incorpora variables de entorno como fuente adicional de configuración.

Línea 110: `.Build();` → materializa la configuración o el objeto encadenado construido en las líneas anteriores.

Línea 112: `var connectionString = configuration.GetConnectionString("AceriaDB")` → obtiene la cadena de conexión AceriaDB desde la configuración.

Línea 113: `?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");` → interrumpe el arranque con un error explícito si la configuración no contiene ConnectionStrings:AceriaDB.

Línea 115: `var services = new ServiceCollection();` → crea la colección donde se registran las dependencias de la aplicación.

Línea 116: `services.AddDbContext<AceriaDbContext>(options =>` → registra el DbContext en el contenedor de inyección de dependencias.

Línea 117: `options` → continúa la configuración del DbContext sobre el parámetro options recibido por la lambda de AddDbContext.

Línea 118: `.UseSqlServer(connectionString, sql =>` → selecciona SQL Server como proveedor de EF Core.

Línea 119: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 120: `sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);` → habilita reintentos ante fallos transitorios del proveedor SQL Server.

Línea 121: `sql.CommandTimeout(60);` → establece el tiempo máximo permitido para los comandos SQL.

Línea 122: `)` → cierra la lambda de configuración actual y devuelve el control a la llamada encadenada que la contiene.

Línea 123: `.LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)` → configura la salida del logging de EF Core.

Línea 124: `.EnableDetailedErrors();` → activa errores detallados para facilitar el diagnóstico del laboratorio.

Línea 126: `services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();` → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.

Línea 127: `services.AddScoped<IServicioOrdenes, ServicioOrdenes>();` → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.

Línea 129: `using var provider = services.BuildServiceProvider(new ServiceProviderOptions` → construye el `ServiceProvider` con validación de servicios y garantiza su liberación al salir del ámbito mediante `using var`.

Línea 130: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 131: `ValidateScopes = true,` → activa la validación de ámbitos para detectar dependencias scoped usadas desde ámbitos incorrectos.

Línea 132: `ValidateOnBuild = true` → obliga al contenedor a validar las resoluciones registradas al construir el `ServiceProvider`.

Línea 133: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 135: `using var scope = provider.CreateScope();` → crea un ámbito de DI para resolver servicios scoped y lo libera automáticamente al terminar el bloque.

Línea 136: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → resuelve un servicio obligatorio desde el contenedor de dependencias.

Línea 137: `global::System.Console.WriteLine("--- MODELO EF CORE 2.1 ---");` → escribe en consola la evidencia usada para verificar el comportamiento del estado.

Línea 138: `foreach (var entity in context.Model.GetEntityTypes().OrderBy(e => e.ClrType.Name))` → ordena el resultado de la consulta antes de materializarlo.

Línea 139: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 140: `var pk = entity.FindPrimaryKey();` → consulta los metadatos de EF Core para obtener la clave primaria de la entidad.

Línea 141: `global::System.Console.WriteLine($"Entidad: {entity.ClrType.Name} | Tabla: {entity.GetTableName()} | PK: {string.Join(", ", pk?.Properties.Select(x => x.Name)) ?? Array.Empty<string>()}");` → escribe en consola la evidencia usada para verificar el comportamiento del estado.

Línea 142: `foreach (var fk in entity.GetForeignKeys())` → enumera las claves foráneas configuradas para la entidad.

Línea 143: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 144: `global::System.Console.WriteLine($" FK: {string.Join(", ", fk.Properties.Select(x => x.Name))} -> {fk.PrincipalEntityType.ClrType.Name}");` → escribe en consola la evidencia usada para verificar el comportamiento del estado.

Línea 145: `}` → cierra el bloque de código actual.

Línea 146: `}` → cierra el bloque de código actual.

Línea 147: `}` → cierra el bloque de código actual.

Línea 148: `}` → cierra el bloque de código actual.

Punto 2.2 - Entidades y propiedades

Ejercicio guiado: enriquecer las entidades de `AceriaData` y configurar explícitamente sus propiedades escalares: requeridos, opcionales, longitudes máximas, precisión decimal y valores por defecto.

Contexto del proyecto: en 2.1 se inspeccionó el modelo que EF Core había construido por convención. Ahora se parte de ese mismo estado y se modifica el modelo de forma controlada. Este punto se limita a **entidades y propiedades**. Las relaciones explícitas se trabajarán desde 2.3, las relaciones muchos-a-muchos en 2.5 y los índices y restricciones de negocio en 2.9.

Objetivos de aprendizaje

- Añadir propiedades de negocio sin reiniciar `AceriaData`.
- Distinguir propiedades requeridas y opcionales.
- Configurar longitudes máximas para cadenas.
- Configurar precisión y escala para valores `decimal`.
- Configurar valores por defecto constantes y generados por SQL Server.
- Comprender qué cambios de propiedades producen un delta de esquema.
- Generar y revisar una migración incremental sobre el snapshot heredado.
- Verificar que el punto no introduce relaciones ni índices reservados para puntos posteriores.

Paso 1: Abrir el estado autónomo de 2.2

```
cd M02/PROYECTO/2.2
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

El checkpoint 2.2 ya contiene el resultado completo del ejercicio. Si estás siguiendo el laboratorio manualmente desde 2.1, aplica sobre tu copia los cambios descritos en los pasos siguientes y genera después la migración incremental.

Paso 2: Añadir Peso y Activa a PlanchaAcero

La entidad incorpora dos propiedades de negocio nuevas:

```
public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    public OrdenFabricacion Orden { get; set; } = null!;
}
```

- `Peso` usa `decimal` porque en este dominio interesa conservar una escala decimal controlada.
- `Activa` representa el estado operativo de la plancha y parte de `true`.
- `OrdenId` y la navegación `Orden` ya existían; en este punto no se reconfigura todavía la relación.

El error que debe evitarse aquí es aprovechar el cambio para configurar `HasOne/WithMany`: esa configuración pertenece a 2.3.

Paso 3: Añadir propiedades de negocio a OrdenFabricacion

El estado 2.2 amplía la orden con fecha de entrega, estado y observaciones:

```
public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    public string Estado { get; set; } = "Pendiente";
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
}
```

FechaEntrega es DateTime? porque una orden puede crearse antes de disponer de una fecha definitiva. Observaciones es string? por la misma razón: no toda orden necesita texto adicional.

Si FechaEntrega se declarase como DateTime, el modelo la trataría como no anulable y se perdería esa regla del dominio.

Paso 4: Completar Aleacion y EstadoOrden sin adelantar relaciones

También se incorporan propiedades escalares a las otras entidades:

```
public class Aleacion
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Codigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    public string? Descripcion { get; set; }
}

public class EstadoOrden
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
}
```

En 2.2 **no se introducen todavía colecciones para una relación muchos-a-muchos**: esa relación se reserva para 2.5, donde se modelará con la entidad intermedia OrdenAleacion.

Paso 5: Configurar las propiedades de OrdenFabricacion

En OnModelCreating, el checkpoint configura únicamente aspectos que pertenecen a propiedades:

```

modelBuilder.Entity<OrdenFabricacion>(entity =>
{
    entity.ToTable("OrdenesFabricacion");
    entity.HasKey(o => o.Id);

    entity.Property(o => o.NumeroOrden)
        .IsRequired()
        .HasMaxLength(50);

    entity.Property(o => o.Cliente)
        .IsRequired()
        .HasMaxLength(200);

    entity.Property(o => o.FechaCreacion)
        .HasDefaultValueSql("GETDATE()");

    entity.Property(o => o.Estado)
        .IsRequired()
        .HasMaxLength(50)
        .HasDefaultValue("Pendiente");

    entity.Property(o => o.Observaciones)
        .HasMaxLength(500);
});

```

Qué debes observar:

1. `NumeroOrden` y `Cliente` dejan de ser cadenas sin límite conocido.
2. `FechaCreacion` obtiene un valor por defecto generado por SQL Server.
3. `Estado` es requerido, tiene longitud máxima y valor por defecto.
4. `Observaciones` conserva la nulabilidad del tipo CLR y limita su longitud.
5. No se crea todavía un índice único para `NumeroOrden`; los índices se estudian en 2.9.

| Paso 6: Configurar precisión y valor por defecto de PlanchaAcero

La configuración relevante de la plancha es:

```

modelBuilder.Entity<PlanchaAcero>(entity =>
{
    entity.ToTable("PlanchasAcero");
    entity.HasKey(x => x.Id);

    entity.Property(x => x.Peso)
        .HasPrecision(18, 3);

    entity.Property(x => x.Activa)
        .HasDefaultValue(true);
});

```

`HasPrecision(18, 3)` significa hasta 18 dígitos en total, de los cuales 3 quedan a la derecha del separador decimal. El objetivo es que valores como `371.250` se representen con la escala prevista.

`HasDefaultValue(true)` define el valor por defecto a nivel de modelo/base de datos. Si una operación envía explícitamente otro valor, prevalece el valor enviado.

| Paso 7: Configurar las propiedades escalares de Aleacion y EstadoOrden

El checkpoint mantiene la configuración de estas entidades centrada en propiedades:

```

modelBuilder.Entity<Aleacion>(entity =>
{
    entity.ToTable("Aleaciones");
    entity.HasKey(a => a.Id);
    entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
    entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
    entity.Property(a => a.Descripcion).HasMaxLength(500);
});

modelBuilder.Entity<EstadoOrden>(entity =>
{
    entity.ToTable("EstadosOrden");
    entity.HasKey(e => e.Id);
    entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
    entity.Property(e => e.Descripcion).HasMaxLength(250);
    entity.Property(e => e.Activo).HasDefaultValue(true);
});

```

En este estado `Codigo` tiene longitud máxima, pero **todavía no tiene un índice único**. La unicidad e índices de negocio se formalizarán en 2.9.

Paso 8: Generar la migración incremental desde 2.1

Si estás reproduciendo el ejercicio manualmente sobre una copia del estado 2.1, después de introducir los cambios anteriores ejecuta:

```
dotnet ef migrations add M2_2_2
```

`migrations add` compara el modelo modificado con el snapshot heredado y genera el delta necesario.

En el checkpoint entregado **no debes volver a ejecutar ese comando**, porque la migración ya existe:

```
Migrations/20260927204717_M2_2_2.cs
```

Generarla de nuevo sobre el estado final produciría otra migración distinta y dejaría de representar el proceso incremental original.

Paso 9: Revisar qué cambia realmente la migración M2_2_2

Abre `Migrations/20260927204717_M2_2_2.cs` antes de aplicar nada. Entre sus operaciones reales están:

- añadir `Activa` y `Peso` a `PlanchasAcero`;
- cambiar `NumeroOrden` a `nvarchar(50)`;
- cambiar `Cliente` a `nvarchar(200)`;
- establecer `GETDATE()` como valor por defecto de `FechaCreacion`;
- añadir `Estado`, `FechaEntrega` y `Observaciones`;
- añadir `Activo` a `EstadosOrden`;
- añadir `Codigo` y `Descripcion` a `Aleaciones`;
- aplicar las longitudes máximas configuradas.

Por ejemplo, la columna `Peso` aparece como:

```

migrationBuilder.AddColumn<decimal>(
    name: "Peso",
    table: "PlanchasAcero",
    type: "decimal(18,3)",
    precision: 18,
    scale: 3,
    nullable: false,
    defaultValue: 0m);

```

La migración de 2.2 **no debe contener índices de negocio ni la configuración explícita de nuevas relaciones**. Si aparecen, se ha adelantado contenido posterior.

Paso 10: Aplicar la migración y verificar el esquema

Sobre el laboratorio construido desde el estado anterior:

```
dotnet ef database update
dotnet ef migrations list
```

Comprueba en SQL Server Object Explorer, al menos:

- `OrdenesFabricacion.NumeroOrden` → longitud máxima 50;
- `OrdenesFabricacion.Cliente` → longitud máxima 200;
- `OrdenesFabricacion.FechaEntrega` → admite `NULL`;
- `OrdenesFabricacion.Observaciones` → admite `NULL`;
- `PlanchasAcero.Peso` → `decimal(18,3)`;
- `PlanchasAcero.Activa` → `bit` con valor por defecto;
- las nuevas propiedades de `Aleaciones` y `EstadosOrden`.

El flujo del curso está gobernado por **Migrations**. No se usa `EnsureCreated()` para sustituir este historial.

Paso 11: Ejecutar el estado completo y comprobar sus datos

Ejecuta:

```
dotnet build AceriaData.sln --configuration Release
dotnet run --project AceriaData.Console.csproj --configuration Release
```

El programa aplica el historial con `Database.Migrate()`, inserta una orden de demostración y termina mostrando una evidencia equivalente a:

2.2 OK | Órdenes: 1 | Planchas: 1

La plancha de prueba utiliza `Peso = 371.250m`, de forma que la ejecución atraviesa realmente la propiedad cuya precisión se ha configurado.

Errores comunes

Error	Causa	Corrección
La migración sale vacía	El modelo no cambió respecto al snapshot	Revisar que las propiedades y su configuración se hayan incorporado antes de generar la migración
<code>FechaEntrega</code> no admite <code>NULL</code>	Se declaró como <code>DateTime</code>	Usar <code>DateTime?</code>
<code>Peso</code> no conserva la escala prevista	Falta <code>HasPrecision(18, 3)</code>	Configurar precisión y escala antes de generar la migración
Aparecen índices únicos en 2.2	Se adelantó contenido de 2.9	Mantener en 2.2 sólo configuración de propiedades
Aparece una nueva relación muchos-a-muchos	Se adelantó contenido de un punto posterior	Reservar <code>OrdenAleacion</code> para 2.5
Se usa <code>EnsureCreated()</code>		

Error	Causa	Corrección
	Se sustituye el historial de migraciones por creación directa	Usar la migración incremental y <code>Database.Migrate() / database update</code>

Retos resueltos: comprobar el modelo mediante metadatos

Retos: verificar desde EF Core que `Observaciones` sigue siendo opcional y que `Peso` tiene precisión 18 y escala 3.

El checkpoint contiene un bloque pedagógico comentado `RETO 2.2 - COMPROBAR PROPIEDADES`. Al activarlo se consultan los metadatos del modelo:

```
var ordenType = context.Model.FindEntityType(typeof(OrdenFabricacion))
?? throw new InvalidOperationException("No se encontró OrdenFabricacion en el modelo.");
var observaciones = ordenType.FindProperty(nameof(OrdenFabricacion.Observaciones))
?? throw new InvalidOperationException("No se encontró Observaciones en el modelo.");

var planchaType = context.Model.FindEntityType(typeof(PlanchaAcero))
?? throw new InvalidOperationException("No se encontró PlanchaAcero en el modelo.");
var peso = planchaType.FindProperty(nameof(PlanchaAcero.Peso))
?? throw new InvalidOperationException("No se encontró Peso en el modelo.");

global::System.Console.WriteLine($"Observaciones nullable: {observaciones.IsNullable}");
global::System.Console.WriteLine(
    $"Peso precision/scale: {peso.GetPrecision()}/{peso.GetScale()}");
```

El resultado esperado debe confirmar:

```
Observaciones nullable: True
Peso precision/scale: 18/3
```

Este reto no añade índices, relaciones ni propiedades de puntos posteriores.

Analogía final

Configurar propiedades se parece a fijar las especificaciones dimensionales de una pieza antes de fabricarla. No basta con saber que existe un campo “peso”: hay que decidir con qué precisión se registra. No basta con tener un “número de orden”: conviene limitar su longitud. Y una fecha de entrega puede no conocerse todavía, del mismo modo que una orden de producción puede abrirse antes de cerrar todos sus datos logísticos. El modelo convierte esas decisiones del dominio en reglas persistentes.

Resultado esperado

Al finalizar 2.2, `AceriaData` conserva el estado heredado de 2.1 y añade únicamente el delta de entidades y propiedades. La migración `M2_2_2` representa ese cambio, el esquema se aplica mediante `Migrations`, el programa termina con `2.2 OK` y el reto confirma la nulabilidad de `Observaciones` y la precisión 18/3 de `Peso`.

Conexión con el siguiente punto

Con las propiedades ya definidas y persistidas, 2.3 podrá centrarse exclusivamente en hacer explícita la relación uno-a-muchos entre `OrdenFabricacion` y `PlanchaAcero`. No se vuelve a crear el proyecto: se continúa desde este estado.

Código acumulativo completo del estado 2.2

El siguiente archivo es el `Program.cs` real del estado ejecutable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

```

using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    public string Estado { get; set; } = "Pendiente";
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
}

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class Aleacion
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public stringCodigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    public string? Descripcion { get; set; }
}

public class EstadoOrden
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<OrdenFabricacion>(entity =>
        {
            entity.ToTable("OrdenesFabricacion");
            entity.HasKey(o => o.Id);
            entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);
            entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);
            entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");
            entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");
        });
    }
}

```

```

        entity.Property(o => o.Observaciones).HasMaxLength(500);
    });
    modelBuilder.Entity<PlanchaAcero>(entity =>
    {
        entity.ToTable("PlanchasAcero");
        entity.HasKey(x => x.Id);
        entity.Property(x => x.Peso).HasPrecision(18, 3);
        entity.Property(x => x.Activa).HasDefaultValue(true);
    });
    modelBuilder.Entity<Aleacion>(entity =>
    {
        entity.ToTable("Aleaciones");
        entity.HasKey(a => a.Id);
        entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
        entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
        entity.Property(a => a.Descripcion).HasMaxLength(500);
    });
    modelBuilder.Entity<EstadoOrden>(entity =>
    {
        entity.ToTable("EstadosOrden");
        entity.HasKey(e => e.Id);
        entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
        entity.Property(e => e.Descripcion).HasMaxLength(250);
        entity.Property(e => e.Activo).HasDefaultValue(true);
    });
}

}

public interface IOrdenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    int Guardar();
}

public sealed class OrdenRepositorio : IOrdenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o =>
o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) =>
_context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOrdenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);

```

```

        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();
        services.AddDbContext<AceriaDbContext>(options =>
            options
                .UseSqlServer(connectionString, sql =>
                {
                    sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                    sql.CommandTimeout(60);
                })
                .LogTo(global::System.Console.WriteLine, new[] {
                    "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)
                .EnableDetailedErrors());

        services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
        services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

        using var provider = services.BuildServiceProvider(new ServiceProviderOptions
        {
            ValidateScopes = true,
            ValidateOnBuild = true
        });

        using var scope = provider.CreateScope();
        var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
        context.Database.EnsureDeleted();
        context.Database.Migrate();

        var orden = new OrdenFabricacion
        {
            NumeroOrden = "OF-M2-0001",
            Cliente = "Constructora del Norte",
            FechaCreacion = DateTime.UtcNow,
            FechaEntrega = DateTime.Today.AddDays(14),
            Estado = "Pendiente",
            Observaciones = "Orden de validación M2",
            Planchas =
            {
                new PlanchaAcero
                {
                    Espesor = 10.5,
                    Ancho = 1500,
                    Largo = 3000,
                    Peso = 371.250m,
                    Activa = true,
                }
            },
        };

        context.OrdenesFabricacion.Add(orden);
        context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa",
        Activo = true });
        context.SaveChanges();

        global::System.Console.WriteLine($"2.2 OK | Órdenes: {context.OrdenesFabricacion.Count()} |

```

```
Planchas: {context.PlanchasAcero.Count()});
    }
}
```

Explicación línea a línea del código acumulativo 2.2

Las referencias siguientes usan la numeración real de `M02/PROYECTO/2.2/Program.cs`. Se explican todas las líneas no vacías para que el documento sea trazable con el archivo que compila.

Línea 1: `using System.ComponentModel.DataAnnotations;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 2: `using System.ComponentModel.DataAnnotations.Schema;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 3: `using Microsoft.EntityFrameworkCore;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 4: `using Microsoft.Extensions.Configuration;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 5: `using Microsoft.Extensions.DependencyInjection;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 6: `using Microsoft.Extensions.Logging;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 8: `namespace AceriaData.ConsoleApp;` → declara el espacio de nombres del estado acumulativo.

Línea 10: `public class OrdenFabricacion` → declara la clase `OrdenFabricacion` que forma parte del estado acumulativo del ejercicio.

Línea 11: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 12: `public int Id { get; set; }` → declara la propiedad `Id` de tipo `int` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 13: `public string NumeroOrden { get; set; } = string.Empty;` → declara la propiedad `NumeroOrden` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 14: `public string Cliente { get; set; } = string.Empty;` → declara la propiedad `Cliente` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 15: `public DateTime FechaCreacion { get; set; }` → declara la propiedad `FechaCreacion` de tipo `DateTime` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 16: `public DateTime? FechaEntrega { get; set; }` → declara la propiedad `FechaEntrega` de tipo `DateTime?` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 17: `public string Estado { get; set; } = "Pendiente";` → declara la propiedad `Estado` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 18: `public string? Observaciones { get; set; }` → declara la propiedad `Observaciones` de tipo `string?` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 19: `public List<PlanchaAcero> Planchas { get; set; } = new();` → declara la propiedad `Planchas` de tipo `List` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 20: `}` → cierra el bloque de código actual.

Línea 22: `public class PlanchaAcero` → declara la clase `PlanchaAcero` que forma parte del estado acumulativo del ejercicio.

Línea 23: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 24: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 25: `public int OrdenId { get; set; }` → declara la propiedad OrdenId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 26: `public double Espesor { get; set; }` → declara la propiedad Espesor de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 27: `public double Ancho { get; set; }` → declara la propiedad Ancho de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 28: `public double Largo { get; set; }` → declara la propiedad Largo de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 29: `public decimal Peso { get; set; }` → declara la propiedad Peso de tipo decimal que EF Core o la lógica de negocio utiliza en este modelo.

Línea 30: `public bool Activa { get; set; } = true;` → declara la propiedad Activa de tipo bool que EF Core o la lógica de negocio utiliza en este modelo.

Línea 31: `public OrdenFabricacion Orden { get; set; } = null!` → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.

Línea 32: `}` → cierra el bloque de código actual.

Línea 34: `public class Aleacion` → declara la clase Aleacion que forma parte del estado acumulativo del ejercicio.

Línea 35: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 36: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 37: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 38: `public string Codigo { get; set; } = string.Empty;` → declara la propiedad Codigo de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 39: `public double PorcentajeCarbono { get; set; }` → declara la propiedad PorcentajeCarbono de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 40: `public double PorcentajeManganeso { get; set; }` → declara la propiedad PorcentajeManganeso de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 41: `public string? Descripcion { get; set; }` → declara la propiedad Descripcion de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 42: `}` → cierra el bloque de código actual.

Línea 44: `public class EstadoOrden` → declara la clase EstadoOrden que forma parte del estado acumulativo del ejercicio.

Línea 45: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 46: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 47: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 48: `public string Descripcion { get; set; } = string.Empty;` → declara la propiedad Descripcion de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 49: `public bool Activo { get; set; } = true;` → declara la propiedad Activo de tipo bool que EF Core o la lógica de negocio utiliza en este modelo.

Línea 50: `}` → cierra el bloque de código actual.

Línea 52: `public class AceriaDbContext : DbContext` → declara la clase `AceriaDbContext` que forma parte del estado acumulativo del ejercicio.

Línea 53: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 54: `public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }` → declara el constructor o método `AceriaDbContext` con los parámetros necesarios para este componente.

Línea 56: `public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 57: `public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 58: `public DbSet<Aleacion> Aleaciones => Set<Aleacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 59: `public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 62: `protected override void OnModelCreating(ModelBuilder modelBuilder)` → sobrescribe `OnModelCreating` para definir configuración explícita del modelo.

Línea 63: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 64: `base.OnModelCreating(modelBuilder);` → sobrescribe `OnModelCreating` para definir configuración explícita del modelo.

Línea 65: `modelBuilder.Entity<OrdenFabricacion>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 66: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 67: `entity.ToTable("OrdenesFabricacion");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 68: `entity.HasKey(o => o.Id);` → define la clave primaria de la entidad.

Línea 69: `entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 70: `entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);` → selecciona una propiedad escalar para continuar su configuración.

Línea 71: `entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");` → selecciona una propiedad escalar para continuar su configuración.

Línea 72: `entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");` → selecciona una propiedad escalar para continuar su configuración.

Línea 73: `entity.Property(o => o.Observaciones).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 74: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 75: `modelBuilder.Entity<PlanchaAcero>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 76: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 77: `entity.ToTable("PlanchasAcero");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 78: `entity.HasKey(x => x.Id);` → define la clave primaria de la entidad.

Línea 79: `entity.Property(x => x.Peso).HasPrecision(18, 3);` → selecciona una propiedad escalar para continuar su configuración.

Línea 80: `entity.Property(x => x.Activa).HasDefaultValue(true);` → selecciona una propiedad escalar para continuar su configuración.

Línea 81: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 82: `modelBuilder.Entity<Aleacion>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 83: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 84: `entity.ToTable("Aleaciones");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 85: `entity.HasKey(a => a.Id);` → define la clave primaria de la entidad.

Línea 86: `entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);` → selecciona una propiedad escalar para continuar su configuración.

Línea 87: `entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);` → selecciona una propiedad escalar para continuar su configuración.

Línea 88: `entity.Property(a => a.Descripcion).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 89: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 90: `modelBuilder.Entity<EstadoOrden>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 91: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 92: `entity.ToTable("EstadosOrden");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 93: `entity.HasKey(e => e.Id);` → define la clave primaria de la entidad.

Línea 94: `entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 95: `entity.Property(e => e.Descripcion).HasMaxLength(250);` → selecciona una propiedad escalar para continuar su configuración.

Línea 96: `entity.Property(e => e.Activo).HasDefaultValue(true);` → selecciona una propiedad escalar para continuar su configuración.

Línea 97: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 100: `}` → cierra el bloque de código actual.

Línea 101: `}` → cierra el bloque de código actual.

Línea 103: `public interface IOOrdenRepositorio` → declara el contrato IOOrdenRepositorio para desacoplar consumidores e implementación.

Línea 104: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 105: `List<OrdenFabricacion> ObtenerTodas();` → declara en el contrato la operación que debe devolver todas las órdenes visibles para el repositorio.

Línea 106: `OrdenFabricacion? ObtenerPorId(int id);` → declara en el contrato la búsqueda de una orden por su clave primaria y permite devolver null si no existe.

Línea 107: `OrdenFabricacion? ObtenerPorNumero(string numeroOrden);` → declara en el contrato la búsqueda por el identificador de negocio `NumeroOrden`.

Línea 108: `void Agregar(OrdenFabricacion orden);` → declara la operación que incorpora una orden al repositorio para que EF Core la siga como nueva.

Línea 109: `void Eliminar(OrdenFabricacion orden);` → declara la operación que marca una orden para eliminación física cuando el escenario lo requiere.

Línea 110: `int Guardar();` → declara la operación que confirma los cambios pendientes y devuelve el número de entradas afectadas.

Línea 111: `}` → cierra el bloque de código actual.

Línea 113: `public sealed class OrdenRepositorio : IOrdenRepositorio` → declara la clase `OrdenRepositorio` que forma parte del estado acumulativo del ejercicio.

Línea 114: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 115: `private readonly AceriaDbContext _context;` → declara el campo privado `_context` de tipo `AceriaDbContext` que conserva una dependencia de la clase.

Línea 116: `public OrdenRepositorio(AceriaDbContext context) => _context = context;` → declara el constructor o método `OrdenRepositorio` con los parámetros necesarios para este componente.

Línea 118: `public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();` → ordena el resultado de la consulta antes de materializarlo.

Línea 119: `public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);` → implementa la búsqueda por clave primaria mediante `Find`, aprovechando primero el `ChangeTracker` y después la base de datos.

Línea 120: `public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);` → recupera la primera coincidencia o null cuando no existe.

Línea 121: `public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 122: `public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);` → marca la entidad indicada para borrado al confirmar los cambios.

Línea 123: `public int Guardar() => _context.SaveChanges();` → persiste en SQL Server los cambios seguidos por el `DbContext`.

Línea 124: `}` → cierra el bloque de código actual.

Línea 126: `public interface IServicioOrdenes` → declara el contrato `IServicioOrdenes` para desacoplar consumidores e implementación.

Línea 127: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 128: `string ObtenerResumen(int ordenId);` → declara el servicio que construye un resumen de una orden a partir de su identificador.

Línea 129: `}` → cierra el bloque de código actual.

Línea 131: `public sealed class ServicioOrdenes : IServicioOrdenes` → declara la clase `ServicioOrdenes` que forma parte del estado acumulativo del ejercicio.

Línea 132: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 133: `private readonly IOrdenRepositorio _repositorio;` → declara el campo privado `_repositorio` de tipo `IOrdenRepositorio` que conserva una dependencia de la clase.

Línea 134: `private readonly AceriaDbContext _context;` → declara el campo privado `_context` de tipo `AceriaDbContext` que conserva una dependencia de la clase.

Línea 135: `public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)` → declara el constructor o método `ServicioOrdenes` con los parámetros necesarios para este componente.

Línea 136: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 137: `_repositorio = repositorio;` → guarda en el servicio el repositorio inyectado para reutilizarlo en sus operaciones.

Línea 138: `_context = context;` → guarda el `DbContext` inyectado para consultar los datos necesarios durante el servicio.

Línea 139: `}` → cierra el bloque de código actual.

Línea 141: `public string ObtenerResumen(int ordenId)` → declara el método que obtiene la orden y compone el resumen solicitado por el servicio.

Línea 142: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 143: `var orden = _repositorio.ObtenerPorId(ordenId);` → consulta la orden mediante el repositorio y conserva el resultado anulable en una variable local.

Línea 144: `if (orden is null) return $"Orden {ordenId} no encontrada";` → evalúa la condición y ejecuta el bloque asociado sólo cuando se cumple.

Línea 145: `var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);` → cuenta las filas que cumplen la consulta construida.

Línea 146: `return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";` → devuelve el resultado calculado al código que invocó el método.

Línea 147: `}` → cierra el bloque de código actual.

Línea 148: `}` → cierra el bloque de código actual.

Línea 150: `public static class Program` → declara la clase `Program` que forma parte del estado acumulativo del ejercicio.

Línea 151: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 152: `public static void Main()` → declara el punto de entrada ejecutable de la aplicación de consola.

Línea 153: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 154: `var configuration = new ConfigurationBuilder()` → inicia la construcción de la configuración de la aplicación.

Línea 155: `.SetBasePath(AppContext.BaseDirectory)` → establece la ruta base usada para localizar los archivos de configuración.

Línea 156: `.AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)` → incorpora `appsettings.json` a la configuración.

Línea 157: `.AddEnvironmentVariables()` → incorpora variables de entorno como fuente adicional de configuración.

Línea 158: `.Build();` → materializa la configuración o el objeto encadenado construido en las líneas anteriores.

Línea 160: `var connectionString = configuration.GetConnectionString("AceriaDB")` → obtiene la cadena de conexión `AceriaDB` desde la configuración.

Línea 161: `?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");` → interrumpe el arranque con un error explícito si la configuración no contiene `ConnectionStrings:AceriaDB`.

Línea 163: `var services = new ServiceCollection();` → crea la colección donde se registran las dependencias de la aplicación.

Línea 164: `services.AddDbContext<AceriaDbContext>(options =>` → registra el `DbContext` en el contenedor de inyección de dependencias.

Línea 165: `options` → continúa la configuración del `DbContext` sobre el parámetro `options` recibido por la lambda de `AddDbContext`.

Línea 166: `.UseSqlServer(connectionString, sql =>` → selecciona SQL Server como proveedor de EF Core.

Línea 167: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 168: `sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);` → habilita reintentos ante fallos transitorios del proveedor SQL Server.

Línea 169: `sql.CommandTimeout(60);` → establece el tiempo máximo permitido para los comandos SQL.

Línea 170: `})` → cierra la lambda de configuración actual y devuelve el control a la llamada encadenada que la contiene.

Línea 171: `.LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)` → configura la salida del logging de EF Core.

Línea 172: `.EnableDetailedErrors();` → activa errores detallados para facilitar el diagnóstico del laboratorio.

Línea 174: `services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();` → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.

Línea 175: `services.AddScoped<IServicioOrdenes, ServicioOrdenes>();` → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.

Línea 177: `using var provider = services.BuildServiceProvider(new ServiceProviderOptions` → construye el `ServiceProvider` con validación de servicios y garantiza su liberación al salir del ámbito mediante `using var`.

Línea 178: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 179: `ValidateScopes = true,` → activa la validación de ámbitos para detectar dependencias scoped usadas desde ámbitos incorrectos.

Línea 180: `ValidateOnBuild = true` → obliga al contenedor a validar las resoluciones registradas al construir el `ServiceProvider`.

Línea 181: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 183: `using var scope = provider.CreateScope();` → crea un ámbito de DI para resolver servicios scoped y lo libera automáticamente al terminar el bloque.

Línea 184: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → resuelve un servicio obligatorio desde el contenedor de dependencias.

Línea 185: `context.Database.EnsureDeleted();` → elimina la base del laboratorio para reproducir el escenario desde un estado limpio.

Línea 186: `context.Database.Migrate();` → aplica las migraciones pendientes y deja AceriaDB en el esquema del punto.

Línea 188: `var orden = new OrdenFabricacion` → crea la entidad `OrdenFabricacion` que concentra el grafo de datos del escenario acumulativo.

Línea 189: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 190: `NumeroOrden = "OF-M2-0001"`, → asigna el identificador de negocio usado por la orden de validación acumulativa del módulo.

Línea 191: `Cliente = "Constructora del Norte"`, → asigna el cliente de la orden utilizada en el escenario práctico del punto.

Línea 192: `FechaCreacion = DateTime.UtcNow`, → asigna a la orden la fecha de creación en UTC para el escenario reproducible del punto.

Línea 193: `FechaEntrega = DateTime.Today.AddDays(14)`, → establece una fecha de entrega prevista catorce días después de la fecha local actual.

Línea 194: `Estado = "Pendiente"`, → inicializa la orden con el estado `Pendiente`, coherente con el flujo del laboratorio.

Línea 195: `Observaciones = "Orden de validación M2"`, → añade texto opcional para comprobar el mapeo de `Observaciones`.

Línea 196: `Planchas =` → inicia el inicializador de la colección de planchas asociadas a la orden.

Línea 197: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 198: `new PlanchaAcero` → crea una nueva plancha dentro del inicializador de la colección `Planchas` de la orden.

Línea 199: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 200: `Espesor = 10.5`, → asigna a la plancha un espesor positivo para satisfacer las reglas del modelo.

Línea 201: `Ancho = 1500`, → asigna el ancho de la plancha usada en el escenario de persistencia.

Línea 202: `Largo = 3000`, → asigna el largo de la plancha usada en el escenario de persistencia.

Línea 203: `Peso = 371.250m`, → asigna un peso decimal con tres posiciones para validar la precisión configurada.

Línea 204: `Activa = true`, → marca la plancha como activa para que sea visible en los estados que aplican ese criterio.

Línea 205: `}` → cierra el bloque de código actual.

Línea 206: `,` → continúa el inicializador o la llamada iniciada en el bloque actual.

Línea 207: `};` → cierra el inicializador del objeto y finaliza la instrucción.

Línea 209: `context.OrdenesFabricacion.Add(orden);` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 210: `context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo = true });` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 211: `context.SaveChanges();` → persiste en SQL Server los cambios seguidos por el `DbContext`.

Línea 213: `global::System.Console.WriteLine($"2.2 OK | Órdenes: {context.OrdenesFabricacion.Count()} | Planchas: {context.PlanchasAcero.Count()}");` → muestra la evidencia de ejecución de 2.2 junto con los recuentos persistidos de órdenes y planchas.

Línea 214: `}` → cierra el bloque de código actual.

Línea 215: `}` → cierra el bloque de código actual.

Punto 2.3 - Relaciones uno a muchos

Ejercicio guiado: configurar explícitamente la relación uno-a-muchos entre `OrdenFabricacion` y `PlanchaAcero`, identificar los extremos principal y dependiente, fijar la clave foránea `OrdenId`, establecer el comportamiento de eliminación y comprobar la carga de la colección con `Include`.

Contexto del proyecto: 2.3 continúa directamente desde 2.2. Las entidades y sus propiedades escalares ya están configuradas. La relación entre orden y plancha ya podía ser descubierta por convención, pero ahora se expresa de forma explícita con Fluent API para que su intención y su comportamiento queden visibles en el modelo. Este punto se limita a la relación `OrdenFabricacion 1 -> N PlanchaAcero`: las relaciones uno-a-uno se estudian en 2.4 y la relación muchos-a-muchos se reserva para 2.5.

Objetivos de aprendizaje

- Comprender qué representa una relación uno-a-muchos.
- Identificar el extremo principal y el extremo dependiente.
- Localizar la clave foránea en la entidad dependiente.
- Configurar la relación con `HasOne`, `WithMany` y `HasForeignKey`.
- Configurar `DeleteBehavior.Cascade` de forma explícita.
- Entender por qué `IsRequired()` es coherente con una FK `int` no anulable.
- Diferenciar un cambio de configuración del modelo de un cambio real del esquema SQL.
- Cargar la colección relacionada con `Include`.
- Comprobar la relación con datos reales en SQL Server LocalDB.
- No adelantar `DetalleOrden`, `CertificadoCalidad` ni `OrdenAleacion`.

Paso 1: Abrir el estado autónomo de 2.3

```
cd M02/PROYECTO/2.3
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

El checkpoint 2.3 contiene el resultado completo del ejercicio. Si estás realizando el laboratorio desde 2.2, aplica los cambios siguientes sobre tu copia del punto anterior y genera después la migración incremental.

Paso 2: Revisar los dos extremos de la relación

Las propiedades relevantes son:

```
public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public List<PlanchaAcero> Planchas { get; set; } = new();
}

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    public OrdenFabricacion Orden { get; set; } = null!;
}
```

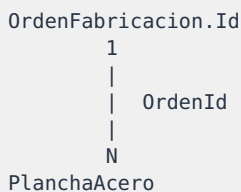
Interpreta el modelo antes de configurarlo:

- `OrdenFabricacion` es el **principal**: una orden puede contener varias planchas.
- `PlanchaAcero` es el **dependiente**: cada plancha pertenece a una orden.
- `OrdenFabricacion.Planchas` es la navegación de colección.
- `PlanchaAcero.Orden` es la navegación de referencia.
- `PlanchaAcero.OrdenId` almacena la clave primaria de la orden asociada.

La clave foránea reside en el dependiente. El nombre `OrdenId` también puede ser reconocido por convención, pero en este punto se declarará explícitamente.

Paso 3: Identificar la convención antes de sustituirla por configuración explícita

EF Core ya puede relacionar estas propiedades:



La colección y la referencia permiten deducir la cardinalidad. La finalidad de 2.3 no es inventar una segunda relación, sino hacer explícita la existente para controlar su comportamiento.

Un nombre como `OrdenFabricacionId` también encaja en las convenciones habituales. Un nombre no convencional, por ejemplo `IdOrden`, requeriría configuración explícita para evitar una FK sombra o una relación distinta de la esperada.

Paso 4: Configurar la relación con Fluent API

En `OnModelCreating`, la configuración efectiva de `PlanchaAcero` queda así:

```

modelBuilder.Entity<PlanchaAcero>(entity =>
{
    entity.ToTable("PlanchasAcero");
    entity.HasKey(x => x.Id);
    entity.Property(x => x.Peso).HasPrecision(18, 3);
    entity.Property(x => x.Activa).HasDefaultValue(true);

    entity.HasOne(x => x.Orden)
        .WithMany(o => o.Planchas)
        .HasForeignKey(x => x.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
});
  
```

Línea a línea:

1. `HasOne(x => x.Orden)` selecciona la navegación de referencia del dependiente.
2. `WithMany(o => o.Planchas)` conecta esa referencia con la colección del principal.
3. `HasForeignKey(x => x.OrdenId)` declara explícitamente qué propiedad actúa como FK.
4. `OnDelete(DeleteBehavior.Cascade)` indica que la eliminación física del principal se propaga a sus planchas.
5. `IsRequired()` expresa que una plancha persistida debe tener una orden válida.

La relación debe configurarse de forma coherente una sola vez. No hace falta repetirla desde `OrdenFabricacion` con otra configuración.

Paso 5: Comprender el comportamiento de eliminación

EF Core permite distintos comportamientos de eliminación. En este ejercicio se utiliza:

```
.onDelete(DeleteBehavior.Cascade)
```

Cascade es coherente con el modelo actual: si una orden se elimina físicamente, sus planchas dependientes no deben quedar huérfanas.

Otros comportamientos existen, pero no se aplican aquí de forma intercambiable:

- **Restrict / NoAction**: impiden la eliminación del principal mientras existan dependientes, según cómo actúe el proveedor.
- **SetNull**: requiere una FK que pueda admitir `NULL`; `OrdenId` es `int` no anulable.
- los comportamientos de cliente afectan a cómo EF Core gestiona entidades rastreadas y no deben confundirse con una regla SQL idéntica en todos los proveedores.

El objetivo es entender por qué se elige `Cascade`, no memorizar una opción para todas las relaciones.

Paso 6: Compilar antes de generar la migración

Después de introducir la configuración:

```
dotnet build AceriaData.sln --configuration Release
```

El build debe terminar sin errores. Si aparece un error relacionado con `WithMany`, `HasForeignKey` o una navegación inexistente, revisa primero que las propiedades de ambos extremos coincidan con el código del paso 2.

Paso 7: Generar la migración M2_2_3 desde el estado 2.2

Si estás reproduciendo el laboratorio sobre una copia de 2.2:

```
dotnet ef migrations add M2_2_3
```

En el checkpoint entregado la migración ya existe:

```
Migrations/20260927204726_M2_2_3.cs
```

No vuelvas a generar una migración con el mismo nombre dentro del checkpoint final. El comando anterior representa el paso que debe realizarse cuando se construye 2.3 incrementalmente desde 2.2.

Paso 8: Revisar por qué M2_2_3 no necesita operaciones SQL nuevas

Abre `Migrations/20260927204726_M2_2_3.cs`. Su contenido relevante es:

```
protected override void Up(MigrationBuilder migrationBuilder)
{
}

protected override void Down(MigrationBuilder migrationBuilder)
{
}
```

Esto no significa que el ejercicio no haya hecho nada. La relación ya estaba presente en el esquema heredado porque las propiedades y navegaciones permitían a EF Core descubrirla por convención. En 2.3 se hace **explícita** la misma forma relacional:

- FK `PlanchasAcero.OrdenId`;
- principal `OrdenesFabricacion.Id`;

- índice sobre `OrdenId`;
- eliminación en cascada.

Como la configuración explícita coincide con el esquema ya representado por el snapshot anterior, no existe un delta SQL adicional que aplicar.

Esta es una diferencia importante: **cambiar cómo se expresa una relación en el modelo no implica necesariamente cambiar la base de datos.**

▮ Paso 9: Aplicar el historial y verificar la relación real

Ejecuta:

```
dotnet ef database update
dotnet ef migrations list
```

La lista debe incluir `M2_2_3`. En SQL Server Object Explorer puedes comprobar en `dbo.PlanchasAcero`:

- la columna `OrdenId`;
- el índice `IX_PlanchasAcero_OrdenId`;
- la FK `FK_PlanchasAcero_OrdenesFabricacion_OrdenId`;
- la referencia a `OrdenesFabricacion(Id)`;
- el comportamiento de eliminación en cascada.

En este punto no debe aparecer una tabla `DetallesOrden` ni `OrdenesAleaciones`: pertenecen a puntos posteriores.

▮ Paso 10: Insertar una orden con una plancha relacionada

El estado final crea el agregado utilizando la navegación de colección:

```
var orden = new OrdenFabricacion
{
    NumeroOrden = "OF-M2-0001",
    Cliente = "Constructora del Norte",
    FechaCreacion = DateTime.UtcNow,
    Estado = "Pendiente",
    Planchas =
    {
        new PlanchaAcero
        {
            Espesor = 10.5,
            Ancho = 1500,
            Largo = 3000,
            Peso = 371.250m,
            Activa = true,
        },
    },
};

context.OrdenesFabricacion.Add(orden);
context.SaveChanges();
```

Al agregar el principal con una plancha en su colección, EF Core mantiene la relación y asigna la FK correspondiente al guardar.

Si se intentase persistir una plancha con una `OrdenId` que no corresponde a una orden existente, SQL Server rechazaría la operación por integridad referencial.

▮ Paso 11: Cargar la colección con Include

La comprobación funcional del punto usa carga eager:

```
var cargada = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .Single(o => o.NumeroOrden == "OF-M2-0001");

global::System.Console.WriteLine(
    $"2.3 OK | {cargada.NumeroOrden} | Planchas: {cargada.Planchas.Count}");
```

La salida esperada del estado base es:

```
2.3 OK | OF-M2-0001 | Planchas: 1
```

`Include(o => o.Planchas)` pide explícitamente que la colección se cargue con la consulta. No debe suponerse que la navegación se cargará automáticamente.

Paso 12: Resolver el reto con dos planchas

Reto: crear otra orden con exactamente dos planchas, recuperarla con `Include` y demostrar que la colección contiene dos elementos.

El checkpoint contiene el bloque pedagógico **RETO 2.3 - DOS PLANCHAS CON INCLUDE**. La parte esencial es:

```
var ordenReto = new OrdenFabricacion
{
    NumeroOrden = "OF-M2-RETO-23",
    Cliente = "Cliente reto 2.3",
    FechaCreacion = DateTime.UtcNow,
    Estado = "Pendiente",
    Planchas =
    {
        new PlanchaAcero
        {
            Espesor = 8,
            Ancho = 1200,
            Largo = 2500,
            Peso = 188.500m,
            Activa = true,
        },
        new PlanchaAcero
        {
            Espesor = 12,
            Ancho = 1500,
            Largo = 3000,
            Peso = 424.125m,
            Activa = true,
        },
    },
};

context.OrdenesFabricacion.Add(ordenReto);
context.SaveChanges();

var retoCargada = context.OrdenesFabricacion
    .AsNoTracking()
    .Include(o => o.Planchas)
    .Single(o => o.NumeroOrden == "OF-M2-RETO-23");

global::System.Console.WriteLine(
    $"Reto 2.3 planchas: {retoCargada.Planchas.Count}");
```

El resultado esperado es:

```
Reto 2.3 planchas: 2
```

El reto amplía la misma relación 1:N; no introduce todavía una relación 1:1 ni N:M.

Errores comunes

Error	Causa	Corrección
La FK no se asocia a la navegación prevista	La propiedad no sigue una convención y no se configuró explícitamente	Usar <code>HasForeignKey(x => x.OrdenId)</code>
Se crea una FK sombra adicional	La navegación y la FK se configuraron de forma incoherente	Relacionar explícitamente <code>Orden</code> , <code>Planchas</code> y <code>OrdenId</code>
Se configura la relación dos veces con opciones diferentes	Se repite desde ambos extremos con reglas contradictorias	Mantener una única configuración coherente
Se usa <code>SetNull</code> con <code>OrdenId</code> no anulable	La FK no puede almacenar <code>NULL</code>	Mantener <code>Cascade</code> en este escenario o rediseñar conscientemente la nulabilidad
La colección no aparece cargada	Se consultó la orden sin <code>Include</code>	Usar <code>Include(o => o.Planchas)</code> cuando se necesita el grafo
Se crea <code>DetalleOrden</code> o <code>OrdenAleacion</code>	Se adelantó contenido de 2.4 o 2.5	Mantener 2.3 exclusivamente en <code>OrdenFabricacion 1 -> N PlanchaAcero</code>
Se usa <code>EnsureCreated()</code>	Se sustituye el historial incremental por creación directa	Mantener el flujo basado en Migrations

Analogía final

La relación se parece a una orden de fabricación y las planchas producidas para ella. Una orden puede agrupar muchas planchas, pero cada plancha conserva una referencia a una sola orden mediante `OrdenId`. La colección `Planchas` funciona como el listado de piezas asociado a la orden; la navegación `Orden` permite recorrer la relación en sentido contrario. Definir la relación explícitamente equivale a dejar escritas las reglas de ese vínculo en el plano del sistema, incluida la política que se aplica si el principal se elimina físicamente.

Resultado esperado

Al finalizar 2.3, `AceriaData` conserva todas las propiedades de 2.2 y hace explícita la relación `OrdenFabricacion 1 -> N PlanchaAcero`. El historial incluye `M2_2_3`, cuya ausencia de operaciones SQL se explica porque la forma relacional ya había sido descubierta por convención. La aplicación termina con `2.3 OK | 0F-M2-0001 | Planchas: 1`, y el reto confirma una segunda orden con exactamente dos planchas cargadas mediante `Include`.

Conexión con el siguiente punto

Con la relación uno-a-muchos ya expresada de forma explícita, 2.4 añadirá las relaciones uno-a-uno del modelo. Hasta entonces, 2.3 no incorpora entidades ni configuraciones propias de esos puntos posteriores.

Código acumulativo completo del estado 2.3

El siguiente archivo es el `Program.cs` real del estado ejecutable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;
```

```

public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    public string Estado { get; set; } = "Pendiente";
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
}

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class Aleacion
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public stringCodigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    public string? Descripcion { get; set; }
}

public class EstadoOrden
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<OrdenFabricacion>(entity =>
        {
            entity.ToTable("OrdenesFabricacion");
            entity.HasKey(o => o.Id);
            entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);
            entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);
            entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");
            entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");
            entity.Property(o => o.Observaciones).HasMaxLength(500);
        });
        modelBuilder.Entity<PlanchaAcero>(entity =>
        {
            entity.ToTable("PlanchasAcero");
            entity.HasKey(x => x.Id);
            entity.Property(x => x.Peso).HasPrecision(18, 3);
            entity.Property(x => x.Activa).HasDefaultValue(true);
            entity.HasOne(x => x.Orden)

```

```

        .WithMany(o => o.Planchas)
        .HasForeignKey(x => x.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
    });
    modelBuilder.Entity<Aleacion>(entity =>
    {
        entity.ToTable("Aleaciones");
        entity.HasKey(a => a.Id);
        entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
        entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
        entity.Property(a => a.Descripcion).HasMaxLength(500);
    });
    modelBuilder.Entity<EstadoOrden>(entity =>
    {
        entity.ToTable("EstadosOrden");
        entity.HasKey(e => e.Id);
        entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
        entity.Property(e => e.Descripcion).HasMaxLength(250);
        entity.Property(e => e.Activo).HasDefaultValue(true);
    });
}

}

public interface IOordenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    int Guardar();
}

public sealed class OrdenRepositorio : IOordenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o =>
o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) =>
_context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOordenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOordenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

```

```

}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();
        services.AddDbContext<AceriaDbContext>(options =>
            options
                .UseSqlServer(connectionString, sql =>
                {
                    sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                    sql.CommandTimeout(60);
                })
                .LogTo(global::System.Console.WriteLine, new[] {
                    "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)
                .EnableDetailedErrors());

        services.AddScoped<IOOrdenRepositorio, OrdenRepositorio>();
        services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

        using var provider = services.BuildServiceProvider(new ServiceProviderOptions
        {
            ValidateScopes = true,
            ValidateOnBuild = true
        });

        using var scope = provider.CreateScope();
        var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
        context.Database.EnsureDeleted();
        context.Database.Migrate();

        var orden = new OrdenFabricacion
        {
            NumeroOrden = "OF-M2-0001",
            Cliente = "Constructora del Norte",
            FechaCreacion = DateTime.UtcNow,
            FechaEntrega = DateTime.Today.AddDays(14),
            Estado = "Pendiente",
            Observaciones = "Orden de validación M2",
            Planchas =
            {
                new PlanchaAcero
                {
                    Espesor = 10.5,
                    Ancho = 1500,
                    Largo = 3000,
                    Peso = 371.250m,
                    Activa = true,
                }
            },
        };

        context OrdenesFabricacion.Add(orden);
        context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa",
        Activo = true });
        context.SaveChanges();

        var cargada = context OrdenesFabricacion.Include(o => o.Planchas).Single(o => o.NumeroOrden ==
        "OF-M2-0001");
        global::System.Console.WriteLine($"2.3 OK | {cargada.NumeroOrden} | Planchas:
        {cargada.Planchas.Count}");
    }
}

```

| Explicación línea a línea del código acumulativo 2.3

Las referencias siguientes usan la numeración real de `M02/PROYECTO/2.3/Program.cs`. Se explican todas las líneas no vacías para que el documento sea trazable con el archivo que compila.

Línea 1: `using System.ComponentModel.DataAnnotations;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 2: `using System.ComponentModel.DataAnnotations.Schema;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 3: `using Microsoft.EntityFrameworkCore;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 4: `using Microsoft.Extensions.Configuration;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 5: `using Microsoft.Extensions.DependencyInjection;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 6: `using Microsoft.Extensions.Logging;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 8: `namespace AceriaData.ConsoleApp;` → declara el espacio de nombres del estado acumulativo.

Línea 10: `public class OrdenFabricacion` → declara la clase `OrdenFabricacion` que forma parte del estado acumulativo del ejercicio.

Línea 11: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 12: `public int Id { get; set; }` → declara la propiedad `Id` de tipo `int` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 13: `public string NumeroOrden { get; set; } = string.Empty;` → declara la propiedad `NumeroOrden` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 14: `public string Cliente { get; set; } = string.Empty;` → declara la propiedad `Cliente` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 15: `public DateTime FechaCreacion { get; set; }` → declara la propiedad `FechaCreacion` de tipo `DateTime` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 16: `public DateTime? FechaEntrega { get; set; }` → declara la propiedad `FechaEntrega` de tipo `DateTime?` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 17: `public string Estado { get; set; } = "Pendiente";` → declara la propiedad `Estado` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 18: `public string? Observaciones { get; set; }` → declara la propiedad `Observaciones` de tipo `string?` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 19: `public List<PlanchaAcero> Planchas { get; set; } = new();` → declara la propiedad `Planchas` de tipo `List` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 20: `}` → cierra el bloque de código actual.

Línea 22: `public class PlanchaAcero` → declara la clase `PlanchaAcero` que forma parte del estado acumulativo del ejercicio.

Línea 23: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 24: `public int Id { get; set; }` → declara la propiedad `Id` de tipo `int` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 25: `public int OrdenId { get; set; }` → declara la propiedad `OrdenId` de tipo `int` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 26: `public double Espesor { get; set; }` → declara la propiedad Espesor de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 27: `public double Ancho { get; set; }` → declara la propiedad Ancho de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 28: `public double Largo { get; set; }` → declara la propiedad Largo de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 29: `public decimal Peso { get; set; }` → declara la propiedad Peso de tipo decimal que EF Core o la lógica de negocio utiliza en este modelo.

Línea 30: `public bool Activa { get; set; } = true;` → declara la propiedad Activa de tipo bool que EF Core o la lógica de negocio utiliza en este modelo.

Línea 31: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.

Línea 32: `}` → cierra el bloque de código actual.

Línea 34: `public class Aleacion` → declara la clase Aleacion que forma parte del estado acumulativo del ejercicio.

Línea 35: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 36: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 37: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 38: `public string Codigo { get; set; } = string.Empty;` → declara la propiedad Codigo de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 39: `public double PorcentajeCarbono { get; set; }` → declara la propiedad PorcentajeCarbono de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 40: `public double PorcentajeManganeso { get; set; }` → declara la propiedad PorcentajeManganeso de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 41: `public string? Descripcion { get; set; }` → declara la propiedad Descripcion de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 42: `}` → cierra el bloque de código actual.

Línea 44: `public class EstadoOrden` → declara la clase EstadoOrden que forma parte del estado acumulativo del ejercicio.

Línea 45: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 46: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 47: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 48: `public string Descripcion { get; set; } = string.Empty;` → declara la propiedad Descripcion de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 49: `public bool Activo { get; set; } = true;` → declara la propiedad Activo de tipo bool que EF Core o la lógica de negocio utiliza en este modelo.

Línea 50: `}` → cierra el bloque de código actual.

Línea 52: `public class AceriaDbContext : DbContext` → declara la clase AceriaDbContext que forma parte del estado acumulativo del ejercicio.

Línea 53: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 54: `public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }` → declara el constructor o método `AceriaDbContext` con los parámetros necesarios para este componente.

Línea 56: `public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 57: `public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 58: `public DbSet<Aleacion> Aleaciones => Set<Aleacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 59: `public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 62: `protected override void OnModelCreating(ModelBuilder modelBuilder)` → sobrescribe `OnModelCreating` para definir configuración explícita del modelo.

Línea 63: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 64: `base.OnModelCreating(modelBuilder);` → sobrescribe `OnModelCreating` para definir configuración explícita del modelo.

Línea 65: `modelBuilder.Entity<OrdenFabricacion>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 66: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 67: `entity.ToTable("OrdenesFabricacion");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 68: `entity.HasKey(o => o.Id);` → define la clave primaria de la entidad.

Línea 69: `entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 70: `entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);` → selecciona una propiedad escalar para continuar su configuración.

Línea 71: `entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");` → selecciona una propiedad escalar para continuar su configuración.

Línea 72: `entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");` → selecciona una propiedad escalar para continuar su configuración.

Línea 73: `entity.Property(o => o.Observaciones).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 74: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 75: `modelBuilder.Entity<PlanchaAcero>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 76: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 77: `entity.ToTable("PlanchasAcero");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 78: `entity.HasKey(x => x.Id);` → define la clave primaria de la entidad.

Línea 79: `entity.Property(x => x.Peso).HasPrecision(18, 3);` → selecciona una propiedad escalar para continuar su configuración.

Línea 80: `entity.Property(x => x.Activa).HasDefaultValue(true);` → selecciona una propiedad escalar para continuar su configuración.

Línea 81: `entity.HasOne(x => x.Orden)` → inicia la configuración de una navegación de referencia.

Línea 82: `.WithMany(o => o.Planchas)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 83: `.HasForeignKey(x => x.OrdenId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 84: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 85: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 86: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 87: `modelBuilder.Entity<Aleacion>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 88: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 89: `entity.ToTable("Aleaciones");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 90: `entity.HasKey(a => a.Id);` → define la clave primaria de la entidad.

Línea 91: `entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);` → selecciona una propiedad escalar para continuar su configuración.

Línea 92: `entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);` → selecciona una propiedad escalar para continuar su configuración.

Línea 93: `entity.Property(a => a.Descripcion).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 94: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 95: `modelBuilder.Entity<EstadoOrden>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 96: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 97: `entity.ToTable("EstadosOrden");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 98: `entity.HasKey(e => e.Id);` → define la clave primaria de la entidad.

Línea 99: `entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 100: `entity.Property(e => e.Descripcion).HasMaxLength(250);` → selecciona una propiedad escalar para continuar su configuración.

Línea 101: `entity.Property(e => e.Activo).HasDefaultValue(true);` → selecciona una propiedad escalar para continuar su configuración.

Línea 102: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 105: `}` → cierra el bloque de código actual.

Línea 106: `}` → cierra el bloque de código actual.

Línea 108: `public interface IOrdenRepositorio` → declara el contrato IOrdenRepositorio para desacoplar consumidores e implementación.

Línea 109: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 110: `List<OrdenFabricacion> ObtenerTodas();` → declara en el contrato la operación que debe devolver todas las órdenes visibles para el repositorio.

Línea 111: `OrdenFabricacion? ObtenerPorId(int id);` → declara en el contrato la búsqueda de una orden por su clave primaria y permite devolver null si no existe.

Línea 112: `OrdenFabricacion? ObtenerPorNumero(string numeroOrden);` → declara en el contrato la búsqueda por el identificador de negocio NumeroOrden.

Línea 113: `void Agregar(OrdenFabricacion orden);` → declara la operación que incorpora una orden al repositorio para que EF Core la siga como nueva.

Línea 114: `void Eliminar(OrdenFabricacion orden);` → declara la operación que marca una orden para eliminación física cuando el escenario lo requiere.

Línea 115: `int Guardar();` → declara la operación que confirma los cambios pendientes y devuelve el número de entradas afectadas.

Línea 116: `}` → cierra el bloque de código actual.

Línea 118: `public sealed class OrdenRepositorio : IOrdenRepositorio` → declara la clase OrdenRepositorio que forma parte del estado acumulativo del ejercicio.

Línea 119: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 120: `private readonly AceriaDbContext _context;` → declara el campo privado _context de tipo AceriaDbContext que conserva una dependencia de la clase.

Línea 121: `public OrdenRepositorio(AceriaDbContext context) => _context = context;` → declara el constructor o método OrdenRepositorio con los parámetros necesarios para este componente.

Línea 123: `public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();` → ordena el resultado de la consulta antes de materializarlo.

Línea 124: `public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);` → implementa la búsqueda por clave primaria mediante Find, aprovechando primero el ChangeTracker y después la base de datos.

Línea 125: `public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);` → recupera la primera coincidencia o null cuando no existe.

Línea 126: `public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);` → registra una entidad o relación nueva en el ChangeTracker para insertarla al guardar.

Línea 127: `public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);` → marca la entidad indicada para borrado al confirmar los cambios.

Línea 128: `public int Guardar() => _context.SaveChanges();` → persiste en SQL Server los cambios seguidos por el DbContext.

Línea 129: `}` → cierra el bloque de código actual.

Línea 131: `public interface IServicioOrdenes` → declara el contrato IServicioOrdenes para desacoplar consumidores e implementación.

Línea 132: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 133: `string ObtenerResumen(int ordenId);` → declara el servicio que construye un resumen de una orden a partir de su identificador.

Línea 134: `}` → cierra el bloque de código actual.

Línea 136: `public sealed class ServicioOrdenes : IServicioOrdenes` → declara la clase `ServicioOrdenes` que forma parte del estado acumulativo del ejercicio.

Línea 137: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 138: `private readonly IOrdenRepositorio _repositorio;` → declara el campo privado `_repositorio` de tipo `IOrdenRepositorio` que conserva una dependencia de la clase.

Línea 139: `private readonly AceriaDbContext _context;` → declara el campo privado `_context` de tipo `AceriaDbContext` que conserva una dependencia de la clase.

Línea 140: `public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)` → declara el constructor o método `ServicioOrdenes` con los parámetros necesarios para este componente.

Línea 141: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 142: `_repositorio = repositorio;` → guarda en el servicio el repositorio inyectado para reutilizarlo en sus operaciones.

Línea 143: `_context = context;` → guarda el `DbContext` inyectado para consultar los datos necesarios durante el servicio.

Línea 144: `}` → cierra el bloque de código actual.

Línea 146: `public string ObtenerResumen(int ordenId)` → declara el método que obtiene la orden y compone el resumen solicitado por el servicio.

Línea 147: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 148: `var orden = _repositorio.ObtenerPorId(ordenId);` → consulta la orden mediante el repositorio y conserva el resultado anulable en una variable local.

Línea 149: `if (orden is null) return $"Orden {ordenId} no encontrada";` → evalúa la condición y ejecuta el bloque asociado sólo cuando se cumple.

Línea 150: `var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);` → cuenta las filas que cumplen la consulta construida.

Línea 151: `return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";` → devuelve el resultado calculado al código que invocó el método.

Línea 152: `}` → cierra el bloque de código actual.

Línea 153: `}` → cierra el bloque de código actual.

Línea 155: `public static class Program` → declara la clase `Program` que forma parte del estado acumulativo del ejercicio.

Línea 156: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 157: `public static void Main()` → declara el punto de entrada ejecutable de la aplicación de consola.

Línea 158: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 159: `var configuration = new ConfigurationBuilder()` → inicia la construcción de la configuración de la aplicación.

Línea 160: `.SetBasePath(AppContext.BaseDirectory)` → establece la ruta base usada para localizar los archivos de configuración.

Línea 161: `.AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)` → incorpora `appsettings.json` a la configuración.

Línea 162: `.AddEnvironmentVariables()` → incorpora variables de entorno como fuente adicional de configuración.

Línea 163: `.Build();` → materializa la configuración o el objeto encadenado construido en las líneas anteriores.

Línea 165: `var connectionString = configuration.GetConnectionString("AceriaDB")` → obtiene la cadena de conexión AceriaDB desde la configuración.

Línea 166: `?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");` → interrumpe el arranque con un error explícito si la configuración no contiene `ConnectionStrings:AceriaDB`.

Línea 168: `var services = new ServiceCollection();` → crea la colección donde se registran las dependencias de la aplicación.

Línea 169: `services.AddDbContext<AceriaDbContext>(options =>` → registra el DbContext en el contenedor de inyección de dependencias.

Línea 170: `options` → continúa la configuración del DbContext sobre el parámetro options recibido por la lambda de `AddDbContext`.

Línea 171: `.UseSqlServer(connectionString, sql =>` → selecciona SQL Server como proveedor de EF Core.

Línea 172: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 173: `sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);` → habilita reintentos ante fallos transitorios del proveedor SQL Server.

Línea 174: `sql.CommandTimeout(60);` → establece el tiempo máximo permitido para los comandos SQL.

Línea 175: `})` → cierra la lambda de configuración actual y devuelve el control a la llamada encadenada que la contiene.

Línea 176: `.LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)` → configura la salida del logging de EF Core.

Línea 177: `.EnableDetailedErrors();` → activa errores detallados para facilitar el diagnóstico del laboratorio.

Línea 179: `services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();` → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.

Línea 180: `services.AddScoped<IServicioOrdenes, ServicioOrdenes>();` → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.

Línea 182: `using var provider = services.BuildServiceProvider(new ServiceProviderOptions` → construye el ServiceProvider con validación de servicios y garantiza su liberación al salir del ámbito mediante `using var`.

Línea 183: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 184: `ValidateScopes = true,` → activa la validación de ámbitos para detectar dependencias scoped usadas desde ámbitos incorrectos.

Línea 185: `ValidateOnBuild = true` → obliga al contenedor a validar las resoluciones registradas al construir el ServiceProvider.

Línea 186: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 188: `using var scope = provider.CreateScope();` → crea un ámbito de DI para resolver servicios scoped y lo libera automáticamente al terminar el bloque.

Línea 189: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → resuelve un servicio obligatorio desde el contenedor de dependencias.

Línea 190: `context.Database.EnsureDeleted();` → elimina la base del laboratorio para reproducir el escenario desde un estado limpio.

Línea 191: `context.Database.Migrate();` → aplica las migraciones pendientes y deja AceriaDB en el esquema del punto.

Línea 193: `var orden = new OrdenFabricacion` → crea la entidad `OrdenFabricacion` que concentra el grafo de datos del escenario acumulativo.

Línea 194: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 195: `NumeroOrden = "0F-M2-0001"`, → asigna el identificador de negocio usado por la orden de validación acumulativa del módulo.

Línea 196: `Cliente = "Constructora del Norte"`, → asigna el cliente de la orden utilizada en el escenario práctico del punto.

Línea 197: `FechaCreacion = DateTime.UtcNow,` → asigna a la orden la fecha de creación en UTC para el escenario reproducible del punto.

Línea 198: `FechaEntrega = DateTime.Today.AddDays(14),` → establece una fecha de entrega prevista catorce días después de la fecha local actual.

Línea 199: `Estado = "Pendiente"`, → inicializa la orden con el estado `Pendiente`, coherente con el flujo del laboratorio.

Línea 200: `Observaciones = "Orden de validación M2"`, → añade texto opcional para comprobar el mapeo de `Observaciones`.

Línea 201: `Planchas =` → inicia el inicializador de la colección de planchas asociadas a la orden.

Línea 202: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 203: `new PlanchaAcero` → crea una nueva plancha dentro del inicializador de la colección `Planchas` de la orden.

Línea 204: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 205: `Espesor = 10.5,` → asigna a la plancha un espesor positivo para satisfacer las reglas del modelo.

Línea 206: `Ancho = 1500,` → asigna el ancho de la plancha usada en el escenario de persistencia.

Línea 207: `Largo = 3000,` → asigna el largo de la plancha usada en el escenario de persistencia.

Línea 208: `Peso = 371.250m,` → asigna un peso decimal con tres posiciones para validar la precisión configurada.

Línea 209: `Activa = true,` → marca la plancha como activa para que sea visible en los estados que aplican ese criterio.

Línea 210: `}` → cierra el bloque de código actual.

Línea 211: `},` → continúa el inicializador o la llamada iniciada en el bloque actual.

Línea 212: `};` → cierra el inicializador del objeto y finaliza la instrucción.

Línea 214: `context.OrdenesFabricacion.Add(orden);` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 215: `context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo = true });` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 216: `context.SaveChanges();` → persiste en SQL Server los cambios seguidos por el `DbContext`.

Línea 218: `var cargada = context.OrdenesFabricacion.Include(o => o.Planchas).Single(o => o.NumeroOrden == "0F-M2-0001");` → incluye de forma eager la navegación relacionada en la consulta.

Línea 219: `global::System.Console.WriteLine($"2.3 OK | {cargada.NumeroOrden} | Planchas: {cargada.Planchas.Count}");` → escribe en consola la evidencia usada para verificar el comportamiento del estado.

Línea 220: `}` → cierra el bloque de código actual.

Línea 221: `}` → cierra el bloque de código actual.

Punto 2.4 - Relaciones uno a uno

Ejercicio guiado: añadir y configurar dos relaciones uno-a-uno en `AceriaData`: `OrdenFabricacion -> DetalleOrden` y `OrdenFabricacion -> CertificadoCalidad`. El ejercicio identifica principal y dependiente, hace explícita la clave foránea, comprueba la unicidad de `OrdenId`, aplica eliminación en cascada y carga ambas referencias con `Include`.

Contexto del proyecto: 2.4 continúa directamente desde 2.3. La relación uno-a-muchos entre orden y planchas ya está configurada. En este punto se incorporan `DetalleOrden` y `CertificadoCalidad` como dependientes uno-a-uno de `OrdenFabricacion`. Una orden puede existir sin detalle o sin certificado; si alguno de esos dependientes existe, su `OrdenId` es obligatorio y debe identificar una única orden.

Objetivos de aprendizaje

- Diferenciar una relación uno-a-uno de una relación uno-a-muchos.
- Identificar principal y dependiente en una relación 1:1.
- Entender por qué el dependiente contiene la clave foránea.
- Configurar `HasOne`, `WithOne` y `HasForeignKey<TDependiente>`.
- Comprender la relación entre `WithOne` y el índice único sobre la FK.
- Permitir que la navegación desde la orden sea opcional sin convertir en opcional la FK del dependiente.
- Configurar `DeleteBehavior.Cascade`.
- Añadir `DetalleOrden` y `CertificadoCalidad` sin duplicar la configuración de la misma relación.
- Revisar la migración real y los dos índices únicos generados.
- Insertar y cargar una orden con detalle y certificado.
- Verificar que SQL Server rechaza un segundo certificado para la misma orden.

Paso 1: Abrir el estado autónomo de 2.4

```
cd M02/PROYECTO/2.4
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

El estado 2.4 contiene la solución completa del punto. Si estás construyendo el ejercicio manualmente desde 2.3, aplica sobre tu copia los cambios de los pasos siguientes y genera después la migración incremental.

Paso 2: Añadir las navegaciones uno-a-uno a `OrdenFabricacion`

La entidad principal incorpora dos referencias opcionales:

```
public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    public string Estado { get; set; } = "Pendiente";
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();

    public DetalleOrden? Detalle { get; set; }
    public CertificadoCalidad? Certificado { get; set; }
}
```

Las dos navegaciones llevan `?` porque una orden puede crearse antes de disponer de su detalle técnico o de su certificado de calidad. Esa opcionalidad pertenece al **principal**: no obliga a que toda orden tenga un dependiente.

No significa que un `DetalleOrden` o un `CertificadoCalidad` existente pueda quedar sin orden. Esa regla se expresará en sus propias configuraciones mediante una FK `int` no anulable y `.IsRequired()`.

Paso 3: Crear `DetalleOrden` y exponerlo en el `DbContext`

La entidad dependiente del detalle es:

```
public class DetalleOrden
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    public string? Notas { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}
```

y el contexto la expone con:

```
public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();
```

Interpreta cada parte:

- `Id` es la clave primaria propia del detalle.
- `OrdenId` es la clave foránea hacia `OrdenFabricacion`.
- `Orden` es la navegación de referencia hacia el principal.
- `ComposicionQuimica` es requerida.
- `Notas` es opcional.

`DetalleOrden` es el dependiente porque contiene `OrdenId`.

Paso 4: Configurar una sola vez la relación `OrdenFabricacion-DetalleOrden`

La configuración definitiva se hace desde el dependiente:

```
modelBuilder.Entity<DetalleOrden>(entity =>
{
    entity.ToTable("DetallesOrden");
    entity.HasKey(d => d.Id);

    entity.Property(d => d.ComposicionQuimica)
        .IsRequired()
        .HasMaxLength(200);

    entity.Property(d => d.Notas)
        .HasMaxLength(500);

    entity.HasOne(d => d.Orden)
        .WithOne(o => o.Detalle)
        .HasForeignKey<DetalleOrden>(d => d.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
});
```

Línea a línea:

1. `HasOne(d => d.Orden)` selecciona la referencia del dependiente al principal.
2. `WithOne(o => o.Detalle)` establece que la navegación opuesta también es una única referencia.

3. `HasForeignKey<DetalleOrden>(d => d.OrdenId)` identifica explícitamente a `DetalleOrden` como dependiente y a `OrdenId` como FK.
4. `OnDelete(DeleteBehavior.Cascade)` propaga al detalle la eliminación física de su orden.
5. `IsRequired()` expresa que un detalle existente debe pertenecer a una orden.

No se configura una segunda vez la misma relación desde `OrdenFabricacion` con `IsRequired(false)`. La navegación `DetalleOrden? Detalle` ya expresa que **la orden puede no tener detalle**, mientras que `DetalleOrden.OrdenId` sigue siendo obligatorio cuando el dependiente existe.

Paso 5: Crear CertificadoCalidad y exponerlo en el DbContext

La segunda relación uno-a-uno usa otra entidad dependiente:

```
public class CertificadoCalidad
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public string NumeroCertificado { get; set; } = string.Empty;
    public DateTime FechaEmision { get; set; }
    public string OrganismoCertificador { get; set; } = string.Empty;
    public OrdenFabricacion Orden { get; set; } = null!;
}
```

El contexto añade:

```
public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();
```

Aquí también `OrdenId` reside en el dependiente. `NumeroCertificado` es un dato de negocio del certificado, pero en 2.4 no se adelantan las claves alternativas del punto 2.8.

Paso 6: Configurar la relación OrdenFabricacion-CertificadoCalidad

La configuración real del estado es:

```
modelBuilder.Entity<CertificadoCalidad>(entity =>
{
    entity.ToTable("CertificadosCalidad");
    entity.HasKey(c => c.Id);

    entity.Property(c => c.NumeroCertificado)
        .IsRequired()
        .HasMaxLength(50);

    entity.Property(c => c.OrganismoCertificador)
        .IsRequired()
        .HasMaxLength(100);

    entity.HasOne(c => c.Orden)
        .WithOne(o => o.Certificado)
        .HasForeignKey<CertificadoCalidad>(c => c.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
});
```

La forma es paralela a `DetalleOrden`:

- `OrdenFabricacion` es principal.
- `CertificadoCalidad` es dependiente.
- `OrdenId` es la FK.
- una orden puede existir sin certificado;
- si el certificado existe, debe apuntar a una orden válida.

Paso 7: Entender qué hace realmente que la relación sea uno-a-uno

Una FK normal permite que varias filas dependientes apunten al mismo principal. Para que la cardinalidad sea 1:1, `OrdenId` debe ser **único** en cada tabla dependiente.

La combinación:

```
.WithOne(...)
.HasForeignKey<DetalleOrden>(d => d.OrdenId)
```

hace que EF Core modele una FK única para `DetallesOrden.OrdenId`. Lo mismo ocurre con `CertificadosCalidad.OrdenId`.

En SQL Server el resultado se materializa mediante índices únicos. Si se intenta insertar un segundo dependiente con la misma `OrdenId`, la base de datos rechaza la operación.

Paso 8: Generar la migración incremental M2_2_4

Si estás reproduciendo el laboratorio desde una copia de 2.3:

```
dotnet build AceriaData.sln --configuration Release
dotnet ef migrations add M2_2_4
```

En el estado entregado la migración ya está generada:

```
Migrations/20260927204735_M2_2_4.cs
```

No vuelvas a ejecutar `migrations add M2_2_4` sobre la solución final del punto. El comando representa el paso incremental que se realiza al construir 2.4 desde 2.3.

Paso 9: Revisar las tablas, claves foráneas e índices únicos de la migración

`M2_2_4` crea dos tablas nuevas:

- `DetallesOrden`;
- `CertificadosCalidad`.

Para `DetallesOrden`, la migración define la FK:

```
table.ForeignKey(
    name: "FK_DetallesOrden_OrdenesFabricacion_OrdenId",
    column: x => x.OrdenId,
    principalTable: "OrdenesFabricacion",
    principalColumn: "Id",
    onDelete: ReferentialAction.Cascade);
```

y crea:

```
migrationBuilder.CreateIndex(
    name: "IX_DetallesOrden_OrdenId",
    table: "DetallesOrden",
    column: "OrdenId",
    unique: true);
```

Para `CertificadosCalidad` aparece la misma estructura:

```
migrationBuilder.CreateIndex(
    name: "IX_CertificadosCalidad_OrdenId",
    table: "CertificadosCalidad",
    column: "OrdenId",
    unique: true);
```

Los dos `unique: true` son evidencia directa de que una misma orden no puede tener dos detalles ni dos certificados.

Paso 10: Aplicar la migración y comprobar el esquema en SQL Server

Ejecuta:

```
dotnet ef database update
dotnet ef migrations list
```

Comprueba en SQL Server Object Explorer:

- tabla `dbo.DetallesOrden`;
- tabla `dbo.CertificadosCalidad`;
- FK de cada tabla hacia `OrdenesFabricacion(Id)`;
- `IX_DetallesOrden_OrdenId` con `Unique = True`;
- `IX_CertificadosCalidad_OrdenId` con `Unique = True`.

El esquema sigue gobernado por Migrations. No se usa `EnsureCreated()` para sustituir el historial.

Paso 11: Insertar una orden con sus dos dependientes

El estado 2.4 crea una única orden y anida ambas referencias:

```
var orden = new OrdenFabricacion
{
    NumeroOrden = "OF-M2-0001",
    Cliente = "Constructora del Norte",
    FechaCreacion = DateTime.UtcNow,
    Estado = "Pendiente",

    Detalle = new DetalleOrden
    {
        ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
        TemperaturaColada = 1550.5,
        Notas = "Colada principal"
    },

    Certificado = new CertificadoCalidad
    {
        NumeroCertificado = "CERT-0001",
        FechaEmision = DateTime.Today,
        OrganismoCertificador = "Laboratorio Aceria"
    }
};

context.OrdenesFabricacion.Add(orden);
context.SaveChanges();
```

Al guardar el grafo, EF Core inserta primero la orden y propaga su clave a `DetalleOrden.OrdenId` y `CertificadoCalidad.OrdenId`.

Paso 12: Cargar Detalle y Certificado con Include

La comprobación funcional carga las dos referencias:

```
var cargada = context.OrdenesFabricacion
    .Include(o => o.Detalle)
    .Include(o => o.Certificado)
    .Single(o => o.NumeroOrden == "OF-M2-0001");

global::System.Console.WriteLine(
    $"2.4 OK | Detalle: {cargada.Detalle?.ComposicionQuimica} | " +
    $"Certificado: {cargada.Certificado?.NumeroCertificado}");
```

La salida debe contener:

```
2.4 OK | Detalle: C: 0.45%, Mn: 0.75% | Certificado: CERT-0001
```

`Include` es necesario porque las referencias no se cargan por arte de magia en una consulta nueva. El ejercicio hace explícito qué parte del grafo necesita.

Paso 13: Resolver el reto de unicidad con un segundo certificado

Reto: intentar guardar un segundo `CertificadoCalidad` con la misma `OrdenId` y comprobar que SQL Server lo rechaza.

La solución del punto contiene el bloque comentado `RETO 2.4 - CERTIFICADO ÚNICO`. Utiliza un ámbito y un `DbContext` nuevos para que la prueba llegue realmente a la restricción única de la base de datos, sin que el relationship fixup del `ChangeTracker` altere el escenario:

```
using var retoScope = provider.CreateScope();
var retoContext =
    retoScope.ServiceProvider.GetRequiredService<AceriaDbContext>();

retoContext.CertificadosCalidad.Add(new CertificadoCalidad
{
    OrdenId = orden.Id,
    NumeroCertificado = "CERT-DUPLICADO",
    FechaEmision = DateTime.Today,
    OrganismoCertificador = "Laboratorio duplicado"
});

try
{
    retoContext.SaveChanges();
    global::System.Console.WriteLine(
        "Reto 2.4 ERROR: se permitió un segundo certificado.");
}
catch (DbUpdateException)
{
    global::System.Console.WriteLine(
        "Reto 2.4: segundo certificado rechazado");
}
```

Resultado esperado:

```
Reto 2.4: segundo certificado rechazado
```

La prueba confirma en el proveedor real que el índice único sobre `CertificadosCalidad.OrdenId` protege la cardinalidad 1:1.

Errores comunes

Error	Causa	Corrección
Se configura la misma relación dos veces con reglas distintas	Se intenta expresar la opcionalidad desde ambos extremos	Configurar una sola vez desde el dependiente
Se usa <code>IsRequired(false)</code> sobre una FK <code>int</code> no anulable	Se confunde “la orden puede no tener detalle” con “un detalle puede no tener orden”	Mantener <code>OrdenId</code> obligatorio y la navegación del principal como nullable
Aparecen dos detalles para una orden	La FK no tiene restricción única	Comprobar <code>IX_DetallesOrden_OrdenId</code> con <code>Unique = True</code>

Error	Causa	Corrección
El certificado duplicado se sustituye en memoria en vez de llegar a SQL Server	Se reutiliza el mismo grafo rastreado	Ejecutar la prueba de unicidad con un <code>DbContext</code> separado
Detalle o certificado aparecen <code>null</code> al consultar	Se omitió <code>Include</code>	Incluir explícitamente las navegaciones necesarias
Se añade <code>OrdenAleacion</code>	Se adelanta el punto 2.5	Mantener 2.4 exclusivamente en relaciones 1:1
Se usa <code>EnsureCreated()</code>	Se evita el historial incremental	Usar Migrations y <code>Database.Migrate()</code> / <code>database update</code>

Analogía final

Una relación uno-a-uno se parece al expediente técnico único de una orden de fabricación. La orden puede existir mientras ese expediente todavía se está preparando, pero, una vez creado, el detalle pertenece a una sola orden y no puede duplicarse para la misma `OrdenId`. El certificado de calidad sigue la misma regla: una orden puede estar pendiente de certificación, pero el certificado que exista queda ligado de forma única a ella. Los índices únicos son la regla física que impide que aparezcan dos expedientes o dos certificados para la misma orden.

Resultado esperado

Al finalizar 2.4, `AceriaData` conserva todo lo construido en 2.3 y añade `DetalleOrden` y `CertificadoCalidad`. La migración `M2_2_4` crea las dos tablas, sus claves foráneas con `Cascade` y los índices únicos sobre `OrdenId`. La aplicación carga ambas referencias mediante `Include` y termina con la evidencia `2.4 OK`. El reto confirma que SQL Server rechaza un segundo certificado asociado a la misma orden.

Conexión con el siguiente punto

Con las relaciones 1:N y 1:1 ya definidas, 2.5 añadirá la relación muchos-a-muchos entre `OrdenFabricacion` y `Aleacion` mediante la entidad intermedia explícita `OrdenAleacion`, donde la propia relación tendrá datos de negocio.

Código acumulativo completo del estado 2.4

El siguiente archivo es el `Program.cs` real del estado ejecutable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    public string Estado { get; set; } = "Pendiente";
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
    public DetalleOrden? Detalle { get; set; }
    public CertificadoCalidad? Certificado { get; set; }
}
```

```

}

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class Aleacion
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Codigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    public string? Descripcion { get; set; }
}

public class EstadoOrden
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
}

public class DetalleOrden
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    public string? Notas { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class CertificadoCalidad
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public string NumeroCertificado { get; set; } = string.Empty;
    public DateTime FechaEmision { get; set; }
    public string OrganismoCertificador { get; set; } = string.Empty;
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
    public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();
    public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<OrdenFabricacion>(entity =>
        {
            entity.ToTable("OrdenesFabricacion");
            entity.HasKey(o => o.Id);
            entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);
            entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);
        });
    }
}

```

```

        entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");
        entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");
        entity.Property(o => o.Observaciones).HasMaxLength(500);
    });
    modelBuilder.Entity<PlanchaAcero>(entity =>
    {
        entity.ToTable("PlanchasAcero");
        entity.HasKey(x => x.Id);
        entity.Property(x => x.Peso).HasPrecision(18, 3);
        entity.Property(x => x.Activa).HasDefaultValue(true);
        entity.HasOne(x => x.Orden)
            .WithMany(o => o.Planchas)
            .HasForeignKey(x => x.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });
    modelBuilder.Entity<Aleacion>(entity =>
    {
        entity.ToTable("Aleaciones");
        entity.HasKey(a => a.Id);
        entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
        entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
        entity.Property(a => a.Descripcion).HasMaxLength(500);
    });
    modelBuilder.Entity<EstadoOrden>(entity =>
    {
        entity.ToTable("EstadosOrden");
        entity.HasKey(e => e.Id);
        entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
        entity.Property(e => e.Descripcion).HasMaxLength(250);
        entity.Property(e => e.Activo).HasDefaultValue(true);
    });
    modelBuilder.Entity<DetalleOrden>(entity =>
    {
        entity.ToTable("DetallesOrden");
        entity.HasKey(d => d.Id);
        entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);
        entity.Property(d => d.Notas).HasMaxLength(500);
        entity.HasOne(d => d.Orden)
            .WithOne(o => o.Detalle)
            .HasForeignKey<DetalleOrden>(d => d.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });

    modelBuilder.Entity<CertificadoCalidad>(entity =>
    {
        entity.ToTable("CertificadosCalidad");
        entity.HasKey(c => c.Id);
        entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);
        entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);
        entity.HasOne(c => c.Orden)
            .WithOne(o => o.Certificado)
            .HasForeignKey<CertificadoCalidad>(c => c.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });
}

}

public interface IOrdenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    int Guardar();
}

public sealed class OrdenRepositorio : IOrdenRepositorio
{

```

```

    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o =>
o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) =>
_context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOOrdenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOOrdenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();
        services.AddDbContext<AceriaDbContext>(options =>
            options
                .UseSqlServer(connectionString, sql =>
                    {
                        sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                        sql.CommandTimeout(60);
                    })
                .LogTo(global::System.Console.WriteLine, new[] {
"Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)
                .EnableDetailedErrors());

        services.AddScoped<IOOrdenRepositorio, OrdenRepositorio>();
        services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

        using var provider = services.BuildServiceProvider(new ServiceProviderOptions
        {
            ValidateScopes = true,
            ValidateOnBuild = true
        });

        using var scope = provider.CreateScope();
        var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();

```

```

context.Database.EnsureDeleted();
context.Database.Migrate();

var orden = new OrdenFabricacion
{
    NumeroOrden = "OF-M2-0001",
    Cliente = "Constructora del Norte",
    FechaCreacion = DateTime.UtcNow,
    FechaEntrega = DateTime.Today.AddDays(14),
    Estado = "Pendiente",
    Observaciones = "Orden de validación M2",
    Planchas =
    {
        new PlanchaAcero
        {
            Espesor = 10.5,
            Ancho = 1500,
            Largo = 3000,
            Peso = 371.250m,
            Activa = true,
        }
    },
    Detalle = new DetalleOrden
    {
        ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
        TemperaturaColada = 1550.5,
        Notas = "Colada principal"
    },
    Certificado = new CertificadoCalidad
    {
        NumeroCertificado = "CERT-0001",
        FechaEmision = DateTime.Today,
        OrganismoCertificador = "Laboratorio Aceria"
    },
};

context.OrdenesFabricacion.Add(orden);
context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa",
Activo = true });
context.SaveChanges();

var cargada = context.OrdenesFabricacion.Include(o => o.Detalle).Include(o =>
o.Certificado).Single(o => o.NumeroOrden == "OF-M2-0001");
global::System.Console.WriteLine($"2.4 OK | Detalle: {cargada.Detalle?.ComposicionQuimica} |
Certificado: {cargada.Certificado?.NumeroCertificado}");
}
}

```

Explicación línea a línea del código acumulativo 2.4

Las referencias siguientes usan la numeración real de M02/PROYECTO/2.4/Program.cs. Se explican todas las líneas no vacías para que el documento sea trazable con el archivo que compila.

Línea 1: `using System.ComponentModel.DataAnnotations;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 2: `using System.ComponentModel.DataAnnotations.Schema;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 3: `using Microsoft.EntityFrameworkCore;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 4: `using Microsoft.Extensions.Configuration;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 5: `using Microsoft.Extensions.DependencyInjection;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 6: `using Microsoft.Extensions.Logging;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 8: `namespace AceriaData.ConsoleApp;` → declara el espacio de nombres del estado acumulativo.

Línea 10: `public class OrdenFabricacion` → declara la clase `OrdenFabricacion` que forma parte del estado acumulativo del ejercicio.

Línea 11: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 12: `public int Id { get; set; }` → declara la propiedad `Id` de tipo `int` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 13: `public string NumeroOrden { get; set; } = string.Empty;` → declara la propiedad `NumeroOrden` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 14: `public string Cliente { get; set; } = string.Empty;` → declara la propiedad `Cliente` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 15: `public DateTime FechaCreacion { get; set; }` → declara la propiedad `FechaCreacion` de tipo `DateTime` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 16: `public DateTime? FechaEntrega { get; set; }` → declara la propiedad `FechaEntrega` de tipo `DateTime?` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 17: `public string Estado { get; set; } = "Pendiente";` → declara la propiedad `Estado` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 18: `public string? Observaciones { get; set; }` → declara la propiedad `Observaciones` de tipo `string?` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 19: `public List<PlanchaAcero> Planchas { get; set; } = new();` → declara la propiedad `Planchas` de tipo `List` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 20: `public DetalleOrden? Detalle { get; set; }` → declara la propiedad `Detalle` de tipo `DetalleOrden?` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 21: `public CertificadoCalidad? Certificado { get; set; }` → declara la propiedad `Certificado` de tipo `CertificadoCalidad?` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 22: `}` → cierra el bloque de código actual.

Línea 24: `public class PlanchaAcero` → declara la clase `PlanchaAcero` que forma parte del estado acumulativo del ejercicio.

Línea 25: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 26: `public int Id { get; set; }` → declara la propiedad `Id` de tipo `int` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 27: `public int OrdenId { get; set; }` → declara la propiedad `OrdenId` de tipo `int` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 28: `public double Espesor { get; set; }` → declara la propiedad `Espesor` de tipo `double` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 29: `public double Ancho { get; set; }` → declara la propiedad `Ancho` de tipo `double` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 30: `public double Largo { get; set; }` → declara la propiedad `Largo` de tipo `double` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 31: `public decimal Peso { get; set; }` → declara la propiedad `Peso` de tipo `decimal` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 32: `public bool Activa { get; set; } = true;` → declara la propiedad `Activa` de tipo `bool` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 33: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad `Orden` de tipo `OrdenFabricacion` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 34: `}` → cierra el bloque de código actual.

Línea 36: `public class Aleacion` → declara la clase Aleacion que forma parte del estado acumulativo del ejercicio.

Línea 37: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 38: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 39: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 40: `public string Codigo { get; set; } = string.Empty;` → declara la propiedad Codigo de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 41: `public double PorcentajeCarbono { get; set; }` → declara la propiedad PorcentajeCarbono de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 42: `public double PorcentajeManganeso { get; set; }` → declara la propiedad PorcentajeManganeso de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 43: `public string? Descripcion { get; set; }` → declara la propiedad Descripcion de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 44: `}` → cierra el bloque de código actual.

Línea 46: `public class EstadoOrden` → declara la clase EstadoOrden que forma parte del estado acumulativo del ejercicio.

Línea 47: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 48: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 49: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 50: `public string Descripcion { get; set; } = string.Empty;` → declara la propiedad Descripcion de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 51: `public bool Activo { get; set; } = true;` → declara la propiedad Activo de tipo bool que EF Core o la lógica de negocio utiliza en este modelo.

Línea 52: `}` → cierra el bloque de código actual.

Línea 54: `public class DetalleOrden` → declara la clase DetalleOrden que forma parte del estado acumulativo del ejercicio.

Línea 55: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 56: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 57: `public int OrdenId { get; set; }` → declara la propiedad OrdenId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 58: `public string ComposicionQuimica { get; set; } = string.Empty;` → declara la propiedad ComposicionQuimica de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 59: `public double TemperaturaColada { get; set; }` → declara la propiedad TemperaturaColada de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 60: `public string? Notas { get; set; }` → declara la propiedad Notas de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 61: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.

Línea 62: `}` → cierra el bloque de código actual.

Línea 64: `public class CertificadoCalidad` → declara la clase CertificadoCalidad que forma parte del estado acumulativo del ejercicio.

Línea 65: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 66: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 67: `public int OrdenId { get; set; }` → declara la propiedad OrdenId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 68: `public string NumeroCertificado { get; set; } = string.Empty;` → declara la propiedad NumeroCertificado de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 69: `public DateTime FechaEmision { get; set; }` → declara la propiedad FechaEmision de tipo DateTime que EF Core o la lógica de negocio utiliza en este modelo.

Línea 70: `public string OrganismoCertificador { get; set; } = string.Empty;` → declara la propiedad OrganismoCertificador de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 71: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.

Línea 72: `}` → cierra el bloque de código actual.

Línea 74: `public class AceriaDbContext : DbContext` → declara la clase AceriaDbContext que forma parte del estado acumulativo del ejercicio.

Línea 75: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 76: `public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }` → declara el constructor o método AceriaDbContext con los parámetros necesarios para este componente.

Línea 78: `public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 79: `public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 80: `public DbSet<Aleacion> Aleaciones => Set<Aleacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 81: `public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 82: `public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 83: `public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 86: `protected override void OnModelCreating(ModelBuilder modelBuilder)` → sobrescribe OnModelCreating para definir configuración explícita del modelo.

Línea 87: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 88: `base.OnModelCreating(modelBuilder);` → sobrescribe OnModelCreating para definir configuración explícita del modelo.

Línea 89: `modelBuilder.Entity<OrdenFabricacion>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 90: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 91: `entity.ToTable("OrdenesFabricacion");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 92: `entity.HasKey(o => o.Id);` → define la clave primaria de la entidad.

Línea 93: `entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 94: `entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);` → selecciona una propiedad escalar para continuar su configuración.

Línea 95: `entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");` → selecciona una propiedad escalar para continuar su configuración.

Línea 96: `entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");` → selecciona una propiedad escalar para continuar su configuración.

Línea 97: `entity.Property(o => o.Observaciones).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 98: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 99: `modelBuilder.Entity<PlanchaAcero>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 100: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 101: `entity.ToTable("PlanchasAcero");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 102: `entity.HasKey(x => x.Id);` → define la clave primaria de la entidad.

Línea 103: `entity.Property(x => x.Peso).HasPrecision(18, 3);` → selecciona una propiedad escalar para continuar su configuración.

Línea 104: `entity.Property(x => x.Activa).HasDefaultValue(true);` → selecciona una propiedad escalar para continuar su configuración.

Línea 105: `entity.HasOne(x => x.Orden)` → inicia la configuración de una navegación de referencia.

Línea 106: `.WithMany(o => o.Planchas)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 107: `.HasForeignKey(x => x.OrdenId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 108: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 109: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 110: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 111: `modelBuilder.Entity<Aleacion>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 112: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 113: `entity.ToTable("Aleaciones");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 114: `entity.HasKey(a => a.Id);` → define la clave primaria de la entidad.

Línea 115: `entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);` → selecciona una propiedad escalar para continuar su configuración.

Línea 116: `entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);` → selecciona una propiedad escalar para continuar su configuración.

Línea 117: `entity.Property(a => a.Descripcion).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 118: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 119: `modelBuilder.Entity<EstadoOrden>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 120: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 121: `entity.ToTable("EstadosOrden");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 122: `entity.HasKey(e => e.Id);` → define la clave primaria de la entidad.

Línea 123: `entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 124: `entity.Property(e => e.Descripcion).HasMaxLength(250);` → selecciona una propiedad escalar para continuar su configuración.

Línea 125: `entity.Property(e => e.Activo).HasDefaultValue(true);` → selecciona una propiedad escalar para continuar su configuración.

Línea 126: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 127: `modelBuilder.Entity<DetalleOrden>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 128: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 129: `entity.ToTable("DetallesOrden");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 130: `entity.HasKey(d => d.Id);` → define la clave primaria de la entidad.

Línea 131: `entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);` → selecciona una propiedad escalar para continuar su configuración.

Línea 132: `entity.Property(d => d.Notas).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 133: `entity.HasOne(d => d.Orden)` → inicia la configuración de una navegación de referencia.

Línea 134: `.WithOne(o => o.Detalle)` → establece el otro extremo de referencia de una relación uno-a-uno.

Línea 135: `.HasForeignKey<DetalleOrden>(d => d.OrdenId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 136: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 137: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 138: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 140: `modelBuilder.Entity<CertificadoCalidad>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 141: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 142: `entity.ToTable("CertificadosCalidad");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 143: `entity.HasKey(c => c.Id);` → define la clave primaria de la entidad.

Línea 144: `entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 145: `entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);` → selecciona una propiedad escalar para continuar su configuración.

Línea 146: `entity.HasOne(c => c.Orden)` → inicia la configuración de una navegación de referencia.

Línea 147: `.WithOne(o => o.Certificado)` → establece el otro extremo de referencia de una relación uno-a-uno.

Línea 148: `.HasForeignKey<CertificadoCalidad>(c => c.OrdenId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 149: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 150: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 151: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 153: `}` → cierra el bloque de código actual.

Línea 154: `}` → cierra el bloque de código actual.

Línea 156: `public interface IOrdenRepositorio` → declara el contrato IOrdenRepositorio para desacoplar consumidores e implementación.

Línea 157: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 158: `List<OrdenFabricacion> ObtenerTodas();` → declara en el contrato la operación que debe devolver todas las órdenes visibles para el repositorio.

Línea 159: `OrdenFabricacion? ObtenerPorId(int id);` → declara en el contrato la búsqueda de una orden por su clave primaria y permite devolver null si no existe.

Línea 160: `OrdenFabricacion? ObtenerPorNumero(string numeroOrden);` → declara en el contrato la búsqueda por el identificador de negocio NumeroOrden.

Línea 161: `void Agregar(OrdenFabricacion orden);` → declara la operación que incorpora una orden al repositorio para que EF Core la siga como nueva.

Línea 162: `void Eliminar(OrdenFabricacion orden);` → declara la operación que marca una orden para eliminación física cuando el escenario lo requiere.

Línea 163: `int Guardar();` → declara la operación que confirma los cambios pendientes y devuelve el número de entradas afectadas.

Línea 164: `}` → cierra el bloque de código actual.

Línea 166: `public sealed class OrdenRepositorio : IOrdenRepositorio` → declara la clase OrdenRepositorio que forma parte del estado acumulativo del ejercicio.

Línea 167: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 168: `private readonly AceriaDbContext _context;` → declara el campo privado _context de tipo AceriaDbContext que conserva una dependencia de la clase.

Línea 169: `public OrdenRepositorio(AceriaDbContext context) => _context = context;` → declara el constructor o método OrdenRepositorio con los parámetros necesarios para este componente.

Línea 171: `public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();` → ordena el resultado de la consulta antes de materializarlo.

Línea 172: `public OrdenFabricacion? ObtenerPorId(int id) =>`
`_context.OrdenesFabricacion.Find(id);` → implementa la búsqueda por clave primaria mediante Find, aprovechando primero el ChangeTracker y después la base de datos.

Línea 173: `public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) =>`
`_context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);` → recupera la primera coincidencia o null cuando no existe.

Línea 174: `public void Agregar(OrdenFabricacion orden) =>`
`_context.OrdenesFabricacion.Add(orden);` → registra una entidad o relación nueva en el ChangeTracker para insertarla al guardar.

Línea 175: `public void Eliminar(OrdenFabricacion orden) =>`
`_context.OrdenesFabricacion.Remove(orden);` → marca la entidad indicada para borrado al confirmar los cambios.

Línea 176: `public int Guardar() => _context.SaveChanges();` → persiste en SQL Server los cambios seguidos por el DbContext.

Línea 177: `}` → cierra el bloque de código actual.

Línea 179: `public interface IServicioOrdenes` → declara el contrato IServicioOrdenes para desacoplar consumidores e implementación.

Línea 180: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 181: `string ObtenerResumen(int ordenId);` → declara el servicio que construye un resumen de una orden a partir de su identificador.

Línea 182: `}` → cierra el bloque de código actual.

Línea 184: `public sealed class ServicioOrdenes : IServicioOrdenes` → declara la clase ServicioOrdenes que forma parte del estado acumulativo del ejercicio.

Línea 185: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 186: `private readonly IOrdenRepositorio _repositorio;` → declara el campo privado _repositorio de tipo IOrdenRepositorio que conserva una dependencia de la clase.

Línea 187: `private readonly AceriaDbContext _context;` → declara el campo privado _context de tipo AceriaDbContext que conserva una dependencia de la clase.

Línea 188: `public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)` → declara el constructor o método ServicioOrdenes con los parámetros necesarios para este componente.

Línea 189: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 190: `_repositorio = repositorio;` → guarda en el servicio el repositorio inyectado para reutilizarlo en sus operaciones.

Línea 191: `_context = context;` → guarda el DbContext inyectado para consultar los datos necesarios durante el servicio.

Línea 192: `}` → cierra el bloque de código actual.

Línea 194: `public string ObtenerResumen(int ordenId)` → declara el método que obtiene la orden y compone el resumen solicitado por el servicio.

Línea 195: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 196: `var orden = _repositorio.ObtenerPorId(ordenId);` → consulta la orden mediante el repositorio y conserva el resultado anulable en una variable local.

Línea 197: `if (orden is null) return $"Orden {ordenId} no encontrada";` → evalúa la condición y ejecuta el bloque asociado sólo cuando se cumple.

Línea 198: `var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);` → cuenta las filas que cumplen la consulta construida.

Línea 199: `return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";` → devuelve el resultado calculado al código que invocó el método.

Línea 200: `}` → cierra el bloque de código actual.

Línea 201: `}` → cierra el bloque de código actual.

Línea 203: `public static class Program` → declara la clase Program que forma parte del estado acumulativo del ejercicio.

Línea 204: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 205: `public static void Main()` → declara el punto de entrada ejecutable de la aplicación de consola.

Línea 206: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 207: `var configuration = new ConfigurationBuilder()` → inicia la construcción de la configuración de la aplicación.

Línea 208: `.SetBasePath(AppContext.BaseDirectory)` → establece la ruta base usada para localizar los archivos de configuración.

Línea 209: `.AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)` → incorpora appsettings.json a la configuración.

Línea 210: `.AddEnvironmentVariables()` → incorpora variables de entorno como fuente adicional de configuración.

Línea 211: `.Build();` → materializa la configuración o el objeto encadenado construido en las líneas anteriores.

Línea 213: `var connectionString = configuration.GetConnectionString("AceriaDB")` → obtiene la cadena de conexión AceriaDB desde la configuración.

Línea 214: `?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");` → interrumpe el arranque con un error explícito si la configuración no contiene ConnectionStrings:AceriaDB.

Línea 216: `var services = new ServiceCollection();` → crea la colección donde se registran las dependencias de la aplicación.

Línea 217: `services.AddDbContext<AceriaDbContext>(options =>` → registra el DbContext en el contenedor de inyección de dependencias.

Línea 218: `options` → continúa la configuración del DbContext sobre el parámetro options recibido por la lambda de AddDbContext.

Línea 219: `.UseSqlServer(connectionString, sql =>` → selecciona SQL Server como proveedor de EF Core.

Línea 220: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 221: `sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);` → habilita reintentos ante fallos transitorios del proveedor SQL Server.

Línea 222: `sql.CommandTimeout(60);` → establece el tiempo máximo permitido para los comandos SQL.

Línea 223: `})` → cierra la lambda de configuración actual y devuelve el control a la llamada encadenada que la contiene.

Línea 224: `.LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)` → configura la salida del logging de EF Core.

Línea 225: `.EnableDetailedErrors();` → activa errores detallados para facilitar el diagnóstico del laboratorio.

Línea 227: `services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();` → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.

Línea 228: `services.AddScoped<IServicioOrdenes, ServicioOrdenes>();` → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.

Línea 230: `using var provider = services.BuildServiceProvider(new ServiceProviderOptions` → construye el ServiceProvider con validación de servicios y garantiza su liberación al salir del ámbito mediante using var.

Línea 231: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 232: `ValidateScopes = true,` → activa la validación de ámbitos para detectar dependencias scoped usadas desde ámbitos incorrectos.

Línea 233: `ValidateOnBuild = true` → obliga al contenedor a validar las resoluciones registradas al construir el ServiceProvider.

Línea 234: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 236: `using var scope = provider.CreateScope();` → crea un ámbito de DI para resolver servicios scoped y lo libera automáticamente al terminar el bloque.

Línea 237: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → resuelve un servicio obligatorio desde el contenedor de dependencias.

Línea 238: `context.Database.EnsureDeleted();` → elimina la base del laboratorio para reproducir el escenario desde un estado limpio.

Línea 239: `context.Database.Migrate();` → aplica las migraciones pendientes y deja AceriaDB en el esquema del punto.

Línea 241: `var orden = new OrdenFabricacion` → crea la entidad OrdenFabricacion que concentra el grafo de datos del escenario acumulativo.

Línea 242: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 243: `NumeroOrden = "OF-M2-0001",` → asigna el identificador de negocio usado por la orden de validación acumulativa del módulo.

Línea 244: `Cliente = "Constructora del Norte",` → asigna el cliente de la orden utilizada en el escenario práctico del punto.

Línea 245: `FechaCreacion = DateTime.UtcNow,` → asigna a la orden la fecha de creación en UTC para el escenario reproducible del punto.

Línea 246: `FechaEntrega = DateTime.Today.AddDays(14),` → establece una fecha de entrega prevista catorce días después de la fecha local actual.

Línea 247: `Estado = "Pendiente",` → inicializa la orden con el estado Pendiente, coherente con el flujo del laboratorio.

Línea 248: `Observaciones = "Orden de validación M2",` → añade texto opcional para comprobar el mapeo de Observaciones.

Línea 249: `Planchas =` → inicia el inicializador de la colección de planchas asociadas a la orden.

Línea 250: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 251: `new PlanchaAcero` → crea una nueva plancha dentro del inicializador de la colección Planchas de la orden.

Línea 252: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 253: `Espesor = 10.5`, → asigna a la plancha un espesor positivo para satisfacer las reglas del modelo.

Línea 254: `Ancho = 1500`, → asigna el ancho de la plancha usada en el escenario de persistencia.

Línea 255: `Largo = 3000`, → asigna el largo de la plancha usada en el escenario de persistencia.

Línea 256: `Peso = 371.250m`, → asigna un peso decimal con tres posiciones para validar la precisión configurada.

Línea 257: `Activa = true`, → marca la plancha como activa para que sea visible en los estados que aplican ese criterio.

Línea 258: `}` → cierra el bloque de código actual.

Línea 259: `,` → continúa el inicializador o la llamada iniciada en el bloque actual.

Línea 260: `Detalle = new DetalleOrden` → crea el detalle uno-a-uno que se persistirá junto con la orden principal.

Línea 261: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 262: `ComposicionQuimica = "C: 0.45%, Mn: 0.75%"`, → asigna la composición química utilizada para verificar la entidad `DetalleOrden`.

Línea 263: `TemperaturaColada = 1550.5`, → asigna la temperatura de colada del detalle de fabricación.

Línea 264: `Notas = "Colada principal"` → asigna las notas opcionales del detalle asociado a la orden.

Línea 265: `,` → continúa el inicializador o la llamada iniciada en el bloque actual.

Línea 266: `Certificado = new CertificadoCalidad` → crea el certificado uno-a-uno asociado a la orden del escenario.

Línea 267: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 268: `NumeroCertificado = "CERT-0001"`, → asigna el identificador natural del certificado de calidad.

Línea 269: `FechaEmision = DateTime.Today`, → asigna la fecha de emisión del certificado usado en la prueba.

Línea 270: `OrganismoCertificador = "Laboratorio Aceria"` → asigna el organismo que emite el certificado de calidad.

Línea 271: `,` → continúa el inicializador o la llamada iniciada en el bloque actual.

Línea 272: `};` → cierra el inicializador del objeto y finaliza la instrucción.

Línea 274: `context.OrdenesFabricacion.Add(orden);` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 275: `context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo = true });` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 276: `context.SaveChanges();` → persiste en SQL Server los cambios seguidos por el `DbContext`.

Línea 278: `var cargada = context.OrdenesFabricacion.Include(o => o.Detalle).Include(o => o.Certificado).Single(o => o.NumeroOrden == "OF-M2-0001");` → incluye de forma eager la navegación relacionada en la consulta.

Línea 279: `global::System.Console.WriteLine($"2.4 OK | Detalle: {cargada.Detalle?.ComposicionQuimica} | Certificado: {cargada.Certificado?.NumeroCertificado}");` → escribe en consola la evidencia usada para verificar el comportamiento del estado.

Línea 280: `}` → cierra el bloque de código actual.

Línea 281: `}` → cierra el bloque de código actual.

Punto 2.5 - Relaciones muchos a muchos

Ejercicio guiado: modelar la relación muchos-a-muchos entre `OrdenFabricacion` y `Aleacion` mediante una entidad intermedia explícita `OrdenAleacion`. La relación no es una simple tabla puente: registra `FechaAsignacion`, `CantidadUtilizada` y `EstadoRelacion`, por lo que debe tratarse como una entidad con significado propio.

Contexto del proyecto: 2.5 continúa directamente desde 2.4. El modelo ya contiene la relación 1:N entre órdenes y planchas y las relaciones 1:1 de detalle y certificado. Hasta este punto `OrdenFabricacion` y `Aleacion` existen como entidades independientes, pero **no existe una relación muchos-a-muchos implícita previa**. En 2.5 se crea por primera vez la entidad `OrdenAleacion`, sus dos relaciones uno-a-muchos y la tabla `OrdenesAleaciones`.

Objetivos de aprendizaje

- Entender cuándo una relación muchos-a-muchos necesita una entidad intermedia explícita.
- Crear `OrdenAleacion` sin una propiedad `Id` artificial.
- Definir una clave primaria compuesta con `OrdenFabricacionId` y `AleacionId`.
- Añadir datos propios de la relación: fecha, cantidad y estado.
- Configurar las colecciones `OrdenesAleaciones` en ambos extremos.
- Configurar las dos relaciones uno-a-muchos que forman el muchos-a-muchos.
- Diferenciar `DeleteBehavior.Cascade` y `DeleteBehavior.Restrict`.
- Generar y leer la migración real `M2_2_5`.
- Comprobar la tabla intermedia y sus restricciones en SQL Server.
- Persistir una relación mediante el grafo de entidades.
- Recuperar la aleación con `Include` y `ThenInclude`.
- Entender cómo la PK compuesta impide duplicar el mismo par orden/aleación.
- Resolver el reto con dos aleaciones y cantidades diferentes.

Paso 1: Abrir el estado autónomo de 2.5

```
cd M02/PROYECTO/2.5
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

El checkpoint contiene el estado completo de 2.5. Si estás reproduciendo el laboratorio de forma incremental, parte de una copia de 2.4 y aplica los cambios siguientes.

Paso 2: Revisar el estado heredado de `OrdenFabricacion` y `Aleacion`

Antes de añadir la relación, recuerda qué existe en 2.4:

- `OrdenFabricacion` ya tiene `Planchas`, `Detalle` y `Certificado`.
- `Aleacion` ya tiene sus propiedades escalares de nombre, código y composición.
- No hay una colección directa `List<Aleacion>` en la orden.
- No hay una colección directa `List<OrdenFabricacion>` en la aleación.
- No existe todavía una tabla implícita `AleacionOrdenFabricacion`.

Esto es importante porque la migración real de 2.5 **no elimina una tabla puente anterior**. Crea `OrdenesAleaciones` directamente a partir del modelo de 2.4.

Paso 3: Crear la entidad intermedia `OrdenAleacion`

La entidad final del punto es:

```
public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }
    public DateTime FechaAsignacion { get; set; } = DateTime.Now;
    public decimal CantidadUtilizada { get; set; }
    public string EstadoRelacion { get; set; } = "Activa";
    public OrdenFabricacion Orden { get; set; } = null!;
    public Aleacion Aleacion { get; set; } = null!;
}
```

Interpreta sus propiedades:

1. `OrdenFabricacionId` identifica la orden y formará parte de la PK.
2. `AleacionId` identifica la aleación y completa la PK compuesta.
3. `FechaAsignacion` registra cuándo se creó la relación.
4. `CantidadUtilizada` pertenece a la relación, no a la orden ni a la aleación por separado.
5. `EstadoRelacion` permite expresar el estado propio de la asignación.
6. `Orden` y `Aleacion` son las navegaciones hacia los dos extremos.

No añadas una propiedad `Id`. El identificador natural de esta entidad es el par (`OrdenFabricacionId`, `AleacionId`).

Paso 4: Añadir las colecciones de entidades intermedias

`OrdenFabricacion` incorpora:

```
public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
```

y `Aleacion` incorpora la misma colección desde su extremo:

```
public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
```

La orden no almacena directamente una lista de `Aleacion`. Almacena las asignaciones, porque cada asignación posee `CantidadUtilizada`, `FechaAsignacion` y `EstadoRelacion`.

Paso 5: Exponer OrdenAleacion desde el DbContext

Añade:

```
public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();
```

El `DbSet` hace visible la entidad intermedia para consultas, inserciones y operaciones directas sobre la relación.

Paso 6: Configurar tabla, clave y propiedades de la relación

En `OnModelCreating`:

```
modelBuilder.Entity<OrdenAleacion>(entity =>
{
    entity.ToTable("OrdenesAleaciones");
    entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });

    entity.Property(x => x.FechaAsignacion)
        .HasDefaultValueSql("GETDATE()");

    entity.Property(x => x.CantidadUtilizada)
        .HasPrecision(18, 3);

    entity.Property(x => x.EstadoRelacion)
        .IsRequired()
        .HasMaxLength(20)
        .HasDefaultValue("Activa");
});
```

La clave:

```
entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });
```

impide dos filas con exactamente el mismo par orden/aleación.

`CantidadUtilizada` usa precisión 18,3, coherente con cantidades decimales que necesitan tres posiciones. `EstadoRelacion` queda requerido, limitado a 20 caracteres y con valor por defecto "Activa".

| Paso 7: Configurar las dos relaciones uno-a-muchos

Dentro del mismo bloque se configuran los dos extremos:

```
entity.HasOne(x => x.Orden)
    .WithMany(o => o.OrdenesAleaciones)
    .HasForeignKey(x => x.OrdenFabricacionId)
    .OnDelete(DeleteBehavior.Cascade);

entity.HasOne(x => x.Aleacion)
    .WithMany(a => a.OrdenesAleaciones)
    .HasForeignKey(x => x.AleacionId)
    .OnDelete(DeleteBehavior.Restrict);
});
```

La relación muchos-a-muchos explícita se compone realmente de:

```
OrdenFabricacion 1 ---- N OrdenAleacion N ---- 1 Aleacion
```

Hacia OrdenFabricacion se usa Cascade. Si una orden se elimina físicamente, sus filas de asignación dejan de tener sentido y pueden eliminarse.

Hacia Aleacion se usa Restrict. Una aleación referenciada no debe desaparecer dejando asignaciones inválidas. Antes habría que eliminar o reasignar conscientemente esas relaciones.

No configures `Cascade` indiscriminadamente en ambos extremos.

| Paso 8: Compilar y generar la migración M2_2_5

Si estás construyendo 2.5 desde una copia de 2.4:

```
dotnet build AceriaData.sln --configuration Release
dotnet ef migrations add M2_2_5
```

En el checkpoint final la migración ya existe:

```
Migrations/20260927204745_M2_2_5.cs
```

No vuelvas a generar otra migración con el mismo nombre dentro de la solución entregada.

Paso 9: Revisar la migración real

La migración crea directamente `OrdenesAleaciones`:

```
migrationBuilder.CreateTable(
    name: "OrdenesAleaciones",
    columns: table => new
    {
        OrdenFabricacionId = table.Column<int>(type: "int", nullable: false),
        AleacionId = table.Column<int>(type: "int", nullable: false),
        FechaAsignacion = table.Column<DateTime>(
            type: "datetime2",
            nullable: false,
            defaultValueSql: "GETDATE()"),
        CantidadUtilizada = table.Column<decimal>(
            type: "decimal(18,3)",
            precision: 18,
            scale: 3,
            nullable: false),
        EstadoRelacion = table.Column<string>(
            type: "nvarchar(20)",
            maxLength: 20,
            nullable: false,
            defaultValue: "Activa")
    },
    constraints: table =>
    {
        table.PrimaryKey(
            "PK_OrdenesAleaciones",
            x => new { x.OrdenFabricacionId, x.AleacionId });
    });
```

La migración **no contiene** `DropTable("AleacionOrdenFabricacion")`, porque 2.4 no tenía una tabla many-to-many implícita. Esperar ese `DropTable` sería describir un estado anterior que este curso no tiene.

Paso 10: Revisar las claves foráneas y el índice

La misma migración crea:

```
table.ForeignKey(
    name: "FK_OrdenesAleaciones_Aleaciones_AleacionId",
    column: x => x.AleacionId,
    principalTable: "Aleaciones",
    principalColumn: "Id",
    onDelete: ReferentialAction.Restrict);

table.ForeignKey(
    name: "FK_OrdenesAleaciones_OrdenesFabricacion_OrdenFabricacionId",
    column: x => x.OrdenFabricacionId,
    principalTable: "OrdenesFabricacion",
    principalColumn: "Id",
    onDelete: ReferentialAction.Cascade);
```

y un índice auxiliar sobre `AleacionId`:

```
migrationBuilder.CreateIndex(
    name: "IX_OrdenesAleaciones_AleacionId",
    table: "OrdenesAleaciones",
    column: "AleacionId");
```

La PK compuesta ya comienza por `OrdenFabricacionId`; el índice adicional facilita la navegación y la FK por `AleacionId`.

Paso 11: Aplicar la migración y comprobar SQL Server

Ejecuta:

```
dotnet ef database update
dotnet ef migrations list
```

En SQL Server Object Explorer comprueba `dbo.OrdenesAleaciones` y verifica:

- `OrdenFabricacionId` no nullable;
- `AleacionId` no nullable;
- `FechaAsignacion`;
- `CantidadUtilizada decimal(18,3)`;
- `EstadoRelacion nvarchar(20)`;
- PK compuesta (`OrdenFabricacionId`, `AleacionId`);
- FK a `OrdenesFabricacion` con eliminación en cascada;
- FK a `Aleaciones` con eliminación restrictiva;
- índice `IX_OrdenesAleaciones_AleacionId`.

El historial sigue gestionado mediante Migrations. No uses `EnsureCreated()`.

Paso 12: Crear una orden con una asignación de aleación

El checkpoint crea primero la orden y una aleación y las conecta mediante la entidad intermedia:

```
var aleacion = new Aleacion
{
    Nombre = "AISI 1045",
   Codigo = "A1045",
    PorcentajeCarbono = 0.45,
    PorcentajeManganeso = 0.75,
    Descripcion = "Acero medio en carbono"
};

orden.OrdenesAleaciones.Add(new OrdenAleacion
{
    Aleacion = aleacion,
    CantidadUtilizada = 1500.500m,
    EstadoRelacion = "Activa"
});

context.OrdenesFabricacion.Add(orden);
context.SaveChanges();
```

No es necesario guardar primero la aleación para obtener manualmente su Id. EF Core conoce el grafo y puede ordenar las inserciones, generar las claves y propagar las FKs.

La fila intermedia queda identificada por el Id de la orden y el Id de la aleación.

Paso 13: Recuperar el grafo con Include y ThenInclude

La comprobación funcional es:

```
var cargada = context.OrdenesFabricacion
    .Include(o => o.OrdenesAleaciones)
    .ThenInclude(x => x.Aleacion)
    .Single(o => o.NumeroOrden == "OF-M2-0001");

global::System.Console.WriteLine(
    $"2.5 OK | Aleaciones: {cargada.OrdenesAleaciones.Count} | " +
    $"Primera: {cargada.OrdenesAleaciones[0].Aleacion.Codigo}");
```

La salida esperada contiene:

2.5 OK | Aleaciones: 1 | Primera: A1045

`Include(o => o.OrdenesAleaciones)` carga las filas de la entidad intermedia. `ThenInclude(x => x.Aleacion)` continúa desde cada asignación hasta la entidad `Aleacion`.

Si omites `ThenInclude`, no debes asumir que `OrdenAleacion.Aleacion` estará cargada en una consulta nueva.

Paso 14: Comprender actualización, eliminación y restricciones

La entidad intermedia permite tratar la asignación como un registro normal.

Cambiar la cantidad significa modificar `OrdenAleacion.CantidadUtilizada` y guardar.

Eliminar una asignación significa eliminar la fila concreta de `OrdenesAleaciones`; no implica borrar ni la orden ni la aleación.

Eliminar la orden elimina en cascada sus filas `OrdenAleacion`.

Eliminar la aleación mientras está referenciada debe ser rechazado por `Restrict`.

Insertar otra vez el mismo par de claves debe fallar por la PK compuesta.

Estas reglas derivan directamente de la configuración y de las restricciones visibles en la migración. No se sustituyen por lógica manual en memoria.

Paso 15: Resolver el reto con dos aleaciones y cantidades diferentes

Reto: asignar dos aleaciones distintas a una nueva orden, cada una con su propia `CantidadUtilizada`, y recuperarlas mediante `Include` + `ThenInclude`.

El checkpoint contiene el bloque pedagógico `RETO 2.5 - DOS ALEACIONES CON THENINCLUDE`. Crea dos aleaciones:

```
var aleacionReto1 = new Aleacion
{
    Nombre = "AISI 1018",
   Codigo = "A1018",
    PorcentajeCarbono = 0.18,
    PorcentajeManganeso = 0.70
};

var aleacionReto2 = new Aleacion
{
    Nombre = "AISI 4140",
   Codigo = "A4140",
    PorcentajeCarbono = 0.40,
    PorcentajeManganeso = 0.90
};
```

y añade dos asignaciones:

```
ordenReto.OrdenesAleaciones.Add(new OrdenAleacion
{
    Aleacion = aleacionReto1,
    CantidadUtilizada = 1000.250m,
    EstadoRelacion = "Activa"
});

ordenReto.OrdenesAleaciones.Add(new OrdenAleacion
{
    Aleacion = aleacionReto2,
    CantidadUtilizada = 500.750m,
    EstadoRelacion = "Activa"
});
```

Después recupera:

```
var retoCargada = context.OrdenesFabricacion
    .AsNoTracking()
    .Include(o => o.OrdenesAleaciones)
    .ThenInclude(x => x.Aleacion)
    .Single(o => o.NumeroOrden == "OF-M2-RET0-25");
```

y debe mostrar:

```
Reto 2.5 aleaciones: 2
A1018 | Cantidad: 1000.250
A4140 | Cantidad: 500.750
```

La PK compuesta permite ambas filas porque cambia `AleacionId`; lo que no permitiría sería repetir dos veces el mismo par orden/aleación.

Paso 16: Verificar EstadoRelacion como dato propio de la relación

`EstadoRelacion` está mapeada en la entidad, forma parte de la migración y posee reglas propias:

```
entity.Property(x => x.EstadoRelacion)
    .IsRequired()
    .HasMaxLength(20)
    .HasDefaultValue("Activa");
```

En el código del laboratorio las relaciones se crean explícitamente como `"Activa"`. En SQL Server también existe el valor por defecto para inserciones que no proporcionen un valor desde el cliente.

El punto importante es conceptual: cuando la relación necesita fecha, cantidad, estado u otros atributos, deja de ser una simple asociación invisible y la entidad intermedia explícita se convierte en parte del dominio persistente.

Errores comunes

Error	Causa	Corrección
Se añade un <code>Id</code> a <code>OrdenAleacion</code>	Se trata la entidad puente como una entidad con PK simple	Mantener la PK compuesta <code>OrdenFabricacionId + AleacionId</code>
Se usan colecciones directas además de <code>OrdenesAleaciones</code>	Se modelan dos relaciones diferentes sin intención	Usar las colecciones de la entidad intermedia
Se espera un <code>DropTable</code> de una tabla implícita	Se supone que 2.4 ya tenía una relación M:N	La migración real de 2.5 crea directamente <code>OrdenesAleaciones</code>
Se omite <code>ThenInclude</code>	Sólo se carga la entidad intermedia	Añadir <code>.ThenInclude(x => x.Aleacion)</code>

Error	Causa	Corrección
Se repite el mismo par orden/aleación	La PK compuesta ya existe	No insertar una relación duplicada; actualizar la existente
Se configura <code>Cascade</code> en ambos lados	Se relaja la protección de la aleación compartida	Mantener <code>Cascade</code> hacia orden y <code>Restrict</code> hacia aleación
Se usa <code>EnsureCreated()</code>	Se evita el historial de migraciones	Aplicar Migrations
Se ignoran <code>CantidadUtilizada</code> o <code>EstadoRelacion</code>	Se pierde el motivo de usar entidad explícita	Tratar los datos de la relación como parte del modelo

Analugía final

`OrdenAleacion` se parece al libro de recetas de la acería. La orden indica qué se fabrica y la aleación describe un material disponible, pero la hoja de asignación registra **qué aleación se utilizó en qué orden, cuándo, en qué cantidad y con qué estado**. La combinación orden + aleación identifica de forma natural cada registro. Si se elimina la orden, sus asignaciones dejan de tener sentido; si una aleación sigue en uso, no debería eliminarse del catálogo.

Resultado esperado

Al finalizar 2.5, `AceriaData` conserva todo lo construido en 2.4 y añade la entidad `OrdenAleacion`, la tabla `OrdenesAleaciones`, una PK compuesta, las dos FKs con comportamientos de borrado diferentes y los datos propios `FechaAsignacion`, `CantidadUtilizada` y `EstadoRelacion`. La ejecución termina con 2.5 OK | Aleaciones: 1 | Primera: A1045, y el reto demuestra dos aleaciones distintas con cantidades diferentes cargadas mediante `Include` y `ThenInclude`.

Conexión con el siguiente punto

El siguiente estado, 2.6, continúa desde esta solución y se centra en **Data Annotations**. La PK compuesta y la entidad intermedia creadas aquí permanecen en el modelo acumulativo.

Código acumulativo completo del estado 2.5

El siguiente archivo es el `Program.cs` real del estado ejecutable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    public string Estado { get; set; } = "Pendiente";
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
    public DetalleOrden? Detalle { get; set; }
    public CertificadoCalidad? Certificado { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}
```

```

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class Aleacion
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Codigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    public string? Descripcion { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

public class EstadoOrden
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
}

public class DetalleOrden
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    public string? Notas { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class CertificadoCalidad
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public string NumeroCertificado { get; set; } = string.Empty;
    public DateTime FechaEmision { get; set; }
    public string OrganismoCertificador { get; set; } = string.Empty;
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }
    public DateTime FechaAsignacion { get; set; } = DateTime.Now;
    public decimal CantidadUtilizada { get; set; }
    public string EstadoRelacion { get; set; } = "Activa";
    public OrdenFabricacion Orden { get; set; } = null!;
    public Aleacion Aleacion { get; set; } = null!;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
    public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();
    public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();
    public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();
}

```

```

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);
    modelBuilder.Entity<OrdenFabricacion>(entity =>
    {
        entity.ToTable("OrdenesFabricacion");
        entity.HasKey(o => o.Id);
        entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);
        entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);
        entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");
        entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");
        entity.Property(o => o.Observaciones).HasMaxLength(500);
    });
    modelBuilder.Entity<PlanchaAcero>(entity =>
    {
        entity.ToTable("PlanchasAcero");
        entity.HasKey(x => x.Id);
        entity.Property(x => x.Peso).HasPrecision(18, 3);
        entity.Property(x => x.Activa).HasDefaultValue(true);
        entity.HasOne(x => x.Orden)
            .WithMany(o => o.Planchas)
            .HasForeignKey(x => x.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });
    modelBuilder.Entity<Aleacion>(entity =>
    {
        entity.ToTable("Aleaciones");
        entity.HasKey(a => a.Id);
        entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
        entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
        entity.Property(a => a.Descripcion).HasMaxLength(500);
    });
    modelBuilder.Entity<EstadoOrden>(entity =>
    {
        entity.ToTable("EstadosOrden");
        entity.HasKey(e => e.Id);
        entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
        entity.Property(e => e.Descripcion).HasMaxLength(250);
        entity.Property(e => e.Activo).HasDefaultValue(true);
    });
    modelBuilder.Entity<DetalleOrden>(entity =>
    {
        entity.ToTable("DetallesOrden");
        entity.HasKey(d => d.Id);
        entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);
        entity.Property(d => d.Notas).HasMaxLength(500);
        entity.HasOne(d => d.Orden)
            .WithOne(o => o.Detalle)
            .HasForeignKey<DetalleOrden>(d => d.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });
    modelBuilder.Entity<CertificadoCalidad>(entity =>
    {
        entity.ToTable("CertificadosCalidad");
        entity.HasKey(c => c.Id);
        entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);
        entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);
        entity.HasOne(c => c.Orden)
            .WithOne(o => o.Certificado)
            .HasForeignKey<CertificadoCalidad>(c => c.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });
    modelBuilder.Entity<OrdenAleacion>(entity =>
    {
        entity.ToTable("OrdenesAleaciones");
        entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });
        entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");
    });
}

```

```

        entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);
        entity.Property(x =>
x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");
        entity.HasOne(x => x.Orden)
            .WithMany(o => o.OrdenesAleaciones)
            .HasForeignKey(x => x.OrdenFabricacionId)
            .OnDelete(DeleteBehavior.Cascade);
        entity.HasOne(x => x.Aleacion)
            .WithMany(a => a.OrdenesAleaciones)
            .HasForeignKey(x => x.AleacionId)
            .OnDelete(DeleteBehavior.Restrict);
    });
}
}

public interface IOordenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    int Guardar();
}

public sealed class OrdenRepositorio : IOordenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o =>
o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) =>
_context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOordenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOordenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();
    }
}

```

```

var connectionString = configuration.GetConnectionString("AceriaDB")
    ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

var services = new ServiceCollection();
services.AddDbContext<AceriaDbContext>(options =>
    options
        .UseSqlServer(connectionString, sql =>
            {
                sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                sql.CommandTimeout(60);
            })
        .LogTo(global::System.Console.WriteLine, new[] {
"Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)
        .EnableDetailedErrors());

services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateScopes = true,
    ValidateOnBuild = true
});

using var scope = provider.CreateScope();
var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();

var orden = new OrdenFabricacion
{
    NumeroOrden = "0F-M2-0001",
    Cliente = "Constructora del Norte",
    FechaCreacion = DateTime.UtcNow,
    FechaEntrega = DateTime.Today.AddDays(14),
    Estado = "Pendiente",
    Observaciones = "Orden de validación M2",
    Planchas =
    {
        new PlanchaAcero
        {
            Espesor = 10.5,
            Ancho = 1500,
            Largo = 3000,
            Peso = 371.250m,
            Activa = true,
        }
    },
    Detalle = new DetalleOrden
    {
        ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
        TemperaturaColada = 1550.5,
        Notas = "Colada principal"
    },
    Certificado = new CertificadoCalidad
    {
        NumeroCertificado = "CERT-0001",
        FechaEmision = DateTime.Today,
        OrganismoCertificador = "Laboratorio Aceria"
    },
};

var aleacion = new Aleacion
{
    Nombre = "AISI 1045",
   Codigo = "A1045",
    PorcentajeCarbono = 0.45,
    PorcentajeManganeso = 0.75,
    Descripcion = "Acero medio en carbono"
};
orden.OrdenesAleaciones.Add(new OrdenAleacion
{

```

```

        Aleacion = aleacion,
        CantidadUtilizada = 1500.500m,
        EstadoRelacion = "Activa"
    });
    context.OrdenesFabricacion.Add(orden);
    context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa",
Activo = true });
    context.SaveChanges();

    var cargada = context.OrdenesFabricacion.Include(o => o.OrdenesAleaciones).ThenInclude(x =>
x.Aleacion).Single(o => o.NumeroOrden == "OF-M2-0001");
    global::System.Console.WriteLine($"2.5 OK | Aleaciones: {cargada.OrdenesAleaciones.Count} |
Primera: {cargada.OrdenesAleaciones[0].Aleacion.Codigo}");
    }
}

```

Explicación línea a línea del código acumulativo 2.5

Las referencias siguientes usan la numeración real de M02/PROYECTO/2.5/Program.cs. Se explican todas las líneas no vacías para que el documento sea trazable con el archivo que compila.

Línea 1: `using System.ComponentModel.DataAnnotations;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 2: `using System.ComponentModel.DataAnnotations.Schema;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 3: `using Microsoft.EntityFrameworkCore;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 4: `using Microsoft.Extensions.Configuration;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 5: `using Microsoft.Extensions.DependencyInjection;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 6: `using Microsoft.Extensions.Logging;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 8: `namespace AceriaData.ConsoleApp;` → declara el espacio de nombres del estado acumulativo.

Línea 10: `public class OrdenFabricacion` → declara la clase OrdenFabricacion que forma parte del estado acumulativo del ejercicio.

Línea 11: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 12: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 13: `public string NumeroOrden { get; set; } = string.Empty;` → declara la propiedad NumeroOrden de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 14: `public string Cliente { get; set; } = string.Empty;` → declara la propiedad Cliente de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 15: `public DateTime FechaCreacion { get; set; }` → declara la propiedad FechaCreacion de tipo DateTime que EF Core o la lógica de negocio utiliza en este modelo.

Línea 16: `public DateTime? FechaEntrega { get; set; }` → declara la propiedad FechaEntrega de tipo DateTime? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 17: `public string Estado { get; set; } = "Pendiente";` → declara la propiedad Estado de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 18: `public string? Observaciones { get; set; }` → declara la propiedad Observaciones de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 19: `public List<PlanchaAcero> Planchas { get; set; } = new();` → declara la propiedad Planchas de tipo List que EF Core o la lógica de negocio utiliza en este modelo.

Línea 20: `public DetalleOrden? Detalle { get; set; }` → declara la propiedad Detalle de tipo DetalleOrden? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 21: `public CertificadoCalidad? Certificado { get; set; }` → declara la propiedad Certificado de tipo CertificadoCalidad? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 22: `public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();` → declara la propiedad OrdenesAleaciones de tipo List que EF Core o la lógica de negocio utiliza en este modelo.

Línea 23: `}` → cierra el bloque de código actual.

Línea 25: `public class PlanchaAcero` → declara la clase PlanchaAcero que forma parte del estado acumulativo del ejercicio.

Línea 26: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 27: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 28: `public int OrdenId { get; set; }` → declara la propiedad OrdenId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 29: `public double Espesor { get; set; }` → declara la propiedad Espesor de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 30: `public double Ancho { get; set; }` → declara la propiedad Ancho de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 31: `public double Largo { get; set; }` → declara la propiedad Largo de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 32: `public decimal Peso { get; set; }` → declara la propiedad Peso de tipo decimal que EF Core o la lógica de negocio utiliza en este modelo.

Línea 33: `public bool Activa { get; set; } = true;` → declara la propiedad Activa de tipo bool que EF Core o la lógica de negocio utiliza en este modelo.

Línea 34: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.

Línea 35: `}` → cierra el bloque de código actual.

Línea 37: `public class Aleacion` → declara la clase Aleacion que forma parte del estado acumulativo del ejercicio.

Línea 38: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 39: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 40: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 41: `public stringCodigo { get; set; } = string.Empty;` → declara la propiedad Codigo de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 42: `public double PorcentajeCarbono { get; set; }` → declara la propiedad PorcentajeCarbono de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 43: `public double PorcentajeManganeso { get; set; }` → declara la propiedad PorcentajeManganeso de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 44: `public string? Descripcion { get; set; }` → declara la propiedad Descripcion de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 45: `public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();` → declara la propiedad `OrdenesAleaciones` de tipo `List` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 46: `}` → cierra el bloque de código actual.

Línea 48: `public class EstadoOrden` → declara la clase `EstadoOrden` que forma parte del estado acumulativo del ejercicio.

Línea 49: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 50: `public int Id { get; set; }` → declara la propiedad `Id` de tipo `int` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 51: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad `Nombre` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 52: `public string Descripcion { get; set; } = string.Empty;` → declara la propiedad `Descripcion` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 53: `public bool Activo { get; set; } = true;` → declara la propiedad `Activo` de tipo `bool` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 54: `}` → cierra el bloque de código actual.

Línea 56: `public class DetalleOrden` → declara la clase `DetalleOrden` que forma parte del estado acumulativo del ejercicio.

Línea 57: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 58: `public int Id { get; set; }` → declara la propiedad `Id` de tipo `int` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 59: `public int OrdenId { get; set; }` → declara la propiedad `OrdenId` de tipo `int` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 60: `public string ComposicionQuimica { get; set; } = string.Empty;` → declara la propiedad `ComposicionQuimica` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 61: `public double TemperaturaColada { get; set; }` → declara la propiedad `TemperaturaColada` de tipo `double` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 62: `public string? Notas { get; set; }` → declara la propiedad `Notas` de tipo `string?` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 63: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad `Orden` de tipo `OrdenFabricacion` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 64: `}` → cierra el bloque de código actual.

Línea 66: `public class CertificadoCalidad` → declara la clase `CertificadoCalidad` que forma parte del estado acumulativo del ejercicio.

Línea 67: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 68: `public int Id { get; set; }` → declara la propiedad `Id` de tipo `int` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 69: `public int OrdenId { get; set; }` → declara la propiedad `OrdenId` de tipo `int` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 70: `public string NumeroCertificado { get; set; } = string.Empty;` → declara la propiedad `NumeroCertificado` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 71: `public DateTime FechaEmision { get; set; }` → declara la propiedad `FechaEmision` de tipo `DateTime` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 72: `public string OrganismoCertificador { get; set; } = string.Empty;` → declara la propiedad `OrganismoCertificador` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 73: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad `Orden` de tipo `OrdenFabricacion` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 74: `}` → cierra el bloque de código actual.

Línea 76: `public class OrdenAleacion` → declara la clase `OrdenAleacion` que forma parte del estado acumulativo del ejercicio.

Línea 77: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 78: `public int OrdenFabricacionId { get; set; }` → declara la propiedad `OrdenFabricacionId` de tipo `int` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 79: `public int AleacionId { get; set; }` → declara la propiedad `AleacionId` de tipo `int` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 80: `public DateTime FechaAsignacion { get; set; } = DateTime.Now;` → declara la propiedad `FechaAsignacion` de tipo `DateTime` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 81: `public decimal CantidadUtilizada { get; set; }` → declara la propiedad `CantidadUtilizada` de tipo `decimal` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 82: `public string EstadoRelacion { get; set; } = "Activa";` → declara la propiedad `EstadoRelacion` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 83: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad `Orden` de tipo `OrdenFabricacion` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 84: `public Aleacion Aleacion { get; set; } = null!;` → declara la propiedad `Aleacion` de tipo `Aleacion` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 85: `}` → cierra el bloque de código actual.

Línea 87: `public class AceriaDbContext : DbContext` → declara la clase `AceriaDbContext` que forma parte del estado acumulativo del ejercicio.

Línea 88: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 89: `public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }` → declara el constructor o método `AceriaDbContext` con los parámetros necesarios para este componente.

Línea 91: `public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 92: `public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 93: `public DbSet<Aleacion> Aleaciones => Set<Aleacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 94: `public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 95: `public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 96: `public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 97: `public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 100: `protected override void OnModelCreating(ModelBuilder modelBuilder)` → sobrescribe `OnModelCreating` para definir configuración explícita del modelo.

Línea 101: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 102: `base.OnModelCreating(modelBuilder);` → sobrescribe `OnModelCreating` para definir configuración explícita del modelo.

Línea 103: `modelBuilder.Entity<OrdenFabricacion>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 104: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 105: `entity.ToTable("OrdenesFabricacion");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 106: `entity.HasKey(o => o.Id);` → define la clave primaria de la entidad.

Línea 107: `entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 108: `entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);` → selecciona una propiedad escalar para continuar su configuración.

Línea 109: `entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");` → selecciona una propiedad escalar para continuar su configuración.

Línea 110: `entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");` → selecciona una propiedad escalar para continuar su configuración.

Línea 111: `entity.Property(o => o.Observaciones).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 112: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 113: `modelBuilder.Entity<PlanchaAcero>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 114: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 115: `entity.ToTable("PlanchasAcero");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 116: `entity.HasKey(x => x.Id);` → define la clave primaria de la entidad.

Línea 117: `entity.Property(x => x.Peso).HasPrecision(18, 3);` → selecciona una propiedad escalar para continuar su configuración.

Línea 118: `entity.Property(x => x.Activa).HasDefaultValue(true);` → selecciona una propiedad escalar para continuar su configuración.

Línea 119: `entity.HasOne(x => x.Orden)` → inicia la configuración de una navegación de referencia.

Línea 120: `.WithMany(o => o.Planchas)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 121: `.HasForeignKey(x => x.OrdenId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 122: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 123: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 124: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 125: `modelBuilder.Entity<Aleacion>(entity => → selecciona una entidad del modelo para aplicar su configuración Fluent API.`

Línea 126: `{ → abre el bloque de código asociado a la declaración o instrucción anterior.`

Línea 127: `entity.ToTable("Aleaciones"); → asigna explícitamente la tabla de SQL Server usada por la entidad.`

Línea 128: `entity.HasKey(a => a.Id); → define la clave primaria de la entidad.`

Línea 129: `entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100); → selecciona una propiedad escalar para continuar su configuración.`

Línea 130: `entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20); → selecciona una propiedad escalar para continuar su configuración.`

Línea 131: `entity.Property(a => a.Descripcion).HasMaxLength(500); → selecciona una propiedad escalar para continuar su configuración.`

Línea 132: `}); → cierra la lambda o configuración encadenada y finaliza la llamada.`

Línea 133: `modelBuilder.Entity<EstadoOrden>(entity => → selecciona una entidad del modelo para aplicar su configuración Fluent API.`

Línea 134: `{ → abre el bloque de código asociado a la declaración o instrucción anterior.`

Línea 135: `entity.ToTable("EstadosOrden"); → asigna explícitamente la tabla de SQL Server usada por la entidad.`

Línea 136: `entity.HasKey(e => e.Id); → define la clave primaria de la entidad.`

Línea 137: `entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50); → selecciona una propiedad escalar para continuar su configuración.`

Línea 138: `entity.Property(e => e.Descripcion).HasMaxLength(250); → selecciona una propiedad escalar para continuar su configuración.`

Línea 139: `entity.Property(e => e.Activo).HasDefaultValue(true); → selecciona una propiedad escalar para continuar su configuración.`

Línea 140: `}); → cierra la lambda o configuración encadenada y finaliza la llamada.`

Línea 141: `modelBuilder.Entity<DetalleOrden>(entity => → selecciona una entidad del modelo para aplicar su configuración Fluent API.`

Línea 142: `{ → abre el bloque de código asociado a la declaración o instrucción anterior.`

Línea 143: `entity.ToTable("DetallesOrden"); → asigna explícitamente la tabla de SQL Server usada por la entidad.`

Línea 144: `entity.HasKey(d => d.Id); → define la clave primaria de la entidad.`

Línea 145: `entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200); → selecciona una propiedad escalar para continuar su configuración.`

Línea 146: `entity.Property(d => d.Notas).HasMaxLength(500); → selecciona una propiedad escalar para continuar su configuración.`

Línea 147: `entity.HasOne(d => d.Orden) → inicia la configuración de una navegación de referencia.`

Línea 148: `.WithOne(o => o.Detalle) → establece el otro extremo de referencia de una relación uno-a-uno.`

Línea 149: `.HasForeignKey<DetalleOrden>(d => d.OrdenId) → declara explícitamente la propiedad o propiedades de clave foránea.`

Línea 150: `.OnDelete(DeleteBehavior.Cascade) → define el comportamiento de eliminación de la relación.`

Línea 151: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 152: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 154: `modelBuilder.Entity<CertificadoCalidad>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 155: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 156: `entity.ToTable("CertificadosCalidad");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 157: `entity.HasKey(c => c.Id);` → define la clave primaria de la entidad.

Línea 158: `entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 159: `entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);` → selecciona una propiedad escalar para continuar su configuración.

Línea 160: `entity.HasOne(c => c.Orden)` → inicia la configuración de una navegación de referencia.

Línea 161: `.WithOne(o => o.Certificado)` → establece el otro extremo de referencia de una relación uno-a-uno.

Línea 162: `.HasForeignKey<CertificadoCalidad>(c => c.OrdenId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 163: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 164: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 165: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 166: `modelBuilder.Entity<OrdenAleacion>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 167: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 168: `entity.ToTable("OrdenesAleaciones");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 169: `entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });` → define la clave primaria de la entidad.

Línea 170: `entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");` → selecciona una propiedad escalar para continuar su configuración.

Línea 171: `entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);` → selecciona una propiedad escalar para continuar su configuración.

Línea 172: `entity.Property(x => x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");` → selecciona una propiedad escalar para continuar su configuración.

Línea 173: `entity.HasOne(x => x.Orden)` → inicia la configuración de una navegación de referencia.

Línea 174: `.WithMany(o => o.OrdenesAleaciones)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 175: `.HasForeignKey(x => x.OrdenFabricacionId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 176: `.OnDelete(DeleteBehavior.Cascade);` → define el comportamiento de eliminación de la relación.

Línea 177: `entity.HasOne(x => x.Aleacion)` → inicia la configuración de una navegación de referencia.

Línea 178: `.WithMany(a => a.OrdenesAleaciones)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 179: `.HasForeignKey(x => x.AleacionId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 180: `.OnDelete(DeleteBehavior.Restrict);` → define el comportamiento de eliminación de la relación.

Línea 181: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 182: `}` → cierra el bloque de código actual.

Línea 183: `}` → cierra el bloque de código actual.

Línea 185: `public interface IOrdenRepositorio` → declara el contrato IOrdenRepositorio para desacoplar consumidores e implementación.

Línea 186: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 187: `List<OrdenFabricacion> ObtenerTodas();` → declara en el contrato la operación que debe devolver todas las órdenes visibles para el repositorio.

Línea 188: `OrdenFabricacion? ObtenerPorId(int id);` → declara en el contrato la búsqueda de una orden por su clave primaria y permite devolver null si no existe.

Línea 189: `OrdenFabricacion? ObtenerPorNumero(string numeroOrden);` → declara en el contrato la búsqueda por el identificador de negocio NumeroOrden.

Línea 190: `void Agregar(OrdenFabricacion orden);` → declara la operación que incorpora una orden al repositorio para que EF Core la siga como nueva.

Línea 191: `void Eliminar(OrdenFabricacion orden);` → declara la operación que marca una orden para eliminación física cuando el escenario lo requiere.

Línea 192: `int Guardar();` → declara la operación que confirma los cambios pendientes y devuelve el número de entradas afectadas.

Línea 193: `}` → cierra el bloque de código actual.

Línea 195: `public sealed class OrdenRepositorio : IOrdenRepositorio` → declara la clase OrdenRepositorio que forma parte del estado acumulativo del ejercicio.

Línea 196: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 197: `private readonly AceriaDbContext _context;` → declara el campo privado _context de tipo AceriaDbContext que conserva una dependencia de la clase.

Línea 198: `public OrdenRepositorio(AceriaDbContext context) => _context = context;` → declara el constructor o método OrdenRepositorio con los parámetros necesarios para este componente.

Línea 200: `public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();` → ordena el resultado de la consulta antes de materializarlo.

Línea 201: `public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);` → implementa la búsqueda por clave primaria mediante Find, aprovechando primero el ChangeTracker y después la base de datos.

Línea 202: `public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);` → recupera la primera coincidencia o null cuando no existe.

Línea 203: `public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);` → registra una entidad o relación nueva en el ChangeTracker para insertarla al guardar.

Línea 204: `public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);` → marca la entidad indicada para borrado al confirmar los cambios.

Línea 205: `public int Guardar() => _context.SaveChanges();` → persiste en SQL Server los cambios seguidos por el DbContext.

Línea 206: `}` → cierra el bloque de código actual.

Línea 208: `public interface IServicioOrdenes` → declara el contrato IServicioOrdenes para desacoplar consumidores e implementación.

Línea 209: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 210: `string ObtenerResumen(int ordenId);` → declara el servicio que construye un resumen de una orden a partir de su identificador.

Línea 211: `}` → cierra el bloque de código actual.

Línea 213: `public sealed class ServicioOrdenes : IServicioOrdenes` → declara la clase ServicioOrdenes que forma parte del estado acumulativo del ejercicio.

Línea 214: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 215: `private readonly IOrdenRepositorio _repositorio;` → declara el campo privado _repositorio de tipo IOrdenRepositorio que conserva una dependencia de la clase.

Línea 216: `private readonly AceriaDbContext _context;` → declara el campo privado _context de tipo AceriaDbContext que conserva una dependencia de la clase.

Línea 217: `public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)` → declara el constructor o método ServicioOrdenes con los parámetros necesarios para este componente.

Línea 218: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 219: `_repositorio = repositorio;` → guarda en el servicio el repositorio inyectado para reutilizarlo en sus operaciones.

Línea 220: `_context = context;` → guarda el DbContext inyectado para consultar los datos necesarios durante el servicio.

Línea 221: `}` → cierra el bloque de código actual.

Línea 223: `public string ObtenerResumen(int ordenId)` → declara el método que obtiene la orden y compone el resumen solicitado por el servicio.

Línea 224: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 225: `var orden = _repositorio.ObtenerPorId(ordenId);` → consulta la orden mediante el repositorio y conserva el resultado anulable en una variable local.

Línea 226: `if (orden is null) return $"Orden {ordenId} no encontrada";` → evalúa la condición y ejecuta el bloque asociado sólo cuando se cumple.

Línea 227: `var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);` → cuenta las filas que cumplen la consulta construida.

Línea 228: `return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";` → devuelve el resultado calculado al código que invocó el método.

Línea 229: `}` → cierra el bloque de código actual.

Línea 230: `}` → cierra el bloque de código actual.

Línea 232: `public static class Program` → declara la clase Program que forma parte del estado acumulativo del ejercicio.

Línea 233: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 234: `public static void Main()` → declara el punto de entrada ejecutable de la aplicación de consola.

Línea 235: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 236: `var configuration = new ConfigurationBuilder()` → inicia la construcción de la configuración de la aplicación.

Línea 237: `.SetBasePath(AppContext.BaseDirectory)` → establece la ruta base usada para localizar los archivos de configuración.

Línea 238: `.AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)` → incorpora `appsettings.json` a la configuración.

Línea 239: `.AddEnvironmentVariables()` → incorpora variables de entorno como fuente adicional de configuración.

Línea 240: `.Build();` → materializa la configuración o el objeto encadenado construido en las líneas anteriores.

Línea 242: `var connectionString = configuration.GetConnectionString("AceriaDB")` → obtiene la cadena de conexión `AceriaDB` desde la configuración.

Línea 243: `?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");` → interrumpe el arranque con un error explícito si la configuración no contiene `ConnectionStrings:AceriaDB`.

Línea 245: `var services = new ServiceCollection();` → crea la colección donde se registran las dependencias de la aplicación.

Línea 246: `services.AddDbContext<AceriaDbContext>(options =>` → registra el `DbContext` en el contenedor de inyección de dependencias.

Línea 247: `options` → continúa la configuración del `DbContext` sobre el parámetro `options` recibido por la lambda de `AddDbContext`.

Línea 248: `.UseSqlServer(connectionString, sql =>` → selecciona `SQL Server` como proveedor de `EF Core`.

Línea 249: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 250: `sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);` → habilita reintentos ante fallos transitorios del proveedor `SQL Server`.

Línea 251: `sql.CommandTimeout(60);` → establece el tiempo máximo permitido para los comandos `SQL`.

Línea 252: `})` → cierra la lambda de configuración actual y devuelve el control a la llamada encadenada que la contiene.

Línea 253: `.LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)` → configura la salida del logging de `EF Core`.

Línea 254: `.EnableDetailedErrors();` → activa errores detallados para facilitar el diagnóstico del laboratorio.

Línea 256: `services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();` → registra un servicio `scoped` para compartir dependencias dentro del mismo ámbito.

Línea 257: `services.AddScoped<IServicioOrdenes, ServicioOrdenes>();` → registra un servicio `scoped` para compartir dependencias dentro del mismo ámbito.

Línea 259: `using var provider = services.BuildServiceProvider(new ServiceProviderOptions` → construye el `ServiceProvider` con validación de servicios y garantiza su liberación al salir del ámbito mediante `using var`.

Línea 260: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 261: `ValidateScopes = true,` → activa la validación de ámbitos para detectar dependencias scoped usadas desde ámbitos incorrectos.

Línea 262: `ValidateOnBuild = true` → obliga al contenedor a validar las resoluciones registradas al construir el `ServiceProvider`.

Línea 263: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 265: `using var scope = provider.CreateScope();` → crea un ámbito de DI para resolver servicios scoped y lo libera automáticamente al terminar el bloque.

Línea 266: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → resuelve un servicio obligatorio desde el contenedor de dependencias.

Línea 267: `context.Database.EnsureDeleted();` → elimina la base del laboratorio para reproducir el escenario desde un estado limpio.

Línea 268: `context.Database.Migrate();` → aplica las migraciones pendientes y deja `AceriaDB` en el esquema del punto.

Línea 270: `var orden = new OrdenFabricacion` → crea la entidad `OrdenFabricacion` que concentra el grafo de datos del escenario acumulativo.

Línea 271: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 272: `NumeroOrden = "0F-M2-0001",` → asigna el identificador de negocio usado por la orden de validación acumulativa del módulo.

Línea 273: `Cliente = "Constructora del Norte",` → asigna el cliente de la orden utilizada en el escenario práctico del punto.

Línea 274: `FechaCreacion = DateTime.UtcNow,` → asigna a la orden la fecha de creación en UTC para el escenario reproducible del punto.

Línea 275: `FechaEntrega = DateTime.Today.AddDays(14),` → establece una fecha de entrega prevista catorce días después de la fecha local actual.

Línea 276: `Estado = "Pendiente",` → inicializa la orden con el estado `Pendiente`, coherente con el flujo del laboratorio.

Línea 277: `Observaciones = "Orden de validación M2",` → añade texto opcional para comprobar el mapeo de `Observaciones`.

Línea 278: `Planchas =` → inicia el inicializador de la colección de planchas asociadas a la orden.

Línea 279: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 280: `new PlanchaAcero` → crea una nueva plancha dentro del inicializador de la colección `Planchas` de la orden.

Línea 281: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 282: `Espesor = 10.5,` → asigna a la plancha un espesor positivo para satisfacer las reglas del modelo.

Línea 283: `Ancho = 1500,` → asigna el ancho de la plancha usada en el escenario de persistencia.

Línea 284: `Largo = 3000,` → asigna el largo de la plancha usada en el escenario de persistencia.

Línea 285: `Peso = 371.250m,` → asigna un peso decimal con tres posiciones para validar la precisión configurada.

Línea 286: `Activa = true,` → marca la plancha como activa para que sea visible en los estados que aplican ese criterio.

Línea 287: `}` → cierra el bloque de código actual.

Línea 288: `},` → continúa el inicializador o la llamada iniciada en el bloque actual.

Línea 289: `Detalle = new DetalleOrden` → crea el detalle uno-a-uno que se persistirá junto con la orden principal.

Línea 290: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 291: `ComposicionQuimica = "C: 0.45%, Mn: 0.75%",` → asigna la composición química utilizada para verificar la entidad `DetalleOrden`.

Línea 292: `TemperaturaColada = 1550.5,` → asigna la temperatura de colada del detalle de fabricación.

Línea 293: `Notas = "Colada principal"` → asigna las notas opcionales del detalle asociado a la orden.

Línea 294: `},` → continúa el inicializador o la llamada iniciada en el bloque actual.

Línea 295: `Certificado = new CertificadoCalidad` → crea el certificado uno-a-uno asociado a la orden del escenario.

Línea 296: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 297: `NumeroCertificado = "CERT-0001",` → asigna el identificador natural del certificado de calidad.

Línea 298: `FechaEmision = DateTime.Today,` → asigna la fecha de emisión del certificado usado en la prueba.

Línea 299: `OrganismoCertificador = "Laboratorio Aceria"` → asigna el organismo que emite el certificado de calidad.

Línea 300: `},` → continúa el inicializador o la llamada iniciada en el bloque actual.

Línea 301: `};` → cierra el inicializador del objeto y finaliza la instrucción.

Línea 303: `var aleacion = new Aleacion` → crea la aleación que se enlazará con la orden mediante la entidad intermedia `OrdenAleacion`.

Línea 304: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 305: `Nombre = "AISI 1045",` → asigna el nombre comercial de la aleación utilizada en el escenario.

Línea 306: `Codigo = "A1045",` → asigna el código natural único de la aleación.

Línea 307: `PorcentajeCarbono = 0.45,` → asigna el porcentaje de carbono dentro del rango permitido por la restricción del modelo.

Línea 308: `PorcentajeManganeso = 0.75,` → asigna el porcentaje de manganeso dentro del rango permitido por la restricción del modelo.

Línea 309: `Descripcion = "Acero medio en carbono"` → asigna una descripción opcional a la aleación.

Línea 310: `};` → cierra el inicializador del objeto y finaliza la instrucción.

Línea 311: `orden.OrdenesAleaciones.Add(new OrdenAleacion` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 312: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 313: `Aleacion = aleacion,` → conecta la entidad intermedia `OrdenAleacion` con la aleación creada para el escenario.

Línea 314: `CantidadUtilizada = 1500.500m,` → registra la cantidad utilizada con tres decimales, coherente con `HasPrecision(18, 3)`.

Línea 315: `EstadoRelacion = "Activa"` → marca como activa la relación entre la orden y la aleación.

Línea 316: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 317: `context.OrdenesFabricacion.Add(orden);` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 318: `context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo = true });` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 319: `context.SaveChanges();` → persiste en SQL Server los cambios seguidos por el `DbContext`.

Línea 321: `var cargada = context.OrdenesFabricacion.Include(o => o.OrdenesAleaciones).ThenInclude(x => x.Aleacion).Single(o => o.NumeroOrden == "OF-M2-0001");` → incluye de forma eager la navegación relacionada en la consulta.

Línea 322: `global::System.Console.WriteLine($"2.5 OK | Aleaciones: {cargada.OrdenesAleaciones.Count} | Primera: {cargada.OrdenesAleaciones[0].Aleacion.Codigo}");` → escribe en consola la evidencia usada para verificar el comportamiento del estado.

Línea 323: `}` → cierra el bloque de código actual.

Línea 324: `}` → cierra el bloque de código actual.

Punto 2.6 - Configuración mediante Data Annotations

Ejercicio guiado: expresar mediante Data Annotations una parte del mapeo ya construido en AceriaData y comprobar cómo conviven los atributos con la Fluent API existente. El objetivo no es rehacer el modelo, sino aprender qué configuración puede viajar junto a las clases, qué configuración debe seguir en OnModelCreating y cuál prevalece cuando ambas expresan la misma regla.

Contexto del proyecto: 2.6 parte del estado completo de 2.5. Ya existen relaciones 1:N, 1:1 y la entidad intermedia OrdenAleacion con clave compuesta. En este punto se añaden atributos a las entidades acumulativas. Las relaciones complejas siguen configurándose con Fluent API. La migración M2_2_6 muestra además una consecuencia importante: mover reglas desde Fluent API a atributos puede cambiar metadatos del modelo, por ejemplo valores por defecto que antes estaban definidos en SQL.

Objetivos de aprendizaje

- Importar y utilizar System.ComponentModel.DataAnnotations y System.ComponentModel.DataAnnotations.Schema.
- Aplicar [Table], [Key], [Required] y [MaxLength].
- Aplicar [ForeignKey] en navegaciones.
- Aplicar [Precision] sobre propiedades decimal.
- Definir la PK compuesta de OrdenAleacion mediante [PrimaryKey] en EF Core 8.
- Comprender por qué dos [Key] no representan correctamente una PK compuesta en este escenario.
- Mantener relaciones complejas en Fluent API.
- Comprender la prioridad: Fluent API > Data Annotations > convenciones.
- Interpretar la migración real M2_2_6 y los defaults que desaparecen.
- Inspeccionar el modelo efectivo construido por EF Core.
- Comparar qué reglas proceden de atributos y cuáles quedan fijadas por Fluent API.

Paso 1: Abrir el estado autónomo de 2.6

```
cd M02/PROYECTO/2.6
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

El checkpoint contiene el estado completo de 2.6. Para reproducir la evolución manualmente, parte de una copia de 2.5 y aplica los cambios descritos a continuación.

Paso 2: Añadir los espacios de nombres necesarios

Al principio de Program.cs aparecen:

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
```

Los dos primeros espacios de nombres proporcionan atributos como Key, Required, MaxLength, Table y ForeignKey. Los atributos específicos de EF Core, como Precision y PrimaryKey, están disponibles a través de Microsoft.EntityFrameworkCore.

Si falta alguno de estos using, el compilador no podrá resolver los atributos correspondientes.

Paso 3: Aplicar atributos a OrdenFabricacion

```
[Table("OrdenesFabricacion")]
public class OrdenFabricacion
{
    [Key]
    public int Id { get; set; }

    [Required]
    [MaxLength(50)]
    public string NumeroOrden { get; set; } = string.Empty;

    [Required]
    [MaxLength(200)]
    public string Cliente { get; set; } = string.Empty;

    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }

    [Required]
    [MaxLength(50)]
    public string Estado { get; set; } = "Pendiente";

    [MaxLength(500)]
    public string? Observaciones { get; set; }
}
```

Table fija el nombre de tabla. Key identifica la PK simple. Required expresa obligatoriedad en propiedades de referencia y MaxLength fija el tamaño máximo de las columnas de texto. Observaciones continúa siendo nullable porque su tipo es string?.

No se añaden aquí índices, columnas calculadas ni propiedades nuevas: esos conceptos quedan fuera del estado ejecutable de 2.6.

Paso 4: Aplicar Table, Key, Precision y ForeignKey a PlanchaAcero

```
[Table("PlanchasAcero")]
public class PlanchaAcero
{
    [Key]
    public int Id { get; set; }

    public int OrdenId { get; set; }

    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }

    [Precision(18, 3)]
    public decimal Peso { get; set; }

    public bool Activa { get; set; } = true;

    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}
```

Precision se aplica a Peso, que es decimal. ForeignKey vincula la navegación Orden con la propiedad OrdenId. La cardinalidad, DeleteBehavior.Cascade y la obligatoriedad de la relación permanecen en Fluent API.

Paso 5: Aplicar atributos a Aleacion y EstadoOrden

```
[Table("Aleaciones")]
public class Aleacion
{
    [Key]
    public int Id { get; set; }

    [Required]
    [MaxLength(100)]
    public string Nombre { get; set; } = string.Empty;

    [Required]
    [MaxLength(20)]
    public string Codigo { get; set; } = string.Empty;

    [MaxLength(500)]
    public string? Descripcion { get; set; }
}
```

EstadoOrden sigue el mismo patrón:

```
[Table("EstadosOrden")]
public class EstadoOrden
{
    [Key]
    public int Id { get; set; }

    [Required]
    [MaxLength(50)]
    public string Nombre { get; set; } = string.Empty;

    [MaxLength(250)]
    public string Descripcion { get; set; } = string.Empty;

    public bool Activo { get; set; } = true;
}
```

Un tipo valor no anulable como bool ya es requerido por su propio tipo CLR; no necesita Required.

Paso 6: Aplicar atributos a DetalleOrden y CertificadoCalidad

```
[Table("DetallesOrden")]
public class DetalleOrden
{
    [Key]
    public int Id { get; set; }

    public int OrdenId { get; set; }

    [Required]
    [MaxLength(200)]
    public string ComposicionQuimica { get; set; } = string.Empty;

    [MaxLength(500)]
    public string? Notas { get; set; }

    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}
```

Y para CertificadoCalidad:

```
[Table("CertificadosCalidad")]
public class CertificadoCalidad
{
    [Key]
    public int Id { get; set; }

    public int OrdenId { get; set; }

    [Required]
    [MaxLength(50)]
    public string NumeroCertificado { get; set; } = string.Empty;

    public DateTime FechaEmision { get; set; }

    [Required]
    [MaxLength(100)]
    public string OrganismoCertificador { get; set; } = string.Empty;

    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}
```

Los atributos permiten identificar propiedades y FK, pero la cardinalidad 1:1 y el índice único derivado continúan definidos por la relación Fluent API heredada.

Paso 7: Declarar correctamente la clave compuesta de OrdenAleacion

```
[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]
[Table("OrdenesAleaciones")]
public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }

    public DateTime FechaAsignacion { get; set; } = DateTime.Now;

    [Precision(18, 3)]
    public decimal CantidadUtilizada { get; set; }

    [MaxLength(20)]
    public string EstadoRelacion { get; set; } = "Activa";

    public OrdenFabricacion Orden { get; set; } = null!;
    public Aleacion Aleacion { get; set; } = null!;
}
```

No coloques dos atributos [Key], uno en cada FK. En EF Core 8 la forma declarativa adecuada para esta PK compuesta es PrimaryKey sobre la clase. La Fluent API conserva también HasKey con las dos propiedades. Si hubiese una contradicción, prevalecería Fluent API.

Paso 8: Mantener en Fluent API las relaciones y reglas complejas

La relación entre plancha y orden continúa configurada con:

```
entity.HasOne(x => x.Orden)
    .WithMany(o => o.Planchas)
    .HasForeignKey(x => x.OrdenId)
    .OnDelete(DeleteBehavior.Cascade)
    .IsRequired();
```

Las relaciones 1:1 siguen usando WithOne y HasForeignKey, y OrdenAleacion mantiene dos relaciones 1:N con distintos comportamientos de borrado:

```
entity.HasOne(x => x.Orden)
    .WithMany(o => o.OrdenesAleaciones)
    .HasForeignKey(x => x.OrdenFabricacionId)
    .OnDelete(DeleteBehavior.Cascade);

entity.HasOne(x => x.Aleacion)
    .WithMany(a => a.OrdenesAleaciones)
    .HasForeignKey(x => x.AleacionId)
    .OnDelete(DeleteBehavior.Restrict);
```

Esta combinación muestra el propósito de ambos mecanismos: atributos para reglas locales y Fluent API para relaciones o configuraciones que requieren más control.

Paso 9: Comprender la prioridad del modelo

EF Core aplica las fuentes de configuración con esta prioridad:

```
Convenciones < Data Annotations < Fluent API
```

Table("OrdenesFabricacion") y ToTable("OrdenesFabricacion") coinciden, por lo que no existe conflicto. La PK compuesta también aparece con PrimaryKey y HasKey y ambas expresiones describen la misma clave.

Si un MaxLength de un atributo y un HasMaxLength Fluent estableciesen valores distintos para la misma propiedad, el valor Fluent formaría parte del modelo final.

Paso 10: Compilar antes de generar la migración

```
dotnet build AceriaData.sln --configuration Release
```

El build debe finalizar correctamente antes de pedir a EF Core que compare el modelo actual con el snapshot de 2.5.

Paso 11: Generar M2_2_6 desde el estado 2.5

```
dotnet ef migrations add M2_2_6
```

El checkpoint ya contiene la migración real:

```
Migrations/20260927204758_M2_2_6.cs
```

No vuelvas a crear una migración con el mismo nombre dentro del estado final.

Paso 12: Revisar qué cambia realmente en M2_2_6

La migración no está vacía. EF Core detecta que varios defaults definidos antes mediante Fluent API ya no están presentes en el modelo 2.6.

FechaCreacion deja de tener el default SQL GETDATE(). También desaparecen defaults heredados de:

- OrdenesFabricacion.Estado, que tenía "Pendiente";
- PlanchasAcero.Activa, que tenía true;
- OrdenesAleaciones.FechaAsignacion, que tenía GETDATE();
- OrdenesAleaciones.EstadoRelacion, que tenía "Activa";
- EstadosOrden.Activo, que tenía true.

Esto demuestra que un inicializador de propiedad en C# no equivale a un DEFAULT definido en SQL Server. Dos configuraciones que parecen similares al leer la clase pueden producir metadata relacional distinto.

Paso 13: Aplicar y listar las migraciones

```
dotnet ef database update
dotnet ef migrations list
```

La lista debe incluir M2_2_6. Después de aplicar el historial, la base de datos refleja el snapshot del punto.

Paso 14: Ejecutar e inspeccionar el modelo efectivo

```
dotnet run --project AceriaData.Console.csproj --configuration Release
```

El estado final consulta context.Model y muestra:

```
2.6 OK | Tabla por annotations: OrdenesFabricacion | NumeroOrden MaxLength: 50
```

La salida observa el modelo efectivo que EF Core construyó después de combinar convenciones, atributos y Fluent API.

Paso 15: Resolver el reto Annotations vs Fluent API

Reto: explicar qué parte de la configuración procede de atributos y qué parte sigue controlada por Fluent API, comprobando el modelo resultante.

El checkpoint contiene el bloque comentado RETO 2.6 - ANNOTATIONS VS FLUENT API. Al activarlo muestra las categorías de configuración y los valores efectivos del modelo:

```
Annotations: Table, Key, Required, MaxLength, ForeignKey, Precision y PrimaryKey
Fluent API: ToTable, HasKey y relaciones explícitas tienen prioridad
Modelo efectivo OrdenesFabricacion | Tabla: OrdenesFabricacion
Modelo efectivo NumeroOrden MaxLength: 50
```

El reto no añade entidades ni cambia el esquema. Su finalidad es aprender a razonar sobre el origen de una configuración y el modelo efectivo que termina usando EF Core.

Errores comunes

Error	Causa	Corrección
Se colocan dos [Key] para OrdenAleacion	Se intenta modelar una PK compuesta como dos PK simples	Usar PrimaryKey con OrdenFabricacionId y AleacionId
Un atributo parece no tener efecto	Fluent API configura después la misma característica	Recordar Fluent API > annotations > convenciones
Se usa Precision esperando cambiar un double a decimal	El tipo CLR sigue siendo double	Aplicar Precision a propiedades decimal o cambiar conscientemente el tipo
Se confunde inicializador C# con default SQL	El valor C# sólo actúa al crear el objeto en .NET	Revisar la migración y distinguir valor CLR de DEFAULT SQL
Se elimina Fluent API de las relaciones	Los atributos no expresan todo el comportamiento necesario	Mantener HasOne, WithMany, WithOne, HasForeignKey y DeleteBehavior

Error	Causa	Corrección
Se introduce una entidad auxiliar nueva	Se amplía el dominio fuera del alcance del punto	Mantener 2.6 sobre las entidades acumuladas de 2.5
Se usa EnsureCreated()	Se evita el historial incremental	Mantener Migrations y Database.Migrate() / database update

Analugía final

Las Data Annotations son etiquetas colocadas directamente sobre las piezas del modelo: permiten ver de inmediato que una propiedad es obligatoria, tiene una longitud máxima o forma parte de una clave. La Fluent API se parece al libro de especificaciones de la planta: permite describir reglas que afectan a varias piezas y relaciones completas. Si la etiqueta y el libro discrepan, el libro de especificaciones tiene prioridad.

Resultado esperado

Al finalizar 2.6, el proyecto conserva todo el modelo de 2.5 y añade Data Annotations a las entidades existentes. OrdenAleacion mantiene su PK compuesta mediante PrimaryKey, las relaciones siguen gobernadas por Fluent API y M2_2_6 registra las diferencias reales del modelo, incluida la retirada de varios defaults SQL heredados. La ejecución termina con 2.6 OK y el reto permite explicar la prioridad entre atributos y Fluent API.

Conexión con el siguiente punto

2.7 continúa desde este mismo modelo y profundiza en Fluent API. Allí se estudiarán configuraciones que requieren un control centralizado mayor que el que ofrecen los atributos.

Código acumulativo completo del estado 2.6

El siguiente archivo es el Program.cs real del estado ejecutable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

[Table("OrdenesFabricacion")]
public class OrdenFabricacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroOrden { get; set; } = string.Empty;
    [Required]
    [MaxLength(200)]
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    [Required]
    [MaxLength(50)]
    public string Estado { get; set; } = "Pendiente";
    [MaxLength(500)]
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
    public DetalleOrden? Detalle { get; set; }
    public CertificadoCalidad? Certificado { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}
```

```

}

[Table("PlanchasAcero")]
public class PlanchaAcero
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    [Precision(18, 3)]
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("Aleaciones")]
public class Aleacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(100)]
    public string Nombre { get; set; } = string.Empty;
    [Required]
    [MaxLength(20)]
    public stringCodigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    [MaxLength(500)]
    public string? Descripcion { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

[Table("EstadosOrden")]
public class EstadoOrden
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string Nombre { get; set; } = string.Empty;
    [MaxLength(250)]
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
}

[Table("DetallesOrden")]
public class DetalleOrden
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(200)]
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    [MaxLength(500)]
    public string? Notas { get; set; }
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("CertificadosCalidad")]
public class CertificadoCalidad
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(50)]

```

```

    public string NumeroCertificado { get; set; } = string.Empty;
    public DateTime FechaEmision { get; set; }
    [Required]
    [MaxLength(100)]
    public string OrganismoCertificador { get; set; } = string.Empty;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]
[Table("OrdenesAleaciones")]
public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }
    public DateTime FechaAsignacion { get; set; } = DateTime.Now;
    [Precision(18, 3)]
    public decimal CantidadUtilizada { get; set; }
    [MaxLength(20)]
    public string EstadoRelacion { get; set; } = "Activa";
    public OrdenFabricacion Orden { get; set; } = null!;
    public Aleacion Aleacion { get; set; } = null!;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
    public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();
    public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();
    public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<OrdenFabricacion>(entity =>
        {
            entity.ToTable("OrdenesFabricacion");
            entity.HasKey(o => o.Id);
        });
        modelBuilder.Entity<PlanchaAcero>(entity =>
        {
            entity.ToTable("PlanchasAcero");
            entity.HasKey(x => x.Id);
            entity.HasOne(x => x.Orden)
                .WithMany(o => o.Planchas)
                .HasForeignKey(x => x.OrdenId)
                .OnDelete(DeleteBehavior.Cascade)
                .IsRequired();
        });
        modelBuilder.Entity<Aleacion>(entity =>
        {
            entity.ToTable("Aleaciones");
            entity.HasKey(a => a.Id);
        });
        modelBuilder.Entity<EstadoOrden>(entity =>
        {
            entity.ToTable("EstadosOrden");
            entity.HasKey(e => e.Id);
        });
        modelBuilder.Entity<DetalleOrden>(entity =>
        {
            entity.ToTable("DetallesOrden");
            entity.HasKey(d => d.Id);
            entity.HasOne(d => d.Orden)
                .WithOne(o => o.Detalle)
                .HasForeignKey<DetalleOrden>(d => d.OrdenId)
                .OnDelete(DeleteBehavior.Cascade)
    
```

```

        .IsRequired();
    });

    modelBuilder.Entity<CertificadoCalidad>(entity =>
    {
        entity.ToTable("CertificadosCalidad");
        entity.HasKey(c => c.Id);
        entity.HasOne(c => c.Orden)
            .WithOne(o => o.Certificado)
            .HasForeignKey<CertificadoCalidad>(c => c.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });

    modelBuilder.Entity<OrdenAleacion>(entity =>
    {
        entity.ToTable("OrdenesAleaciones");
        entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });
        entity.HasOne(x => x.Orden)
            .WithMany(o => o.OrdenesAleaciones)
            .HasForeignKey(x => x.OrdenFabricacionId)
            .OnDelete(DeleteBehavior.Cascade);
        entity.HasOne(x => x.Aleacion)
            .WithMany(a => a.OrdenesAleaciones)
            .HasForeignKey(x => x.AleacionId)
            .OnDelete(DeleteBehavior.Restrict);
    });
}

public interface IOordenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    int Guardar();
}

public sealed class OrdenRepositorio : IOordenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) =>
        _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOordenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOordenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null) return $"Orden {ordenId} no encontrada";
    }
}

```

```

        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();
        services.AddDbContext<AceriaDbContext>(options =>
            options
                .UseSqlServer(connectionString, sql =>
                    {
                        sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                        sql.CommandTimeout(60);
                    }
                ))
            .LogTo(global::System.Console.WriteLine, new[] {
                "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)
            .EnableDetailedErrors());

        services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
        services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

        using var provider = services.BuildServiceProvider(new ServiceProviderOptions
        {
            ValidateScopes = true,
            ValidateOnBuild = true
        });

        using var scope = provider.CreateScope();
        var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
        context.Database.EnsureDeleted();
        context.Database.Migrate();

        var orden = new OrdenFabricacion
        {
            NumeroOrden = "OF-M2-0001",
            Cliente = "Constructora del Norte",
            FechaCreacion = DateTime.UtcNow,
            FechaEntrega = DateTime.Today.AddDays(14),
            Estado = "Pendiente",
            Observaciones = "Orden de validación M2",
            Planchas =
            {
                new PlanchaAcero
                {
                    Espesor = 10.5,
                    Ancho = 1500,
                    Largo = 3000,
                    Peso = 371.250m,
                    Activa = true,
                }
            },
            Detalle = new DetalleOrden
            {
                ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
                TemperaturaColada = 1550.5,
                Notas = "Colada principal"
            },
            Certificado = new CertificadoCalidad
            {
                NumeroCertificado = "CERT-0001",
                FechaEmision = DateTime.Today,
            }
        }
    }
}

```

```

        OrganismoCertificador = "Laboratorio Aceria"
    },
};

var aleacion = new Aleacion
{
    Nombre = "AISI 1045",
   Codigo = "A1045",
    PorcentajeCarbono = 0.45,
    PorcentajeManganeso = 0.75,
    Descripcion = "Acero medio en carbono"
};
orden.OrganizacionesAleaciones.Add(new OrdenAleacion
{
    Aleacion = aleacion,
    CantidadUtilizada = 1500.500m,
    EstadoRelacion = "Activa"
});
context.OrganizacionesFabricacion.Add(orden);
context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa",
Activo = true });
context.SaveChanges();

var entity = context.Model.FindEntityType(typeof(OrdenFabricacion));
global::System.Console.WriteLine($"2.6 OK | Tabla por annotations: {entity.GetTableName()} |
NumeroOrden MaxLength: {entity.FindProperty(nameof(OrdenFabricacion.NumeroOrden)).GetMaxLength()}");
}
}

```

Explicación línea a línea del código acumulativo 2.6

Las referencias siguientes usan la numeración real de M02/PROYECTO/2.6/Program.cs. Se explican todas las líneas no vacías para que el documento sea trazable con el archivo que compila.

Línea 1: `using System.ComponentModel.DataAnnotations;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 2: `using System.ComponentModel.DataAnnotations.Schema;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 3: `using Microsoft.EntityFrameworkCore;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 4: `using Microsoft.Extensions.Configuration;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 5: `using Microsoft.Extensions.DependencyInjection;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 6: `using Microsoft.Extensions.Logging;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 8: `namespace AceriaData.ConsoleApp;` → declara el espacio de nombres del estado acumulativo.

Línea 10: `[Table("OrdenesFabricacion")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 11: `public class OrdenFabricacion` → declara la clase OrdenFabricacion que forma parte del estado acumulativo del ejercicio.

Línea 12: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 13: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 14: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 15: `[Required]` → marca la propiedad siguiente como requerida.

Línea 16: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 17: `public string NumeroOrden { get; set; } = string.Empty;` → declara la propiedad NumeroOrden de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 18: `[Required]` → marca la propiedad siguiente como requerida.

Línea 19: `[MaxLength(200)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 20: `public string Cliente { get; set; } = string.Empty;` → declara la propiedad Cliente de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 21: `public DateTime FechaCreacion { get; set; }` → declara la propiedad FechaCreacion de tipo DateTime que EF Core o la lógica de negocio utiliza en este modelo.

Línea 22: `public DateTime? FechaEntrega { get; set; }` → declara la propiedad FechaEntrega de tipo DateTime? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 23: `[Required]` → marca la propiedad siguiente como requerida.

Línea 24: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 25: `public string Estado { get; set; } = "Pendiente";` → declara la propiedad Estado de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 26: `[MaxLength(500)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 27: `public string? Observaciones { get; set; }` → declara la propiedad Observaciones de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 28: `public List<PlanchaAcero> Planchas { get; set; } = new();` → declara la propiedad Planchas de tipo List que EF Core o la lógica de negocio utiliza en este modelo.

Línea 29: `public DetalleOrden? Detalle { get; set; }` → declara la propiedad Detalle de tipo DetalleOrden? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 30: `public CertificadoCalidad? Certificado { get; set; }` → declara la propiedad Certificado de tipo CertificadoCalidad? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 31: `public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();` → declara la propiedad OrdenesAleaciones de tipo List que EF Core o la lógica de negocio utiliza en este modelo.

Línea 32: `}` → cierra el bloque de código actual.

Línea 34: `[Table("PlanchasAcero")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 35: `public class PlanchaAcero` → declara la clase PlanchaAcero que forma parte del estado acumulativo del ejercicio.

Línea 36: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 37: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 38: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 39: `public int OrdenId { get; set; }` → declara la propiedad OrdenId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 40: `public double Espesor { get; set; }` → declara la propiedad Espesor de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 41: `public double Ancho { get; set; }` → declara la propiedad Ancho de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 42: `public double Largo { get; set; }` → declara la propiedad Largo de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 43: `[Precision(18, 3)]` → configura precisión y escala para la propiedad numérica siguiente.

Línea 44: `public decimal Peso { get; set; }` → declara la propiedad Peso de tipo decimal que EF Core o la lógica de negocio utiliza en este modelo.

Línea 45: `public bool Activa { get; set; } = true;` → declara la propiedad Activa de tipo bool que EF Core o la lógica de negocio utiliza en este modelo.

Línea 46: `[ForeignKey(nameof(OrdenId))]` → vincula explícitamente la navegación siguiente con su clave foránea.

Línea 47: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.

Línea 48: `}` → cierra el bloque de código actual.

Línea 50: `[Table("Aleaciones")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 51: `public class Aleacion` → declara la clase Aleacion que forma parte del estado acumulativo del ejercicio.

Línea 52: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 53: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 54: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 55: `[Required]` → marca la propiedad siguiente como requerida.

Línea 56: `[MaxLength(100)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 57: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 58: `[Required]` → marca la propiedad siguiente como requerida.

Línea 59: `[MaxLength(20)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 60: `public stringCodigo { get; set; } = string.Empty;` → declara la propiedad Codigo de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 61: `public double PorcentajeCarbono { get; set; }` → declara la propiedad PorcentajeCarbono de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 62: `public double PorcentajeManganeso { get; set; }` → declara la propiedad PorcentajeManganeso de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 63: `[MaxLength(500)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 64: `public string? Descripcion { get; set; }` → declara la propiedad Descripcion de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 65: `public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();` → declara la propiedad OrdenesAleaciones de tipo List que EF Core o la lógica de negocio utiliza en este modelo.

Línea 66: `}` → cierra el bloque de código actual.

Línea 68: `[Table("EstadosOrden")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 69: `public class EstadoOrden` → declara la clase EstadoOrden que forma parte del estado acumulativo del ejercicio.

Línea 70: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 71: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 72: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 73: `[Required]` → marca la propiedad siguiente como requerida.

Línea 74: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 75: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 76: `[MaxLength(250)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 77: `public string Descripcion { get; set; } = string.Empty;` → declara la propiedad Descripcion de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 78: `public bool Activo { get; set; } = true;` → declara la propiedad Activo de tipo bool que EF Core o la lógica de negocio utiliza en este modelo.

Línea 79: `}` → cierra el bloque de código actual.

Línea 81: `[Table("DetallesOrden")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 82: `public class DetalleOrden` → declara la clase DetalleOrden que forma parte del estado acumulativo del ejercicio.

Línea 83: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 84: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 85: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 86: `public int OrdenId { get; set; }` → declara la propiedad OrdenId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 87: `[Required]` → marca la propiedad siguiente como requerida.

Línea 88: `[MaxLength(200)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 89: `public string ComposicionQuimica { get; set; } = string.Empty;` → declara la propiedad ComposicionQuimica de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 90: `public double TemperaturaColada { get; set; }` → declara la propiedad TemperaturaColada de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 91: `[MaxLength(500)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 92: `public string? Notas { get; set; }` → declara la propiedad Notas de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 93: `[ForeignKey(nameof(OrdenId))]` → vincula explícitamente la navegación siguiente con su clave foránea.

Línea 94: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.

Línea 95: `}` → cierra el bloque de código actual.

Línea 97: `[Table("CertificadosCalidad")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 98: `public class CertificadoCalidad` → declara la clase CertificadoCalidad que forma parte del estado acumulativo del ejercicio.

Línea 99: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 100: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 101: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 102: `public int OrdenId { get; set; }` → declara la propiedad OrdenId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 103: `[Required]` → marca la propiedad siguiente como requerida.

Línea 104: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 105: `public string NumeroCertificado { get; set; } = string.Empty;` → declara la propiedad NumeroCertificado de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 106: `public DateTime FechaEmision { get; set; }` → declara la propiedad FechaEmision de tipo DateTime que EF Core o la lógica de negocio utiliza en este modelo.

Línea 107: `[Required]` → marca la propiedad siguiente como requerida.

Línea 108: `[MaxLength(100)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 109: `public string OrganismoCertificador { get; set; } = string.Empty;` → declara la propiedad OrganismoCertificador de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 110: `[ForeignKey(nameof(OrdenId))]` → vincula explícitamente la navegación siguiente con su clave foránea.

Línea 111: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.

Línea 112: `}` → cierra el bloque de código actual.

Línea 114: `[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]` → declara una clave primaria compuesta con las propiedades indicadas.

Línea 115: `[Table("OrdenesAleaciones")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 116: `public class OrdenAleacion` → declara la clase OrdenAleacion que forma parte del estado acumulativo del ejercicio.

Línea 117: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 118: `public int OrdenFabricacionId { get; set; }` → declara la propiedad OrdenFabricacionId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 119: `public int AleacionId { get; set; }` → declara la propiedad AleacionId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 120: `public DateTime FechaAsignacion { get; set; } = DateTime.Now;` → declara la propiedad FechaAsignacion de tipo DateTime que EF Core o la lógica de negocio utiliza en este modelo.

Línea 121: `[Precision(18, 3)]` → configura precisión y escala para la propiedad numérica siguiente.

Línea 122: `public decimal CantidadUtilizada { get; set; }` → declara la propiedad CantidadUtilizada de tipo decimal que EF Core o la lógica de negocio utiliza en este modelo.

Línea 123: `[MaxLength(20)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 124: `public string EstadoRelacion { get; set; } = "Activa";` → declara la propiedad EstadoRelacion de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 125: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.

Línea 126: `public Aleacion Aleacion { get; set; } = null!;` → declara la propiedad Aleacion de tipo Aleacion que EF Core o la lógica de negocio utiliza en este modelo.

Línea 127: `}` → cierra el bloque de código actual.

Línea 129: `public class AceriaDbContext : DbContext` → declara la clase `AceriaDbContext` que forma parte del estado acumulativo del ejercicio.

Línea 130: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 131:

`public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }` → declara el constructor o método `AceriaDbContext` con los parámetros necesarios para este componente.

Línea 133: `public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 134: `public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 135: `public DbSet<Aleacion> Aleaciones => Set<Aleacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 136: `public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 137: `public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 138: `public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 139: `public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 142: `protected override void OnModelCreating(ModelBuilder modelBuilder)` → sobrescribe `OnModelCreating` para definir configuración explícita del modelo.

Línea 143: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 144: `base.OnModelCreating(modelBuilder);` → sobrescribe `OnModelCreating` para definir configuración explícita del modelo.

Línea 145: `modelBuilder.Entity<OrdenFabricacion>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 146: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 147: `entity.ToTable("OrdenesFabricacion");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 148: `entity.HasKey(o => o.Id);` → define la clave primaria de la entidad.

Línea 149: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 150: `modelBuilder.Entity<PlanchaAcero>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 151: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 152: `entity.ToTable("PlanchasAcero");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 153: `entity.HasKey(x => x.Id);` → define la clave primaria de la entidad.

Línea 154: `entity.HasOne(x => x.Orden)` → inicia la configuración de una navegación de referencia.

Línea 155: `.WithMany(o => o.Planchas)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 156: `.HasForeignKey(x => x.OrdenId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 157: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 158: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 159: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 160: `modelBuilder.Entity<Aleacion>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 161: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 162: `entity.ToTable("Aleaciones");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 163: `entity.HasKey(a => a.Id);` → define la clave primaria de la entidad.

Línea 164: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 165: `modelBuilder.Entity<EstadoOrden>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 166: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 167: `entity.ToTable("EstadosOrden");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 168: `entity.HasKey(e => e.Id);` → define la clave primaria de la entidad.

Línea 169: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 170: `modelBuilder.Entity<DetalleOrden>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 171: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 172: `entity.ToTable("DetallesOrden");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 173: `entity.HasKey(d => d.Id);` → define la clave primaria de la entidad.

Línea 174: `entity.HasOne(d => d.Orden)` → inicia la configuración de una navegación de referencia.

Línea 175: `.WithOne(o => o.Detalle)` → establece el otro extremo de referencia de una relación uno-a-uno.

Línea 176: `.HasForeignKey<DetalleOrden>(d => d.OrdenId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 177: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 178: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 179: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 181: `modelBuilder.Entity<CertificadoCalidad>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 182: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 183: `entity.ToTable("CertificadosCalidad");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 184: `entity.HasKey(c => c.Id);` → define la clave primaria de la entidad.

Línea 185: `entity.HasOne(c => c.Orden)` → inicia la configuración de una navegación de referencia.

Línea 186: `.WithOne(o => o.Certificado)` → establece el otro extremo de referencia de una relación uno-a-uno.

Línea 187: `.HasForeignKey<CertificadoCalidad>(c => c.OrdenId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 188: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 189: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 190: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 191: `modelBuilder.Entity<OrdenAleacion>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 192: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 193: `entity.ToTable("OrdenesAleaciones");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 194: `entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });` → define la clave primaria de la entidad.

Línea 195: `entity.HasOne(x => x.Orden)` → inicia la configuración de una navegación de referencia.

Línea 196: `.WithMany(o => o.OrdenesAleaciones)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 197: `.HasForeignKey(x => x.OrdenFabricacionId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 198: `.OnDelete(DeleteBehavior.Cascade);` → define el comportamiento de eliminación de la relación.

Línea 199: `entity.HasOne(x => x.Aleacion)` → inicia la configuración de una navegación de referencia.

Línea 200: `.WithMany(a => a.OrdenesAleaciones)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 201: `.HasForeignKey(x => x.AleacionId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 202: `.OnDelete(DeleteBehavior.Restrict);` → define el comportamiento de eliminación de la relación.

Línea 203: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 204: `}` → cierra el bloque de código actual.

Línea 205: `}` → cierra el bloque de código actual.

Línea 207: `public interface IOrdenRepositorio` → declara el contrato IOrdenRepositorio para desacoplar consumidores e implementación.

Línea 208: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 209: `List<OrdenFabricacion> ObtenerTodas();` → declara en el contrato la operación que debe devolver todas las órdenes visibles para el repositorio.

Línea 210: `OrdenFabricacion? ObtenerPorId(int id);` → declara en el contrato la búsqueda de una orden por su clave primaria y permite devolver null si no existe.

Línea 211: `OrdenFabricacion? ObtenerPorNumero(string numeroOrden);` → declara en el contrato la búsqueda por el identificador de negocio NumeroOrden.

Línea 212: `void Agregar(OrdenFabricacion orden);` → declara la operación que incorpora una orden al repositorio para que EF Core la siga como nueva.

Línea 213: `void Eliminar(OrdenFabricacion orden);` → declara la operación que marca una orden para eliminación física cuando el escenario lo requiere.

Línea 214: `int Guardar();` → declara la operación que confirma los cambios pendientes y devuelve el número de entradas afectadas.

Línea 215: `}` → cierra el bloque de código actual.

Línea 217: `public sealed class OrdenRepositorio : IOrdenRepositorio` → declara la clase `OrdenRepositorio` que forma parte del estado acumulativo del ejercicio.

Línea 218: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 219: `private readonly AceriaDbContext _context;` → declara el campo privado `_context` de tipo `AceriaDbContext` que conserva una dependencia de la clase.

Línea 220: `public OrdenRepositorio(AceriaDbContext context) => _context = context;` → declara el constructor o método `OrdenRepositorio` con los parámetros necesarios para este componente.

Línea 222: `public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();` → ordena el resultado de la consulta antes de materializarlo.

Línea 223: `public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);` → implementa la búsqueda por clave primaria mediante `Find`, aprovechando primero el `ChangeTracker` y después la base de datos.

Línea 224: `public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);` → recupera la primera coincidencia o `null` cuando no existe.

Línea 225: `public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 226: `public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);` → marca la entidad indicada para borrado al confirmar los cambios.

Línea 227: `public int Guardar() => _context.SaveChanges();` → persiste en SQL Server los cambios seguidos por el `DbContext`.

Línea 228: `}` → cierra el bloque de código actual.

Línea 230: `public interface IServicioOrdenes` → declara el contrato `IServicioOrdenes` para desacoplar consumidores e implementación.

Línea 231: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 232: `string ObtenerResumen(int ordenId);` → declara el servicio que construye un resumen de una orden a partir de su identificador.

Línea 233: `}` → cierra el bloque de código actual.

Línea 235: `public sealed class ServicioOrdenes : IServicioOrdenes` → declara la clase `ServicioOrdenes` que forma parte del estado acumulativo del ejercicio.

Línea 236: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 237: `private readonly IOrdenRepositorio _repositorio;` → declara el campo privado `_repositorio` de tipo `IOrdenRepositorio` que conserva una dependencia de la clase.

Línea 238: `private readonly AceriaDbContext _context;` → declara el campo privado `_context` de tipo `AceriaDbContext` que conserva una dependencia de la clase.

Línea 239: `public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)` → declara el constructor o método `ServicioOrdenes` con los parámetros necesarios para este componente.

Línea 240: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 241: `_repositorio = repositorio;` → guarda en el servicio el repositorio inyectado para reutilizarlo en sus operaciones.

Línea 242: `_context = context;` → guarda el DbContext inyectado para consultar los datos necesarios durante el servicio.

Línea 243: `}` → cierra el bloque de código actual.

Línea 245: `public string ObtenerResumen(int ordenId)` → declara el método que obtiene la orden y compone el resumen solicitado por el servicio.

Línea 246: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 247: `var orden = _repositorio.ObtenerPorId(ordenId);` → consulta la orden mediante el repositorio y conserva el resultado anulable en una variable local.

Línea 248: `if (orden is null) return $"Orden {ordenId} no encontrada";` → evalúa la condición y ejecuta el bloque asociado sólo cuando se cumple.

Línea 249: `var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);` → cuenta las filas que cumplen la consulta construida.

Línea 250: `return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";` → devuelve el resultado calculado al código que invocó el método.

Línea 251: `}` → cierra el bloque de código actual.

Línea 252: `}` → cierra el bloque de código actual.

Línea 254: `public static class Program` → declara la clase Program que forma parte del estado acumulativo del ejercicio.

Línea 255: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 256: `public static void Main()` → declara el punto de entrada ejecutable de la aplicación de consola.

Línea 257: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 258: `var configuration = new ConfigurationBuilder()` → inicia la construcción de la configuración de la aplicación.

Línea 259: `.SetBasePath(AppContext.BaseDirectory)` → establece la ruta base usada para localizar los archivos de configuración.

Línea 260: `.AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)` → incorpora appsettings.json a la configuración.

Línea 261: `.AddEnvironmentVariables()` → incorpora variables de entorno como fuente adicional de configuración.

Línea 262: `.Build();` → materializa la configuración o el objeto encadenado construido en las líneas anteriores.

Línea 264: `var connectionString = configuration.GetConnectionString("AceriaDB")` → obtiene la cadena de conexión AceriaDB desde la configuración.

Línea 265: `?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");` → interrumpe el arranque con un error explícito si la configuración no contiene ConnectionStrings:AceriaDB.

Línea 267: `var services = new ServiceCollection();` → crea la colección donde se registran las dependencias de la aplicación.

Línea 268: `services.AddDbContext<AceriaDbContext>(options => → registra el DbContext en el contenedor de inyección de dependencias.`

Línea 269: `options → continúa la configuración del DbContext sobre el parámetro options recibido por la lambda de AddDbContext.`

Línea 270: `.UseSqlServer(connectionString, sql => → selecciona SQL Server como proveedor de EF Core.`

Línea 271: `{ → abre el bloque de código asociado a la declaración o instrucción anterior.`

Línea 272: `sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null); → habilita reintentos ante fallos transitorios del proveedor SQL Server.`

Línea 273: `sql.CommandTimeout(60); → establece el tiempo máximo permitido para los comandos SQL.`

Línea 274: `}) → cierra la lambda de configuración actual y devuelve el control a la llamada encadenada que la contiene.`

Línea 275: `.LogTo(global::System.Console.WriteLine, new[]
{ "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information) → configura la salida del logging de EF Core.`

Línea 276: `.EnableDetailedErrors(); → activa errores detallados para facilitar el diagnóstico del laboratorio.`

Línea 278: `services.AddScoped<IOrdenRepositorio, OrdenRepositorio>(); → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.`

Línea 279: `services.AddScoped<IServicioOrdenes, ServicioOrdenes>(); → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.`

Línea 281: `using var provider = services.BuildServiceProvider(new ServiceProviderOptions → construye el ServiceProvider con validación de servicios y garantiza su liberación al salir del ámbito mediante using var.`

Línea 282: `{ → abre el bloque de código asociado a la declaración o instrucción anterior.`

Línea 283: `ValidateScopes = true, → activa la validación de ámbitos para detectar dependencias scoped usadas desde ámbitos incorrectos.`

Línea 284: `ValidateOnBuild = true → obliga al contenedor a validar las resoluciones registradas al construir el ServiceProvider.`

Línea 285: `}); → cierra la lambda o configuración encadenada y finaliza la llamada.`

Línea 287: `using var scope = provider.CreateScope(); → crea un ámbito de DI para resolver servicios scoped y lo libera automáticamente al terminar el bloque.`

Línea 288: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>(); → resuelve un servicio obligatorio desde el contenedor de dependencias.`

Línea 289: `context.Database.EnsureDeleted(); → elimina la base del laboratorio para reproducir el escenario desde un estado limpio.`

Línea 290: `context.Database.Migrate(); → aplica las migraciones pendientes y deja AceriaDB en el esquema del punto.`

Línea 292: `var orden = new OrdenFabricacion → crea la entidad OrdenFabricacion que concentra el grafo de datos del escenario acumulativo.`

Línea 293: `{ → abre el bloque de código asociado a la declaración o instrucción anterior.`

Línea 294: `NumeroOrden = "0F-M2-0001", → asigna el identificador de negocio usado por la orden de validación acumulativa del módulo.`

Línea 295: `Cliente = "Constructora del Norte"`, → asigna el cliente de la orden utilizada en el escenario práctico del punto.

Línea 296: `FechaCreacion = DateTime.UtcNow`, → asigna a la orden la fecha de creación en UTC para el escenario reproducible del punto.

Línea 297: `FechaEntrega = DateTime.Today.AddDays(14)`, → establece una fecha de entrega prevista catorce días después de la fecha local actual.

Línea 298: `Estado = "Pendiente"`, → inicializa la orden con el estado Pendiente, coherente con el flujo del laboratorio.

Línea 299: `Observaciones = "Orden de validación M2"`, → añade texto opcional para comprobar el mapeo de Observaciones.

Línea 300: `Planchas =` → inicia el inicializador de la colección de planchas asociadas a la orden.

Línea 301: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 302: `new PlanchaAcero` → crea una nueva plancha dentro del inicializador de la colección Planchas de la orden.

Línea 303: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 304: `Espesor = 10.5`, → asigna a la plancha un espesor positivo para satisfacer las reglas del modelo.

Línea 305: `Ancho = 1500`, → asigna el ancho de la plancha usada en el escenario de persistencia.

Línea 306: `Largo = 3000`, → asigna el largo de la plancha usada en el escenario de persistencia.

Línea 307: `Peso = 371.250m`, → asigna un peso decimal con tres posiciones para validar la precisión configurada.

Línea 308: `Activa = true`, → marca la plancha como activa para que sea visible en los estados que aplican ese criterio.

Línea 309: `}` → cierra el bloque de código actual.

Línea 310: `}`, → continúa el inicializador o la llamada iniciada en el bloque actual.

Línea 311: `Detalle = new DetalleOrden` → crea el detalle uno-a-uno que se persistirá junto con la orden principal.

Línea 312: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 313: `ComposicionQuimica = "C: 0.45%, Mn: 0.75%"`, → asigna la composición química utilizada para verificar la entidad DetalleOrden.

Línea 314: `TemperaturaColada = 1550.5`, → asigna la temperatura de colada del detalle de fabricación.

Línea 315: `Notas = "Colada principal"` → asigna las notas opcionales del detalle asociado a la orden.

Línea 316: `}`, → continúa el inicializador o la llamada iniciada en el bloque actual.

Línea 317: `Certificado = new CertificadoCalidad` → crea el certificado uno-a-uno asociado a la orden del escenario.

Línea 318: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 319: `NumeroCertificado = "CERT-0001"`, → asigna el identificador natural del certificado de calidad.

Línea 320: `FechaEmision = DateTime.Today`, → asigna la fecha de emisión del certificado usado en la prueba.

Línea 321: `OrganismoCertificador = "Laboratorio Aceria"` → asigna el organismo que emite el certificado de calidad.

Línea 322: `}`, → continúa el inicializador o la llamada iniciada en el bloque actual.

Línea 323: `};` → cierra el inicializador del objeto y finaliza la instrucción.

Línea 325: `var aleacion = new Aleacion` → crea la aleación que se enlazará con la orden mediante la entidad intermedia `OrdenAleacion`.

Línea 326: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 327: `Nombre = "AISI 1045"`, → asigna el nombre comercial de la aleación utilizada en el escenario.

Línea 328: `Codigo = "A1045"`, → asigna el código natural único de la aleación.

Línea 329: `PorcentajeCarbono = 0.45`, → asigna el porcentaje de carbono dentro del rango permitido por la restricción del modelo.

Línea 330: `PorcentajeManganeso = 0.75`, → asigna el porcentaje de manganeso dentro del rango permitido por la restricción del modelo.

Línea 331: `Descripcion = "Acero medio en carbono"` → asigna una descripción opcional a la aleación.

Línea 332: `};` → cierra el inicializador del objeto y finaliza la instrucción.

Línea 333: `orden.OrdenesAleaciones.Add(new OrdenAleacion` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 334: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 335: `Aleacion = aleacion`, → conecta la entidad intermedia `OrdenAleacion` con la aleación creada para el escenario.

Línea 336: `CantidadUtilizada = 1500.500m`, → registra la cantidad utilizada con tres decimales, coherente con `HasPrecision(18, 3)`.

Línea 337: `EstadoRelacion = "Activa"` → marca como activa la relación entre la orden y la aleación.

Línea 338: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 339: `context.OrdenesFabricacion.Add(orden);` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 340: `context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo = true });` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 341: `context.SaveChanges();` → persiste en SQL Server los cambios seguidos por el `DbContext`.

Línea 343: `var entity = context.Model.FindEntityType(typeof(OrdenFabricacion))!;` → obtiene los metadatos EF Core de `OrdenFabricacion` para inspeccionar claves, índices o filtros del modelo construido.

Línea 344: `global::System.Console.WriteLine($"2.6 OK | Tabla por annotations: {entity.GetTableName()} | NumeroOrden MaxLength: {entity.FindProperty(nameof(OrdenFabricacion.NumeroOrden))?.GetMaxLength()}");` → escribe en consola la evidencia usada para verificar el comportamiento del estado.

Línea 345: `}` → cierra el bloque de código actual.

Línea 346: `}` → cierra el bloque de código actual.

Punto 2.7 - Configuración mediante Fluent API

Ejercicio guiado: recuperar y centralizar con Fluent API las reglas del modelo que en 2.6 se habían expresado parcialmente mediante Data Annotations, comprobando qué cambia en el modelo relacional y por qué Fluent API prevalece cuando ambas fuentes configuran la misma característica.

Contexto del proyecto: 2.7 parte íntegramente del estado 2.6. Las Data Annotations continúan presentes en las clases, pero OnModelCreating vuelve a expresar explícitamente propiedades, defaults y relaciones. Este punto no introduce todavía índices avanzados, restricciones CHECK ni filtros globales: esos conceptos se reservan para sus puntos específicos.

Objetivos de aprendizaje

- Configurar tablas y claves con ToTable y HasKey.
- Configurar propiedades con Property, IsRequired y HasMaxLength.
- Restaurar defaults de base de datos con HasDefaultValue y HasDefaultValueSql.
- Configurar precisión decimal con HasPrecision.
- Configurar relaciones 1:N y 1:1 con HasOne, WithMany, WithOne y HasForeignKey.
- Configurar DeleteBehavior de forma explícita.
- Mantener la clave compuesta y relaciones de OrdenAleacion.
- Comprender por qué Fluent API tiene prioridad sobre Data Annotations.
- Interpretar la migración real M2_2_7.
- Comprobar el modelo efectivo mediante context.Model.
- Trasladar una regla MaxLength desde annotation a Fluent sin cambiar el resultado final.

Paso 1: Abrir el estado autónomo de 2.7

```
cd M02/PROYECTO/2.7
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

El checkpoint contiene el estado completo del punto. Si reproduces la evolución manualmente, parte de 2.6.

Paso 2: Revisar la coexistencia con Data Annotations

Las clases conservan los atributos añadidos en 2.6, por ejemplo:

```
[Table("OrdenesFabricacion")]
public class OrdenFabricacion
{
    [Key]
    public int Id { get; set; }

    [Required]
    [MaxLength(50)]
    public string NumeroOrden { get; set; } = string.Empty;

    [Required]
    [MaxLength(200)]
    public string Cliente { get; set; } = string.Empty;
}
```

La Fluent API no obliga a eliminar estos atributos. Pueden coexistir. Si ambos mecanismos expresan la misma regla, el modelo final es coherente; si expresan valores diferentes, la configuración Fluent aplicada en OnModelCreating prevalece.

Paso 3: Configurar OrdenFabricacion con Fluent API

El bloque central del punto es:

```
modelBuilder.Entity<OrdenFabricacion>(entity =>
{
    entity.ToTable("OrdenesFabricacion");
    entity.HasKey(o => o.Id);

    entity.Property(o => o.NumeroOrden)
        .IsRequired()
        .HasMaxLength(50);

    entity.Property(o => o.Cliente)
        .IsRequired()
        .HasMaxLength(200);

    entity.Property(o => o.FechaCreacion)
        .HasDefaultValueSql("GETDATE()");

    entity.Property(o => o.Estado)
        .IsRequired()
        .HasMaxLength(50)
        .HasDefaultValue("Pendiente");

    entity.Property(o => o.Observaciones)
        .HasMaxLength(500);
});
```

ToTable y HasKey hacen explícitos tabla y PK. Property selecciona una propiedad concreta. IsRequired y HasMaxLength describen nulabilidad y longitud. HasDefaultValueSql delega el valor por defecto a SQL Server; HasDefaultValue define un valor constante del esquema.

Paso 4: Comprender la diferencia entre inicializador C# y default SQL

En la clase puede existir:

```
public string Estado { get; set; } = "Pendiente";
```

pero esa asignación sólo actúa cuando .NET crea una instancia. Fluent API añade además:

```
entity.Property(o => o.Estado)
    .HasDefaultValue("Pendiente");
```

Este segundo valor forma parte del modelo relacional y de la migración. Lo mismo ocurre con:

```
entity.Property(o => o.FechaCreacion)
    .HasDefaultValueSql("GETDATE()");
```

Por eso 2.7 produce cambios de esquema aunque los objetos C# ya tengan inicializadores.

Paso 5: Restaurar precisión y defaults de PlanchaAcero

```
modelBuilder.Entity<PlanchaAcero>(entity =>
{
    entity.ToTable("PlanchasAcero");
    entity.HasKey(x => x.Id);

    entity.Property(x => x.Peso)
        .HasPrecision(18, 3);

    entity.Property(x => x.Activa)
        .HasDefaultValue(true);

    entity.HasOne(x => x.Orden)
        .WithMany(o => o.Planchas)
        .HasForeignKey(x => x.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
});
```

Aquí Fluent API complementa la annotation Precision y vuelve a introducir el DEFAULT SQL de Activa.

Paso 6: Configurar Aleacion y EstadoOrden

```
modelBuilder.Entity<Aleacion>(entity =>
{
    entity.ToTable("Aleaciones");
    entity.HasKey(a => a.Id);
    entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
    entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
    entity.Property(a => a.Descripcion).HasMaxLength(500);
});

modelBuilder.Entity<EstadoOrden>(entity =>
{
    entity.ToTable("EstadosOrden");
    entity.HasKey(e => e.Id);
    entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
    entity.Property(e => e.Descripcion).HasMaxLength(250);
    entity.Property(e => e.Activo).HasDefaultValue(true);
});
```

El default true de Activo vuelve a formar parte del esquema de SQL Server.

Paso 7: Mantener las relaciones uno-a-uno

DetalleOrden conserva:

```
entity.HasOne(d => d.Orden)
    .WithOne(o => o.Detalle)
    .HasForeignKey<DetalleOrden>(d => d.OrdenId)
    .OnDelete(DeleteBehavior.Cascade)
    .IsRequired();
```

CertificadoCalidad usa la misma estructura con su propia FK. Fluent API es especialmente útil aquí porque permite señalar de forma inequívoca el dependiente, la FK y el comportamiento de eliminación.

Paso 8: Configurar OrdenAleacion de forma explícita

```
modelBuilder.Entity<OrdenAleacion>(entity =>
{
    entity.ToTable("OrdenesAleaciones");
    entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });

    entity.Property(x => x.FechaAsignacion)
        .HasDefaultValueSql("GETDATE()");

    entity.Property(x => x.CantidadUtilizada)
        .HasPrecision(18, 3);

    entity.Property(x => x.EstadoRelacion)
        .IsRequired()
        .HasMaxLength(20)
        .HasDefaultValue("Activa");

    entity.HasOne(x => x.Orden)
        .WithMany(o => o.OrdenesAleaciones)
        .HasForeignKey(x => x.OrdenFabricacionId)
        .OnDelete(DeleteBehavior.Cascade);

    entity.HasOne(x => x.Aleacion)
        .WithMany(a => a.OrdenesAleaciones)
        .HasForeignKey(x => x.AleacionId)
        .OnDelete(DeleteBehavior.Restrict);
});
```

La configuración mantiene la PK compuesta de 2.5/2.6, restaura el default SQL de FechaAsignacion y el default de EstadoRelacion y conserva Cascade hacia Orden y Restrict hacia Aleacion.

Paso 9: Compilar antes de generar la migración

```
dotnet build AceriaData.sln --configuration Release
```

El build debe ser correcto antes de comparar el modelo con el snapshot de 2.6.

Paso 10: Generar M2_2_7

```
dotnet ef migrations add M2_2_7
```

El checkpoint ya contiene:

```
Migrations/20260927204806_M2_2_7.cs
```

No vuelvas a crear la misma migración dentro del estado final.

Paso 11: Revisar la migración real M2_2_7

La migración restaura exactamente los defaults que 2.6 había retirado:

- PlanchasAcero.Activa vuelve a tener default true.
- OrdenesFabricacion.FechaCreacion vuelve a tener default SQL GETDATE().
- OrdenesFabricacion.Estado vuelve a tener default "Pendiente".
- OrdenesAleaciones.FechaAsignacion vuelve a tener default SQL GETDATE().
- OrdenesAleaciones.EstadoRelacion vuelve a tener default "Activa".
- EstadosOrden.Activo vuelve a tener default true.

Por ejemplo:

```
migrationBuilder.AlterColumn<DateTime>(
    name: "FechaCreacion",
    table: "OrdenesFabricacion",
    type: "datetime2",
    nullable: false,
    defaultValueSql: "GETDATE()",
    oldClrType: typeof(DateTime),
    oldType: "datetime2");
```

La migración es la evidencia tangible de que Fluent API no es sólo una sintaxis alternativa: puede cambiar el metadata relacional que EF Core compara con el snapshot.

Paso 12: Aplicar y verificar el historial

```
dotnet ef database update
dotnet ef migrations list
```

La lista debe incluir M2_2_7 después de M2_2_6.

Paso 13: Ejecutar el estado y comprobar el default efectivo

```
dotnet run --project AceriaData.Console.csproj --configuration Release
```

La salida del checkpoint contiene:

```
2.7 OK | Fluent API | Estado default: Pendiente
```

La aplicación obtiene ese valor desde context.Model, no desde una constante escrita para la demostración.

Paso 14: Resolver el reto MaxLength de annotation a Fluent

Reto: trasladar la regla MaxLength de Cliente desde la annotation a Fluent API sin cambiar el modelo efectivo.

El checkpoint contiene el bloque RETO 2.7 - MAXLENGTH DE ANNOTATION A FLUENT. Para reproducirlo, comenta temporalmente:

```
[MaxLength(200)]
```

sobre Cliente. Mantén activa la regla Fluent:

```
entity.Property(o => o.Cliente)
    .IsRequired()
    .HasMaxLength(200);
```

y consulta el metadata:

```
var clienteMetadata = entity.FindProperty(nameof(OrdenFabricacion.Cliente))
    ?? throw new InvalidOperationException("No se encontró Cliente en el modelo.");

Console.WriteLine(
    $"Reto 2.7 MaxLength Cliente: {clienteMetadata.GetMaxLength()}");
```

Resultado esperado:

```
Reto 2.7 MaxLength Cliente: 200
```

Esto demuestra que la misma regla puede vivir en Fluent API sin alterar el modelo final.

Paso 15: Delimitar el alcance del punto

En 2.7 no se añaden:

- índices compuestos o únicos nuevos;
- restricciones CHECK;
- filtros globales HasQueryFilter;
- collation;
- owned types;
- table splitting;
- nuevas entidades.

Esos conceptos tienen sus propios puntos y se introducirán cuando corresponda. Aquí el objetivo es dominar la sintaxis y la prioridad de Fluent API sobre el modelo acumulado existente.

Errores comunes

Error	Causa	Corrección
Fluent contradice una annotation sin intención	Dos fuentes configuran valores distintos	Elegir conscientemente la regla final; Fluent prevalece
Se confunde inicializador C# con default SQL	Se observa el valor del objeto y no el esquema	Revisar la migración y el metadata del modelo
Se elimina PrimaryKey de OrdenAleacion sin mantener HasKey	Se rompe la PK compuesta	Conservar al menos una configuración correcta y coherente
Se cambia Cascade/Restrict al centralizar la configuración	Se altera el comportamiento de relaciones ya certificado	Mantener los DeleteBehavior heredados
Se añaden índices o CHECK	Se adelantan puntos posteriores	Reservarlos para sus apartados específicos
Se usa EnsureCreated()	Se evita el historial de Migrations	Mantener Database.Migrate() / database update

Analogía final

La Fluent API funciona como el plano central de configuración de la planta. Las etiquetas de las Data Annotations siguen pegadas a cada pieza, pero el plano central puede confirmar o sobrescribir una especificación y, además, describir relaciones completas entre componentes. La migración muestra qué decisiones del plano terminan afectando realmente a la instalación.

Resultado esperado

Al finalizar 2.7, AceriaData conserva las Data Annotations de 2.6 pero centraliza de nuevo propiedades, defaults y relaciones en Fluent API. M2_2_7 restaura los defaults SQL retirados en el punto anterior y la ejecución confirma que Estado tiene el default Pendiente. El reto demuestra que MaxLength de Cliente puede trasladarse de annotation a Fluent sin cambiar el resultado efectivo.

Conexión con el siguiente punto

2.8 partirá de este estado para trabajar específicamente con claves primarias, alternativas y compuestas, sin rehacer el proyecto.

Código acumulativo completo del estado 2.7

El siguiente archivo es el `Program.cs` real del estado ejecutable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

[Table("OrdenesFabricacion")]
public class OrdenFabricacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroOrden { get; set; } = string.Empty;
    [Required]
    [MaxLength(200)]
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    [Required]
    [MaxLength(50)]
    public string Estado { get; set; } = "Pendiente";
    [MaxLength(500)]
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
    public DetalleOrden? Detalle { get; set; }
    public CertificadoCalidad? Certificado { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

[Table("PlanchasAcero")]
public class PlanchaAcero
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    [Precision(18, 3)]
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("Aleaciones")]
public class Aleacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(100)]
    public string Nombre { get; set; } = string.Empty;
    [Required]
    [MaxLength(20)]
    public string Codigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    [MaxLength(500)]
    public string? Descripcion { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}
```

```

[Table("EstadosOrden")]
public class EstadoOrden
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string Nombre { get; set; } = string.Empty;
    [MaxLength(250)]
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
}

[Table("DetallesOrden")]
public class DetalleOrden
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(200)]
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    [MaxLength(500)]
    public string? Notas { get; set; }
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("CertificadosCalidad")]
public class CertificadoCalidad
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroCertificado { get; set; } = string.Empty;
    public DateTime FechaEmision { get; set; }
    [Required]
    [MaxLength(100)]
    public string OrganismoCertificador { get; set; } = string.Empty;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]
[Table("OrdenesAleaciones")]
public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }
    public DateTime FechaAsignacion { get; set; } = DateTime.Now;
    [Precision(18, 3)]
    public decimal CantidadUtilizada { get; set; }
    [MaxLength(20)]
    public string EstadoRelacion { get; set; } = "Activa";
    public OrdenFabricacion Orden { get; set; } = null!;
    public Aleacion Aleacion { get; set; } = null!;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
    public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();
    public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();
    public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();

```

```

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);
    modelBuilder.Entity<OrdenFabricacion>(entity =>
    {
        entity.ToTable("OrdenesFabricacion");
        entity.HasKey(o => o.Id);
        entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);
        entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);
        entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");
        entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");
        entity.Property(o => o.Observaciones).HasMaxLength(500);
    });
    modelBuilder.Entity<PlanchaAcero>(entity =>
    {
        entity.ToTable("PlanchasAcero");
        entity.HasKey(x => x.Id);
        entity.Property(x => x.Peso).HasPrecision(18, 3);
        entity.Property(x => x.Activa).HasDefaultValue(true);
        entity.HasOne(x => x.Orden)
            .WithMany(o => o.Planchas)
            .HasForeignKey(x => x.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });
    modelBuilder.Entity<Aleacion>(entity =>
    {
        entity.ToTable("Aleaciones");
        entity.HasKey(a => a.Id);
        entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
        entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
        entity.Property(a => a.Descripcion).HasMaxLength(500);
    });
    modelBuilder.Entity<EstadoOrden>(entity =>
    {
        entity.ToTable("EstadosOrden");
        entity.HasKey(e => e.Id);
        entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
        entity.Property(e => e.Descripcion).HasMaxLength(250);
        entity.Property(e => e.Activo).HasDefaultValue(true);
    });
    modelBuilder.Entity<DetalleOrden>(entity =>
    {
        entity.ToTable("DetallesOrden");
        entity.HasKey(d => d.Id);
        entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);
        entity.Property(d => d.Notas).HasMaxLength(500);
        entity.HasOne(d => d.Orden)
            .WithOne(o => o.Detalle)
            .HasForeignKey<DetalleOrden>(d => d.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });
    modelBuilder.Entity<CertificadoCalidad>(entity =>
    {
        entity.ToTable("CertificadosCalidad");
        entity.HasKey(c => c.Id);
        entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);
        entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);
        entity.HasOne(c => c.Orden)
            .WithOne(o => o.Certificado)
            .HasForeignKey<CertificadoCalidad>(c => c.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });
    modelBuilder.Entity<OrdenAleacion>(entity =>
    {
        entity.ToTable("OrdenesAleaciones");
        entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });
        entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");
        entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);
    });
}

```

```

        entity.Property(x =>
x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");
        entity.HasOne(x => x.Orden)
            .WithMany(o => o.OrdenesAleaciones)
            .HasForeignKey(x => x.OrdenFabricacionId)
            .OnDelete(DeleteBehavior.Cascade);
        entity.HasOne(x => x.Aleacion)
            .WithMany(a => a.OrdenesAleaciones)
            .HasForeignKey(x => x.AleacionId)
            .OnDelete(DeleteBehavior.Restrict);
    });
}
}

public interface IOordenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    int Guardar();
}

public sealed class OrdenRepositorio : IOordenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o =>
o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) =>
_context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOordenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOordenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();
    }
}

```

```

var connectionString = configuration.GetConnectionString("AceriaDB")
?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

var services = new ServiceCollection();
services.AddDbContext<AceriaDbContext>(options =>
    options
        .UseSqlServer(connectionString, sql =>
            {
                sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                sql.CommandTimeout(60);
            })
        .LogTo(global::System.Console.WriteLine, new[] {
            "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)
        .EnableDetailedErrors());

services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateScopes = true,
    ValidateOnBuild = true
});

using var scope = provider.CreateScope();
var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();

var orden = new OrdenFabricacion
{
    NumeroOrden = "OF-M2-0001",
    Cliente = "Constructora del Norte",
    FechaCreacion = DateTime.UtcNow,
    FechaEntrega = DateTime.Today.AddDays(14),
    Estado = "Pendiente",
    Observaciones = "Orden de validación M2",
    Planchas =
    {
        new PlanchaAcero
        {
            Espesor = 10.5,
            Ancho = 1500,
            Largo = 3000,
            Peso = 371.250m,
            Activa = true,
        }
    },
    Detalle = new DetalleOrden
    {
        ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
        TemperaturaColada = 1550.5,
        Notas = "Colada principal"
    },
    Certificado = new CertificadoCalidad
    {
        NumeroCertificado = "CERT-0001",
        FechaEmision = DateTime.Today,
        OrganismoCertificador = "Laboratorio Aceria"
    },
};

var aleacion = new Aleacion
{
    Nombre = "AISI 1045",
   Codigo = "A1045",
    PorcentajeCarbono = 0.45,
    PorcentajeManganeso = 0.75,
    Descripcion = "Acero medio en carbono"
};
orden.OrdenesAleaciones.Add(new OrdenAleacion
{
    Aleacion = aleacion,

```

```

        CantidadUtilizada = 1500.500m,
        EstadoRelacion = "Activa"
    });
    context.OrdenesFabricacion.Add(orden);
    context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa",
Activo = true });
    context.SaveChanges();

    var entity = context.Model.FindEntityType(typeof(OrdenFabricacion))!;
    global::System.Console.WriteLine($"2.7 OK | Fluent API | Estado default:
{entity.FindProperty(nameof(OrdenFabricacion.Estado))?.GetDefaultValue()}");
    }
}

```

Explicación línea a línea del código acumulativo 2.7

Las referencias siguientes usan la numeración real de `M02/PROYECTO/2.7/Program.cs`. Se explican todas las líneas no vacías para que el documento sea trazable con el archivo que compila.

Línea 1: `using System.ComponentModel.DataAnnotations;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 2: `using System.ComponentModel.DataAnnotations.Schema;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 3: `using Microsoft.EntityFrameworkCore;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 4: `using Microsoft.Extensions.Configuration;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 5: `using Microsoft.Extensions.DependencyInjection;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 6: `using Microsoft.Extensions.Logging;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 8: `namespace AceriaData.ConsoleApp;` → declara el espacio de nombres del estado acumulativo.

Línea 10: `[Table("OrdenesFabricacion")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 11: `public class OrdenFabricacion` → declara la clase `OrdenFabricacion` que forma parte del estado acumulativo del ejercicio.

Línea 12: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 13: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 14: `public int Id { get; set; }` → declara la propiedad `Id` de tipo `int` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 15: `[Required]` → marca la propiedad siguiente como requerida.

Línea 16: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 17: `public string NumeroOrden { get; set; } = string.Empty;` → declara la propiedad `NumeroOrden` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 18: `[Required]` → marca la propiedad siguiente como requerida.

Línea 19: `[MaxLength(200)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 20: `public string Cliente { get; set; } = string.Empty;` → declara la propiedad `Cliente` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 21: `public DateTime FechaCreacion { get; set; }` → declara la propiedad `FechaCreacion` de tipo `DateTime` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 22: `public DateTime? FechaEntrega { get; set; }` → declara la propiedad FechaEntrega de tipo DateTime? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 23: `[Required]` → marca la propiedad siguiente como requerida.

Línea 24: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 25: `public string Estado { get; set; } = "Pendiente";` → declara la propiedad Estado de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 26: `[MaxLength(500)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 27: `public string? Observaciones { get; set; }` → declara la propiedad Observaciones de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 28: `public List<PlanchaAcero> Planchas { get; set; } = new();` → declara la propiedad Planchas de tipo List que EF Core o la lógica de negocio utiliza en este modelo.

Línea 29: `public DetalleOrden? Detalle { get; set; }` → declara la propiedad Detalle de tipo DetalleOrden? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 30: `public CertificadoCalidad? Certificado { get; set; }` → declara la propiedad Certificado de tipo CertificadoCalidad? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 31: `public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();` → declara la propiedad OrdenesAleaciones de tipo List que EF Core o la lógica de negocio utiliza en este modelo.

Línea 32: `}` → cierra el bloque de código actual.

Línea 34: `[Table("PlanchasAcero")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 35: `public class PlanchaAcero` → declara la clase PlanchaAcero que forma parte del estado acumulativo del ejercicio.

Línea 36: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 37: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 38: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 39: `public int OrdenId { get; set; }` → declara la propiedad OrdenId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 40: `public double Espesor { get; set; }` → declara la propiedad Espesor de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 41: `public double Ancho { get; set; }` → declara la propiedad Ancho de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 42: `public double Largo { get; set; }` → declara la propiedad Largo de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 43: `[Precision(18, 3)]` → configura precisión y escala para la propiedad numérica siguiente.

Línea 44: `public decimal Peso { get; set; }` → declara la propiedad Peso de tipo decimal que EF Core o la lógica de negocio utiliza en este modelo.

Línea 45: `public bool Activa { get; set; } = true;` → declara la propiedad Activa de tipo bool que EF Core o la lógica de negocio utiliza en este modelo.

Línea 46: `[ForeignKey(nameof(OrdenId))]` → vincula explícitamente la navegación siguiente con su clave foránea.

Línea 47: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.

Línea 48: `}` → cierra el bloque de código actual.

Línea 50: `[Table("Aleaciones")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 51: `public class Aleacion` → declara la clase Aleacion que forma parte del estado acumulativo del ejercicio.

Línea 52: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 53: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 54: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 55: `[Required]` → marca la propiedad siguiente como requerida.

Línea 56: `[MaxLength(100)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 57: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 58: `[Required]` → marca la propiedad siguiente como requerida.

Línea 59: `[MaxLength(20)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 60: `public string Codigo { get; set; } = string.Empty;` → declara la propiedad Codigo de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 61: `public double PorcentajeCarbono { get; set; }` → declara la propiedad PorcentajeCarbono de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 62: `public double PorcentajeManganeso { get; set; }` → declara la propiedad PorcentajeManganeso de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 63: `[MaxLength(500)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 64: `public string? Descripcion { get; set; }` → declara la propiedad Descripcion de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 65: `public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();` → declara la propiedad OrdenesAleaciones de tipo List que EF Core o la lógica de negocio utiliza en este modelo.

Línea 66: `}` → cierra el bloque de código actual.

Línea 68: `[Table("EstadosOrden")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 69: `public class EstadoOrden` → declara la clase EstadoOrden que forma parte del estado acumulativo del ejercicio.

Línea 70: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 71: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 72: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 73: `[Required]` → marca la propiedad siguiente como requerida.

Línea 74: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 75: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 76: `[MaxLength(250)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 77: `public string Descripcion { get; set; } = string.Empty;` → declara la propiedad Descripcion de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 78: `public bool Activo { get; set; } = true;` → declara la propiedad Activo de tipo bool que EF Core o la lógica de negocio utiliza en este modelo.

Línea 79: `}` → cierra el bloque de código actual.

Línea 81: `[Table("DetallesOrden")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 82: `public class DetalleOrden` → declara la clase DetalleOrden que forma parte del estado acumulativo del ejercicio.

Línea 83: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 84: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 85: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 86: `public int OrdenId { get; set; }` → declara la propiedad OrdenId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 87: `[Required]` → marca la propiedad siguiente como requerida.

Línea 88: `[MaxLength(200)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 89: `public string ComposicionQuimica { get; set; } = string.Empty;` → declara la propiedad ComposicionQuimica de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 90: `public double TemperaturaColada { get; set; }` → declara la propiedad TemperaturaColada de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 91: `[MaxLength(500)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 92: `public string? Notas { get; set; }` → declara la propiedad Notas de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 93: `[ForeignKey(nameof(OrdenId))]` → vincula explícitamente la navegación siguiente con su clave foránea.

Línea 94: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.

Línea 95: `}` → cierra el bloque de código actual.

Línea 97: `[Table("CertificadosCalidad")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 98: `public class CertificadoCalidad` → declara la clase CertificadoCalidad que forma parte del estado acumulativo del ejercicio.

Línea 99: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 100: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 101: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 102: `public int OrdenId { get; set; }` → declara la propiedad OrdenId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 103: `[Required]` → marca la propiedad siguiente como requerida.

Línea 104: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 105: `public string NumeroCertificado { get; set; } = string.Empty;` → declara la propiedad NumeroCertificado de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 106: `public DateTime FechaEmision { get; set; }` → declara la propiedad FechaEmision de tipo DateTime que EF Core o la lógica de negocio utiliza en este modelo.

Línea 107: `[Required]` → marca la propiedad siguiente como requerida.

Línea 108: `[MaxLength(100)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 109: `public string OrganismoCertificador { get; set; } = string.Empty;` → declara la propiedad OrganismoCertificador de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 110: `[ForeignKey(nameof(OrdenId))]` → vincula explícitamente la navegación siguiente con su clave foránea.

Línea 111: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.

Línea 112: `}` → cierra el bloque de código actual.

Línea 114: `[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]` → declara una clave primaria compuesta con las propiedades indicadas.

Línea 115: `[Table("OrdenesAleaciones")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 116: `public class OrdenAleacion` → declara la clase OrdenAleacion que forma parte del estado acumulativo del ejercicio.

Línea 117: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 118: `public int OrdenFabricacionId { get; set; }` → declara la propiedad OrdenFabricacionId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 119: `public int AleacionId { get; set; }` → declara la propiedad AleacionId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 120: `public DateTime FechaAsignacion { get; set; } = DateTime.Now;` → declara la propiedad FechaAsignacion de tipo DateTime que EF Core o la lógica de negocio utiliza en este modelo.

Línea 121: `[Precision(18, 3)]` → configura precisión y escala para la propiedad numérica siguiente.

Línea 122: `public decimal CantidadUtilizada { get; set; }` → declara la propiedad CantidadUtilizada de tipo decimal que EF Core o la lógica de negocio utiliza en este modelo.

Línea 123: `[MaxLength(20)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 124: `public string EstadoRelacion { get; set; } = "Activa";` → declara la propiedad EstadoRelacion de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 125: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.

Línea 126: `public Aleacion Aleacion { get; set; } = null!;` → declara la propiedad Aleacion de tipo Aleacion que EF Core o la lógica de negocio utiliza en este modelo.

Línea 127: `}` → cierra el bloque de código actual.

Línea 129: `public class AceriaDbContext : DbContext` → declara la clase AceriaDbContext que forma parte del estado acumulativo del ejercicio.

Línea 130: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 131:

`public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }` → declara el constructor o método AceriaDbContext con los parámetros necesarios para este componente.

Línea 133: `public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 134: `public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 135: `public DbSet<Aleacion> Aleaciones => Set<Aleacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 136: `public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 137: `public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 138: `public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 139: `public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 142: `protected override void OnModelCreating(ModelBuilder modelBuilder)` → sobrescribe `OnModelCreating` para definir configuración explícita del modelo.

Línea 143: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 144: `base.OnModelCreating(modelBuilder);` → sobrescribe `OnModelCreating` para definir configuración explícita del modelo.

Línea 145: `modelBuilder.Entity<OrdenFabricacion>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 146: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 147: `entity.ToTable("OrdenesFabricacion");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 148: `entity.HasKey(o => o.Id);` → define la clave primaria de la entidad.

Línea 149: `entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 150: `entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);` → selecciona una propiedad escalar para continuar su configuración.

Línea 151: `entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");` → selecciona una propiedad escalar para continuar su configuración.

Línea 152: `entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");` → selecciona una propiedad escalar para continuar su configuración.

Línea 153: `entity.Property(o => o.Observaciones).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 154: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 155: `modelBuilder.Entity<PlanchaAcero>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 156: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 157: `entity.ToTable("PlanchasAcero");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 158: `entity.HasKey(x => x.Id);` → define la clave primaria de la entidad.

Línea 159: `entity.Property(x => x.Peso).HasPrecision(18, 3);` → selecciona una propiedad escalar para continuar su configuración.

Línea 160: `entity.Property(x => x.Activa).HasDefaultValue(true);` → selecciona una propiedad escalar para continuar su configuración.

Línea 161: `entity.HasOne(x => x.Orden)` → inicia la configuración de una navegación de referencia.

Línea 162: `.WithMany(o => o.Planchas)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 163: `.HasForeignKey(x => x.OrdenId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 164: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 165: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 166: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 167: `modelBuilder.Entity<Aleacion>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 168: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 169: `entity.ToTable("Aleaciones");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 170: `entity.HasKey(a => a.Id);` → define la clave primaria de la entidad.

Línea 171: `entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);` → selecciona una propiedad escalar para continuar su configuración.

Línea 172: `entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);` → selecciona una propiedad escalar para continuar su configuración.

Línea 173: `entity.Property(a => a.Descripcion).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 174: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 175: `modelBuilder.Entity<EstadoOrden>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 176: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 177: `entity.ToTable("EstadosOrden");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 178: `entity.HasKey(e => e.Id);` → define la clave primaria de la entidad.

Línea 179: `entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 180: `entity.Property(e => e.Descripcion).HasMaxLength(250);` → selecciona una propiedad escalar para continuar su configuración.

Línea 181: `entity.Property(e => e.Activo).HasDefaultValue(true);` → selecciona una propiedad escalar para continuar su configuración.

Línea 182: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 183: `modelBuilder.Entity<DetalleOrden>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 184: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 185: `entity.ToTable("DetallesOrden");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 186: `entity.HasKey(d => d.Id);` → define la clave primaria de la entidad.

Línea 187: `entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);` → selecciona una propiedad escalar para continuar su configuración.

Línea 188: `entity.Property(d => d.Notas).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 189: `entity.HasOne(d => d.Orden)` → inicia la configuración de una navegación de referencia.

Línea 190: `.WithOne(o => o.Detalle)` → establece el otro extremo de referencia de una relación uno-a-uno.

Línea 191: `.HasForeignKey<DetalleOrden>(d => d.OrdenId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 192: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 193: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 194: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 196: `modelBuilder.Entity<CertificadoCalidad>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 197: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 198: `entity.ToTable("CertificadosCalidad");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 199: `entity.HasKey(c => c.Id);` → define la clave primaria de la entidad.

Línea 200: `entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 201: `entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);` → selecciona una propiedad escalar para continuar su configuración.

Línea 202: `entity.HasOne(c => c.Orden)` → inicia la configuración de una navegación de referencia.

Línea 203: `.WithOne(o => o.Certificado)` → establece el otro extremo de referencia de una relación uno-a-uno.

Línea 204: `.HasForeignKey<CertificadoCalidad>(c => c.OrdenId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 205: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 206: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 207: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 208: `modelBuilder.Entity<OrdenAleacion>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 209: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 210: `entity.ToTable("OrdenesAleaciones");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 211: `entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });` → define la clave primaria de la entidad.

Línea 212: `entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");` → selecciona una propiedad escalar para continuar su configuración.

Línea 213: `entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);` → selecciona una propiedad escalar para continuar su configuración.

Línea 214: `entity.Property(x => x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");` → selecciona una propiedad escalar para continuar su configuración.

Línea 215: `entity.HasOne(x => x.Orden)` → inicia la configuración de una navegación de referencia.

Línea 216: `.WithMany(o => o.OrdenesAleaciones)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 217: `.HasForeignKey(x => x.OrdenFabricacionId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 218: `.OnDelete(DeleteBehavior.Cascade);` → define el comportamiento de eliminación de la relación.

Línea 219: `entity.HasOne(x => x.Aleacion)` → inicia la configuración de una navegación de referencia.

Línea 220: `.WithMany(a => a.OrdenesAleaciones)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 221: `.HasForeignKey(x => x.AleacionId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 222: `.OnDelete(DeleteBehavior.Restrict);` → define el comportamiento de eliminación de la relación.

Línea 223: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 224: `}` → cierra el bloque de código actual.

Línea 225: `}` → cierra el bloque de código actual.

Línea 227: `public interface IOrdenRepositorio` → declara el contrato IOrdenRepositorio para desacoplar consumidores e implementación.

Línea 228: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 229: `List<OrdenFabricacion> ObtenerTodas();` → declara en el contrato la operación que debe devolver todas las órdenes visibles para el repositorio.

Línea 230: `OrdenFabricacion? ObtenerPorId(int id);` → declara en el contrato la búsqueda de una orden por su clave primaria y permite devolver null si no existe.

Línea 231: `OrdenFabricacion? ObtenerPorNumero(string numeroOrden);` → declara en el contrato la búsqueda por el identificador de negocio NumeroOrden.

Línea 232: `void Agregar(OrdenFabricacion orden);` → declara la operación que incorpora una orden al repositorio para que EF Core la siga como nueva.

Línea 233: `void Eliminar(OrdenFabricacion orden);` → declara la operación que marca una orden para eliminación física cuando el escenario lo requiere.

Línea 234: `int Guardar();` → declara la operación que confirma los cambios pendientes y devuelve el número de entradas afectadas.

Línea 235: `}` → cierra el bloque de código actual.

Línea 237: `public sealed class OrdenRepositorio : IOrdenRepositorio` → declara la clase OrdenRepositorio que forma parte del estado acumulativo del ejercicio.

Línea 238: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 239: `private readonly AceriaDbContext _context;` → declara el campo privado _context de tipo AceriaDbContext que conserva una dependencia de la clase.

Línea 240: `public OrdenRepositorio(AceriaDbContext context) => _context = context;` → declara el constructor o método `OrdenRepositorio` con los parámetros necesarios para este componente.

Línea 242: `public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();` → ordena el resultado de la consulta antes de materializarlo.

Línea 243: `public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);` → implementa la búsqueda por clave primaria mediante `Find`, aprovechando primero el `ChangeTracker` y después la base de datos.

Línea 244: `public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);` → recupera la primera coincidencia o `null` cuando no existe.

Línea 245: `public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 246: `public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);` → marca la entidad indicada para borrado al confirmar los cambios.

Línea 247: `public int Guardar() => _context.SaveChanges();` → persiste en SQL Server los cambios seguidos por el `DbContext`.

Línea 248: `}` → cierra el bloque de código actual.

Línea 250: `public interface IServicioOrdenes` → declara el contrato `IServicioOrdenes` para desacoplar consumidores e implementación.

Línea 251: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 252: `string ObtenerResumen(int ordenId);` → declara el servicio que construye un resumen de una orden a partir de su identificador.

Línea 253: `}` → cierra el bloque de código actual.

Línea 255: `public sealed class ServicioOrdenes : IServicioOrdenes` → declara la clase `ServicioOrdenes` que forma parte del estado acumulativo del ejercicio.

Línea 256: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 257: `private readonly IOrdenRepositorio _repositorio;` → declara el campo privado `_repositorio` de tipo `IOrdenRepositorio` que conserva una dependencia de la clase.

Línea 258: `private readonly AceriaDbContext _context;` → declara el campo privado `_context` de tipo `AceriaDbContext` que conserva una dependencia de la clase.

Línea 259: `public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)` → declara el constructor o método `ServicioOrdenes` con los parámetros necesarios para este componente.

Línea 260: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 261: `_repositorio = repositorio;` → guarda en el servicio el repositorio inyectado para reutilizarlo en sus operaciones.

Línea 262: `_context = context;` → guarda el `DbContext` inyectado para consultar los datos necesarios durante el servicio.

Línea 263: `}` → cierra el bloque de código actual.

Línea 265: `public string ObtenerResumen(int ordenId)` → declara el método que obtiene la orden y compone el resumen solicitado por el servicio.

Línea 266: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 267: `var orden = _repositorio.ObtenerPorId(ordenId);` → consulta la orden mediante el repositorio y conserva el resultado anulable en una variable local.

Línea 268: `if (orden is null) return $"Orden {ordenId} no encontrada";` → evalúa la condición y ejecuta el bloque asociado sólo cuando se cumple.

Línea 269: `var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);` → cuenta las filas que cumplen la consulta construida.

Línea 270: `return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";` → devuelve el resultado calculado al código que invocó el método.

Línea 271: `}` → cierra el bloque de código actual.

Línea 272: `}` → cierra el bloque de código actual.

Línea 274: `public static class Program` → declara la clase Program que forma parte del estado acumulativo del ejercicio.

Línea 275: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 276: `public static void Main()` → declara el punto de entrada ejecutable de la aplicación de consola.

Línea 277: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 278: `var configuration = new ConfigurationBuilder()` → inicia la construcción de la configuración de la aplicación.

Línea 279: `.SetBasePath(AppContext.BaseDirectory)` → establece la ruta base usada para localizar los archivos de configuración.

Línea 280: `.AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)` → incorpora appsettings.json a la configuración.

Línea 281: `.AddEnvironmentVariables()` → incorpora variables de entorno como fuente adicional de configuración.

Línea 282: `.Build();` → materializa la configuración o el objeto encadenado construido en las líneas anteriores.

Línea 284: `var connectionString = configuration.GetConnectionString("AceriaDB")` → obtiene la cadena de conexión AceriaDB desde la configuración.

Línea 285: `?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");` → interrumpe el arranque con un error explícito si la configuración no contiene ConnectionStrings:AceriaDB.

Línea 287: `var services = new ServiceCollection();` → crea la colección donde se registran las dependencias de la aplicación.

Línea 288: `services.AddDbContext<AceriaDbContext>(options =>` → registra el DbContext en el contenedor de inyección de dependencias.

Línea 289: `options` → continúa la configuración del DbContext sobre el parámetro options recibido por la lambda de AddDbContext.

Línea 290: `.UseSqlServer(connectionString, sql =>` → selecciona SQL Server como proveedor de EF Core.

Línea 291: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 292: `sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);` → habilita reintentos ante fallos transitorios del proveedor SQL Server.

Línea 293: `sql.CommandTimeout(60);` → establece el tiempo máximo permitido para los comandos SQL.

Línea 294: `})` → cierra la lambda de configuración actual y devuelve el control a la llamada encadenada que la contiene.

Línea 295: `.LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)` → configura la salida del logging de EF Core.

Línea 296: `.EnableDetailedErrors();` → activa errores detallados para facilitar el diagnóstico del laboratorio.

Línea 298: `services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();` → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.

Línea 299: `services.AddScoped<IServicioOrdenes, ServicioOrdenes>();` → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.

Línea 301: `using var provider = services.BuildServiceProvider(new ServiceProviderOptions` → construye el ServiceProvider con validación de servicios y garantiza su liberación al salir del ámbito mediante using var.

Línea 302: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 303: `ValidateScopes = true,` → activa la validación de ámbitos para detectar dependencias scoped usadas desde ámbitos incorrectos.

Línea 304: `ValidateOnBuild = true` → obliga al contenedor a validar las resoluciones registradas al construir el ServiceProvider.

Línea 305: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 307: `using var scope = provider.CreateScope();` → crea un ámbito de DI para resolver servicios scoped y lo libera automáticamente al terminar el bloque.

Línea 308: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → resuelve un servicio obligatorio desde el contenedor de dependencias.

Línea 309: `context.Database.EnsureDeleted();` → elimina la base del laboratorio para reproducir el escenario desde un estado limpio.

Línea 310: `context.Database.Migrate();` → aplica las migraciones pendientes y deja AceriaDB en el esquema del punto.

Línea 312: `var orden = new OrdenFabricacion` → crea la entidad OrdenFabricacion que concentra el grafo de datos del escenario acumulativo.

Línea 313: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 314: `NumeroOrden = "0F-M2-0001",` → asigna el identificador de negocio usado por la orden de validación acumulativa del módulo.

Línea 315: `Cliente = "Constructora del Norte",` → asigna el cliente de la orden utilizada en el escenario práctico del punto.

Línea 316: `FechaCreacion = DateTime.UtcNow,` → asigna a la orden la fecha de creación en UTC para el escenario reproducible del punto.

Línea 317: `FechaEntrega = DateTime.Today.AddDays(14),` → establece una fecha de entrega prevista catorce días después de la fecha local actual.

Línea 318: `Estado = "Pendiente",` → inicializa la orden con el estado Pendiente, coherente con el flujo del laboratorio.

Línea 319: `Observaciones = "Orden de validación M2",` → añade texto opcional para comprobar el mapeo de Observaciones.

Línea 320: `Planchas =` → inicia el inicializador de la colección de planchas asociadas a la orden.

Línea 321: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 322: `new PlanchaAcero` → crea una nueva plancha dentro del inicializador de la colección Planchas de la orden.

Línea 323: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 324: `Espesor = 10.5,` → asigna a la plancha un espesor positivo para satisfacer las reglas del modelo.

Línea 325: `Ancho = 1500,` → asigna el ancho de la plancha usada en el escenario de persistencia.

Línea 326: `Largo = 3000,` → asigna el largo de la plancha usada en el escenario de persistencia.

Línea 327: `Peso = 371.250m,` → asigna un peso decimal con tres posiciones para validar la precisión configurada.

Línea 328: `Activa = true,` → marca la plancha como activa para que sea visible en los estados que aplican ese criterio.

Línea 329: `}` → cierra el bloque de código actual.

Línea 330: `},` → continúa el inicializador o la llamada iniciada en el bloque actual.

Línea 331: `Detalle = new DetalleOrden` → crea el detalle uno-a-uno que se persistirá junto con la orden principal.

Línea 332: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 333: `ComposicionQuimica = "C: 0.45%, Mn: 0.75%",` → asigna la composición química utilizada para verificar la entidad DetalleOrden.

Línea 334: `TemperaturaColada = 1550.5,` → asigna la temperatura de colada del detalle de fabricación.

Línea 335: `Notas = "Colada principal"` → asigna las notas opcionales del detalle asociado a la orden.

Línea 336: `},` → continúa el inicializador o la llamada iniciada en el bloque actual.

Línea 337: `Certificado = new CertificadoCalidad` → crea el certificado uno-a-uno asociado a la orden del escenario.

Línea 338: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 339: `NumeroCertificado = "CERT-0001",` → asigna el identificador natural del certificado de calidad.

Línea 340: `FechaEmision = DateTime.Today,` → asigna la fecha de emisión del certificado usado en la prueba.

Línea 341: `OrganismoCertificador = "Laboratorio Aceria"` → asigna el organismo que emite el certificado de calidad.

Línea 342: `},` → continúa el inicializador o la llamada iniciada en el bloque actual.

Línea 343: `};` → cierra el inicializador del objeto y finaliza la instrucción.

Línea 345: `var aleacion = new Aleacion` → crea la aleación que se enlazará con la orden mediante la entidad intermedia OrdenAleacion.

Línea 346: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 347: `Nombre = "AISI 1045",` → asigna el nombre comercial de la aleación utilizada en el escenario.

Línea 348: `Codigo = "A1045",` → asigna el código natural único de la aleación.

Línea 349: `PorcentajeCarbono = 0.45,` → asigna el porcentaje de carbono dentro del rango permitido por la restricción del modelo.

Línea 350: `PorcentajeManganeso = 0.75,` → asigna el porcentaje de manganeso dentro del rango permitido por la restricción del modelo.

Línea 351: `Descripcion = "Acero medio en carbono"` → asigna una descripción opcional a la aleación.

Línea 352: `};` → cierra el inicializador del objeto y finaliza la instrucción.

Línea 353: `orden.OrdenesAleaciones.Add(new OrdenAleacion` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 354: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 355: `Aleacion = aleacion,` → conecta la entidad intermedia `OrdenAleacion` con la aleación creada para el escenario.

Línea 356: `CantidadUtilizada = 1500.500m,` → registra la cantidad utilizada con tres decimales, coherente con `HasPrecision(18, 3)`.

Línea 357: `EstadoRelacion = "Activa"` → marca como activa la relación entre la orden y la aleación.

Línea 358: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 359: `context.OrdenesFabricacion.Add(orden);` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 360: `context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo = true });` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 361: `context.SaveChanges();` → persiste en SQL Server los cambios seguidos por el `DbContext`.

Línea 363: `var entity = context.Model.FindEntityType(typeof(OrdenFabricacion));` → obtiene los metadatos EF Core de `OrdenFabricacion` para inspeccionar claves, índices o filtros del modelo construido.

Línea 364: `global::System.Console.WriteLine($"2.7 OK | Fluent API | Estado default: {entity.FindProperty(nameof(OrdenFabricacion.Estado)).GetDefaultValue()}");` → escribe en consola la evidencia usada para verificar el comportamiento del estado.

Línea 365: `}` → cierra el bloque de código actual.

Línea 366: `}` → cierra el bloque de código actual.

Punto 2.8 - Claves primarias, alternativas y compuestas

Ejercicio guiado: consolidar las reglas de identidad del modelo AceriaData distinguiendo qué papel cumple una clave primaria, cuándo una clave alternativa representa un identificador de negocio y por qué una clave compuesta identifica una relación mediante la combinación de varias propiedades.

Contexto del proyecto: 2.8 parte íntegramente del estado 2.7. No se rehacen las relaciones ni la configuración Fluent ya existente. El cambio de esquema de este punto consiste en añadir tres claves alternativas reales: `NumeroOrden` en `OrdenFabricacion`, `Codigo` en `Aleacion` y `NumeroCertificado` en `CertificadoCalidad`. La clave compuesta de `OrdenAleacion` ya existía y se conserva. Los índices adicionales y las restricciones CHECK se reservan para 2.9.

Objetivos de aprendizaje

- Distinguir clave primaria, clave alternativa y clave compuesta.
- Reconocer una clave alternativa como clave candidata del modelo, no como simple índice de rendimiento.
- Configurar `HasAlternateKey` y asignar un nombre estable a la restricción.
- Mantener la PK simple `Id` de las entidades principales.
- Mantener la PK compuesta `OrdenFabricacionId + AleacionId` de `OrdenAleacion`.
- Añadir las claves alternativas reales de `OrdenFabricacion`, `Aleacion` y `CertificadoCalidad`.
- Interpretar la migración real `M2_2_8` y sus tres `AddUniqueConstraint`.
- Inspeccionar claves con el metadata de EF Core mediante `GetKeys()` y `FindPrimaryKey()`.
- Verificar las restricciones en SQL Server LocalDB.
- Comprobar que una clave alternativa rechaza un identificador de negocio duplicado.
- Diferenciar una alternate key de los índices que se estudiarán en 2.9.

Paso 1: Abrir el estado autónomo de 2.8

```
cd M02/PROYECTO/2.8
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

El checkpoint contiene el estado completo y ejecutable del punto. Si reproduces la evolución manualmente, parte de 2.7 y aplica únicamente los cambios descritos aquí.

Paso 2: Identificar las tres formas de identidad presentes

En AceriaData conviven tres situaciones diferentes:

- **PK simple:** `OrdenFabricacion.Id`, `Aleacion.Id` o `CertificadoCalidad.Id` identifican técnicamente una fila.
- **Clave alternativa:** `NumeroOrden`, `Codigo` y `NumeroCertificado` son identificadores de negocio que también deben ser únicos.
- **PK compuesta:** `OrdenAleacion` se identifica por la combinación `OrdenFabricacionId + AleacionId`.

La diferencia es importante: una alternate key forma parte del modelo de identidad de EF Core y puede ser objetivo de relaciones. No debe añadirse sólo porque se quiera acelerar una búsqueda; para eso existen los índices.

Paso 3: Mantener la PK y añadir la alternate key de OrdenFabricacion

El bloque relevante del modelo efectivo es:

```
modelBuilder.Entity<OrdenFabricacion>(entity =>
{
    entity.ToTable("OrdenesFabricacion");
    entity.HasKey(o => o.Id);

    entity.HasAlternateKey(o => o.NumeroOrden)
        .HasName("AK_OrdenesFabricacion_NumeroOrden");
});
```

`HasKey(o => o.Id)` mantiene `Id` como clave primaria. `HasAlternateKey(o => o.NumeroOrden)` declara `NumeroOrden` como segunda clave candidata y `HasName` fija el nombre de la restricción que aparecerá en SQL Server.

No se añade además un `HasIndex(...).IsUnique()` sobre `NumeroOrden` en este punto: sería mezclar una regla de identidad con el contenido específico de índices de 2.9.

Paso 4: Mantener la PK de PlanchaAcero

`PlanchaAcero` continúa usando una PK simple:

```
modelBuilder.Entity<PlanchaAcero>(entity =>
{
    entity.ToTable("PlanchasAcero");
    entity.HasKey(x => x.Id);
});
```

No hay una nueva alternate key para esta entidad en 2.8. El resto de su configuración —precisión, default de `Activa` y relación con `OrdenFabricacion`— se hereda sin cambios desde 2.7.

Paso 5: Añadir Codigo como clave alternativa de Aleacion

```
modelBuilder.Entity<Aleacion>(entity =>
{
    entity.ToTable("Aleaciones");
    entity.HasKey(a => a.Id);

    entity.HasAlternateKey(a => a.Codigo)
        .HasName("AK_Aleaciones_Codigo");
});
```

`Id` sigue siendo la PK técnica. `Codigo` pasa a ser un identificador de negocio único. La migración real de este punto materializa esa decisión como una restricción UNIQUE llamada `AK_Aleaciones_Codigo`.

No se añade una segunda clave alternativa compuesta `Nombre + Codigo`: no forma parte del estado final 2.8 y cambiaría el modelo que debe recibir 2.9.

Paso 6: Revisar EstadoOrden y no inventar una clave nueva

`EstadoOrden` mantiene:

```
modelBuilder.Entity<EstadoOrden>(entity =>
{
    entity.ToTable("EstadosOrden");
    entity.HasKey(e => e.Id);
});
```

`Nombre` continúa siendo una propiedad requerida con longitud máxima, pero **no se convierte en alternate key en este punto**. Esto permite que la práctica coincida exactamente con el checkpoint y con la migración `M2_2_8`.

Paso 7: Mantener la PK de DetalleOrden y su relación 1:1

```
modelBuilder.Entity<DetalleOrden>(entity =>
{
    entity.ToTable("DetallesOrden");
    entity.HasKey(d => d.Id);

    entity.HasOne(d => d.Orden)
        .WithOne(o => o.Detalle)
        .HasForeignKey<DetalleOrden>(d => d.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
});
```

La identidad principal sigue en `Id`. La unicidad necesaria para la relación 1:1 se deriva de la propia relación; no se crea una alternate key artificial sobre `OrdenId`.

Paso 8: Añadir NumeroCertificado como clave alternativa

```
modelBuilder.Entity<CertificadoCalidad>(entity =>
{
    entity.ToTable("CertificadosCalidad");
    entity.HasKey(c => c.Id);

    entity.HasAlternateKey(c => c.NumeroCertificado)
        .HasName("AK_CertificadosCalidad_NumeroCertificado");
});
```

Un certificado conserva su `Id` técnico y, al mismo tiempo, `NumeroCertificado` se convierte en identificador de negocio único.

Paso 9: Mantener la clave primaria compuesta de OrdenAleacion

```
modelBuilder.Entity<OrdenAleacion>(entity =>
{
    entity.ToTable("OrdenesAleaciones");

    entity.HasKey(x => new
    {
        x.OrdenFabricacionId,
        x.AleacionId
    });
});
```

La combinación de ambas FK identifica una sola asignación de una aleación a una orden. No se cambia el orden de las columnas ni se introduce una segunda PK: se conserva la definición ya existente y coherente con los estados anteriores.

Paso 10: Compilar antes de generar la migración

```
dotnet build AceriaData.sln --configuration Release
```

El build debe ser correcto antes de pedir a EF Core que compare el modelo 2.8 con el snapshot de 2.7.

Paso 11: Generar la migración M2_2_8 al reproducir la evolución

Si partes manualmente del estado 2.7:

```
dotnet ef migrations add M2_2_8
```

El checkpoint final ya contiene:

```
Migrations/20260927204815_M2_2_8.cs
```

No generes de nuevo la misma migración dentro de `M02/PROYECTO/2.8`.

Paso 12: Leer la migración real, no una migración hipotética

`M2_2_8` añada exactamente tres restricciones:

```
migrationBuilder.AddUniqueConstraint(
    name: "AK_OrdenesFabricacion_NumeroOrden",
    table: "OrdenesFabricacion",
    column: "NumeroOrden");

migrationBuilder.AddUniqueConstraint(
    name: "AK_CertificadosCalidad_NumeroCertificado",
    table: "CertificadosCalidad",
    column: "NumeroCertificado");

migrationBuilder.AddUniqueConstraint(
    name: "AK_Aleaciones_Codigo",
    table: "Aleaciones",
    column: "Codigo");
```

No elimina ni recrea las PK existentes y no añade una alternate key a `EstadoOrden.Nombre`. Esa diferencia permite separar el objetivo pedagógico de 2.8 de configuraciones que no pertenecen al modelo final.

Paso 13: Aplicar el historial y verificar SQL Server

```
dotnet ef database update
dotnet ef migrations list
```

La lista debe contener `M2_2_8` después de `M2_2_7`.

En Visual Studio puedes abrir **Ver → Explorador de objetos de SQL Server**, expandir `(localdb)\MSSQLLocalDB` → **Bases de datos** → **AceriaDB** y revisar las restricciones de las tablas implicadas.

Comprueba especialmente:

- `AK_OrdenesFabricacion_NumeroOrden`;
- `AK_Aleaciones_Codigo`;
- `AK_CertificadosCalidad_NumeroCertificado`;
- la PK compuesta de `OrdenesAleaciones`.

Paso 14: Ejecutar e inspeccionar las claves del modelo

```
dotnet run --project AceriaData.Console.csproj --configuration Release
```

El checkpoint obtiene los metadatos reales:

```
var entity = context.Model.FindEntityType(typeof(OrdenFabricacion))!;

Console.WriteLine(
    $"2.8 OK | Alternate keys: {entity.GetKeys().Count()} | " +
    $"PK: {string.Join(", ", entity.FindPrimaryKey().Properties.Select(x => x.Name))}");
```

`GetKeys()` devuelve las claves configuradas para la entidad, incluida la primaria y las alternativas. `FindPrimaryKey()` permite comprobar qué propiedades forman la PK efectiva.

Paso 15: Resolver el reto de NumeroOrden duplicado

Reto: demostrar que la alternate key de `NumeroOrden` protege la identidad de negocio y no es sólo metadata.

El checkpoint contiene el bloque comentado `RETO 2.8 - NUMEROORDEN DUPLICADO`. Al activarlo se guarda primero una orden con:

```
NumeroOrden = "0F-M2-RETO-28"
```

y, desde un segundo ámbito y otro `DbContext`, se intenta guardar otra orden con el mismo valor.

La segunda operación se envuelve en:

```
try
{
    retoContext.SaveChanges();
    Console.WriteLine(
        "Reto 2.8 ERROR: se permitió NumeroOrden duplicado.");
}
catch (DbUpdateException)
{
    Console.WriteLine(
        "Reto 2.8: NumeroOrden duplicado rechazado");
}
```

Resultado esperado:

```
Reto 2.8: NumeroOrden duplicado rechazado
```

Se usan dos operaciones y un segundo contexto para que la demostración llegue realmente a SQL Server. El esquema se prepara mediante `Migrations`; no se sustituye por `EnsureCreated()`.

Paso 16: Razonar sobre la PK compuesta de OrdenAleacion

La PK de `OrdenAleacion` impide que exista dos veces la misma pareja `OrdenFabricacionId + AleacionId`. La regla se puede inspeccionar sin cambiar el modelo final:

```
var ordenAleacionEntity =
    context.Model.FindEntityType(typeof(OrdenAleacion));

var pk = ordenAleacionEntity.FindPrimaryKey();

Console.WriteLine(string.Join(
    ", ",
    pk.Properties.Select(p => p.Name)));
```

El resultado debe identificar las dos propiedades de la clave. Si se intenta persistir otra fila con la misma combinación, SQL Server protege la unicidad de la PK compuesta.

Paso 17: Delimitar el alcance antes de pasar a 2.9

En 2.8 **no** se añaden:

- índices únicos o compuestos adicionales con `HasIndex`;
- restricciones CHECK;
- una alternate key en `EstadoOrden.Nombre`;
- una alternate key compuesta extra en `Aleacion`;
- cambios de `DeleteBehavior`;
- nuevas entidades;
- `EnsureCreated()` como sustituto del historial de `Migrations`.

El objetivo es cerrar correctamente las reglas de identidad antes de estudiar índices y restricciones en el punto siguiente.

Errores comunes

Error	Causa	Corrección
Confundir alternate key con índice único	Ambas pueden imponer unicidad, pero expresan intenciones distintas	Usar <code>HasAlternateKey</code> para una clave candidata; estudiar <code>HasIndex</code> en 2.9
Duplicar <code>HasAlternateKey</code> y <code>HasIndex(...).IsUnique()</code> sobre la misma propiedad	Se mezclan dos configuraciones de unicidad	Mantener sólo la regla que corresponde al objetivo del punto
Omitir una propiedad de la PK compuesta	La identidad de <code>OrdenAleacion</code> queda incompleta	Mantener <code>OrdenFabricacionId + AleacionId</code>
Añadir una alternate key que no aparece en la migración real	El MD deja de representar el checkpoint	Contrastar siempre modelo, snapshot y <code>M2_2_8</code>
Aplicar una UNIQUE con datos duplicados existentes	SQL Server no puede crear la restricción	Limpiar o corregir los datos antes de aplicar la migración
Probar duplicados en el mismo <code>ChangeTracker</code> y confundir el error	EF Core puede detectar conflictos antes de llegar a SQL Server	Usar operaciones separadas cuando se quiera demostrar la restricción de base de datos
Usar <code>EnsureCreated()</code>	Se evita el historial acumulativo de Migrations	Mantener <code>Database.Migrate()</code> / <code>dotnet ef database update</code>

Analogía final

En una acería, la PK es el identificador técnico grabado en el registro interno de una pieza. Una alternate key es otro identificador reconocido por el negocio —por ejemplo, el número oficial de una orden o certificado— que también debe ser irrepetible. La PK compuesta de `OrdenAleacion` se parece a una ficha cuya identidad depende simultáneamente de la orden y de la aleación: repetir exactamente la misma pareja significaría duplicar la misma asignación.

Resultado esperado

Al finalizar 2.8, `AceriaData` conserva todas las reglas válidas de 2.7, mantiene las PK simples y la PK compuesta de `OrdenAleacion`, y añade tres claves alternativas reales: `NumeroOrden`, `Codigo` y `NumeroCertificado`. La migración `M2_2_8` materializa esas tres restricciones UNIQUE. La ejecución permite inspeccionar las claves del modelo y el reto confirma que SQL Server rechaza un `NumeroOrden` duplicado.

Conexión con el siguiente punto

2.9 partirá de este estado para trabajar específicamente con índices y restricciones, sin confundir una clave candidata con una estructura creada únicamente para búsqueda, ordenación o integridad adicional.

Código acumulativo completo del estado 2.8

El siguiente archivo es el `Program.cs` real del estado ejecutable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

```

using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

[Table("OrdenesFabricacion")]
public class OrdenFabricacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroOrden { get; set; } = string.Empty;
    [Required]
    [MaxLength(200)]
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    [Required]
    [MaxLength(50)]
    public string Estado { get; set; } = "Pendiente";
    [MaxLength(500)]
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
    public DetalleOrden? Detalle { get; set; }
    public CertificadoCalidad? Certificado { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

[Table("PlanchasAcero")]
public class PlanchaAcero
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    [Precision(18, 3)]
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("Aleaciones")]
public class Aleacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(100)]
    public string Nombre { get; set; } = string.Empty;
    [Required]
    [MaxLength(20)]
    public string Codigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    [MaxLength(500)]
    public string? Descripcion { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

[Table("EstadosOrden")]
public class EstadoOrden
{
    [Key]
    public int Id { get; set; }

```

```

[Required]
[MaxLength(50)]
public string Nombre { get; set; } = string.Empty;
[MaxLength(250)]
public string Descripcion { get; set; } = string.Empty;
public bool Activo { get; set; } = true;
}

[Table("DetallesOrden")]
public class DetalleOrden
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(200)]
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    [MaxLength(500)]
    public string? Notas { get; set; }
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("CertificadosCalidad")]
public class CertificadoCalidad
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroCertificado { get; set; } = string.Empty;
    public DateTime FechaEmision { get; set; }
    [Required]
    [MaxLength(100)]
    public string OrganismoCertificador { get; set; } = string.Empty;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]
[Table("OrdenesAleaciones")]
public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }
    public DateTime FechaAsignacion { get; set; } = DateTime.Now;
    [Precision(18, 3)]
    public decimal CantidadUtilizada { get; set; }
    [MaxLength(20)]
    public string EstadoRelacion { get; set; } = "Activa";
    public OrdenFabricacion Orden { get; set; } = null!;
    public Aleacion Aleacion { get; set; } = null!;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
    public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();
    public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();
    public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<OrdenFabricacion>(entity =>

```

```

{
    entity.ToTable("OrdenesFabricacion");
    entity.HasKey(o => o.Id);
    entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);
    entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);
    entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");
    entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");
    entity.Property(o => o.Observaciones).HasMaxLength(500);
    entity.HasAlternateKey(o => o.NumeroOrden).HasName("AK_OrdenesFabricacion_NumeroOrden");
});
modelBuilder.Entity<PlanchaAcero>(entity =>
{
    entity.ToTable("PlanchasAcero");
    entity.HasKey(x => x.Id);
    entity.Property(x => x.Peso).HasPrecision(18, 3);
    entity.Property(x => x.Activa).HasDefaultValue(true);
    entity.HasOne(x => x.Orden)
        .WithMany(o => o.Planchas)
        .HasForeignKey(x => x.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
});
modelBuilder.Entity<Aleacion>(entity =>
{
    entity.ToTable("Aleaciones");
    entity.HasKey(a => a.Id);
    entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
    entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
    entity.Property(a => a.Descripcion).HasMaxLength(500);
    entity.HasAlternateKey(a => a.Codigo).HasName("AK_Aleaciones_Codigo");
});
modelBuilder.Entity<EstadoOrden>(entity =>
{
    entity.ToTable("EstadosOrden");
    entity.HasKey(e => e.Id);
    entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
    entity.Property(e => e.Descripcion).HasMaxLength(250);
    entity.Property(e => e.Activo).HasDefaultValue(true);
});
modelBuilder.Entity<DetalleOrden>(entity =>
{
    entity.ToTable("DetallesOrden");
    entity.HasKey(d => d.Id);
    entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);
    entity.Property(d => d.Notas).HasMaxLength(500);
    entity.HasOne(d => d.Orden)
        .WithOne(o => o.Detalle)
        .HasForeignKey<DetalleOrden>(d => d.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
});

modelBuilder.Entity<CertificadoCalidad>(entity =>
{
    entity.ToTable("CertificadosCalidad");
    entity.HasKey(c => c.Id);
    entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);
    entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);
    entity.HasAlternateKey(c =>
c.NumeroCertificado).HasName("AK_CertificadosCalidad_NumeroCertificado");
    entity.HasOne(c => c.Orden)
        .WithOne(o => o.Certificado)
        .HasForeignKey<CertificadoCalidad>(c => c.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
});
modelBuilder.Entity<OrdenAleacion>(entity =>
{
    entity.ToTable("OrdenesAleaciones");
    entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });
    entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");
    entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);
    entity.Property(x =>

```

```

x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");
    entity.HasOne(x => x.Orden)
        .WithMany(o => o.OrdenesAleaciones)
        .HasForeignKey(x => x.OrdenFabricacionId)
        .OnDelete(DeleteBehavior.Cascade);
    entity.HasOne(x => x.Aleacion)
        .WithMany(a => a.OrdenesAleaciones)
        .HasForeignKey(x => x.AleacionId)
        .OnDelete(DeleteBehavior.Restrict);
    });
}
}

public interface IOordenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    int Guardar();
}

public sealed class OrdenRepositorio : IOordenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o =>
o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) =>
_context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOordenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOordenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")

```

```

        ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

var services = new ServiceCollection();
services.AddDbContext<AceriaDbContext>(options =>
    options
        .UseSqlServer(connectionString, sql =>
            {
                sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                sql.CommandTimeout(60);
            })
        .LogTo(global::System.Console.WriteLine, new[] {
            "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)
        .EnableDetailedErrors());

services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateScopes = true,
    ValidateOnBuild = true
});

using var scope = provider.CreateScope();
var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();

var orden = new OrdenFabricacion
{
    NumeroOrden = "OF-M2-0001",
    Cliente = "Constructora del Norte",
    FechaCreacion = DateTime.UtcNow,
    FechaEntrega = DateTime.Today.AddDays(14),
    Estado = "Pendiente",
    Observaciones = "Orden de validación M2",
    Planchas =
    {
        new PlanchaAcero
        {
            Espesor = 10.5,
            Ancho = 1500,
            Largo = 3000,
            Peso = 371.250m,
            Activa = true,
        }
    },
    Detalle = new DetalleOrden
    {
        ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
        TemperaturaColada = 1550.5,
        Notas = "Colada principal"
    },
    Certificado = new CertificadoCalidad
    {
        NumeroCertificado = "CERT-0001",
        FechaEmision = DateTime.Today,
        OrganismoCertificador = "Laboratorio Aceria"
    },
};

var aleacion = new Aleacion
{
    Nombre = "AISI 1045",
   Codigo = "A1045",
    PorcentajeCarbono = 0.45,
    PorcentajeManganeso = 0.75,
    Descripcion = "Acero medio en carbono"
};
orden.OrdenesAleaciones.Add(new OrdenAleacion
{
    Aleacion = aleacion,
    CantidadUtilizada = 1500.500m,

```

```

        EstadoRelacion = "Activa"
    });
    context.OrdenesFabricacion.Add(orden);
    context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa",
Activo = true });
    context.SaveChanges();

    var entity = context.Model.FindEntityType(typeof(OrdenFabricacion));
    global::System.Console.WriteLine($"2.8 OK | Alternate keys: {entity.GetKeys().Count()} | PK:
{string.Join(", ", entity.FindPrimaryKey().Properties.Select(x => x.Name))}");
    }
}

```

Explicación línea a línea del código acumulativo 2.8

Las referencias siguientes usan la numeración real de M02/PROYECTO/2.8/Program.cs. Se explican todas las líneas no vacías para que el documento sea trazable con el archivo que compila.

Línea 1: `using System.ComponentModel.DataAnnotations;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 2: `using System.ComponentModel.DataAnnotations.Schema;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 3: `using Microsoft.EntityFrameworkCore;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 4: `using Microsoft.Extensions.Configuration;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 5: `using Microsoft.Extensions.DependencyInjection;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 6: `using Microsoft.Extensions.Logging;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 8: `namespace AceriaData.ConsoleApp;` → declara el espacio de nombres del estado acumulativo.

Línea 10: `[Table("OrdenesFabricacion")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 11: `public class OrdenFabricacion` → declara la clase OrdenFabricacion que forma parte del estado acumulativo del ejercicio.

Línea 12: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 13: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 14: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 15: `[Required]` → marca la propiedad siguiente como requerida.

Línea 16: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 17: `public string NumeroOrden { get; set; } = string.Empty;` → declara la propiedad NumeroOrden de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 18: `[Required]` → marca la propiedad siguiente como requerida.

Línea 19: `[MaxLength(200)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 20: `public string Cliente { get; set; } = string.Empty;` → declara la propiedad Cliente de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 21: `public DateTime FechaCreacion { get; set; }` → declara la propiedad FechaCreacion de tipo DateTime que EF Core o la lógica de negocio utiliza en este modelo.

Línea 22: `public DateTime? FechaEntrega { get; set; }` → declara la propiedad FechaEntrega de tipo DateTime? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 23: `[Required]` → marca la propiedad siguiente como requerida.

Línea 24: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 25: `public string Estado { get; set; } = "Pendiente";` → declara la propiedad Estado de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 26: `[MaxLength(500)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 27: `public string? Observaciones { get; set; }` → declara la propiedad Observaciones de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 28: `public List<PlanchaAcero> Planchas { get; set; } = new();` → declara la propiedad Planchas de tipo List que EF Core o la lógica de negocio utiliza en este modelo.

Línea 29: `public DetalleOrden? Detalle { get; set; }` → declara la propiedad Detalle de tipo DetalleOrden? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 30: `public CertificadoCalidad? Certificado { get; set; }` → declara la propiedad Certificado de tipo CertificadoCalidad? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 31: `public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();` → declara la propiedad OrdenesAleaciones de tipo List que EF Core o la lógica de negocio utiliza en este modelo.

Línea 32: `}` → cierra el bloque de código actual.

Línea 34: `[Table("PlanchasAcero")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 35: `public class PlanchaAcero` → declara la clase PlanchaAcero que forma parte del estado acumulativo del ejercicio.

Línea 36: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 37: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 38: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 39: `public int OrdenId { get; set; }` → declara la propiedad OrdenId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 40: `public double Espesor { get; set; }` → declara la propiedad Espesor de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 41: `public double Ancho { get; set; }` → declara la propiedad Ancho de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 42: `public double Largo { get; set; }` → declara la propiedad Largo de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 43: `[Precision(18, 3)]` → configura precisión y escala para la propiedad numérica siguiente.

Línea 44: `public decimal Peso { get; set; }` → declara la propiedad Peso de tipo decimal que EF Core o la lógica de negocio utiliza en este modelo.

Línea 45: `public bool Activa { get; set; } = true;` → declara la propiedad Activa de tipo bool que EF Core o la lógica de negocio utiliza en este modelo.

Línea 46: `[ForeignKey(nameof(OrdenId))]` → vincula explícitamente la navegación siguiente con su clave foránea.

Línea 47: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.

Línea 48: `}` → cierra el bloque de código actual.

Línea 50: `[Table("Aleaciones")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 51: `public class Aleacion` → declara la clase Aleacion que forma parte del estado acumulativo del ejercicio.

Línea 52: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 53: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 54: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 55: `[Required]` → marca la propiedad siguiente como requerida.

Línea 56: `[MaxLength(100)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 57: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 58: `[Required]` → marca la propiedad siguiente como requerida.

Línea 59: `[MaxLength(20)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 60: `public string Codigo { get; set; } = string.Empty;` → declara la propiedad Codigo de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 61: `public double PorcentajeCarbono { get; set; }` → declara la propiedad PorcentajeCarbono de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 62: `public double PorcentajeManganeso { get; set; }` → declara la propiedad PorcentajeManganeso de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 63: `[MaxLength(500)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 64: `public string? Descripcion { get; set; }` → declara la propiedad Descripcion de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 65: `public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();` → declara la propiedad OrdenesAleaciones de tipo List que EF Core o la lógica de negocio utiliza en este modelo.

Línea 66: `}` → cierra el bloque de código actual.

Línea 68: `[Table("EstadosOrden")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 69: `public class EstadoOrden` → declara la clase EstadoOrden que forma parte del estado acumulativo del ejercicio.

Línea 70: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 71: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 72: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 73: `[Required]` → marca la propiedad siguiente como requerida.

Línea 74: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 75: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 76: `[MaxLength(250)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 77: `public string Descripcion { get; set; } = string.Empty;` → declara la propiedad Descripcion de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 78: `public bool Activo { get; set; } = true;` → declara la propiedad Activo de tipo bool que EF Core o la lógica de negocio utiliza en este modelo.

Línea 79: `}` → cierra el bloque de código actual.

Línea 81: `[Table("DetallesOrden")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 82: `public class DetalleOrden` → declara la clase DetalleOrden que forma parte del estado acumulativo del ejercicio.

Línea 83: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 84: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 85: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 86: `public int OrdenId { get; set; }` → declara la propiedad OrdenId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 87: `[Required]` → marca la propiedad siguiente como requerida.

Línea 88: `[MaxLength(200)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 89: `public string ComposicionQuimica { get; set; } = string.Empty;` → declara la propiedad ComposicionQuimica de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 90: `public double TemperaturaColada { get; set; }` → declara la propiedad TemperaturaColada de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 91: `[MaxLength(500)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 92: `public string? Notas { get; set; }` → declara la propiedad Notas de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 93: `[ForeignKey(nameof(OrdenId))]` → vincula explícitamente la navegación siguiente con su clave foránea.

Línea 94: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.

Línea 95: `}` → cierra el bloque de código actual.

Línea 97: `[Table("CertificadosCalidad")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 98: `public class CertificadoCalidad` → declara la clase CertificadoCalidad que forma parte del estado acumulativo del ejercicio.

Línea 99: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 100: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 101: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 102: `public int OrdenId { get; set; }` → declara la propiedad OrdenId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 103: `[Required]` → marca la propiedad siguiente como requerida.

Línea 104: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 105: `public string NumeroCertificado { get; set; } = string.Empty;` → declara la propiedad NumeroCertificado de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 106: `public DateTime FechaEmision { get; set; }` → declara la propiedad FechaEmision de tipo DateTime que EF Core o la lógica de negocio utiliza en este modelo.

Línea 107: `[Required]` → marca la propiedad siguiente como requerida.

Línea 108: `[MaxLength(100)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 109: `public string OrganismoCertificador { get; set; } = string.Empty;` → declara la propiedad OrganismoCertificador de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 110: `[ForeignKey(nameof(OrdenId))]` → vincula explícitamente la navegación siguiente con su clave foránea.

Línea 111: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.

Línea 112: `}` → cierra el bloque de código actual.

Línea 114: `[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]` → declara una clave primaria compuesta con las propiedades indicadas.

Línea 115: `[Table("OrdenesAleaciones")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 116: `public class OrdenAleacion` → declara la clase OrdenAleacion que forma parte del estado acumulativo del ejercicio.

Línea 117: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 118: `public int OrdenFabricacionId { get; set; }` → declara la propiedad OrdenFabricacionId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 119: `public int AleacionId { get; set; }` → declara la propiedad AleacionId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 120: `public DateTime FechaAsignacion { get; set; } = DateTime.Now;` → declara la propiedad FechaAsignacion de tipo DateTime que EF Core o la lógica de negocio utiliza en este modelo.

Línea 121: `[Precision(18, 3)]` → configura precisión y escala para la propiedad numérica siguiente.

Línea 122: `public decimal CantidadUtilizada { get; set; }` → declara la propiedad CantidadUtilizada de tipo decimal que EF Core o la lógica de negocio utiliza en este modelo.

Línea 123: `[MaxLength(20)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 124: `public string EstadoRelacion { get; set; } = "Activa";` → declara la propiedad EstadoRelacion de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 125: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.

Línea 126: `public Aleacion Aleacion { get; set; } = null!;` → declara la propiedad Aleacion de tipo Aleacion que EF Core o la lógica de negocio utiliza en este modelo.

Línea 127: `}` → cierra el bloque de código actual.

Línea 129: `public class AceriaDbContext : DbContext` → declara la clase AceriaDbContext que forma parte del estado acumulativo del ejercicio.

Línea 130: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 131:

`public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }` → declara el constructor o método AceriaDbContext con los parámetros necesarios para este componente.

Línea 133: `public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 134: `public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 135: `public DbSet<Aleacion> Aleaciones => Set<Aleacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 136: `public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 137: `public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 138: `public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 139: `public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 142: `protected override void OnModelCreating(ModelBuilder modelBuilder)` → sobrescribe `OnModelCreating` para definir configuración explícita del modelo.

Línea 143: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 144: `base.OnModelCreating(modelBuilder);` → sobrescribe `OnModelCreating` para definir configuración explícita del modelo.

Línea 145: `modelBuilder.Entity<OrdenFabricacion>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 146: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 147: `entity.ToTable("OrdenesFabricacion");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 148: `entity.HasKey(o => o.Id);` → define la clave primaria de la entidad.

Línea 149: `entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 150: `entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);` → selecciona una propiedad escalar para continuar su configuración.

Línea 151: `entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");` → selecciona una propiedad escalar para continuar su configuración.

Línea 152: `entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");` → selecciona una propiedad escalar para continuar su configuración.

Línea 153: `entity.Property(o => o.Observaciones).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 154: `entity.HasAlternateKey(o => o.NumeroOrden).HasName("AK_OrdenesFabricacion_NumeroOrden");` → define una clave alternativa que también debe ser única.

Línea 155: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 156: `modelBuilder.Entity<PlanchaAcero>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 157: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 158: `entity.ToTable("PlanchasAcero");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 159: `entity.HasKey(x => x.Id);` → define la clave primaria de la entidad.

Línea 160: `entity.Property(x => x.Peso).HasPrecision(18, 3);` → selecciona una propiedad escalar para continuar su configuración.

Línea 161: `entity.Property(x => x.Activa).HasDefaultValue(true);` → selecciona una propiedad escalar para continuar su configuración.

Línea 162: `entity.HasOne(x => x.Orden)` → inicia la configuración de una navegación de referencia.

Línea 163: `.WithMany(o => o.Planchas)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 164: `.HasForeignKey(x => x.OrdenId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 165: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 166: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 167: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 168: `modelBuilder.Entity<Aleacion>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 169: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 170: `entity.ToTable("Aleaciones");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 171: `entity.HasKey(a => a.Id);` → define la clave primaria de la entidad.

Línea 172: `entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);` → selecciona una propiedad escalar para continuar su configuración.

Línea 173: `entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);` → selecciona una propiedad escalar para continuar su configuración.

Línea 174: `entity.Property(a => a.Descripcion).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 175: `entity.HasAlternateKey(a => a.Codigo).HasName("AK_Aleaciones_Codigo");` → define una clave alternativa que también debe ser única.

Línea 176: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 177: `modelBuilder.Entity<EstadoOrden>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 178: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 179: `entity.ToTable("EstadosOrden");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 180: `entity.HasKey(e => e.Id);` → define la clave primaria de la entidad.

Línea 181: `entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 182: `entity.Property(e => e.Descripcion).HasMaxLength(250);` → selecciona una propiedad escalar para continuar su configuración.

Línea 183: `entity.Property(e => e.Activo).HasDefaultValue(true);` → selecciona una propiedad escalar para continuar su configuración.

Línea 184: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 185: `modelBuilder.Entity<DetalleOrden>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 186: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 187: `entity.ToTable("DetallesOrden");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 188: `entity.HasKey(d => d.Id);` → define la clave primaria de la entidad.

Línea 189: `entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);` → selecciona una propiedad escalar para continuar su configuración.

Línea 190: `entity.Property(d => d.Notas).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 191: `entity.HasOne(d => d.Orden)` → inicia la configuración de una navegación de referencia.

Línea 192: `.WithOne(o => o.Detalle)` → establece el otro extremo de referencia de una relación uno-a-uno.

Línea 193: `.HasForeignKey<DetalleOrden>(d => d.OrdenId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 194: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 195: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 196: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 198: `modelBuilder.Entity<CertificadoCalidad>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 199: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 200: `entity.ToTable("CertificadosCalidad");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 201: `entity.HasKey(c => c.Id);` → define la clave primaria de la entidad.

Línea 202: `entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 203: `entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);` → selecciona una propiedad escalar para continuar su configuración.

Línea 204: `entity.HasAlternateKey(c => c.NumeroCertificado).HasName("AK_CertificadosCalidad_NumeroCertificado");` → define una clave alternativa que también debe ser única.

Línea 205: `entity.HasOne(c => c.Orden)` → inicia la configuración de una navegación de referencia.

Línea 206: `.WithOne(o => o.Certificado)` → establece el otro extremo de referencia de una relación uno-a-uno.

Línea 207: `.HasForeignKey<CertificadoCalidad>(c => c.OrdenId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 208: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 209: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 210: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 211: `modelBuilder.Entity<OrdenAleacion>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 212: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 213: `entity.ToTable("OrdenesAleaciones");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 214: `entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });` → define la clave primaria de la entidad.

Línea 215: `entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");` → selecciona una propiedad escalar para continuar su configuración.

Línea 216: `entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);` → selecciona una propiedad escalar para continuar su configuración.

Línea 217: `entity.Property(x => x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");` → selecciona una propiedad escalar para continuar su configuración.

Línea 218: `entity.HasOne(x => x.Orden)` → inicia la configuración de una navegación de referencia.

Línea 219: `.WithMany(o => o.OrdenesAleaciones)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 220: `.HasForeignKey(x => x.OrdenFabricacionId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 221: `.OnDelete(DeleteBehavior.Cascade);` → define el comportamiento de eliminación de la relación.

Línea 222: `entity.HasOne(x => x.Aleacion)` → inicia la configuración de una navegación de referencia.

Línea 223: `.WithMany(a => a.OrdenesAleaciones)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 224: `.HasForeignKey(x => x.AleacionId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 225: `.OnDelete(DeleteBehavior.Restrict);` → define el comportamiento de eliminación de la relación.

Línea 226: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 227: `}` → cierra el bloque de código actual.

Línea 228: `}` → cierra el bloque de código actual.

Línea 230: `public interface IOrdenRepositorio` → declara el contrato IOrdenRepositorio para desacoplar consumidores e implementación.

Línea 231: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 232: `List<OrdenFabricacion> ObtenerTodas();` → declara en el contrato la operación que debe devolver todas las órdenes visibles para el repositorio.

Línea 233: `OrdenFabricacion? ObtenerPorId(int id);` → declara en el contrato la búsqueda de una orden por su clave primaria y permite devolver null si no existe.

Línea 234: `OrdenFabricacion? ObtenerPorNumero(string numeroOrden);` → declara en el contrato la búsqueda por el identificador de negocio NumeroOrden.

Línea 235: `void Agregar(OrdenFabricacion orden);` → declara la operación que incorpora una orden al repositorio para que EF Core la siga como nueva.

Línea 236: `void Eliminar(OrdenFabricacion orden);` → declara la operación que marca una orden para eliminación física cuando el escenario lo requiere.

Línea 237: `int Guardar();` → declara la operación que confirma los cambios pendientes y devuelve el número de entradas afectadas.

Línea 238: `}` → cierra el bloque de código actual.

Línea 240: `public sealed class OrdenRepositorio : IOrdenRepositorio` → declara la clase `OrdenRepositorio` que forma parte del estado acumulativo del ejercicio.

Línea 241: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 242: `private readonly AceriaDbContext _context;` → declara el campo privado `_context` de tipo `AceriaDbContext` que conserva una dependencia de la clase.

Línea 243: `public OrdenRepositorio(AceriaDbContext context) => _context = context;` → declara el constructor o método `OrdenRepositorio` con los parámetros necesarios para este componente.

Línea 245: `public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();` → ordena el resultado de la consulta antes de materializarlo.

Línea 246: `public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);` → implementa la búsqueda por clave primaria mediante `Find`, aprovechando primero el `ChangeTracker` y después la base de datos.

Línea 247: `public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);` → recupera la primera coincidencia o `null` cuando no existe.

Línea 248: `public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 249: `public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);` → marca la entidad indicada para borrado al confirmar los cambios.

Línea 250: `public int Guardar() => _context.SaveChanges();` → persiste en SQL Server los cambios seguidos por el `DbContext`.

Línea 251: `}` → cierra el bloque de código actual.

Línea 253: `public interface IServicioOrdenes` → declara el contrato `IServicioOrdenes` para desacoplar consumidores e implementación.

Línea 254: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 255: `string ObtenerResumen(int ordenId);` → declara el servicio que construye un resumen de una orden a partir de su identificador.

Línea 256: `}` → cierra el bloque de código actual.

Línea 258: `public sealed class ServicioOrdenes : IServicioOrdenes` → declara la clase `ServicioOrdenes` que forma parte del estado acumulativo del ejercicio.

Línea 259: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 260: `private readonly IOrdenRepositorio _repositorio;` → declara el campo privado `_repositorio` de tipo `IOrdenRepositorio` que conserva una dependencia de la clase.

Línea 261: `private readonly AceriaDbContext _context;` → declara el campo privado `_context` de tipo `AceriaDbContext` que conserva una dependencia de la clase.

Línea 262: `public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)` → declara el constructor o método `ServicioOrdenes` con los parámetros necesarios para este componente.

Línea 263: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 264: `_repositorio = repositorio;` → guarda en el servicio el repositorio inyectado para reutilizarlo en sus operaciones.

Línea 265: `_context = context;` → guarda el DbContext inyectado para consultar los datos necesarios durante el servicio.

Línea 266: `}` → cierra el bloque de código actual.

Línea 268: `public string ObtenerResumen(int ordenId)` → declara el método que obtiene la orden y compone el resumen solicitado por el servicio.

Línea 269: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 270: `var orden = _repositorio.ObtenerPorId(ordenId);` → consulta la orden mediante el repositorio y conserva el resultado anulable en una variable local.

Línea 271: `if (orden is null) return $"Orden {ordenId} no encontrada";` → evalúa la condición y ejecuta el bloque asociado sólo cuando se cumple.

Línea 272: `var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);` → cuenta las filas que cumplen la consulta construida.

Línea 273: `return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";` → devuelve el resultado calculado al código que invocó el método.

Línea 274: `}` → cierra el bloque de código actual.

Línea 275: `}` → cierra el bloque de código actual.

Línea 277: `public static class Program` → declara la clase Program que forma parte del estado acumulativo del ejercicio.

Línea 278: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 279: `public static void Main()` → declara el punto de entrada ejecutable de la aplicación de consola.

Línea 280: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 281: `var configuration = new ConfigurationBuilder()` → inicia la construcción de la configuración de la aplicación.

Línea 282: `.SetBasePath(AppContext.BaseDirectory)` → establece la ruta base usada para localizar los archivos de configuración.

Línea 283: `.AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)` → incorpora appsettings.json a la configuración.

Línea 284: `.AddEnvironmentVariables()` → incorpora variables de entorno como fuente adicional de configuración.

Línea 285: `.Build();` → materializa la configuración o el objeto encadenado construido en las líneas anteriores.

Línea 287: `var connectionString = configuration.GetConnectionString("AceriaDB")` → obtiene la cadena de conexión AceriaDB desde la configuración.

Línea 288: `?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");` → interrumpe el arranque con un error explícito si la configuración no contiene ConnectionStrings:AceriaDB.

Línea 290: `var services = new ServiceCollection();` → crea la colección donde se registran las dependencias de la aplicación.

Línea 291: `services.AddDbContext<AceriaDbContext>(options =>` → registra el DbContext en el contenedor de inyección de dependencias.

Línea 292: `options` → continúa la configuración del DbContext sobre el parámetro options recibido por la lambda de AddDbContext.

Línea 293: `.UseSqlServer(connectionString, sql =>` → selecciona SQL Server como proveedor de EF Core.

Línea 294: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 295: `sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);` → habilita reintentos ante fallos transitorios del proveedor SQL Server.

Línea 296: `sql.CommandTimeout(60);` → establece el tiempo máximo permitido para los comandos SQL.

Línea 297: `})` → cierra la lambda de configuración actual y devuelve el control a la llamada encadenada que la contiene.

Línea 298: `.LogTo(global::System.Console.WriteLine, new[]
{ "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)` → configura la salida del logging de EF Core.

Línea 299: `.EnableDetailedErrors();` → activa errores detallados para facilitar el diagnóstico del laboratorio.

Línea 301: `services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();` → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.

Línea 302: `services.AddScoped<IServicioOrdenes, ServicioOrdenes>();` → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.

Línea 304: `using var provider = services.BuildServiceProvider(new ServiceProviderOptions` → construye el ServiceProvider con validación de servicios y garantiza su liberación al salir del ámbito mediante using var.

Línea 305: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 306: `ValidateScopes = true,` → activa la validación de ámbitos para detectar dependencias scoped usadas desde ámbitos incorrectos.

Línea 307: `ValidateOnBuild = true` → obliga al contenedor a validar las resoluciones registradas al construir el ServiceProvider.

Línea 308: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 310: `using var scope = provider.CreateScope();` → crea un ámbito de DI para resolver servicios scoped y lo libera automáticamente al terminar el bloque.

Línea 311: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → resuelve un servicio obligatorio desde el contenedor de dependencias.

Línea 312: `context.Database.EnsureDeleted();` → elimina la base del laboratorio para reproducir el escenario desde un estado limpio.

Línea 313: `context.Database.Migrate();` → aplica las migraciones pendientes y deja AceriaDB en el esquema del punto.

Línea 315: `var orden = new OrdenFabricacion` → crea la entidad OrdenFabricacion que concentra el grafo de datos del escenario acumulativo.

Línea 316: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 317: `NumeroOrden = "OF-M2-0001",` → asigna el identificador de negocio usado por la orden de validación acumulativa del módulo.

Línea 318: `Cliente = "Constructora del Norte",` → asigna el cliente de la orden utilizada en el escenario práctico del punto.

Línea 319: `FechaCreacion = DateTime.UtcNow,` → asigna a la orden la fecha de creación en UTC para el escenario reproducible del punto.

Línea 320: `FechaEntrega = DateTime.Today.AddDays(14)`, → establece una fecha de entrega prevista catorce días después de la fecha local actual.

Línea 321: `Estado = "Pendiente"`, → inicializa la orden con el estado Pendiente, coherente con el flujo del laboratorio.

Línea 322: `Observaciones = "Orden de validación M2"`, → añade texto opcional para comprobar el mapeo de Observaciones.

Línea 323: `Planchas =` → inicia el inicializador de la colección de planchas asociadas a la orden.

Línea 324: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 325: `new PlanchaAcero` → crea una nueva plancha dentro del inicializador de la colección Planchas de la orden.

Línea 326: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 327: `Espesor = 10.5`, → asigna a la plancha un espesor positivo para satisfacer las reglas del modelo.

Línea 328: `Ancho = 1500`, → asigna el ancho de la plancha usada en el escenario de persistencia.

Línea 329: `Largo = 3000`, → asigna el largo de la plancha usada en el escenario de persistencia.

Línea 330: `Peso = 371.250m`, → asigna un peso decimal con tres posiciones para validar la precisión configurada.

Línea 331: `Activa = true`, → marca la plancha como activa para que sea visible en los estados que aplican ese criterio.

Línea 332: `}` → cierra el bloque de código actual.

Línea 333: `},` → continúa el inicializador o la llamada iniciada en el bloque actual.

Línea 334: `Detalle = new DetalleOrden` → crea el detalle uno-a-uno que se persistirá junto con la orden principal.

Línea 335: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 336: `ComposicionQuimica = "C: 0.45%, Mn: 0.75%"`, → asigna la composición química utilizada para verificar la entidad DetalleOrden.

Línea 337: `TemperaturaColada = 1550.5`, → asigna la temperatura de colada del detalle de fabricación.

Línea 338: `Notas = "Colada principal"` → asigna las notas opcionales del detalle asociado a la orden.

Línea 339: `},` → continúa el inicializador o la llamada iniciada en el bloque actual.

Línea 340: `Certificado = new CertificadoCalidad` → crea el certificado uno-a-uno asociado a la orden del escenario.

Línea 341: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 342: `NumeroCertificado = "CERT-0001"`, → asigna el identificador natural del certificado de calidad.

Línea 343: `FechaEmision = DateTime.Today`, → asigna la fecha de emisión del certificado usado en la prueba.

Línea 344: `OrganismoCertificador = "Laboratorio Aceria"` → asigna el organismo que emite el certificado de calidad.

Línea 345: `},` → continúa el inicializador o la llamada iniciada en el bloque actual.

Línea 346: `};` → cierra el inicializador del objeto y finaliza la instrucción.

Línea 348: `var aleacion = new Aleacion` → crea la aleación que se enlazará con la orden mediante la entidad intermedia OrdenAleacion.

Línea 349: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 350: `Nombre = "AISI 1045"`, → asigna el nombre comercial de la aleación utilizada en el escenario.

Línea 351: `Codigo = "A1045"`, → asigna el código natural único de la aleación.

Línea 352: `PorcentajeCarbono = 0.45`, → asigna el porcentaje de carbono dentro del rango permitido por la restricción del modelo.

Línea 353: `PorcentajeManganeso = 0.75`, → asigna el porcentaje de manganeso dentro del rango permitido por la restricción del modelo.

Línea 354: `Descripcion = "Acero medio en carbono"` → asigna una descripción opcional a la aleación.

Línea 355: `};` → cierra el inicializador del objeto y finaliza la instrucción.

Línea 356: `orden.Organizaciones.Add(new OrdenOrganizacion` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 357: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 358: `Aleacion = aleacion`, → conecta la entidad intermedia `OrdenAleacion` con la aleación creada para el escenario.

Línea 359: `CantidadUtilizada = 1500.500m`, → registra la cantidad utilizada con tres decimales, coherente con `HasPrecision(18, 3)`.

Línea 360: `EstadoRelacion = "Activa"` → marca como activa la relación entre la orden y la aleación.

Línea 361: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 362: `context.OrganizacionesFabricacion.Add(orden);` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 363: `context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo = true });` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 364: `context.SaveChanges();` → persiste en SQL Server los cambios seguidos por el `DbContext`.

Línea 366: `var entity = context.Model.FindEntityType(typeof(OrdenFabricacion));` → obtiene los metadatos EF Core de `OrdenFabricacion` para inspeccionar claves, índices o filtros del modelo construido.

Línea 367: `global::System.Console.WriteLine($"2.8 OK | Alternate keys: {entity.GetKeys().Count()} | PK: {string.Join(", ", entity.FindPrimaryKey().Properties.Select(x => x.Name))}");` → muestra la evidencia de ejecución de 2.8, el número de claves del modelo y las propiedades de la clave primaria.

Línea 368: `}` → cierra el bloque de código actual.

Línea 369: `}` → cierra el bloque de código actual.

Punto 2.9 - Índices y restricciones

Ejercicio guiado: diseñar índices simples, compuestos, filtrados y con columnas incluidas para los patrones de consulta del proyecto AceriaData, y añadir restricciones CHECK que protejan reglas de integridad directamente en SQL Server.

Contexto del proyecto: 2.9 parte íntegramente de 2.8. Las claves primarias y alternativas ya están resueltas y no se duplican con índices equivalentes. En este punto se añaden estructuras de acceso y restricciones de integridad: índices sobre OrdenFabricacion, PlanchaAcero, Aleacion, CertificadoCalidad y OrdenAleacion, además de CHECK para dimensiones/peso de planchas y porcentajes de aleaciones. Los defaults existentes proceden de 2.7 y las alternate keys de 2.8.

Objetivos de aprendizaje

- Diferenciar una clave candidata de un índice creado para acelerar consultas.
- Crear índices simples con `HasIndex`.
- Crear índices compuestos y razonar sobre el orden de sus columnas.
- Crear índices filtrados con `HasFilter`.
- Crear índices con columnas incluidas mediante `IncludeProperties`.
- Evitar índices redundantes cuando otro índice comienza por la misma columna.
- Comprender por qué la PK compuesta no sustituye un índice que empieza por `AleacionId`.
- Configurar CHECK constraints mediante `HasCheckConstraint`.
- Interpretar la migración real `M2_2_9`.
- Verificar índices, filtros, columnas incluidas y CHECK en SQL Server LocalDB.
- Probar que una restricción CHECK rechaza datos inválidos.
- Entender que un índice puede ayudar a una consulta sin garantizar que SQL Server vaya a elegirlo.

Paso 1: Abrir el estado autónomo de 2.9

```
cd M02/PROYECTO/2.9
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

El estado 2.9 contiene todo lo construido en 2.8. Si reproduces la evolución manualmente, parte del checkpoint anterior y añade únicamente la configuración de este punto.

Paso 2: Separar identidad, acceso e integridad

Antes de escribir índices conviene distinguir tres conceptos:

- una **PK o alternate key** identifica una fila o una clave candidata;
- un **índice** es una estructura de acceso que SQL Server puede usar para localizar datos;
- un **CHECK** impide almacenar valores que violan una regla.

Por eso no se crea en 2.9 otro índice único sobre `NumeroOrden`, `Codigo` o `NumeroCertificado`: esas propiedades ya son alternate keys desde 2.8.

Paso 3: Configurar los cuatro índices de OrdenFabricacion

```
entity.HasIndex(o => o.Cliente)
    .HasDatabaseName("IX_OrdenesFabricacion_Cliente");

entity.HasIndex(o => new { o.Cliente, o.FechaCreacion })
    .HasDatabaseName(
        "IX_OrdenesFabricacion_Cliente_FechaCreacion");

entity.HasIndex(o => o.FechaEntrega)
    .HasFilter("[Estado] = 'Pendiente'")
    .HasDatabaseName(
        "IX_OrdenesFabricacion_FechaEntrega_Pendientes");

entity.HasIndex(o => o.Estado)
    .IncludeProperties(o => new
    {
        o.NumeroOrden,
        o.Cliente,
        o.FechaCreacion
    })
    .HasDatabaseName(
        "IX_OrdenesFabricacion_Estado_Incluye");
```

Cada índice responde a un patrón distinto:

- `Cliente` permite búsquedas directas por cliente.
- `Cliente + FechaCreacion` sirve para filtrar por cliente y ordenar o acotar por fecha.
- `FechaEntrega` filtrado contiene sólo filas cuyo `Estado` es `Pendiente`.
- `Estado` con columnas incluidas puede cubrir consultas que necesitan devolver `NumeroOrden`, `Cliente` y `FechaCreacion` sin convertir esas columnas en parte de la clave del índice.

Paso 4: Comprender el orden del índice compuesto

En:

```
entity.HasIndex(o => new
{
    o.Cliente,
    o.FechaCreacion
});
```

`Cliente` es la primera columna. El índice puede ser útil para consultas que comienzan filtrando por `Cliente` y, después, usan `FechaCreacion`. No debe interpretarse como dos índices independientes.

El índice simple sobre `Cliente` se conserva deliberadamente en el modelo final porque forma parte del diseño del punto; al evaluar redundancia en un sistema real habría que contrastar cargas, planes y mantenimiento antes de eliminar uno.

Paso 5: Sustituir el índice simple de la FK de PlanchaAcero

La relación 1:N había generado un índice simple sobre `OrdenId`. 2.9 configura:

```
entity.HasIndex(x => new
{
    x.OrdenId,
    x.Activa
})
    .HasDatabaseName(
        "IX_PlanchasAcero_OrdenId_Activa");
```

La migración real elimina primero:

```
migrationBuilder.DropIndex(
    name: "IX_PlanchasAcero_OrdenId",
    table: "PlanchasAcero");
```

y crea el compuesto `OrdenId + Activa`. Como `OrdenId` es la primera columna, el compuesto también puede servir a búsquedas que comienzan por esa FK y evita mantener ambos índices en este diseño.

Paso 6: Añadir un índice filtrado sobre Espesor

```
entity.HasIndex(x => x.Espesor)
    .HasFilter("[Activa] = 1")
    .HasDatabaseName(
        "IX_PlanchasAcero_Espesor_Activas");
```

El índice sólo contiene planchas activas. Para que SQL Server pueda aprovecharlo, la consulta debe ser compatible con el predicado del filtro; crear un índice filtrado no obliga al optimizador a usarlo.

Paso 7: Proteger las dimensiones de PlanchaAcero con CHECK

```
entity.ToTable("PlanchasAcero", t =>
{
    t.HasCheckConstraint(
        "CK_PlanchasAcero_Espesor",
        "[Espesor] > 0");

    t.HasCheckConstraint(
        "CK_PlanchasAcero_Ancho",
        "[Ancho] > 0");

    t.HasCheckConstraint(
        "CK_PlanchasAcero_Largo",
        "[Largo] > 0");

    t.HasCheckConstraint(
        "CK_PlanchasAcero_Peso",
        "[Peso] > 0");
});
```

Estas reglas se ejecutan en SQL Server. Aunque una aplicación valide previamente los valores, la base mantiene su propia barrera de integridad.

Paso 8: Configurar índices y CHECK de Aleacion

```
entity.HasIndex(a => a.Nombre)
    .HasDatabaseName("IX_Aleaciones_Nombre");

entity.HasIndex(a => new
{
    a.PorcentajeCarbono,
    a.PorcentajeManganeso
})
    .HasDatabaseName("IX_Aleaciones_Porcentajes");
```

Y sobre la tabla:

```
entity.ToTable("Aleaciones", t =>
{
    t.HasCheckConstraint(
        "CK_Aleaciones_PorcentajeCarbono",
        "[PorcentajeCarbono] >= 0 AND " +
        "[PorcentajeCarbono] <= 2");

    t.HasCheckConstraint(
        "CK_Aleaciones_PorcentajeManganeso",
        "[PorcentajeManganeso] >= 0 AND " +
        "[PorcentajeManganeso] <= 5");
});
```

Código no recibe otro índice: sigue protegido por la alternate key configurada en 2.8.

Paso 9: Mantener las relaciones 1:1 sin duplicar índices manuales

`DetalleOrden.OrdenId` y `CertificadoCalidad.OrdenId` pertenecen a relaciones 1:1 configuradas con `WithOne` y `HasForeignKey`. EF Core ya genera la unicidad necesaria para esas relaciones.

En 2.9 no se vuelve a declarar manualmente otro índice único sobre esas FK. Para `CertificadoCalidad` el cambio nuevo es:

```
entity.HasIndex(c => c.FechaEmision)
    .HasDatabaseName(
        "IX_CertificadosCalidad_FechaEmision");
```

Así se mantiene separada la unicidad estructural de la relación del índice adicional destinado a consultas por fecha.

Paso 10: Configurar los índices de OrdenAleacion

```
entity.HasIndex(x => x.AleacionId)
    .HasDatabaseName(
        "IX_OrdenesAleaciones_AleacionId");

entity.HasIndex(x => x.EstadoRelacion)
    .HasFilter("[EstadoRelacion] = 'Activa'")
    .HasDatabaseName(
        "IX_OrdenesAleaciones_EstadoRelacion_Activas");
```

La PK compuesta empieza por `OrdenFabricacionId`. Por eso no sustituye un índice cuyo primer criterio sea `AleacionId`. El segundo índice reduce el conjunto físico a relaciones cuyo estado es Activa.

Paso 11: Compilar antes de generar la migración

```
dotnet build AceriaData.sln --configuration Release
```

La compilación debe ser correcta antes de comparar el modelo 2.9 con el snapshot de 2.8.

Paso 12: Generar M2_2_9 al reproducir la evolución

Si partes manualmente de 2.8:

```
dotnet ef migrations add M2_2_9
```

El checkpoint final ya contiene:

```
Migrations/20260927204824_M2_2_9.cs
```

No vuelvas a generar esa migración dentro del estado terminado.

Paso 13: Revisar la migración real

M2_2_9 no es una migración genérica. Entre sus operaciones reales aparecen:

```
migrationBuilder.DropIndex(
    name: "IX_PlanchasAcero_OrdenId",
    table: "PlanchasAcero");

migrationBuilder.CreateIndex(
    name: "IX_PlanchasAcero_OrdenId_Activa",
    table: "PlanchasAcero",
    columns: new[] { "OrdenId", "Activa" });

migrationBuilder.CreateIndex(
    name: "IX_OrdenesFabricacion_Estado_Incluye",
    table: "OrdenesFabricacion",
    column: "Estado")
.Annotation(
    "SqlServer:Include",
    new[]
    {
        "NumeroOrden",
        "Cliente",
        "FechaCreacion"
    });

migrationBuilder.CreateIndex(
    name:
        "IX_OrdenesFabricacion_FechaEntrega_Pendientes",
    table: "OrdenesFabricacion",
    column: "FechaEntrega",
    filter: "[Estado] = 'Pendiente'");

migrationBuilder.AddCheckConstraint(
    name: "CK_PlanchasAcero_Espesor",
    table: "PlanchasAcero",
    sql: "[Espesor] > 0");
```

La migración también crea el resto de índices descritos y los CHECK de PlanchaAcero/Aleacion. Su método `Down` revierte esas operaciones y restaura el índice simple `IX_PlanchasAcero_OrdenId`.

Paso 14: Aplicar la migración y comprobar el historial

```
dotnet ef database update
dotnet ef migrations list
```

La lista debe mostrar `M2_2_9` después de `M2_2_8`.

Paso 15: Verificar índices y CHECK en SQL Server

En Visual Studio abre **Ver → Explorador de objetos de SQL Server** y navega a `(localdb)\MSSQLLocalDB → Bases de datos → AceriaDB → Tablas`.

En `OrdenesFabricacion → Índices` comprueba:

- `IX_OrdenesFabricacion_Cliente`;
- `IX_OrdenesFabricacion_Cliente_FechaCreacion`;
- `IX_OrdenesFabricacion_FechaEntrega_Pendientes`;
- `IX_OrdenesFabricacion_Estado_Incluye`.

Repita la inspección en `PlanchasAcero`, `Aleaciones`, `CertificadosCalidad` y `OrdenesAleaciones`. En las propiedades de las tablas verifica también los CHECK generados por `M2_2_9`.

Paso 16: Verificar el filtro y las columnas incluidas

En las propiedades de `IX_OrdenesFabricacion_FechaEntrega_Pendientes` el filtro debe ser:

```
[Estado] = 'Pendiente'
```

En `IX_OrdenesFabricacion_Estado_Incluye` deben figurar como columnas incluidas:

```
NumeroOrden
Cliente
FechaCreacion
```

Las columnas incluidas no cambian la clave lógica del índice; amplían la información que SQL Server puede obtener de él sin volver necesariamente a la tabla base.

Paso 17: Ejecutar e inspeccionar los índices del modelo

```
dotnet run --project AceriaData.Console.csproj --configuration Release
```

El checkpoint consulta el metadata de `OrdenFabricacion`:

```
var entity =
    context.Model.FindEntityType(
        typeof(OrdenFabricacion));

Console.WriteLine(
    $"2.9 OK | Índices: " +
    $"{string.Join(", ", " +
    "entity.GetIndexes()" +
    ".Select(i => i.GetDatabaseName()))}");
```

La salida enumera los índices que EF Core conoce para esa entidad. Esta inspección complementa, pero no sustituye, la verificación del esquema real en SQL Server.

Paso 18: Resolver el reto de Espesor negativo

Reto: demostrar que `CK_PlanchasAcero_Espesor` protege la base incluso si se intenta guardar un valor inválido.

El checkpoint contiene el bloque comentado `RETO 2.9 - ESPESOR NEGATIVO`:

```
context.PlanchasAcero.Add(
    new PlanchaAcero
    {
        OrdenId = orden.Id,
        Espesor = -5.0,
        Ancho = 1000,
        Largo = 2000,
        Peso = 100.000m,
        Activa = true
    });
```

La operación de guardado se prueba de forma controlada:

```
try
{
    context.SaveChanges();

    Console.WriteLine(
        "Reto 2.9 ERROR: SQL Server " +
        "permitió Espesor negativo.");
}
catch (DbUpdateException)
{
    Console.WriteLine(
        "Reto 2.9: Espesor negativo rechazado");
}
```

Resultado esperado:

Reto 2.9: Espesor negativo rechazado

La base ya está preparada mediante Migrations. No se usa `EnsureCreated()` para sustituir el esquema acumulativo.

Paso 19: Observar planes de ejecución sin convertirlos en una promesa

Para estudiar el efecto de los índices puedes ejecutar consultas representativas desde las herramientas de SQL Server y activar el **plan de ejecución real**.

Ejemplos de patrones a observar:

```
SELECT Cliente, FechaCreacion
FROM OrdenesFabricacion
WHERE Cliente = @cliente
ORDER BY FechaCreacion;

SELECT NumeroOrden, Cliente, FechaCreacion
FROM OrdenesFabricacion
WHERE Estado = 'Pendiente';
```

Comprueba qué operadores e índices elige SQL Server. El optimizador decide según estadísticas, cardinalidad, selectividad y coste; la presencia de un índice no garantiza que se utilice.

Paso 20: Delimitar el alcance antes de pasar a 2.10

En 2.9 no se:

- duplican índices sobre las alternate keys de 2.8;
- cambian PK ni relaciones;
- añaden nuevos defaults que ya proceden de 2.7;
- usan índices como sustituto de una regla de identidad;
- fuerza un plan de ejecución concreto;
- usa `EnsureCreated()` para evitar Migrations.

El resultado debe ser exactamente el esquema descrito por `M2_2_9`.

Errores comunes

Error	Causa	Corrección
Duplicar un índice sobre una alternate key	Se confunde identidad con rendimiento	Mantener la alternate key y añadir sólo índices con un propósito distinto

Error	Causa	Corrección
Conservar <code>IX_PlanchasAcero_OrdenId</code> además del compuesto sin necesidad	El compuesto empieza por <code>OrdenId</code>	Seguir la migración real, que elimina el simple
Invertir columnas de un índice compuesto sin analizar consultas	Cambia el prefijo útil del índice	Diseñar el orden según filtros y ordenaciones
Escribir un filtro que no representa las filas buscadas	El índice filtrado deja de ser aplicable	Mantener el predicado coherente con las consultas
Confundir <code>IncludeProperties</code> con columnas de clave	Se interpreta mal el índice cubriente	Diferenciar key columns e included columns
Añadir CHECK sólo en C#	La base acepta datos inválidos desde otros clientes	Mantener la regla también en SQL Server
Suponer que SQL Server siempre usará el índice	El optimizador puede elegir otro plan	Verificar planes con datos y estadísticas reales
Usar <code>EnsureCreated()</code>	Se pierde la evolución del esquema	Aplicar <code>M2_2_9</code> mediante Migrations

Analogía final

Los índices son como distintos catálogos de una acería. Un catálogo por cliente permite localizar rápidamente sus órdenes; otro ordenado por cliente y fecha sirve a búsquedas más específicas; un catálogo filtrado guarda sólo las órdenes pendientes. Las columnas incluidas son información adicional que viaja en el catálogo para evitar consultar otra ficha. Los CHECK son controles de calidad a la entrada: una plancha con espesor o peso imposibles no atraviesa la puerta, venga de la aplicación que venga.

Resultado esperado

Al finalizar 2.9, `AceriaData` conserva las claves de 2.8 y añade los índices definidos por el modelo final: simples, compuestos, filtrados y con columnas incluidas. `M2_2_9` elimina el índice simple redundante de `PlanchaAcero.OrdenId`, crea el compuesto `OrdenId + Activa`, añade el resto de índices y materializa seis CHECK: cuatro para dimensiones/peso de planchas y dos para porcentajes de aleaciones. El reto confirma que SQL Server rechaza un espesor negativo.

Conexión con el siguiente punto

2.10 partirá de este esquema para trabajar con filtros globales de consulta mediante `HasQueryFilter` e `IgnoreQueryFilters`, sin rehacer los índices ni las restricciones ya consolidadas.

Código acumulativo completo del estado 2.9

El siguiente archivo es el `Program.cs` real del estado ejecutable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

[Table("OrdenesFabricacion")]
public class OrdenFabricacion
{
```

```

    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroOrden { get; set; } = string.Empty;
    [Required]
    [MaxLength(200)]
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    [Required]
    [MaxLength(50)]
    public string Estado { get; set; } = "Pendiente";
    [MaxLength(500)]
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
    public DetalleOrden? Detalle { get; set; }
    public CertificadoCalidad? Certificado { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

[Table("PlanchasAcero")]
public class PlanchaAcero
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    [Precision(18, 3)]
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("Aleaciones")]
public class Aleacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(100)]
    public string Nombre { get; set; } = string.Empty;
    [Required]
    [MaxLength(20)]
    public string Codigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    [MaxLength(500)]
    public string? Descripcion { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

[Table("EstadosOrden")]
public class EstadoOrden
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string Nombre { get; set; } = string.Empty;
    [MaxLength(250)]
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
}

[Table("DetallesOrden")]
public class DetalleOrden
{
    [Key]
    public int Id { get; set; }

```

```

    public int OrdenId { get; set; }
    [Required]
    [MaxLength(200)]
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    [MaxLength(500)]
    public string? Notas { get; set; }
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("CertificadosCalidad")]
public class CertificadoCalidad
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroCertificado { get; set; } = string.Empty;
    public DateTime FechaEmision { get; set; }
    [Required]
    [MaxLength(100)]
    public string OrganismoCertificador { get; set; } = string.Empty;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]
[Table("OrdenesAleaciones")]
public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }
    public DateTime FechaAsignacion { get; set; } = DateTime.Now;
    [Precision(18, 3)]
    public decimal CantidadUtilizada { get; set; }
    [MaxLength(20)]
    public string EstadoRelacion { get; set; } = "Activa";
    public OrdenFabricacion Orden { get; set; } = null!;
    public Aleacion Aleacion { get; set; } = null!;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
    public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();
    public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();
    public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<OrdenFabricacion>(entity =>
        {
            entity.ToTable("OrdenesFabricacion");
            entity.HasKey(o => o.Id);
            entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);
            entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);
            entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");
            entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");
            entity.Property(o => o.Observaciones).HasMaxLength(500);
            entity.HasAlternateKey(o => o.NumeroOrden).HasName("AK_OrdenesFabricacion_NumeroOrden");
            entity.HasIndex(o => o.Cliente).HasDatabaseName("IX_OrdenesFabricacion_Cliente");
            entity.HasIndex(o => new { o.Cliente,
o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Cliente_FechaCreacion");
            entity.HasIndex(o => o.FechaEntrega).HasFilter("[Estado] =

```

```

'Pendiente')).HasDatabaseName("IX_OrdenesFabricacion_FechaEntrega_Pendientes");
    entity.HasIndex(o => o.Estado).IncludeProperties(o => new { o.NumeroOrden, o.Cliente,
o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Estado_Incluye");
});
modelBuilder.Entity<PlanchaAcero>(entity =>
{
    entity.ToTable("PlanchasAcero", t =>
    {
        t.HasCheckConstraint("CK_PlanchasAcero_Espesor", "[Espesor] > 0");
        t.HasCheckConstraint("CK_PlanchasAcero_Ancho", "[Ancho] > 0");
        t.HasCheckConstraint("CK_PlanchasAcero_Largo", "[Largo] > 0");
        t.HasCheckConstraint("CK_PlanchasAcero_Peso", "[Peso] > 0");
    });
    entity.HasKey(x => x.Id);
    entity.Property(x => x.Peso).HasPrecision(18, 3);
    entity.Property(x => x.Activa).HasDefaultValue(true);
    entity.HasOne(x => x.Orden)
        .WithMany(o => o.Planchas)
        .HasForeignKey(x => x.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
    entity.HasIndex(x => new { x.OrdenId,
x.Activa }).HasDatabaseName("IX_PlanchasAcero_OrdenId_Activa");
    entity.HasIndex(x => x.Espesor).HasFilter("[Activa] =
1").HasDatabaseName("IX_PlanchasAcero_Espesor_Activas");
});
modelBuilder.Entity<Aleacion>(entity =>
{
    entity.ToTable("Aleaciones", t =>
    {
        t.HasCheckConstraint("CK_Aleaciones_PorcentajeCarbono", "[PorcentajeCarbono] >= 0 AND
[PorcentajeCarbono] <= 2");
        t.HasCheckConstraint("CK_Aleaciones_PorcentajeManganeso", "[PorcentajeManganeso] >= 0
AND [PorcentajeManganeso] <= 5");
    });
    entity.HasKey(a => a.Id);
    entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
    entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
    entity.Property(a => a.Descripcion).HasMaxLength(500);
    entity.HasAlternateKey(a => a.Codigo).HasName("AK_Aleaciones_Codigo");
    entity.HasIndex(a => a.Nombre).HasDatabaseName("IX_Aleaciones_Nombre");
    entity.HasIndex(a => new { a.PorcentajeCarbono,
a.PorcentajeManganeso }).HasDatabaseName("IX_Aleaciones_Porcentajes");
});
modelBuilder.Entity<EstadoOrden>(entity =>
{
    entity.ToTable("EstadosOrden");
    entity.HasKey(e => e.Id);
    entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
    entity.Property(e => e.Descripcion).HasMaxLength(250);
    entity.Property(e => e.Activo).HasDefaultValue(true);
});
modelBuilder.Entity<DetalleOrden>(entity =>
{
    entity.ToTable("DetallesOrden");
    entity.HasKey(d => d.Id);
    entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);
    entity.Property(d => d.Notas).HasMaxLength(500);
    entity.HasOne(d => d.Orden)
        .WithOne(o => o.Detalle)
        .HasForeignKey<DetalleOrden>(d => d.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
});

modelBuilder.Entity<CertificadoCalidad>(entity =>
{
    entity.ToTable("CertificadosCalidad");
    entity.HasKey(c => c.Id);
    entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);
    entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);
    entity.HasAlternateKey(c =>
c.NumeroCertificado).HasName("AK_CertificadosCalidad_NumeroCertificado");

```

```

        entity.HasIndex(c => c.FechaEmision).HasDatabaseName("IX_CertificadosCalidad_FechaEmision");
        entity.HasOne(c => c.Orden)
            .WithOne(o => o.Certificado)
            .HasForeignKey<CertificadoCalidad>(c => c.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });
    modelBuilder.Entity<OrdenAleacion>(entity =>
    {
        entity.ToTable("OrdenesAleaciones");
        entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });
        entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");
        entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);
        entity.Property(x =>
x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");
        entity.HasOne(x => x.Orden)
            .WithMany(o => o.OrdenesAleaciones)
            .HasForeignKey(x => x.OrdenFabricacionId)
            .OnDelete(DeleteBehavior.Cascade);
        entity.HasOne(x => x.Aleacion)
            .WithMany(a => a.OrdenesAleaciones)
            .HasForeignKey(x => x.AleacionId)
            .OnDelete(DeleteBehavior.Restrict);
        entity.HasIndex(x => x.AleacionId).HasDatabaseName("IX_OrdenesAleaciones_AleacionId");
        entity.HasIndex(x => x.EstadoRelacion).HasFilter("[EstadoRelacion] =
'Activa'").HasDatabaseName("IX_OrdenesAleaciones_EstadoRelacion_Activas");
    });
    }
}

public interface IOordenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    int Guardar();
}

public sealed class OrdenRepositorio : IOordenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o =>
o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) =>
_context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOordenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOordenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
    }
}

```

```

        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();
        services.AddDbContext<AceriaDbContext>(options =>
            options
                .UseSqlServer(connectionString, sql =>
                {
                    sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                    sql.CommandTimeout(60);
                })
                .LogTo(global::System.Console.WriteLine, new[] {
                    "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)
                .EnableDetailedErrors());

        services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
        services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

        using var provider = services.BuildServiceProvider(new ServiceProviderOptions
        {
            ValidateScopes = true,
            ValidateOnBuild = true
        });

        using var scope = provider.CreateScope();
        var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
        context.Database.EnsureDeleted();
        context.Database.Migrate();

        var orden = new OrdenFabricacion
        {
            NumeroOrden = "OF-M2-0001",
            Cliente = "Constructora del Norte",
            FechaCreacion = DateTime.UtcNow,
            FechaEntrega = DateTime.Today.AddDays(14),
            Estado = "Pendiente",
            Observaciones = "Orden de validación M2",
            Planchas =
            {
                new PlanchaAcero
                {
                    Espesor = 10.5,
                    Ancho = 1500,
                    Largo = 3000,
                    Peso = 371.250m,
                    Activa = true,
                }
            },
            Detalle = new DetalleOrden
            {
                ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
                TemperaturaColada = 1550.5,
                Notas = "Colada principal"
            },
            Certificado = new CertificadoCalidad
            {
                NumeroCertificado = "CERT-0001",
            }
        };
    }
}

```

```

        FechaEmision = DateTime.Today,
        OrganismoCertificador = "Laboratorio Aceria"
    },
};

var aleacion = new Aleacion
{
    Nombre = "AISI 1045",
   Codigo = "A1045",
    PorcentajeCarbono = 0.45,
    PorcentajeManganeso = 0.75,
    Descripcion = "Acero medio en carbono"
};
orden.OrdenesAleaciones.Add(new OrdenAleacion
{
    Aleacion = aleacion,
    CantidadUtilizada = 1500.500m,
    EstadoRelacion = "Activa"
});
context.OrdenesFabricacion.Add(orden);
context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa",
Activo = true });
context.SaveChanges();

var entity = context.Model.FindEntityType(typeof(OrdenFabricacion))!;
global::System.Console.WriteLine($"2.9 OK | Índices: {string.Join(", ",
entity.GetIndexes().Select(i => i.GetDatabaseName()))}");
}
}

```

Explicación línea a línea del código acumulativo 2.9

Las referencias usan la numeración real de `M02/PROYECTO/2.9/Program.cs`. Se explican todas las líneas no vacías del archivo que compila.

Línea 1: `using System.ComponentModel.DataAnnotations;` → importa el espacio de nombres necesario para este archivo.

Línea 2: `using System.ComponentModel.DataAnnotations.Schema;` → importa el espacio de nombres necesario para este archivo.

Línea 3: `using Microsoft.EntityFrameworkCore;` → importa el espacio de nombres necesario para este archivo.

Línea 4: `using Microsoft.Extensions.Configuration;` → importa el espacio de nombres necesario para este archivo.

Línea 5: `using Microsoft.Extensions.DependencyInjection;` → importa el espacio de nombres necesario para este archivo.

Línea 6: `using Microsoft.Extensions.Logging;` → importa el espacio de nombres necesario para este archivo.

Línea 8: `namespace AceriaData.ConsoleApp;` → declara el espacio de nombres del estado acumulativo.

Línea 10: `[Table("OrdenesFabricacion")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 11: `public class OrdenFabricacion` → declara la clase OrdenFabricacion que forma parte del estado acumulativo.

Línea 12: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 13: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 14: `public int Id { get; set; }` → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.

Línea 15: `[Required]` → marca la propiedad siguiente como requerida.

Línea 16: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 17: `public string NumeroOrden { get; set; } = string.Empty;` → declara la propiedad `NumeroOrden` de tipo `string` usada por el modelo o la lógica de negocio.

Línea 18: `[Required]` → marca la propiedad siguiente como requerida.

Línea 19: `[MaxLength(200)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 20: `public string Cliente { get; set; } = string.Empty;` → declara la propiedad `Cliente` de tipo `string` usada por el modelo o la lógica de negocio.

Línea 21: `public DateTime FechaCreacion { get; set; }` → declara la propiedad `FechaCreacion` de tipo `DateTime` usada por el modelo o la lógica de negocio.

Línea 22: `public DateTime? FechaEntrega { get; set; }` → declara la propiedad `FechaEntrega` de tipo `DateTime?` usada por el modelo o la lógica de negocio.

Línea 23: `[Required]` → marca la propiedad siguiente como requerida.

Línea 24: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 25: `public string Estado { get; set; } = "Pendiente";` → declara la propiedad `Estado` de tipo `string` usada por el modelo o la lógica de negocio.

Línea 26: `[MaxLength(500)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 27: `public string? Observaciones { get; set; }` → declara la propiedad `Observaciones` de tipo `string?` usada por el modelo o la lógica de negocio.

Línea 28: `public List<PlanchaAcero> Planchas { get; set; } = new();` → declara la propiedad `Planchas` de tipo `List` usada por el modelo o la lógica de negocio.

Línea 29: `public DetalleOrden? Detalle { get; set; }` → declara la propiedad `Detalle` de tipo `DetalleOrden?` usada por el modelo o la lógica de negocio.

Línea 30: `public CertificadoCalidad? Certificado { get; set; }` → declara la propiedad `Certificado` de tipo `CertificadoCalidad?` usada por el modelo o la lógica de negocio.

Línea 31: `public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();` → declara la propiedad `OrdenesAleaciones` de tipo `List` usada por el modelo o la lógica de negocio.

Línea 32: `}` → cierra el bloque de código actual.

Línea 34: `[Table("PlanchasAcero")]` → fija mediante `Data Annotations` la tabla relacional de la entidad siguiente.

Línea 35: `public class PlanchaAcero` → declara la clase `PlanchaAcero` que forma parte del estado acumulativo.

Línea 36: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 37: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 38: `public int Id { get; set; }` → declara la propiedad `Id` de tipo `int` usada por el modelo o la lógica de negocio.

Línea 39: `public int OrdenId { get; set; }` → declara la propiedad `OrdenId` de tipo `int` usada por el modelo o la lógica de negocio.

Línea 40: `public double Espesor { get; set; }` → declara la propiedad `Espesor` de tipo `double` usada por el modelo o la lógica de negocio.

Línea 41: `public double Ancho { get; set; }` → declara la propiedad `Ancho` de tipo `double` usada por el modelo o la lógica de negocio.

Línea 42: `public double Largo { get; set; }` → declara la propiedad `Largo` de tipo `double` usada por el modelo o la lógica de negocio.

Línea 43: `[Precision(18, 3)]` → configura precisión y escala para la propiedad numérica siguiente.

Línea 44: `public decimal Peso { get; set; }` → declara la propiedad Peso de tipo decimal usada por el modelo o la lógica de negocio.

Línea 45: `public bool Activa { get; set; } = true;` → declara la propiedad Activa de tipo bool usada por el modelo o la lógica de negocio.

Línea 46: `[ForeignKey(nameof(OrdenId))]` → vincula la navegación siguiente con su clave foránea.

Línea 47: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo OrdenFabricacion usada por el modelo o la lógica de negocio.

Línea 48: `}` → cierra el bloque de código actual.

Línea 50: `[Table("Aleaciones")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 51: `public class Aleacion` → declara la clase Aleacion que forma parte del estado acumulativo.

Línea 52: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 53: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 54: `public int Id { get; set; }` → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.

Línea 55: `[Required]` → marca la propiedad siguiente como requerida.

Línea 56: `[MaxLength(100)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 57: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad Nombre de tipo string usada por el modelo o la lógica de negocio.

Línea 58: `[Required]` → marca la propiedad siguiente como requerida.

Línea 59: `[MaxLength(20)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 60: `public string Codigo { get; set; } = string.Empty;` → declara la propiedad Codigo de tipo string usada por el modelo o la lógica de negocio.

Línea 61: `public double PorcentajeCarbono { get; set; }` → declara la propiedad PorcentajeCarbono de tipo double usada por el modelo o la lógica de negocio.

Línea 62: `public double PorcentajeManganeso { get; set; }` → declara la propiedad PorcentajeManganeso de tipo double usada por el modelo o la lógica de negocio.

Línea 63: `[MaxLength(500)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 64: `public string? Descripcion { get; set; }` → declara la propiedad Descripcion de tipo string? usada por el modelo o la lógica de negocio.

Línea 65: `public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();` → declara la propiedad OrdenesAleaciones de tipo List usada por el modelo o la lógica de negocio.

Línea 66: `}` → cierra el bloque de código actual.

Línea 68: `[Table("EstadosOrden")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 69: `public class EstadoOrden` → declara la clase EstadoOrden que forma parte del estado acumulativo.

Línea 70: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 71: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 72: `public int Id { get; set; }` → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.

Línea 73: `[Required]` → marca la propiedad siguiente como requerida.

Línea 74: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 75: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad Nombre de tipo string usada por el modelo o la lógica de negocio.

Línea 76: `[MaxLength(250)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 77: `public string Descripcion { get; set; } = string.Empty;` → declara la propiedad Descripcion de tipo string usada por el modelo o la lógica de negocio.

Línea 78: `public bool Activo { get; set; } = true;` → declara la propiedad Activo de tipo bool usada por el modelo o la lógica de negocio.

Línea 79: `}` → cierra el bloque de código actual.

Línea 81: `[Table("DetallesOrden")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 82: `public class DetalleOrden` → declara la clase DetalleOrden que forma parte del estado acumulativo.

Línea 83: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 84: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 85: `public int Id { get; set; }` → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.

Línea 86: `public int OrdenId { get; set; }` → declara la propiedad OrdenId de tipo int usada por el modelo o la lógica de negocio.

Línea 87: `[Required]` → marca la propiedad siguiente como requerida.

Línea 88: `[MaxLength(200)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 89: `public string ComposicionQuimica { get; set; } = string.Empty;` → declara la propiedad ComposicionQuimica de tipo string usada por el modelo o la lógica de negocio.

Línea 90: `public double TemperaturaColada { get; set; }` → declara la propiedad TemperaturaColada de tipo double usada por el modelo o la lógica de negocio.

Línea 91: `[MaxLength(500)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 92: `public string? Notas { get; set; }` → declara la propiedad Notas de tipo string? usada por el modelo o la lógica de negocio.

Línea 93: `[ForeignKey(nameof(OrdenId))]` → vincula la navegación siguiente con su clave foránea.

Línea 94: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo OrdenFabricacion usada por el modelo o la lógica de negocio.

Línea 95: `}` → cierra el bloque de código actual.

Línea 97: `[Table("CertificadosCalidad")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 98: `public class CertificadoCalidad` → declara la clase CertificadoCalidad que forma parte del estado acumulativo.

Línea 99: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 100: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 101: `public int Id { get; set; }` → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.

Línea 102: `public int OrdenId { get; set; }` → declara la propiedad `OrdenId` de tipo `int` usada por el modelo o la lógica de negocio.

Línea 103: `[Required]` → marca la propiedad siguiente como requerida.

Línea 104: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 105: `public string NumeroCertificado { get; set; } = string.Empty;` → declara la propiedad `NumeroCertificado` de tipo `string` usada por el modelo o la lógica de negocio.

Línea 106: `public DateTime FechaEmision { get; set; }` → declara la propiedad `FechaEmision` de tipo `DateTime` usada por el modelo o la lógica de negocio.

Línea 107: `[Required]` → marca la propiedad siguiente como requerida.

Línea 108: `[MaxLength(100)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 109: `public string OrganismoCertificador { get; set; } = string.Empty;` → declara la propiedad `OrganismoCertificador` de tipo `string` usada por el modelo o la lógica de negocio.

Línea 110: `[ForeignKey(nameof(OrdenId))]` → vincula la navegación siguiente con su clave foránea.

Línea 111: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad `Orden` de tipo `OrdenFabricacion` usada por el modelo o la lógica de negocio.

Línea 112: `}` → cierra el bloque de código actual.

Línea 114: `[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]` → declara una clave primaria compuesta.

Línea 115: `[Table("OrdenesAleaciones")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 116: `public class OrdenAleacion` → declara la clase `OrdenAleacion` que forma parte del estado acumulativo.

Línea 117: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 118: `public int OrdenFabricacionId { get; set; }` → declara la propiedad `OrdenFabricacionId` de tipo `int` usada por el modelo o la lógica de negocio.

Línea 119: `public int AleacionId { get; set; }` → declara la propiedad `AleacionId` de tipo `int` usada por el modelo o la lógica de negocio.

Línea 120: `public DateTime FechaAsignacion { get; set; } = DateTime.Now;` → declara la propiedad `FechaAsignacion` de tipo `DateTime` usada por el modelo o la lógica de negocio.

Línea 121: `[Precision(18, 3)]` → configura precisión y escala para la propiedad numérica siguiente.

Línea 122: `public decimal CantidadUtilizada { get; set; }` → declara la propiedad `CantidadUtilizada` de tipo `decimal` usada por el modelo o la lógica de negocio.

Línea 123: `[MaxLength(20)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 124: `public string EstadoRelacion { get; set; } = "Activa";` → declara la propiedad `EstadoRelacion` de tipo `string` usada por el modelo o la lógica de negocio.

Línea 125: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad `Orden` de tipo `OrdenFabricacion` usada por el modelo o la lógica de negocio.

Línea 126: `public Aleacion Aleacion { get; set; } = null!;` → declara la propiedad `Aleacion` de tipo `Aleacion` usada por el modelo o la lógica de negocio.

Línea 127: `}` → cierra el bloque de código actual.

Línea 129: `public class AceriaDbContext : DbContext` → declara la clase `AceriaDbContext` que forma parte del estado acumulativo.

Línea 130: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 131:

`public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }` → recibe las opciones del DbContext y las pasa a la clase base.

Línea 133: `public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 134: `public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 135: `public DbSet<Aleacion> Aleaciones => Set<Aleacion>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 136: `public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 137: `public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 138: `public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 139: `public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 142: `protected override void OnModelCreating(ModelBuilder modelBuilder)` → entra en la configuración explícita del modelo.

Línea 143: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 144: `base.OnModelCreating(modelBuilder);` → entra en la configuración explícita del modelo.

Línea 145: `modelBuilder.Entity<OrdenFabricacion>(entity =>` → selecciona una entidad para configurarla con Fluent API.

Línea 146: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 147: `entity.ToTable("OrdenesFabricacion");` → asigna explícitamente la tabla usada por la entidad.

Línea 148: `entity.HasKey(o => o.Id);` → define la clave primaria.

Línea 149: `entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 150: `entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);` → selecciona una propiedad escalar para continuar su configuración.

Línea 151: `entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");` → selecciona una propiedad escalar para continuar su configuración.

Línea 152: `entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");` → selecciona una propiedad escalar para continuar su configuración.

Línea 153: `entity.Property(o => o.Observaciones).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 154: `entity.HasAlternateKey(o => o.NumeroOrden).HasName("AK_OrdenesFabricacion_NumeroOrden");` → define una clave alternativa única.

Línea 155: `entity.HasIndex(o => o.Cliente).HasDatabaseName("IX_OrdenesFabricacion_Cliente");` → define un índice sobre las propiedades indicadas.

Línea 156: `entity.HasIndex(o => new { o.Cliente, o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Cliente_FechaCreacion");` → define un índice sobre las propiedades indicadas.

Línea 157: `entity.HasIndex(o => o.FechaEntrega).HasFilter("[Estado] = 'Pendiente']").HasDatabaseName("IX_OrdenesFabricacion_FechaEntrega_Pendientes");` → define un índice sobre las propiedades indicadas.

Línea 158: `entity.HasIndex(o => o.Estado).IncludeProperties(o => new { o.NumeroOrden, o.Cliente, o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Estado_Incluye");` → define un índice sobre las propiedades indicadas.

Línea 159: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 160: `modelBuilder.Entity<PlanchaAcero>(entity =>` → selecciona una entidad para configurarla con Fluent API.

Línea 161: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 162: `entity.ToTable("PlanchasAcero", t =>` → asigna explícitamente la tabla usada por la entidad.

Línea 163: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 164: `t.HasCheckConstraint("CK_PlanchasAcero_Espesor", "[Espesor] > 0");` → añade una restricción CHECK en SQL Server.

Línea 165: `t.HasCheckConstraint("CK_PlanchasAcero_Ancho", "[Ancho] > 0");` → añade una restricción CHECK en SQL Server.

Línea 166: `t.HasCheckConstraint("CK_PlanchasAcero_Largo", "[Largo] > 0");` → añade una restricción CHECK en SQL Server.

Línea 167: `t.HasCheckConstraint("CK_PlanchasAcero_Peso", "[Peso] > 0");` → añade una restricción CHECK en SQL Server.

Línea 168: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 169: `entity.HasKey(x => x.Id);` → define la clave primaria.

Línea 170: `entity.Property(x => x.Peso).HasPrecision(18, 3);` → selecciona una propiedad escalar para continuar su configuración.

Línea 171: `entity.Property(x => x.Activa).HasDefaultValue(true);` → selecciona una propiedad escalar para continuar su configuración.

Línea 172: `entity.HasOne(x => x.Orden)` → inicia la configuración de una navegación de referencia.

Línea 173: `.WithMany(o => o.Planchas)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 174: `.HasForeignKey(x => x.OrdenId)` → declara explícitamente la clave foránea.

Línea 175: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 176: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 177: `entity.HasIndex(x => new { x.OrdenId, x.Activa }).HasDatabaseName("IX_PlanchasAcero_OrdenId_Activa");` → define un índice sobre las propiedades indicadas.

Línea 178: `entity.HasIndex(x => x.Espesor).HasFilter("[Activa] = 1").HasDatabaseName("IX_PlanchasAcero_Espesor_Activas");` → define un índice sobre las propiedades indicadas.

Línea 179: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 180: `modelBuilder.Entity<Aleacion>(entity => → selecciona una entidad para configurarla con Fluent API.`

Línea 181: `{ → abre el bloque de código asociado a la declaración o instrucción anterior.`

Línea 182: `entity.ToTable("Aleaciones", t => → asigna explícitamente la tabla usada por la entidad.`

Línea 183: `{ → abre el bloque de código asociado a la declaración o instrucción anterior.`

Línea 184: `t.HasCheckConstraint("CK_Aleaciones_PorcentajeCarbono", "[PorcentajeCarbono] >= 0 AND [PorcentajeCarbono] <= 2"); → añade una restricción CHECK en SQL Server.`

Línea 185: `t.HasCheckConstraint("CK_Aleaciones_PorcentajeManganeso", "[PorcentajeManganeso] >= 0 AND [PorcentajeManganeso] <= 5"); → añade una restricción CHECK en SQL Server.`

Línea 186: `}); → cierra la lambda o configuración encadenada y finaliza la llamada.`

Línea 187: `entity.HasKey(a => a.Id); → define la clave primaria.`

Línea 188: `entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100); → selecciona una propiedad escalar para continuar su configuración.`

Línea 189: `entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20); → selecciona una propiedad escalar para continuar su configuración.`

Línea 190: `entity.Property(a => a.Descripcion).HasMaxLength(500); → selecciona una propiedad escalar para continuar su configuración.`

Línea 191: `entity.HasAlternateKey(a => a.Codigo).HasName("AK_Aleaciones_Codigo"); → define una clave alternativa única.`

Línea 192: `entity.HasIndex(a => a.Nombre).HasDatabaseName("IX_Aleaciones_Nombre"); → define un índice sobre las propiedades indicadas.`

Línea 193: `entity.HasIndex(a => new { a.PorcentajeCarbono, a.PorcentajeManganeso }).HasDatabaseName("IX_Aleaciones_Porcentajes"); → define un índice sobre las propiedades indicadas.`

Línea 194: `}); → cierra la lambda o configuración encadenada y finaliza la llamada.`

Línea 195: `modelBuilder.Entity<EstadoOrden>(entity => → selecciona una entidad para configurarla con Fluent API.`

Línea 196: `{ → abre el bloque de código asociado a la declaración o instrucción anterior.`

Línea 197: `entity.ToTable("EstadosOrden"); → asigna explícitamente la tabla usada por la entidad.`

Línea 198: `entity.HasKey(e => e.Id); → define la clave primaria.`

Línea 199: `entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50); → selecciona una propiedad escalar para continuar su configuración.`

Línea 200: `entity.Property(e => e.Descripcion).HasMaxLength(250); → selecciona una propiedad escalar para continuar su configuración.`

Línea 201: `entity.Property(e => e.Activo).HasDefaultValue(true); → selecciona una propiedad escalar para continuar su configuración.`

Línea 202: `}); → cierra la lambda o configuración encadenada y finaliza la llamada.`

Línea 203: `modelBuilder.Entity<DetalleOrden>(entity => → selecciona una entidad para configurarla con Fluent API.`

Línea 204: `{ → abre el bloque de código asociado a la declaración o instrucción anterior.`

Línea 205: `entity.ToTable("DetallesOrden"); → asigna explícitamente la tabla usada por la entidad.`

Línea 206: `entity.HasKey(d => d.Id); → define la clave primaria.`

Línea 207: `entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);` → selecciona una propiedad escalar para continuar su configuración.

Línea 208: `entity.Property(d => d.Notas).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 209: `entity.HasOne(d => d.Orden)` → inicia la configuración de una navegación de referencia.

Línea 210: `.WithOne(o => o.Detalle)` → establece el extremo de referencia de una relación uno-a-uno.

Línea 211: `.HasForeignKey<DetalleOrden>(d => d.OrdenId)` → declara explícitamente la clave foránea.

Línea 212: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 213: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 214: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 216: `modelBuilder.Entity<CertificadoCalidad>(entity =>` → selecciona una entidad para configurarla con Fluent API.

Línea 217: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 218: `entity.ToTable("CertificadosCalidad");` → asigna explícitamente la tabla usada por la entidad.

Línea 219: `entity.HasKey(c => c.Id);` → define la clave primaria.

Línea 220: `entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 221: `entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);` → selecciona una propiedad escalar para continuar su configuración.

Línea 222: `entity.HasAlternateKey(c => c.NumeroCertificado).HasName("AK_CertificadosCalidad_NumeroCertificado");` → define una clave alternativa única.

Línea 223: `entity.HasIndex(c => c.FechaEmision).HasDatabaseName("IX_CertificadosCalidad_FechaEmision");` → define un índice sobre las propiedades indicadas.

Línea 224: `entity.HasOne(c => c.Orden)` → inicia la configuración de una navegación de referencia.

Línea 225: `.WithOne(o => o.Certificado)` → establece el extremo de referencia de una relación uno-a-uno.

Línea 226: `.HasForeignKey<CertificadoCalidad>(c => c.OrdenId)` → declara explícitamente la clave foránea.

Línea 227: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 228: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 229: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 230: `modelBuilder.Entity<OrdenAleacion>(entity =>` → selecciona una entidad para configurarla con Fluent API.

Línea 231: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 232: `entity.ToTable("OrdenesAleaciones");` → asigna explícitamente la tabla usada por la entidad.

Línea 233: `entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });` → define la clave primaria.

Línea 234: `entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");` → selecciona una propiedad escalar para continuar su configuración.

Línea 235: `entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);` → selecciona una propiedad escalar para continuar su configuración.

Línea 236: `entity.Property(x => x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");` → selecciona una propiedad escalar para continuar su configuración.

Línea 237: `entity.HasOne(x => x.Orden)` → inicia la configuración de una navegación de referencia.

Línea 238: `.WithMany(o => o.OrdenesAleaciones)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 239: `.HasForeignKey(x => x.OrdenFabricacionId)` → declara explícitamente la clave foránea.

Línea 240: `.OnDelete(DeleteBehavior.Cascade);` → define el comportamiento de eliminación de la relación.

Línea 241: `entity.HasOne(x => x.Aleacion)` → inicia la configuración de una navegación de referencia.

Línea 242: `.WithMany(a => a.OrdenesAleaciones)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 243: `.HasForeignKey(x => x.AleacionId)` → declara explícitamente la clave foránea.

Línea 244: `.OnDelete(DeleteBehavior.Restrict);` → define el comportamiento de eliminación de la relación.

Línea 245: `entity.HasIndex(x => x.AleacionId).HasDatabaseName("IX_OrdenesAleaciones_AleacionId");` → define un índice sobre las propiedades indicadas.

Línea 246: `entity.HasIndex(x => x.EstadoRelacion).HasFilter("[EstadoRelacion] = 'Activa'").HasDatabaseName("IX_OrdenesAleaciones_EstadoRelacion_Activas");` → define un índice sobre las propiedades indicadas.

Línea 247: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 248: `}` → cierra el bloque de código actual.

Línea 249: `}` → cierra el bloque de código actual.

Línea 251: `public interface IOrdenRepositorio` → declara el contrato IOrdenRepositorio usado por la aplicación.

Línea 252: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 253: `List<OrdenFabricacion> ObtenerTodas();` → declara en el contrato la operación que debe devolver todas las órdenes visibles para el repositorio.

Línea 254: `OrdenFabricacion? ObtenerPorId(int id);` → declara en el contrato la búsqueda de una orden por su clave primaria y permite devolver null si no existe.

Línea 255: `OrdenFabricacion? ObtenerPorNumero(string numeroOrden);` → declara en el contrato la búsqueda por el identificador de negocio NumeroOrden.

Línea 256: `void Agregar(OrdenFabricacion orden);` → declara la operación que incorpora una orden al repositorio para que EF Core la siga como nueva.

Línea 257: `void Eliminar(OrdenFabricacion orden);` → declara la operación que marca una orden para eliminación física cuando el escenario lo requiere.

Línea 258: `int Guardar();` → declara la operación que confirma los cambios pendientes y devuelve el número de entradas afectadas.

Línea 259: `}` → cierra el bloque de código actual.

Línea 261: `public sealed class OrdenRepositorio : IOrdenRepositorio` → declara la clase `OrdenRepositorio` que forma parte del estado acumulativo.

Línea 262: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 263: `private readonly AceriaDbContext _context;` → declara el campo privado `_context` de tipo `AceriaDbContext`.

Línea 264: `public OrdenRepositorio(AceriaDbContext context) => _context = context;` → inyecta `AceriaDbContext` en el repositorio y lo conserva como dependencia de persistencia.

Línea 266: `public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();` → ordena el resultado antes de materializarlo.

Línea 267: `public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);` → implementa la búsqueda por clave primaria mediante `Find`, aprovechando primero el `ChangeTracker` y después la base de datos.

Línea 268: `public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);` → recupera la primera coincidencia o `null`.

Línea 269: `public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);` → registra una entidad nueva para insertarla al guardar.

Línea 270: `public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);` → marca una entidad para borrado al guardar.

Línea 271: `public int Guardar() => _context.SaveChanges();` → confirma en SQL Server los cambios del `ChangeTracker`.

Línea 272: `}` → cierra el bloque de código actual.

Línea 274: `public interface IServicioOrdenes` → declara el contrato `IServicioOrdenes` usado por la aplicación.

Línea 275: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 276: `string ObtenerResumen(int ordenId);` → declara el servicio que construye un resumen de una orden a partir de su identificador.

Línea 277: `}` → cierra el bloque de código actual.

Línea 279: `public sealed class ServicioOrdenes : IServicioOrdenes` → declara la clase `ServicioOrdenes` que forma parte del estado acumulativo.

Línea 280: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 281: `private readonly IOrdenRepositorio _repositorio;` → declara el campo privado `_repositorio` de tipo `IOrdenRepositorio`.

Línea 282: `private readonly AceriaDbContext _context;` → declara el campo privado `_context` de tipo `AceriaDbContext`.

Línea 283: `public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)` → declara el constructor que recibe por inyección el repositorio de órdenes y el `DbContext`.

Línea 284: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 285: `_repositorio = repositorio;` → guarda en el servicio el repositorio inyectado para reutilizarlo en sus operaciones.

Línea 286: `_context = context;` → guarda el `DbContext` inyectado para consultar los datos necesarios durante el servicio.

Línea 287: `}` → cierra el bloque de código actual.

Línea 289: `public string ObtenerResumen(int ordenId)` → declara el método que obtiene la orden y compone el resumen solicitado por el servicio.

Línea 290: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 291: `var orden = _repositorio.ObtenerPorId(ordenId);` → declara una variable local y almacena el resultado de la expresión.

Línea 292: `if (orden is null) return $"Orden {ordenId} no encontrada";` → evalúa una condición antes de ejecutar el bloque asociado.

Línea 293: `var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);` → cuenta las filas que cumplen la consulta.

Línea 294: `return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";` → devuelve el resultado al código llamador.

Línea 295: `}` → cierra el bloque de código actual.

Línea 296: `}` → cierra el bloque de código actual.

Línea 298: `public static class Program` → declara la clase Program que forma parte del estado acumulativo.

Línea 299: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 300: `public static void Main()` → declara el punto de entrada ejecutable de la aplicación de consola.

Línea 301: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 302: `var configuration = new ConfigurationBuilder()` → inicia la construcción de la configuración de la aplicación.

Línea 303: `.SetBasePath(AppContext.BaseDirectory)` → establece la ruta base para los archivos de configuración.

Línea 304: `.AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)` → incorpora appsettings.json.

Línea 305: `.AddEnvironmentVariables()` → incorpora variables de entorno.

Línea 306: `.Build();` → materializa el objeto construido por la cadena anterior.

Línea 308: `var connectionString = configuration.GetConnectionString("AceriaDB")` → obtiene la cadena AceriaDB desde configuración.

Línea 309: `?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");` → interrumpe el arranque con un error explícito si la configuración no contiene ConnectionStrings:AceriaDB.

Línea 311: `var services = new ServiceCollection();` → crea la colección de servicios.

Línea 312: `services.AddDbContext<AceriaDbContext>(options =>` → registra el DbContext en inyección de dependencias.

Línea 313: `options` → continúa la configuración del DbContext sobre el parámetro options recibido por la lambda de AddDbContext.

Línea 314: `.UseSqlServer(connectionString, sql =>` → selecciona SQL Server como proveedor de EF Core.

Línea 315: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 316: `sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);` → habilita reintentos ante fallos transitorios.

Línea 317: `sql.CommandTimeout(60);` → establece el timeout de los comandos SQL.

Línea 318: `})` → cierra la lambda de configuración actual y devuelve el control a la llamada encadenada que la contiene.

Línea 319: `.LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)` → configura el logging de EF Core.

Línea 320: `.EnableDetailedErrors();` → activa errores detallados.

Línea 322: `services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();` → registra un servicio con ciclo de vida scoped.

Línea 323: `services.AddScoped<IServicioOrdenes, ServicioOrdenes>();` → registra un servicio con ciclo de vida scoped.

Línea 325: `using var provider = services.BuildServiceProvider(new ServiceProviderOptions` → construye el ServiceProvider con validación de servicios y garantiza su liberación al salir del ámbito mediante using var.

Línea 326: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 327: `ValidateScopes = true,` → activa la validación de ámbitos para detectar dependencias scoped usadas desde ámbitos incorrectos.

Línea 328: `ValidateOnBuild = true` → obliga al contenedor a validar las resoluciones registradas al construir el ServiceProvider.

Línea 329: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 331: `using var scope = provider.CreateScope();` → crea un ámbito de DI para resolver servicios scoped y lo libera automáticamente al terminar el bloque.

Línea 332: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → resuelve un servicio obligatorio del contenedor.

Línea 333: `context.Database.EnsureDeleted();` → reinicia la base del laboratorio para una ejecución reproducible.

Línea 334: `context.Database.Migrate();` → aplica el historial de migraciones pendiente.

Línea 336: `var orden = new OrdenFabricacion` → declara una variable local y almacena el resultado de la expresión.

Línea 337: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 338: `NumeroOrden = "0F-M2-0001",` → asigna el identificador de negocio usado por la orden de validación acumulativa del módulo.

Línea 339: `Cliente = "Constructora del Norte",` → asigna el cliente de la orden utilizada en el escenario práctico del punto.

Línea 340: `FechaCreacion = DateTime.UtcNow,` → asigna a la orden la fecha de creación en UTC para el escenario reproducible del punto.

Línea 341: `FechaEntrega = DateTime.Today.AddDays(14),` → establece una fecha de entrega prevista catorce días después de la fecha local actual.

Línea 342: `Estado = "Pendiente",` → inicializa la orden con el estado Pendiente, coherente con el flujo del laboratorio.

Línea 343: `Observaciones = "Orden de validación M2",` → añade texto opcional para comprobar el mapeo de Observaciones.

Línea 344: `Planchas =` → inicia el inicializador de la colección de planchas asociadas a la orden.

Línea 345: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 346: `new PlanchaAcero` → crea una nueva plancha dentro del inicializador de la colección Planchas de la orden.

Línea 347: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 348: `Espesor = 10.5,` → asigna a la plancha un espesor positivo para satisfacer las reglas del modelo.

Línea 349: `Ancho = 1500,` → asigna el ancho de la plancha usada en el escenario de persistencia.

Línea 350: `Largo = 3000,` → asigna el largo de la plancha usada en el escenario de persistencia.

Línea 351: `Peso = 371.250m,` → asigna un peso decimal con tres posiciones para validar la precisión configurada.

Línea 352: `Activa = true,` → marca la plancha como activa para que sea visible en los estados que aplican ese criterio.

Línea 353: `}` → cierra el bloque de código actual.

Línea 354: `},` → continúa el inicializador o la llamada del bloque actual.

Línea 355: `Detalle = new DetalleOrden` → crea el detalle uno-a-uno que se persistirá junto con la orden principal.

Línea 356: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 357: `ComposicionQuimica = "C: 0.45%, Mn: 0.75%",` → asigna la composición química utilizada para verificar la entidad DetalleOrden.

Línea 358: `TemperaturaColada = 1550.5,` → asigna la temperatura de colada del detalle de fabricación.

Línea 359: `Notas = "Colada principal"` → asigna las notas opcionales del detalle asociado a la orden.

Línea 360: `},` → continúa el inicializador o la llamada del bloque actual.

Línea 361: `Certificado = new CertificadoCalidad` → crea el certificado uno-a-uno asociado a la orden del escenario.

Línea 362: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 363: `NumeroCertificado = "CERT-0001",` → asigna el identificador natural del certificado de calidad.

Línea 364: `FechaEmision = DateTime.Today,` → asigna la fecha de emisión del certificado usado en la prueba.

Línea 365: `OrganismoCertificador = "Laboratorio Aceria"` → asigna el organismo que emite el certificado de calidad.

Línea 366: `},` → continúa el inicializador o la llamada del bloque actual.

Línea 367: `};` → cierra el inicializador del objeto y finaliza la instrucción.

Línea 369: `var aleacion = new Aleacion` → declara una variable local y almacena el resultado de la expresión.

Línea 370: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 371: `Nombre = "AISI 1045",` → asigna el nombre comercial de la aleación utilizada en el escenario.

Línea 372: `Codigo = "A1045",` → asigna el código natural único de la aleación.

Línea 373: `PorcentajeCarbono = 0.45,` → asigna el porcentaje de carbono dentro del rango permitido por la restricción del modelo.

Línea 374: `PorcentajeManganeso = 0.75,` → asigna el porcentaje de manganeso dentro del rango permitido por la restricción del modelo.

Línea 375: `Descripcion = "Acero medio en carbono"` → asigna una descripción opcional a la aleación.

Línea 376: `};` → cierra el inicializador del objeto y finaliza la instrucción.

Línea 377: `orden.OrdenesAleaciones.Add(new OrdenAleacion` → registra una entidad nueva para insertarla al guardar.

Línea 378: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 379: `Aleacion = aleacion,` → conecta la entidad intermedia `OrdenAleacion` con la aleación creada para el escenario.

Línea 380: `CantidadUtilizada = 1500.500m,` → registra la cantidad utilizada con tres decimales, coherente con `HasPrecision(18, 3)`.

Línea 381: `EstadoRelacion = "Activa"` → marca como activa la relación entre la orden y la aleación.

Línea 382: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 383: `context.OrdenesFabricacion.Add(orden);` → registra una entidad nueva para insertarla al guardar.

Línea 384: `context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo = true });` → registra una entidad nueva para insertarla al guardar.

Línea 385: `context.SaveChanges();` → confirma en SQL Server los cambios del `ChangeTracker`.

Línea 387: `var entity = context.Model.FindEntityType(typeof(OrdenFabricacion))!;` → declara una variable local y almacena el resultado de la expresión.

Línea 388: `global::System.Console.WriteLine($"2.9 OK | Índices: {string.Join(", ", entity.GetIndexes().Select(i => i.GetDatabaseName()))}");` → escribe la evidencia de ejecución usada para validar este estado.

Línea 389: `}` → cierra el bloque de código actual.

Línea 390: `}` → cierra el bloque de código actual.

Punto 2.10 - Filtros globales de consulta

Ejercicio guiado: introducir filtros globales de consulta en el modelo acumulativo de AceriaData y comprobar que EF Core oculta determinadas filas en las consultas normales sin eliminarlas físicamente de SQL Server.

Contexto del proyecto: 2.10 parte íntegramente del estado 2.9. Se conservan claves, relaciones, índices y restricciones CHECK. En este punto no se añaden propiedades de Soft Delete: se reutilizan propiedades de negocio ya existentes para definir cuatro filtros reales con `HasQueryFilter`. El siguiente punto, 2.11, será el que introduzca `IsDeleted` y `DeletedAt`.

Objetivos de aprendizaje

- Comprender qué hace un filtro global de consulta en EF Core.
- Configurar `HasQueryFilter` sobre varias entidades.
- Filtrar órdenes cuyo `Estado` sea `Cancelada`.
- Mostrar sólo `PlanchaAcero` con `Activa == true`.
- Mostrar sólo `EstadoOrden` con `Activo == true`.
- Mostrar sólo relaciones `OrdenAleacion` cuyo `EstadoRelacion` sea `Activa`.
- Usar `IgnoreQueryFilters()` de forma explícita.
- Diferenciar ocultación lógica de eliminación física.
- Comprobar la diferencia entre una consulta normal y la misma consulta sin filtros.
- Entender por qué los filtros globales no requieren modificar el esquema físico.
- Interpretar correctamente la migración histórica vacía `M2_2_10`.
- Mantener separado este punto del Soft Delete que comienza en 2.11.

Paso 1: Abrir el estado autónomo de 2.10

```
cd M02/PROYECTO/2.10
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

El checkpoint contiene el estado completo del punto. Si reproduces manualmente la evolución, parte de 2.9 y añade únicamente los filtros descritos aquí.

Paso 2: Delimitar el cambio respecto a 2.9

En 2.9 el esquema quedó consolidado con claves, índices y restricciones CHECK. En 2.10 **no se rehace ninguna de esas configuraciones**.

El cambio nuevo es de comportamiento de consulta:

- `OrdenFabricacion`: ocultar estados `Cancelada`;
- `PlanchaAcero`: ocultar planchas inactivas;
- `EstadoOrden`: ocultar estados cuyo `Activo` sea falso;
- `OrdenAleacion`: ocultar relaciones cuyo `EstadoRelacion` no sea `Activa`.

Todavía no existen `IsDeleted` ni `DeletedAt`. Añadirlas aquí adelantaría el contenido de 2.11 y haría divergir la práctica del checkpoint real.

Paso 3: Configurar el filtro de `OrdenFabricacion`

El modelo efectivo contiene:

```
modelBuilder.Entity<OrdenFabricacion>(entity =>
{
    // ... configuración heredada ...
    entity.HasQueryFilter(
        o => o.Estado != "Cancelada");
});
```

La condición se incorpora automáticamente a las consultas LINQ normales de `OrdenFabricacion`. La fila cancelada sigue almacenada en la tabla; simplemente queda fuera de la vista habitual de EF Core.

Paso 4: Configurar el filtro de PlanchaAcero

```
modelBuilder.Entity<PlanchaAcero>(entity =>
{
    // ... configuración heredada ...
    entity.HasQueryFilter(x => x.Activa);
});
```

La propiedad `Activa` ya formaba parte del modelo antes de este punto. El filtro reutiliza esa propiedad: no añade una columna nueva.

Una consulta normal sobre `PlanchasAcero` devuelve sólo filas activas. Para una operación administrativa que necesite ver también las inactivas se puede usar `IgnoreQueryFilters()`.

Paso 5: Configurar el filtro de EstadoOrden

El checkpoint real contiene también:

```
modelBuilder.Entity<EstadoOrden>(entity =>
{
    // ... configuración heredada ...
    entity.HasQueryFilter(e => e.Activo);
});
```

Por tanto, 2.10 no trabaja únicamente con órdenes y planchas. Los estados marcados como no activos también quedan ocultos en consultas normales de `EstadoOrden`.

Paso 6: Configurar el filtro de OrdenAleacion

```
modelBuilder.Entity<OrdenAleacion>(entity =>
{
    // ... configuración heredada ...
    entity.HasQueryFilter(
        x => x.EstadoRelacion == "Activa");
});
```

La relación muchos-a-muchos con payload conserva su PK compuesta, sus FK y sus índices de 2.9. El único cambio de este punto es la regla de visibilidad basada en `EstadoRelacion`.

Paso 7: Comprender qué entidades no reciben un filtro nuevo

En el estado 2.10 no se configura un `HasQueryFilter` nuevo para:

- `Aleacion`;
- `DetalleOrden`;
- `CertificadoCalidad`.

Esto es deliberado. La práctica debe representar el modelo real y no añadir reglas que no aparecen en `M02/PROYECTO0/2.10/Program.cs`.

Paso 8: Comparar una consulta normal con IgnoreQueryFilters

El ejemplo ejecutable usa:

```
var visibles =
    context.OrdenesFabricacion.Count();

var todas =
    context.OrdenesFabricacion
        .IgnoreQueryFilters()
        .Count();
```

La primera consulta aplica `Estado != "Cancelada"`. La segunda desactiva expresamente los filtros globales para esa consulta.

`IgnoreQueryFilters()` debe utilizarse con intención: sirve para diagnósticos, administración o escenarios que necesitan acceder a filas normalmente ocultas. No conviene convertirlo en la forma habitual de consultar porque neutraliza la regla transversal.

Paso 9: Preparar una fila visible y otra cancelada

El checkpoint crea primero la orden normal de validación y después una segunda orden:

```
var cancelada = new OrdenFabricacion
{
    NumeroOrden = "OF-M2-CANCELADA",
    Cliente = "Cliente Histórico",
    FechaCreacion = DateTime.UtcNow,
    Estado = "Cancelada"
};

context.OrdenesFabricacion.Add(cancelada);
context.SaveChanges();
```

La operación `SaveChanges()` persiste la fila. El filtro no impide insertar una orden cancelada: actúa cuando EF Core compone consultas.

Paso 10: Compilar antes de revisar el historial

```
dotnet build AceriaData.sln --configuration Release
```

La compilación debe terminar correctamente antes de ejecutar las herramientas de EF Core.

Paso 11: Entender la migración histórica M2_2_10

El checkpoint contiene:

```
Migrations/20260927204833_M2_2_10.cs
```

pero su contenido real es:

```
protected override void Up(
    MigrationBuilder migrationBuilder)
{
}

protected override void Down(
    MigrationBuilder migrationBuilder)
{
}
```

Esto es coherente con el cambio introducido: `HasQueryFilter` modifica el modelo de consulta de EF Core, pero **no crea columnas, índices, claves ni restricciones en SQL Server**.

Si reproduces literalmente el historial del curso desde 2.9, `dotnet ef migrations add M2_2_10` genera una migración sin operaciones de esquema. El checkpoint la conserva como hito histórico; no debe confundirse con una migración que altere físicamente la base.

Paso 12: Aplicar y revisar el historial acumulativo

```
dotnet ef database update
dotnet ef migrations list
```

La lista incluye `M2_2_10` después de `M2_2_9`, aunque `Up()` esté vacío. El esquema físico sigue siendo el consolidado en 2.9.

Durante la ejecución del proyecto se mantiene `Database.Migrate()`. No se sustituye el historial por `EnsureCreated()`.

Paso 13: Ejecutar el escenario completo

```
dotnet run --project AceriaData.Console.csproj --configuration Release
```

El escenario crea una orden visible y otra cancelada, ejecuta ambos recuentos y termina con:

```
2.10 OK | Visibles: 1 | Sin filtro: 2
```

La diferencia demuestra que la fila cancelada existe pero el filtro global la excluye de la consulta normal.

Paso 14: Inspeccionar el SQL generado

El proyecto mantiene logging de comandos de EF Core. Al ejecutar el escenario, compara el SQL del recuento normal con el recuento que usa `IgnoreQueryFilters()`.

La consulta normal incorpora el predicado derivado de:

```
o => o.Estado != "Cancelada"
```

La consulta con `IgnoreQueryFilters()` no aplica ese filtro global. Esta evidencia permite comprobar el comportamiento real sin confundirlo con borrado físico.

Paso 15: Probar los otros filtros sin cambiar el modelo final

La misma idea puede comprobarse sobre las otras entidades configuradas:

```
var planchasVisibles =
    context.PlanchasAcero.Count();

var planchasTotales =
    context.PlanchasAcero
        .IgnoreQueryFilters()
        .Count();

var estadosVisibles =
    context.EstadosOrden.Count();

var estadosTotales =
    context.EstadosOrden
        .IgnoreQueryFilters()
        .Count();
```

No es necesario añadir nuevas propiedades ni otra migración. El objetivo es observar cómo el mismo modelo cambia la visibilidad según se apliquen o no los filtros.

Paso 16: Resolver el reto de la orden cancelada

El checkpoint contiene el bloque comentado `RETO 2.10 - ORDEN CANCELADA E IGNOREQUERYFILTERS`.

Al activarlo se inserta otra orden cancelada:

```
var canceladaReto = new OrdenFabricacion
{
    NumeroOrden = "OF-M2-CANCELADA-RETO",
    Cliente = "Cliente Histórico Reto",
    FechaCreacion = DateTime.UtcNow,
    Estado = "Cancelada"
};

context.OrdenesFabricacion.Add(canceladaReto);
context.SaveChanges();

var visiblesReto =
    context.OrdenesFabricacion.Count();

var todasReto =
    context.OrdenesFabricacion
        .IgnoreQueryFilters()
        .Count();
```

La consulta normal mantiene el mismo número de órdenes visibles; la consulta sin filtros aumenta porque incorpora también la nueva fila cancelada.

Paso 17: Delimitar el alcance antes de 2.11

Al terminar 2.10:

- no se añaden `IsDeleted` ni `DeletedAt`;
- no se modifica el esquema consolidado en 2.9;
- no se sustituyen los filtros de negocio por Soft Delete;
- no se elimina físicamente una orden por estar cancelada;
- no se usa `IgnoreQueryFilters()` de forma indiscriminada;
- no se usa `EnsureCreated()` para evitar Migrations.

2.11 partirá de este estado y añadirá el Soft Delete real del curso.

Errores comunes

Error	Causa	Corrección
Pensar que una fila filtrada fue eliminada	La consulta normal no la devuelve	Comprobarla con <code>IgnoreQueryFilters()</code>
Esperar cambios de tablas al añadir <code>HasQueryFilter</code>	Un filtro global modifica consultas, no el esquema físico	Revisar que <code>M2_2_10.Up()</code> y <code>Down()</code> están vacíos
Añadir <code>IsDeleted</code> en 2.10	Se adelanta el contenido de 2.11	Mantener sólo propiedades de negocio ya existentes
Olvidar los filtros de <code>EstadoOrden</code> u <code>OrdenAleacion</code>	Se documenta sólo una parte del modelo real	Contrastar el MD con <code>Program.cs</code> del checkpoint
Usar <code>IgnoreQueryFilters()</code> en todas las consultas	Se neutralizan las reglas transversales	Reservarlo para escenarios que realmente necesiten filas ocultas

Error	Causa	Corrección
Usar <code>EnsureCreated()</code>	Se rompe la evolución acumulativa	Mantener <code>Migrations</code> y <code>Database.Migrate()</code>

Analugía final

Un filtro global se parece a una vista de trabajo automática: las filas siguen en el almacén, pero EF Core aplica una regla de visibilidad cada vez que consulta esa entidad. `IgnoreQueryFilters()` equivale a abrir temporalmente la vista completa.

Resultado esperado

Al finalizar 2.10, `AceriaData` conserva íntegro el esquema de 2.9 y añade cuatro reglas de visibilidad en el modelo de EF Core. La ejecución demuestra que una orden cancelada permanece en SQL Server pero queda fuera de la consulta normal. El marcador esperado es:

2.10 OK | Visibles: 1 | Sin filtro: 2

Conexión con el siguiente punto

2.11 parte de este estado para introducir Soft Delete con `IsDeleted`, `DeletedAt` y los cambios de esquema correspondientes, sin perder los conceptos de filtros globales aprendidos aquí.

Código acumulativo completo del estado 2.10

El siguiente archivo es el `Program.cs` real del estado ejecutable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

[Table("OrdenesFabricacion")]
public class OrdenFabricacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroOrden { get; set; } = string.Empty;
    [Required]
    [MaxLength(200)]
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    [Required]
    [MaxLength(50)]
    public string Estado { get; set; } = "Pendiente";
    [MaxLength(500)]
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
    public DetalleOrden? Detalle { get; set; }
    public CertificadoCalidad? Certificado { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

[Table("PlanchasAcero")]
public class PlanchaAcero
{

```

```

    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    [Precision(18, 3)]
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("Aleaciones")]
public class Aleacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(100)]
    public string Nombre { get; set; } = string.Empty;
    [Required]
    [MaxLength(20)]
    public string Codigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    [MaxLength(500)]
    public string? Descripcion { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

[Table("EstadosOrden")]
public class EstadoOrden
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string Nombre { get; set; } = string.Empty;
    [MaxLength(250)]
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
}

[Table("DetallesOrden")]
public class DetalleOrden
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(200)]
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    [MaxLength(500)]
    public string? Notas { get; set; }
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("CertificadosCalidad")]
public class CertificadoCalidad
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroCertificado { get; set; } = string.Empty;
    public DateTime FechaEmision { get; set; }
    [Required]
    [MaxLength(100)]
    public string OrganismoCertificador { get; set; } = string.Empty;
}

```

```

        [ForeignKey(nameof(OrdenId))]
        public OrdenFabricacion Orden { get; set; } = null!;
    }

    [PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]
    [Table("OrdenesAleaciones")]
    public class OrdenAleacion
    {
        public int OrdenFabricacionId { get; set; }
        public int AleacionId { get; set; }
        public DateTime FechaAsignacion { get; set; } = DateTime.Now;
        [Precision(18, 3)]
        public decimal CantidadUtilizada { get; set; }
        [MaxLength(20)]
        public string EstadoRelacion { get; set; } = "Activa";
        public OrdenFabricacion Orden { get; set; } = null!;
        public Aleacion Aleacion { get; set; } = null!;
    }

    public class AceriaDbContext : DbContext
    {
        public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

        public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
        public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
        public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
        public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
        public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();
        public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();
        public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();

        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);
            modelBuilder.Entity<OrdenFabricacion>(entity =>
            {
                entity.ToTable("OrdenesFabricacion");
                entity.HasKey(o => o.Id);
                entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);
                entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);
                entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");
                entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");
                entity.Property(o => o.Observaciones).HasMaxLength(500);
                entity.HasAlternateKey(o => o.NumeroOrden).HasName("AK_OrdenesFabricacion_NumeroOrden");
                entity.HasIndex(o => o.Cliente).HasDatabaseName("IX_OrdenesFabricacion_Cliente");
                entity.HasIndex(o => new { o.Cliente,
o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Cliente_FechaCreacion");
                entity.HasIndex(o => o.FechaEntrega).HasFilter("[Estado] = 'Pendiente'").HasDatabaseName("IX_OrdenesFabricacion_FechaEntrega_Pendientes");
                entity.HasIndex(o => o.Estado).IncludeProperties(o => new { o.NumeroOrden, o.Cliente,
o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Estado_Incluye");
                entity.HasQueryFilter(o => o.Estado != "Cancelada");
            });
            modelBuilder.Entity<PlanchaAcero>(entity =>
            {
                entity.ToTable("PlanchasAcero", t =>
                {
                    t.HasCheckConstraint("CK_PlanchasAcero_Espesor", "[Espesor] > 0");
                    t.HasCheckConstraint("CK_PlanchasAcero_Ancho", "[Ancho] > 0");
                    t.HasCheckConstraint("CK_PlanchasAcero_Largo", "[Largo] > 0");
                    t.HasCheckConstraint("CK_PlanchasAcero_Peso", "[Peso] > 0");
                });
                entity.HasKey(x => x.Id);
                entity.Property(x => x.Peso).HasPrecision(18, 3);
                entity.Property(x => x.Activa).HasDefaultValue(true);
                entity.HasOne(x => x.Orden)
                    .WithMany(o => o.Planchas)
                    .HasForeignKey(x => x.OrdenId)
                    .OnDelete(DeleteBehavior.Cascade)
                    .IsRequired();
                entity.HasIndex(x => new { x.OrdenId,
x.Activa }).HasDatabaseName("IX_PlanchasAcero_OrdenId_Activa");
            });
        }
    }

```

```

        entity.HasIndex(x => x.Espesor).HasFilter("[Activa] = 1").HasDatabaseName("IX_PlanchasAcero_Espesor_Activas");
        entity.HasQueryFilter(x => x.Activa);
    });
    modelBuilder.Entity<Aleacion>(entity =>
    {
        entity.ToTable("Aleaciones", t =>
        {
            t.HasCheckConstraint("CK_Aleaciones_PorcentajeCarbono", "[PorcentajeCarbono] >= 0 AND [PorcentajeCarbono] <= 2");
            t.HasCheckConstraint("CK_Aleaciones_PorcentajeManganeso", "[PorcentajeManganeso] >= 0 AND [PorcentajeManganeso] <= 5");
        });
        entity.HasKey(a => a.Id);
        entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
        entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
        entity.Property(a => a.Descripcion).HasMaxLength(500);
        entity.HasAlternateKey(a => a.Codigo).HasName("AK_Aleaciones_Codigo");
        entity.HasIndex(a => a.Nombre).HasDatabaseName("IX_Aleaciones_Nombre");
        entity.HasIndex(a => new { a.PorcentajeCarbono, a.PorcentajeManganeso }).HasDatabaseName("IX_Aleaciones_Porcentajes");
    });
    modelBuilder.Entity<EstadoOrden>(entity =>
    {
        entity.ToTable("EstadosOrden");
        entity.HasKey(e => e.Id);
        entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
        entity.Property(e => e.Descripcion).HasMaxLength(250);
        entity.Property(e => e.Activo).HasDefaultValue(true);
        entity.HasQueryFilter(e => e.Activo);
    });
    modelBuilder.Entity<DetalleOrden>(entity =>
    {
        entity.ToTable("DetallesOrden");
        entity.HasKey(d => d.Id);
        entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);
        entity.Property(d => d.Notas).HasMaxLength(500);
        entity.HasOne(d => d.Orden)
            .WithOne(o => o.Detalle)
            .HasForeignKey<DetalleOrden>(d => d.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });
    modelBuilder.Entity<CertificadoCalidad>(entity =>
    {
        entity.ToTable("CertificadosCalidad");
        entity.HasKey(c => c.Id);
        entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);
        entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);
        entity.HasAlternateKey(c => c.NumeroCertificado).HasName("AK_CertificadosCalidad_NumeroCertificado");
        entity.HasIndex(c => c.FechaEmision).HasDatabaseName("IX_CertificadosCalidad_FechaEmision");
        entity.HasOne(c => c.Orden)
            .WithOne(o => o.Certificado)
            .HasForeignKey<CertificadoCalidad>(c => c.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });
    modelBuilder.Entity<OrdenAleacion>(entity =>
    {
        entity.ToTable("OrdenesAleaciones");
        entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });
        entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");
        entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);
        entity.Property(x => x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");
        entity.HasOne(x => x.Orden)
            .WithMany(o => o.OrdenesAleaciones)
            .HasForeignKey(x => x.OrdenFabricacionId)
            .OnDelete(DeleteBehavior.Cascade);
        entity.HasOne(x => x.Aleacion)
            .WithMany(a => a.OrdenesAleaciones)

```

```

        .HasForeignKey(x => x.AleacionId)
        .onDelete(DeleteBehavior.Restrict);
        entity.HasIndex(x => x.AleacionId).HasDatabaseName("IX_OrdenesAleaciones_AleacionId");
        entity.HasIndex(x => x.EstadoRelacion).HasFilter("[EstadoRelacion] = 'Activa'").HasDatabaseName("IX_OrdenesAleaciones_EstadoRelacion_Activas");
        entity.HasQueryFilter(x => x.EstadoRelacion == "Activa");
    });
}

}

public interface IOordenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    int Guardar();
}

public sealed class OrdenRepositorio : IOordenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) =>
        _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOordenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOordenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();

```

```

services.AddDbContext<AceriaDbContext>(options =>
    options
        .UseSqlServer(connectionString, sql =>
            {
                sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                sql.CommandTimeout(60);
            })
        .LogTo(global::System.Console.WriteLine, new[] {
            "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)
        .EnableDetailedErrors());

services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateScopes = true,
    ValidateOnBuild = true
});

using var scope = provider.CreateScope();
var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();

var orden = new OrdenFabricacion
{
    NumeroOrden = "OF-M2-0001",
    Cliente = "Constructora del Norte",
    FechaCreacion = DateTime.UtcNow,
    FechaEntrega = DateTime.Today.AddDays(14),
    Estado = "Pendiente",
    Observaciones = "Orden de validación M2",
    Planchas =
    {
        new PlanchaAcero
        {
            Espesor = 10.5,
            Ancho = 1500,
            Largo = 3000,
            Peso = 371.250m,
            Activa = true,
        }
    },
    Detalle = new DetalleOrden
    {
        ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
        TemperaturaColada = 1550.5,
        Notas = "Colada principal"
    },
    Certificado = new CertificadoCalidad
    {
        NumeroCertificado = "CERT-0001",
        FechaEmision = DateTime.Today,
        OrganismoCertificador = "Laboratorio Aceria"
    },
};

var aleacion = new Aleacion
{
    Nombre = "AISI 1045",
   Codigo = "A1045",
    PorcentajeCarbono = 0.45,
    PorcentajeManganeso = 0.75,
    Descripcion = "Acero medio en carbono"
};
orden.OrdenesAleaciones.Add(new OrdenAleacion
{
    Aleacion = aleacion,
    CantidadUtilizada = 1500.500m,
    EstadoRelacion = "Activa"
});
context.OrdenesFabricacion.Add(orden);

```

```

        context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa",
Activo = true });
        context.SaveChanges();

        var cancelada = new OrdenFabricacion
        {
            NumeroOrden = "OF-M2-CANCELADA",
            Cliente = "Cliente Histórico",
            FechaCreacion = DateTime.UtcNow,
            Estado = "Cancelada"
        };
        context.OrdenesFabricacion.Add(cancelada);
        context.SaveChanges();
        var visibles = context.OrdenesFabricacion.Count();
        var todas = context.OrdenesFabricacion.IgnoreQueryFilters().Count();
        global::System.Console.WriteLine($"2.10 OK | Visibles: {visibles} | Sin filtro: {todas}");
    }
}

```

Explicación línea a línea del código acumulativo 2.10

Las referencias usan la numeración real de M02/PROYECTO/2.10/Program.cs. Se explican todas las líneas no vacías del archivo que compila.

Línea 1: `using System.ComponentModel.DataAnnotations;` → importa el espacio de nombres necesario para este archivo.

Línea 2: `using System.ComponentModel.DataAnnotations.Schema;` → importa el espacio de nombres necesario para este archivo.

Línea 3: `using Microsoft.EntityFrameworkCore;` → importa el espacio de nombres necesario para este archivo.

Línea 4: `using Microsoft.Extensions.Configuration;` → importa el espacio de nombres necesario para este archivo.

Línea 5: `using Microsoft.Extensions.DependencyInjection;` → importa el espacio de nombres necesario para este archivo.

Línea 6: `using Microsoft.Extensions.Logging;` → importa el espacio de nombres necesario para este archivo.

Línea 8: `namespace AceriaData.ConsoleApp;` → declara el espacio de nombres del estado acumulativo.

Línea 10: `[Table("OrdenesFabricacion")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 11: `public class OrdenFabricacion` → declara la clase OrdenFabricacion que forma parte del estado acumulativo.

Línea 12: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 13: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 14: `public int Id { get; set; }` → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.

Línea 15: `[Required]` → marca la propiedad siguiente como requerida.

Línea 16: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 17: `public string NumeroOrden { get; set; } = string.Empty;` → declara la propiedad NumeroOrden de tipo string usada por el modelo o la lógica de negocio.

Línea 18: `[Required]` → marca la propiedad siguiente como requerida.

Línea 19: `[MaxLength(200)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 20: `public string Cliente { get; set; } = string.Empty;` → declara la propiedad Cliente de tipo string usada por el modelo o la lógica de negocio.

Línea 21: `public DateTime FechaCreacion { get; set; }` → declara la propiedad FechaCreacion de tipo DateTime usada por el modelo o la lógica de negocio.

Línea 22: `public DateTime? FechaEntrega { get; set; }` → declara la propiedad FechaEntrega de tipo DateTime? usada por el modelo o la lógica de negocio.

Línea 23: `[Required]` → marca la propiedad siguiente como requerida.

Línea 24: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 25: `public string Estado { get; set; } = "Pendiente";` → declara la propiedad Estado de tipo string usada por el modelo o la lógica de negocio.

Línea 26: `[MaxLength(500)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 27: `public string? Observaciones { get; set; }` → declara la propiedad Observaciones de tipo string? usada por el modelo o la lógica de negocio.

Línea 28: `public List<PlanchaAcero> Planchas { get; set; } = new();` → declara la propiedad Planchas de tipo List usada por el modelo o la lógica de negocio.

Línea 29: `public DetalleOrden? Detalle { get; set; }` → declara la propiedad Detalle de tipo DetalleOrden? usada por el modelo o la lógica de negocio.

Línea 30: `public CertificadoCalidad? Certificado { get; set; }` → declara la propiedad Certificado de tipo CertificadoCalidad? usada por el modelo o la lógica de negocio.

Línea 31: `public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();` → declara la propiedad OrdenesAleaciones de tipo List usada por el modelo o la lógica de negocio.

Línea 32: `}` → cierra el bloque de código actual.

Línea 34: `[Table("PlanchasAcero")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 35: `public class PlanchaAcero` → declara la clase PlanchaAcero que forma parte del estado acumulativo.

Línea 36: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 37: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 38: `public int Id { get; set; }` → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.

Línea 39: `public int OrdenId { get; set; }` → declara la propiedad OrdenId de tipo int usada por el modelo o la lógica de negocio.

Línea 40: `public double Espesor { get; set; }` → declara la propiedad Espesor de tipo double usada por el modelo o la lógica de negocio.

Línea 41: `public double Ancho { get; set; }` → declara la propiedad Ancho de tipo double usada por el modelo o la lógica de negocio.

Línea 42: `public double Largo { get; set; }` → declara la propiedad Largo de tipo double usada por el modelo o la lógica de negocio.

Línea 43: `[Precision(18, 3)]` → configura precisión y escala para la propiedad numérica siguiente.

Línea 44: `public decimal Peso { get; set; }` → declara la propiedad Peso de tipo decimal usada por el modelo o la lógica de negocio.

Línea 45: `public bool Activa { get; set; } = true;` → declara la propiedad Activa de tipo bool usada por el modelo o la lógica de negocio.

Línea 46: `[ForeignKey(nameof(OrdenId))]` → vincula la navegación siguiente con su clave foránea.

Línea 47: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo OrdenFabricacion usada por el modelo o la lógica de negocio.

Línea 48: `}` → cierra el bloque de código actual.

Línea 50: `[Table("Aleaciones")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 51: `public class Aleacion` → declara la clase Aleacion que forma parte del estado acumulativo.

Línea 52: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 53: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 54: `public int Id { get; set; }` → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.

Línea 55: `[Required]` → marca la propiedad siguiente como requerida.

Línea 56: `[MaxLength(100)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 57: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad Nombre de tipo string usada por el modelo o la lógica de negocio.

Línea 58: `[Required]` → marca la propiedad siguiente como requerida.

Línea 59: `[MaxLength(20)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 60: `public string Codigo { get; set; } = string.Empty;` → declara la propiedad Codigo de tipo string usada por el modelo o la lógica de negocio.

Línea 61: `public double PorcentajeCarbono { get; set; }` → declara la propiedad PorcentajeCarbono de tipo double usada por el modelo o la lógica de negocio.

Línea 62: `public double PorcentajeManganeso { get; set; }` → declara la propiedad PorcentajeManganeso de tipo double usada por el modelo o la lógica de negocio.

Línea 63: `[MaxLength(500)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 64: `public string? Descripcion { get; set; }` → declara la propiedad Descripcion de tipo string? usada por el modelo o la lógica de negocio.

Línea 65: `public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();` → declara la propiedad OrdenesAleaciones de tipo List usada por el modelo o la lógica de negocio.

Línea 66: `}` → cierra el bloque de código actual.

Línea 68: `[Table("EstadosOrden")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 69: `public class EstadoOrden` → declara la clase EstadoOrden que forma parte del estado acumulativo.

Línea 70: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 71: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 72: `public int Id { get; set; }` → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.

Línea 73: `[Required]` → marca la propiedad siguiente como requerida.

Línea 74: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 75: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad Nombre de tipo string usada por el modelo o la lógica de negocio.

Línea 76: `[MaxLength(250)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 77: `public string Descripcion { get; set; } = string.Empty;` → declara la propiedad Descripcion de tipo string usada por el modelo o la lógica de negocio.

Línea 78: `public bool Activo { get; set; } = true;` → declara la propiedad Activo de tipo bool usada por el modelo o la lógica de negocio.

Línea 79: `}` → cierra el bloque de código actual.

Línea 81: `[Table("DetallesOrden")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 82: `public class DetalleOrden` → declara la clase DetalleOrden que forma parte del estado acumulativo.

Línea 83: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 84: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 85: `public int Id { get; set; }` → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.

Línea 86: `public int OrdenId { get; set; }` → declara la propiedad OrdenId de tipo int usada por el modelo o la lógica de negocio.

Línea 87: `[Required]` → marca la propiedad siguiente como requerida.

Línea 88: `[MaxLength(200)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 89: `public string ComposicionQuimica { get; set; } = string.Empty;` → declara la propiedad ComposicionQuimica de tipo string usada por el modelo o la lógica de negocio.

Línea 90: `public double TemperaturaColada { get; set; }` → declara la propiedad TemperaturaColada de tipo double usada por el modelo o la lógica de negocio.

Línea 91: `[MaxLength(500)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 92: `public string? Notas { get; set; }` → declara la propiedad Notas de tipo string? usada por el modelo o la lógica de negocio.

Línea 93: `[ForeignKey(nameof(OrdenId))]` → vincula la navegación siguiente con su clave foránea.

Línea 94: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo OrdenFabricacion usada por el modelo o la lógica de negocio.

Línea 95: `}` → cierra el bloque de código actual.

Línea 97: `[Table("CertificadosCalidad")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 98: `public class CertificadoCalidad` → declara la clase CertificadoCalidad que forma parte del estado acumulativo.

Línea 99: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 100: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 101: `public int Id { get; set; }` → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.

Línea 102: `public int OrdenId { get; set; }` → declara la propiedad OrdenId de tipo int usada por el modelo o la lógica de negocio.

Línea 103: `[Required]` → marca la propiedad siguiente como requerida.

Línea 104: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 105: `public string NumeroCertificado { get; set; } = string.Empty;` → declara la propiedad `NumeroCertificado` de tipo `string` usada por el modelo o la lógica de negocio.

Línea 106: `public DateTime FechaEmision { get; set; }` → declara la propiedad `FechaEmision` de tipo `DateTime` usada por el modelo o la lógica de negocio.

Línea 107: `[Required]` → marca la propiedad siguiente como requerida.

Línea 108: `[MaxLength(100)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 109: `public string OrganismoCertificador { get; set; } = string.Empty;` → declara la propiedad `OrganismoCertificador` de tipo `string` usada por el modelo o la lógica de negocio.

Línea 110: `[ForeignKey(nameof(OrdenId))]` → vincula la navegación siguiente con su clave foránea.

Línea 111: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad `Orden` de tipo `OrdenFabricacion` usada por el modelo o la lógica de negocio.

Línea 112: `}` → cierra el bloque de código actual.

Línea 114: `[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]` → declara una clave primaria compuesta.

Línea 115: `[Table("OrdenesAleaciones")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 116: `public class OrdenAleacion` → declara la clase `OrdenAleacion` que forma parte del estado acumulativo.

Línea 117: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 118: `public int OrdenFabricacionId { get; set; }` → declara la propiedad `OrdenFabricacionId` de tipo `int` usada por el modelo o la lógica de negocio.

Línea 119: `public int AleacionId { get; set; }` → declara la propiedad `AleacionId` de tipo `int` usada por el modelo o la lógica de negocio.

Línea 120: `public DateTime FechaAsignacion { get; set; } = DateTime.Now;` → declara la propiedad `FechaAsignacion` de tipo `DateTime` usada por el modelo o la lógica de negocio.

Línea 121: `[Precision(18, 3)]` → configura precisión y escala para la propiedad numérica siguiente.

Línea 122: `public decimal CantidadUtilizada { get; set; }` → declara la propiedad `CantidadUtilizada` de tipo `decimal` usada por el modelo o la lógica de negocio.

Línea 123: `[MaxLength(20)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 124: `public string EstadoRelacion { get; set; } = "Activa";` → declara la propiedad `EstadoRelacion` de tipo `string` usada por el modelo o la lógica de negocio.

Línea 125: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad `Orden` de tipo `OrdenFabricacion` usada por el modelo o la lógica de negocio.

Línea 126: `public Aleacion Aleacion { get; set; } = null!;` → declara la propiedad `Aleacion` de tipo `Aleacion` usada por el modelo o la lógica de negocio.

Línea 127: `}` → cierra el bloque de código actual.

Línea 129: `public class AceriaDbContext : DbContext` → declara la clase `AceriaDbContext` que forma parte del estado acumulativo.

Línea 130: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 131:

`public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }` → recibe las opciones del `DbContext` y las pasa a la clase base.

Línea 133: `public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 134: `public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 135: `public DbSet<Aleacion> Aleaciones => Set<Aleacion>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 136: `public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 137: `public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 138: `public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 139: `public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 142: `protected override void OnModelCreating(ModelBuilder modelBuilder)` → entra en la configuración explícita del modelo.

Línea 143: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 144: `base.OnModelCreating(modelBuilder);` → entra en la configuración explícita del modelo.

Línea 145: `modelBuilder.Entity<OrdenFabricacion>(entity =>` → selecciona una entidad para configurarla con Fluent API.

Línea 146: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 147: `entity.ToTable("OrdenesFabricacion");` → asigna explícitamente la tabla usada por la entidad.

Línea 148: `entity.HasKey(o => o.Id);` → define la clave primaria.

Línea 149: `entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 150: `entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);` → selecciona una propiedad escalar para continuar su configuración.

Línea 151: `entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");` → selecciona una propiedad escalar para continuar su configuración.

Línea 152: `entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");` → selecciona una propiedad escalar para continuar su configuración.

Línea 153: `entity.Property(o => o.Observaciones).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 154: `entity.HasAlternateKey(o => o.NumeroOrden).HasName("AK_OrdenesFabricacion_NumeroOrden");` → define una clave alternativa única.

Línea 155: `entity.HasIndex(o => o.Cliente).HasDatabaseName("IX_OrdenesFabricacion_Cliente");` → define un índice sobre las propiedades indicadas.

Línea 156: `entity.HasIndex(o => new { o.Cliente, o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Cliente_FechaCreacion");` → define un índice sobre las propiedades indicadas.

Línea 157: `entity.HasIndex(o => o.FechaEntrega).HasFilter("[Estado] = 'Pendiente']").HasDatabaseName("IX_OrdenesFabricacion_FechaEntrega_Pendientes");` → define un índice sobre las propiedades indicadas.

Línea 158: `entity.HasIndex(o => o.Estado).IncludeProperties(o => new { o.NumeroOrden, o.Cliente, o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Estado_Incluye");` → define un índice sobre las propiedades indicadas.

Línea 159: `entity.HasQueryFilter(o => o.Estado != "Cancelada");` → aplica un filtro global de consulta.

Línea 160: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 161: `modelBuilder.Entity<PlanchaAcero>(entity =>` → selecciona una entidad para configurarla con Fluent API.

Línea 162: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 163: `entity.ToTable("PlanchasAcero", t =>` → asigna explícitamente la tabla usada por la entidad.

Línea 164: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 165: `t.HasCheckConstraint("CK_PlanchasAcero_Espesor", "[Espesor] > 0");` → añade una restricción CHECK en SQL Server.

Línea 166: `t.HasCheckConstraint("CK_PlanchasAcero_Ancho", "[Ancho] > 0");` → añade una restricción CHECK en SQL Server.

Línea 167: `t.HasCheckConstraint("CK_PlanchasAcero_Largo", "[Largo] > 0");` → añade una restricción CHECK en SQL Server.

Línea 168: `t.HasCheckConstraint("CK_PlanchasAcero_Peso", "[Peso] > 0");` → añade una restricción CHECK en SQL Server.

Línea 169: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 170: `entity.HasKey(x => x.Id);` → define la clave primaria.

Línea 171: `entity.Property(x => x.Peso).HasPrecision(18, 3);` → selecciona una propiedad escalar para continuar su configuración.

Línea 172: `entity.Property(x => x.Activa).HasDefaultValue(true);` → selecciona una propiedad escalar para continuar su configuración.

Línea 173: `entity.HasOne(x => x.Orden)` → inicia la configuración de una navegación de referencia.

Línea 174: `.WithMany(o => o.Planchas)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 175: `.HasForeignKey(x => x.OrdenId)` → declara explícitamente la clave foránea.

Línea 176: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 177: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 178: `entity.HasIndex(x => new { x.OrdenId, x.Activa }).HasDatabaseName("IX_PlanchasAcero_OrdenId_Activa");` → define un índice sobre las propiedades indicadas.

Línea 179: `entity.HasIndex(x => x.Espesor).HasFilter("[Activa] = 1").HasDatabaseName("IX_PlanchasAcero_Espesor_Activas");` → define un índice sobre las propiedades indicadas.

Línea 180: `entity.HasQueryFilter(x => x.Activa);` → aplica un filtro global de consulta.

Línea 181: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 182: `modelBuilder.Entity<Aleacion>(entity =>` → selecciona una entidad para configurarla con Fluent API.

Línea 183: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 184: `entity.ToTable("Aleaciones", t =>` → asigna explícitamente la tabla usada por la entidad.

Línea 185: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 186: `t.HasCheckConstraint("CK_Aleaciones_PorcentajeCarbono", "[PorcentajeCarbono] >= 0 AND [PorcentajeCarbono] <= 2");` → añade una restricción CHECK en SQL Server.

Línea 187: `t.HasCheckConstraint("CK_Aleaciones_PorcentajeManganeso", "[PorcentajeManganeso] >= 0 AND [PorcentajeManganeso] <= 5");` → añade una restricción CHECK en SQL Server.

Línea 188: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 189: `entity.HasKey(a => a.Id);` → define la clave primaria.

Línea 190: `entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);` → selecciona una propiedad escalar para continuar su configuración.

Línea 191: `entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);` → selecciona una propiedad escalar para continuar su configuración.

Línea 192: `entity.Property(a => a.Descripcion).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 193: `entity.HasAlternateKey(a => a.Codigo).HasName("AK_Aleaciones_Codigo");` → define una clave alternativa única.

Línea 194: `entity.HasIndex(a => a.Nombre).HasDatabaseName("IX_Aleaciones_Nombre");` → define un índice sobre las propiedades indicadas.

Línea 195: `entity.HasIndex(a => new { a.PorcentajeCarbono, a.PorcentajeManganeso }).HasDatabaseName("IX_Aleaciones_Porcentajes");` → define un índice sobre las propiedades indicadas.

Línea 196: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 197: `modelBuilder.Entity<EstadoOrden>(entity =>` → selecciona una entidad para configurarla con Fluent API.

Línea 198: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 199: `entity.ToTable("EstadosOrden");` → asigna explícitamente la tabla usada por la entidad.

Línea 200: `entity.HasKey(e => e.Id);` → define la clave primaria.

Línea 201: `entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 202: `entity.Property(e => e.Descripcion).HasMaxLength(250);` → selecciona una propiedad escalar para continuar su configuración.

Línea 203: `entity.Property(e => e.Activo).HasDefaultValue(true);` → selecciona una propiedad escalar para continuar su configuración.

Línea 204: `entity.HasQueryFilter(e => e.Activo);` → aplica un filtro global de consulta.

Línea 205: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 206: `modelBuilder.Entity<DetalleOrden>(entity =>` → selecciona una entidad para configurarla con Fluent API.

Línea 207: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 208: `entity.ToTable("DetallesOrden");` → asigna explícitamente la tabla usada por la entidad.

Línea 209: `entity.HasKey(d => d.Id);` → define la clave primaria.

Línea 210: `entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);` → selecciona una propiedad escalar para continuar su configuración.

Línea 211: `entity.Property(d => d.Notas).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 212: `entity.HasOne(d => d.Orden)` → inicia la configuración de una navegación de referencia.

Línea 213: `.WithOne(o => o.Detalle)` → establece el extremo de referencia de una relación uno-a-uno.

Línea 214: `.HasForeignKey<DetalleOrden>(d => d.OrdenId)` → declara explícitamente la clave foránea.

Línea 215: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 216: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 217: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 219: `modelBuilder.Entity<CertificadoCalidad>(entity =>` → selecciona una entidad para configurarla con Fluent API.

Línea 220: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 221: `entity.ToTable("CertificadosCalidad");` → asigna explícitamente la tabla usada por la entidad.

Línea 222: `entity.HasKey(c => c.Id);` → define la clave primaria.

Línea 223: `entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 224: `entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);` → selecciona una propiedad escalar para continuar su configuración.

Línea 225: `entity.HasAlternateKey(c => c.NumeroCertificado).HasName("AK_CertificadosCalidad_NumeroCertificado");` → define una clave alternativa única.

Línea 226: `entity.HasIndex(c => c.FechaEmision).HasDatabaseName("IX_CertificadosCalidad_FechaEmision");` → define un índice sobre las propiedades indicadas.

Línea 227: `entity.HasOne(c => c.Orden)` → inicia la configuración de una navegación de referencia.

Línea 228: `.WithOne(o => o.Certificado)` → establece el extremo de referencia de una relación uno-a-uno.

Línea 229: `.HasForeignKey<CertificadoCalidad>(c => c.OrdenId)` → declara explícitamente la clave foránea.

Línea 230: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 231: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 232: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 233: `modelBuilder.Entity<OrdenAleacion>(entity =>` → selecciona una entidad para configurarla con Fluent API.

Línea 234: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 235: `entity.ToTable("OrdenesAleaciones");` → asigna explícitamente la tabla usada por la entidad.

Línea 236: `entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });` → define la clave primaria.

Línea 237: `entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");` → selecciona una propiedad escalar para continuar su configuración.

Línea 238: `entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);` → selecciona una propiedad escalar para continuar su configuración.

Línea 239: `entity.Property(x => x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");` → selecciona una propiedad escalar para continuar su configuración.

Línea 240: `entity.HasOne(x => x.Orden)` → inicia la configuración de una navegación de referencia.

Línea 241: `.WithMany(o => o.OrdenesAleaciones)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 242: `.HasForeignKey(x => x.OrdenFabricacionId)` → declara explícitamente la clave foránea.

Línea 243: `.OnDelete(DeleteBehavior.Cascade);` → define el comportamiento de eliminación de la relación.

Línea 244: `entity.HasOne(x => x.Aleacion)` → inicia la configuración de una navegación de referencia.

Línea 245: `.WithMany(a => a.OrdenesAleaciones)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 246: `.HasForeignKey(x => x.AleacionId)` → declara explícitamente la clave foránea.

Línea 247: `.OnDelete(DeleteBehavior.Restrict);` → define el comportamiento de eliminación de la relación.

Línea 248: `entity.HasIndex(x => x.AleacionId).HasDatabaseName("IX_OrdenesAleaciones_AleacionId");` → define un índice sobre las propiedades indicadas.

Línea 249: `entity.HasIndex(x => x.EstadoRelacion).HasFilter("[EstadoRelacion] = 'Activa'").HasDatabaseName("IX_OrdenesAleaciones_EstadoRelacion_Activas");` → define un índice sobre las propiedades indicadas.

Línea 250: `entity.HasQueryFilter(x => x.EstadoRelacion == "Activa");` → aplica un filtro global de consulta.

Línea 251: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 252: `}` → cierra el bloque de código actual.

Línea 253: `}` → cierra el bloque de código actual.

Línea 255: `public interface IOrdenRepositorio` → declara el contrato `IOrdenRepositorio` usado por la aplicación.

Línea 256: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 257: `List<OrdenFabricacion> ObtenerTodas();` → declara en el contrato la operación que debe devolver todas las órdenes visibles para el repositorio.

Línea 258: `OrdenFabricacion? ObtenerPorId(int id);` → declara en el contrato la búsqueda de una orden por su clave primaria y permite devolver null si no existe.

Línea 259: `OrdenFabricacion? ObtenerPorNumero(string numeroOrden);` → declara en el contrato la búsqueda por el identificador de negocio `NumeroOrden`.

Línea 260: `void Agregar(OrdenFabricacion orden);` → declara la operación que incorpora una orden al repositorio para que EF Core la siga como nueva.

Línea 261: `void Eliminar(OrdenFabricacion orden);` → declara la operación que marca una orden para eliminación física cuando el escenario lo requiere.

Línea 262: `int Guardar();` → declara la operación que confirma los cambios pendientes y devuelve el número de entradas afectadas.

Línea 263: `}` → cierra el bloque de código actual.

Línea 265: `public sealed class OrdenRepositorio : IOrdenRepositorio` → declara la clase `OrdenRepositorio` que forma parte del estado acumulativo.

Línea 266: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 267: `private readonly AceriaDbContext _context;` → declara el campo privado `_context` de tipo `AceriaDbContext`.

Línea 268: `public OrdenRepositorio(AceriaDbContext context) => _context = context;` → inyecta `AceriaDbContext` en el repositorio y lo conserva como dependencia de persistencia.

Línea 270: `public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();` → ordena el resultado antes de materializarlo.

Línea 271: `public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);` → implementa la búsqueda por clave primaria mediante `Find`, aprovechando primero el `ChangeTracker` y después la base de datos.

Línea 272: `public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);` → recupera la primera coincidencia o `null`.

Línea 273: `public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);` → registra una entidad nueva para insertarla al guardar.

Línea 274: `public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);` → marca una entidad para borrado al guardar.

Línea 275: `public int Guardar() => _context.SaveChanges();` → confirma en SQL Server los cambios del `ChangeTracker`.

Línea 276: `}` → cierra el bloque de código actual.

Línea 278: `public interface IServicioOrdenes` → declara el contrato `IServicioOrdenes` usado por la aplicación.

Línea 279: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 280: `string ObtenerResumen(int ordenId);` → declara el servicio que construye un resumen de una orden a partir de su identificador.

Línea 281: `}` → cierra el bloque de código actual.

Línea 283: `public sealed class ServicioOrdenes : IServicioOrdenes` → declara la clase `ServicioOrdenes` que forma parte del estado acumulativo.

Línea 284: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 285: `private readonly IOrdenRepositorio _repositorio;` → declara el campo privado `_repositorio` de tipo `IOrdenRepositorio`.

Línea 286: `private readonly AceriaDbContext _context;` → declara el campo privado `_context` de tipo `AceriaDbContext`.

Línea 287: `public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)` → declara el constructor que recibe por inyección el repositorio de órdenes y el `DbContext`.

Línea 288: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 289: `_repositorio = repositorio;` → guarda en el servicio el repositorio inyectado para reutilizarlo en sus operaciones.

Línea 290: `_context = context;` → guarda el DbContext inyectado para consultar los datos necesarios durante el servicio.

Línea 291: `}` → cierra el bloque de código actual.

Línea 293: `public string ObtenerResumen(int ordenId)` → declara el método que obtiene la orden y compone el resumen solicitado por el servicio.

Línea 294: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 295: `var orden = _repositorio.ObtenerPorId(ordenId);` → declara una variable local y almacena el resultado de la expresión.

Línea 296: `if (orden is null) return $"Orden {ordenId} no encontrada";` → evalúa una condición antes de ejecutar el bloque asociado.

Línea 297: `var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);` → cuenta las filas que cumplen la consulta.

Línea 298: `return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";` → devuelve el resultado al código llamador.

Línea 299: `}` → cierra el bloque de código actual.

Línea 300: `}` → cierra el bloque de código actual.

Línea 302: `public static class Program` → declara la clase Program que forma parte del estado acumulativo.

Línea 303: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 304: `public static void Main()` → declara el punto de entrada ejecutable de la aplicación de consola.

Línea 305: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 306: `var configuration = new ConfigurationBuilder()` → inicia la construcción de la configuración de la aplicación.

Línea 307: `.SetBasePath(AppContext.BaseDirectory)` → establece la ruta base para los archivos de configuración.

Línea 308: `.AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)` → incorpora appsettings.json.

Línea 309: `.AddEnvironmentVariables()` → incorpora variables de entorno.

Línea 310: `.Build();` → materializa el objeto construido por la cadena anterior.

Línea 312: `var connectionString = configuration.GetConnectionString("AceriaDB")` → obtiene la cadena AceriaDB desde configuración.

Línea 313: `?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");` → interrumpe el arranque con un error explícito si la configuración no contiene ConnectionStrings:AceriaDB.

Línea 315: `var services = new ServiceCollection();` → crea la colección de servicios.

Línea 316: `services.AddDbContext<AceriaDbContext>(options =>` → registra el DbContext en inyección de dependencias.

Línea 317: `options` → continúa la configuración del DbContext sobre el parámetro options recibido por la lambda de AddDbContext.

Línea 318: `.UseSqlServer(connectionString, sql =>` → selecciona SQL Server como proveedor de EF Core.

Línea 319: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 320: `sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);` → habilita reintentos ante fallos transitorios.

Línea 321: `sql.CommandTimeout(60);` → establece el timeout de los comandos SQL.

Línea 322: `})` → cierra la lambda de configuración actual y devuelve el control a la llamada encadenada que la contiene.

Línea 323: `.LogTo(global::System.Console.WriteLine, new[]
{ "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)` → configura el logging de EF Core.

Línea 324: `.EnableDetailedErrors();` → activa errores detallados.

Línea 326: `services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();` → registra un servicio con ciclo de vida scoped.

Línea 327: `services.AddScoped<IServicioOrdenes, ServicioOrdenes>();` → registra un servicio con ciclo de vida scoped.

Línea 329: `using var provider = services.BuildServiceProvider(new ServiceProviderOptions` → construye el ServiceProvider con validación de servicios y garantiza su liberación al salir del ámbito mediante using var.

Línea 330: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 331: `ValidateScopes = true,` → activa la validación de ámbitos para detectar dependencias scoped usadas desde ámbitos incorrectos.

Línea 332: `ValidateOnBuild = true` → obliga al contenedor a validar las resoluciones registradas al construir el ServiceProvider.

Línea 333: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 335: `using var scope = provider.CreateScope();` → crea un ámbito de DI para resolver servicios scoped y lo libera automáticamente al terminar el bloque.

Línea 336: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → resuelve un servicio obligatorio del contenedor.

Línea 337: `context.Database.EnsureDeleted();` → reinicia la base del laboratorio para una ejecución reproducible.

Línea 338: `context.Database.Migrate();` → aplica el historial de migraciones pendiente.

Línea 340: `var orden = new OrdenFabricacion` → declara una variable local y almacena el resultado de la expresión.

Línea 341: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 342: `NumeroOrden = "OF-M2-0001",` → asigna el identificador de negocio usado por la orden de validación acumulativa del módulo.

Línea 343: `Cliente = "Constructora del Norte",` → asigna el cliente de la orden utilizada en el escenario práctico del punto.

Línea 344: `FechaCreacion = DateTime.UtcNow,` → asigna a la orden la fecha de creación en UTC para el escenario reproducible del punto.

Línea 345: `FechaEntrega = DateTime.Today.AddDays(14),` → establece una fecha de entrega prevista catorce días después de la fecha local actual.

Línea 346: `Estado = "Pendiente"`, → inicializa la orden con el estado Pendiente, coherente con el flujo del laboratorio.

Línea 347: `Observaciones = "Orden de validación M2"`, → añade texto opcional para comprobar el mapeo de Observaciones.

Línea 348: `Planchas =` → inicia el inicializador de la colección de planchas asociadas a la orden.

Línea 349: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 350: `new PlanchaAcero` → crea una nueva plancha dentro del inicializador de la colección Planchas de la orden.

Línea 351: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 352: `Espesor = 10.5`, → asigna a la plancha un espesor positivo para satisfacer las reglas del modelo.

Línea 353: `Ancho = 1500`, → asigna el ancho de la plancha usada en el escenario de persistencia.

Línea 354: `Largo = 3000`, → asigna el largo de la plancha usada en el escenario de persistencia.

Línea 355: `Peso = 371.250m`, → asigna un peso decimal con tres posiciones para validar la precisión configurada.

Línea 356: `Activa = true`, → marca la plancha como activa para que sea visible en los estados que aplican ese criterio.

Línea 357: `}` → cierra el bloque de código actual.

Línea 358: `},` → continúa el inicializador o la llamada del bloque actual.

Línea 359: `Detalle = new DetalleOrden` → crea el detalle uno-a-uno que se persistirá junto con la orden principal.

Línea 360: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 361: `ComposicionQuimica = "C: 0.45%, Mn: 0.75%"`, → asigna la composición química utilizada para verificar la entidad DetalleOrden.

Línea 362: `TemperaturaColada = 1550.5`, → asigna la temperatura de colada del detalle de fabricación.

Línea 363: `Notas = "Colada principal"` → asigna las notas opcionales del detalle asociado a la orden.

Línea 364: `},` → continúa el inicializador o la llamada del bloque actual.

Línea 365: `Certificado = new CertificadoCalidad` → crea el certificado uno-a-uno asociado a la orden del escenario.

Línea 366: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 367: `NumeroCertificado = "CERT-0001"`, → asigna el identificador natural del certificado de calidad.

Línea 368: `FechaEmision = DateTime.Today`, → asigna la fecha de emisión del certificado usado en la prueba.

Línea 369: `OrganismoCertificador = "Laboratorio Aceria"` → asigna el organismo que emite el certificado de calidad.

Línea 370: `},` → continúa el inicializador o la llamada del bloque actual.

Línea 371: `};` → cierra el inicializador del objeto y finaliza la instrucción.

Línea 373: `var aleacion = new Aleacion` → declara una variable local y almacena el resultado de la expresión.

Línea 374: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 375: `Nombre = "AISI 1045"`, → asigna el nombre comercial de la aleación utilizada en el escenario.

Línea 376: `Codigo = "A1045"`, → asigna el código natural único de la aleación.

Línea 377: `PorcentajeCarbono = 0.45`, → asigna el porcentaje de carbono dentro del rango permitido por la restricción del modelo.

Línea 378: `PorcentajeManganeso = 0.75`, → asigna el porcentaje de manganeso dentro del rango permitido por la restricción del modelo.

Línea 379: `Descripcion = "Acero medio en carbono"` → asigna una descripción opcional a la aleación.

Línea 380: `};` → cierra el inicializador del objeto y finaliza la instrucción.

Línea 381: `orden.OrdenesAleaciones.Add(new OrdenAleacion` → registra una entidad nueva para insertarla al guardar.

Línea 382: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 383: `Aleacion = aleacion`, → conecta la entidad intermedia `OrdenAleacion` con la aleación creada para el escenario.

Línea 384: `CantidadUtilizada = 1500.500m`, → registra la cantidad utilizada con tres decimales, coherente con `HasPrecision(18, 3)`.

Línea 385: `EstadoRelacion = "Activa"` → marca como activa la relación entre la orden y la aleación.

Línea 386: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 387: `context.OrdenesFabricacion.Add(orden);` → registra una entidad nueva para insertarla al guardar.

Línea 388: `context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo = true });` → registra una entidad nueva para insertarla al guardar.

Línea 389: `context.SaveChanges();` → confirma en SQL Server los cambios del `ChangeTracker`.

Línea 391: `var cancelada = new OrdenFabricacion` → declara una variable local y almacena el resultado de la expresión.

Línea 392: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 393: `NumeroOrden = "0F-M2-CANCELADA"`, → asigna el número de la orden de control que se crea con estado Cancelada.

Línea 394: `Cliente = "Cliente Histórico"`, → asigna el cliente de la orden histórica utilizada para comprobar el filtro global.

Línea 395: `FechaCreacion = DateTime.UtcNow`, → asigna a la orden la fecha de creación en UTC para el escenario reproducible del punto.

Línea 396: `Estado = "Cancelada"` → establece el estado Cancelada para verificar que `HasQueryFilter` excluye la orden de las consultas normales.

Línea 397: `};` → cierra el inicializador del objeto y finaliza la instrucción.

Línea 398: `context.OrdenesFabricacion.Add(cancelada);` → registra una entidad nueva para insertarla al guardar.

Línea 399: `context.SaveChanges();` → confirma en SQL Server los cambios del `ChangeTracker`.

Línea 400: `var visibles = context.OrdenesFabricacion.Count();` → cuenta las filas que cumplen la consulta.

Línea 401: `var todas = context.OrdenesFabricacion.IgnoreQueryFilters().Count();` → omite de forma explícita los filtros globales para esta consulta.

Línea 402: `global::System.Console.WriteLine($"2.10 OK | Visibles: {visibles} | Sin filtro: {todas}");` → escribe la evidencia de ejecución usada para validar este estado.

Línea 403: `}` → cierra el bloque de código actual.

Línea 404: `}` → cierra el bloque de código actual.

Punto 2.11 - Migraciones en el modelado: ciclo completo con Soft Delete

Audiencia: Desarrolladores que ya han completado los puntos 2.1 a 2.10. Proyecto: Se utiliza el cambio real de Soft Delete para aprender el ciclo completo de una migración incremental: generar, revisar, aplicar, inspeccionar el historial, revertir, reaplicar, retirar una migración local y generar scripts SQL normales e idempotentes.

Objetivos de aprendizaje

Generar una migración incremental sobre el estado 2.10. Revisar línea a línea la migración `M2_2_11`. Comprobar el snapshot y `__EFMigrationsHistory`. Aplicar y revertir el esquema hasta una migración concreta. Entender el uso seguro de `migrations remove`. Generar scripts SQL e idempotentes. Validar que el esquema resultante soporta el Soft Delete y su restauración.

Paso 1: Abrir la solución autónoma del punto

```
cd M02/PROYECTO/2.11
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

`cd` → entra en el estado acumulativo 2.11. `dotnet restore` → restaura EF Core 8 y el proveedor SQL Server. `dotnet build` → comprueba que modelo, migraciones y fábrica de diseño compilan juntos.

Paso 2: Comparar el modelo 2.10 con el cambio funcional 2.11

En 2.10 existen filtros globales, pero todavía no existen las columnas persistentes del borrado lógico. El estado 2.11 añade:

```
public bool IsDeleted { get; set; }
public DateTime? DeletedAt { get; set; }
```

Estas propiedades aparecen en `OrdenFabricacion`, `PlanchaAcero`, `Aleacion` y `EstadoOrden`. Después se combinan con `HasQueryFilter` para que las consultas normales excluyan registros eliminados.

Paso 3: Generar la migración incremental al construir el estado 2.11

Al partir físicamente del estado 2.10, después de introducir las propiedades y filtros de 2.11, el comando es:

```
dotnet ef migrations add M2_2_11
```

`dotnet ef migrations add` → compara el modelo modificado con el snapshot de 2.10. `M2_2_11` → identifica el checkpoint que introduce Soft Delete. En el repositorio entregado la migración ya está generada porque 2.11 es un estado completo y reproducible. No se debe volver a ejecutar `migrations add M2_2_11` sobre el estado final: el delta original ya está registrado.

Paso 4: Revisar la migración real antes de aplicarla

Abrir `Migrations/20260927204841_M2_2_11.cs`:

```
using System;
using Microsoft.EntityFrameworkCore.Migrations;

#nullable disable

namespace AceriaData.ConsoleApp.Migrations
{
```

```

/// <inheritdoc />
public partial class M2_2_11 : Migration
{
    /// <inheritdoc />
    protected override void Up(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.AddColumn<DateTime>(
            name: "DeletedAt",
            table: "PlanchasAcero",
            type: "datetime2",
            nullable: true);

        migrationBuilder.AddColumn<bool>(
            name: "IsDeleted",
            table: "PlanchasAcero",
            type: "bit",
            nullable: false,
            defaultValue: false);

        migrationBuilder.AddColumn<DateTime>(
            name: "DeletedAt",
            table: "OrdenesFabricacion",
            type: "datetime2",
            nullable: true);

        migrationBuilder.AddColumn<bool>(
            name: "IsDeleted",
            table: "OrdenesFabricacion",
            type: "bit",
            nullable: false,
            defaultValue: false);

        migrationBuilder.AddColumn<DateTime>(
            name: "DeletedAt",
            table: "EstadosOrden",
            type: "datetime2",
            nullable: true);

        migrationBuilder.AddColumn<bool>(
            name: "IsDeleted",
            table: "EstadosOrden",
            type: "bit",
            nullable: false,
            defaultValue: false);

        migrationBuilder.AddColumn<DateTime>(
            name: "DeletedAt",
            table: "Aleaciones",
            type: "datetime2",
            nullable: true);

        migrationBuilder.AddColumn<bool>(
            name: "IsDeleted",
            table: "Aleaciones",
            type: "bit",
            nullable: false,
            defaultValue: false);
    }

    /// <inheritdoc />
    protected override void Down(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.DropColumn(
            name: "DeletedAt",
            table: "PlanchasAcero");

        migrationBuilder.DropColumn(
            name: "IsDeleted",
            table: "PlanchasAcero");

        migrationBuilder.DropColumn(
            name: "DeletedAt",
            table: "OrdenesFabricacion");
    }
}

```

```

        migrationBuilder.DropColumn(
            name: "IsDeleted",
            table: "OrdenesFabricacion");

        migrationBuilder.DropColumn(
            name: "DeletedAt",
            table: "EstadosOrden");

        migrationBuilder.DropColumn(
            name: "IsDeleted",
            table: "EstadosOrden");

        migrationBuilder.DropColumn(
            name: "DeletedAt",
            table: "Aleaciones");

        migrationBuilder.DropColumn(
            name: "IsDeleted",
            table: "Aleaciones");
    }
}

```

Explicación línea a línea de la migración M2_2_11

Línea 1: `using System;` → importa los tipos base de .NET necesarios para las columnas DateTime.

Línea 2: `using Microsoft.EntityFrameworkCore.Migrations;` → importa `Migration` y `MigrationBuilder`, la API que expresa cambios de esquema.

Línea 4: `#nullable disable` → mantiene el contexto de nulabilidad generado por EF Core para este archivo.

Línea 6: `namespace AceriaData.ConsoleApp.Migrations` → declara el espacio de nombres del historial de migraciones del contexto.

Línea 7: `{` → abre el bloque de la declaración o método anterior.

Línea 8: `/// <inheritdoc />` → comentario XML generado automáticamente para documentar el miembro siguiente.

Línea 9: `public partial class M2_2_11 : Migration` → declara la migración incremental correspondiente al checkpoint 2.11.

Línea 10: `{` → abre el bloque de la declaración o método anterior.

Línea 11: `/// <inheritdoc />` → comentario XML generado automáticamente para documentar el miembro siguiente.

Línea 12: `protected override void Up(MigrationBuilder migrationBuilder)` → define las operaciones que llevan el esquema desde 2.10 hasta 2.11.

Línea 13: `{` → abre el bloque de la declaración o método anterior.

Línea 14: `migrationBuilder.AddColumn<DateTime>()` → inicia la adición de una columna `DeletedAt` de fecha/hora.

Línea 15: `name: "DeletedAt",` → identifica la columna afectada como `DeletedAt`.

Línea 16: `table: "PlanchasAcero",` → dirige la operación a la tabla `PlanchasAcero`.

Línea 17: `type: "datetime2",` → fija el tipo SQL Server `datetime2`.

Línea 18: `nullable: true);` → `DeletedAt` admite NULL mientras el registro no esté eliminado.

Línea 20: `migrationBuilder.AddColumn<bool>()` → inicia la adición de una columna `IsDeleted` booleana.

Línea 21: `name: "IsDeleted",` → identifica la columna afectada como `IsDeleted`.

Línea 22: `table: "PlanchasAcero",` → dirige la operación a la tabla `PlanchasAcero`.

Línea 23: `type: "bit",` → fija el tipo SQL Server `bit`.

Línea 24: `nullable: false,` → `IsDeleted` es obligatorio para todas las filas.

Línea 25: `defaultValue: false);` → rellena las filas existentes con `false` para mantenerlas activas al aplicar el cambio.

Línea 27: `migrationBuilder.AddColumn<DateTime>(` → inicia la adición de una columna `DeletedAt` de fecha/hora.

Línea 28: `name: "DeletedAt",` → identifica la columna afectada como `DeletedAt`.

Línea 29: `table: "OrdenesFabricacion",` → dirige la operación a la tabla `OrdenesFabricacion`.

Línea 30: `type: "datetime2",` → fija el tipo SQL Server `datetime2`.

Línea 31: `nullable: true);` → `DeletedAt` admite NULL mientras el registro no esté eliminado.

Línea 33: `migrationBuilder.AddColumn<bool>(` → inicia la adición de una columna `IsDeleted` booleana.

Línea 34: `name: "IsDeleted",` → identifica la columna afectada como `IsDeleted`.

Línea 35: `table: "OrdenesFabricacion",` → dirige la operación a la tabla `OrdenesFabricacion`.

Línea 36: `type: "bit",` → fija el tipo SQL Server `bit`.

Línea 37: `nullable: false,` → `IsDeleted` es obligatorio para todas las filas.

Línea 38: `defaultValue: false);` → rellena las filas existentes con `false` para mantenerlas activas al aplicar el cambio.

Línea 40: `migrationBuilder.AddColumn<DateTime>(` → inicia la adición de una columna `DeletedAt` de fecha/hora.

Línea 41: `name: "DeletedAt",` → identifica la columna afectada como `DeletedAt`.

Línea 42: `table: "EstadosOrden",` → dirige la operación a la tabla `EstadosOrden`.

Línea 43: `type: "datetime2",` → fija el tipo SQL Server `datetime2`.

Línea 44: `nullable: true);` → `DeletedAt` admite NULL mientras el registro no esté eliminado.

Línea 46: `migrationBuilder.AddColumn<bool>(` → inicia la adición de una columna `IsDeleted` booleana.

Línea 47: `name: "IsDeleted",` → identifica la columna afectada como `IsDeleted`.

Línea 48: `table: "EstadosOrden",` → dirige la operación a la tabla `EstadosOrden`.

Línea 49: `type: "bit",` → fija el tipo SQL Server `bit`.

Línea 50: `nullable: false,` → `IsDeleted` es obligatorio para todas las filas.

Línea 51: `defaultValue: false);` → rellena las filas existentes con `false` para mantenerlas activas al aplicar el cambio.

Línea 53: `migrationBuilder.AddColumn<DateTime>(` → inicia la adición de una columna `DeletedAt` de fecha/hora.

Línea 54: `name: "DeletedAt",` → identifica la columna afectada como `DeletedAt`.

Línea 55: `table: "Aleaciones",` → dirige la operación a la tabla `Aleaciones`.

Línea 56: `type: "datetime2",` → fija el tipo SQL Server `datetime2`.

Línea 57: `nullable: true);` → `DeletedAt` admite NULL mientras el registro no esté eliminado.

Línea 59: `migrationBuilder.AddColumn<bool>(` → inicia la adición de una columna `IsDeleted` booleana.

Línea 60: `name: "IsDeleted",` → identifica la columna afectada como `IsDeleted`.

Línea 61: `table: "Aleaciones"`, → dirige la operación a la tabla `Aleaciones`.

Línea 62: `type: "bit"`, → fija el tipo SQL Server `bit`.

Línea 63: `nullable: false`, → `IsDeleted` es obligatorio para todas las filas.

Línea 64: `defaultValue: false`); → rellena las filas existentes con `false` para mantenerlas activas al aplicar el cambio.

Línea 65: `}` → cierra el bloque de la declaración o método anterior.

Línea 67: `/// <inheritdoc />` → comentario XML generado automáticamente para documentar el miembro siguiente.

Línea 68: `protected override void Down(MigrationBuilder migrationBuilder)` → define las operaciones inversas que devuelven el esquema desde 2.11 hasta 2.10.

Línea 69: `{` → abre el bloque de la declaración o método anterior.

Línea 70: `migrationBuilder.DropColumn(` → inicia la eliminación de una de las columnas añadidas por esta migración.

Línea 71: `name: "DeletedAt"`, → identifica la columna afectada como `DeletedAt`.

Línea 72: `table: "PlanchasAcero"`); → dirige la operación a la tabla `PlanchasAcero`.

Línea 74: `migrationBuilder.DropColumn(` → inicia la eliminación de una de las columnas añadidas por esta migración.

Línea 75: `name: "IsDeleted"`, → identifica la columna afectada como `IsDeleted`.

Línea 76: `table: "PlanchasAcero"`); → dirige la operación a la tabla `PlanchasAcero`.

Línea 78: `migrationBuilder.DropColumn(` → inicia la eliminación de una de las columnas añadidas por esta migración.

Línea 79: `name: "DeletedAt"`, → identifica la columna afectada como `DeletedAt`.

Línea 80: `table: "OrdenesFabricacion"`); → dirige la operación a la tabla `OrdenesFabricacion`.

Línea 82: `migrationBuilder.DropColumn(` → inicia la eliminación de una de las columnas añadidas por esta migración.

Línea 83: `name: "IsDeleted"`, → identifica la columna afectada como `IsDeleted`.

Línea 84: `table: "OrdenesFabricacion"`); → dirige la operación a la tabla `OrdenesFabricacion`.

Línea 86: `migrationBuilder.DropColumn(` → inicia la eliminación de una de las columnas añadidas por esta migración.

Línea 87: `name: "DeletedAt"`, → identifica la columna afectada como `DeletedAt`.

Línea 88: `table: "EstadosOrden"`); → dirige la operación a la tabla `EstadosOrden`.

Línea 90: `migrationBuilder.DropColumn(` → inicia la eliminación de una de las columnas añadidas por esta migración.

Línea 91: `name: "IsDeleted"`, → identifica la columna afectada como `IsDeleted`.

Línea 92: `table: "EstadosOrden"`); → dirige la operación a la tabla `EstadosOrden`.

Línea 94: `migrationBuilder.DropColumn(` → inicia la eliminación de una de las columnas añadidas por esta migración.

Línea 95: `name: "DeletedAt"`, → identifica la columna afectada como `DeletedAt`.

Línea 96: `table: "Aleaciones"`); → dirige la operación a la tabla `Aleaciones`.

Línea 98: `migrationBuilder.DropColumn()` → inicia la eliminación de una de las columnas añadidas por esta migración.

Línea 99: `name: "IsDeleted"`, → identifica la columna afectada como `IsDeleted`.

Línea 100: `table: "Aleaciones"`; → dirige la operación a la tabla `Aleaciones`.

Línea 101: `}` → cierra el bloque de la declaración o método anterior.

Línea 102: `}` → cierra el bloque de la declaración o método anterior.

Línea 103: `}` → cierra el bloque de la declaración o método anterior.

Las columnas `IsDeleted` usan `defaultValue: false`. Así, las filas existentes permanecen activas después del cambio de esquema.

Paso 5: Aplicar la migración

```
dotnet ef database update --configuration Release
```

Resultado esperado: EF Core aplica las migraciones pendientes hasta `M2_2_11`. Las cuatro tablas afectadas contienen `IsDeleted` y `DeletedAt`.

Paso 6: Verificar la cadena y la tabla de historial

```
dotnet ef migrations list --configuration Release
```

Consultar además en SQL Server LocalDB:

```
SELECT MigrationId, ProductVersion
FROM __EFMigrationsHistory
ORDER BY MigrationId;
```

Resultado esperado: `M2_2_11` aparece como última migración aplicada.

Paso 7: Revertir exactamente hasta 2.10

```
dotnet ef database update 20260927204833_M2_2_10 --configuration Release
```

Resultado esperado: EF Core ejecuta el `Down` de `M2_2_11`, elimina las columnas añadidas en este punto y retira la migración de `__EFMigrationsHistory`.

Paso 8: Reaplicar 2.11

```
dotnet ef database update --configuration Release
```

Resultado esperado: `Up` vuelve a crear las columnas y el historial vuelve a registrar `M2_2_11`.

Paso 9: Comprender migrations remove sin dañar el repositorio canónico

`migrations remove` modifica archivos fuente y snapshot; por eso no se ejecuta directamente sobre la copia canónica ya validada. Para practicarlo, trabajar sobre una copia desechable de `2.11`, revertir primero la base de laboratorio a `2.10` y ejecutar:

```
dotnet ef database update 20260927204833_M2_2_10 --configuration Release
dotnet ef migrations remove --configuration Release
```

Resultado esperado en la copia: desaparecen los archivos de la última migración y el snapshot vuelve al modelo anterior. Después se descarta la copia. Si una migración ya se ha compartido o desplegado, se crea una migración correctiva nueva en lugar de reescribir la historia.

Paso 10: Revisar el ModelSnapshot

Localizar en `Migrations/AceriaDbContextModelSnapshot.cs` las propiedades nuevas. Un fragmento real del snapshot es:

```
b.Property<DateTime?>("DeletedAt")
    .HasColumnType("datetime2");

b.Property<string>("Descripcion")
    .HasMaxLength(500)
    .HasColumnType("nvarchar(500)");

b.Property<bool>("IsDeleted")
    .HasColumnType("bit");

b.Property<string>("Nombre")
    .IsRequired()
    .HasMaxLength(100)
    .HasColumnType("nvarchar(100)");

b.Property<double>("PorcentajeCarbono")
    .HasColumnType("float");

b.Property<double>("PorcentajeManganeso")
    .HasColumnType("float");

b.HasKey("Id");
```

El snapshot no es la base ni el historial aplicado: es la referencia de modelo que la siguiente ejecución de `migrations add` comparará con el nuevo modelo.

Paso 11: Generar scripts SQL normal e idempotente

```
dotnet ef migrations script 20260927204833_M2_2_10 20260927204841_M2_2_11 --configuration Release --
output softdelete.sql
dotnet ef migrations script --idempotent --configuration Release --output migraciones_idempotentes.sql
```

`softdelete.sql` → contiene sólo el tramo 2.10 → 2.11. `migraciones_idempotentes.sql` → comprueba `__EFMigrationsHistory` antes de ejecutar cada migración.

Paso 12: Ejecutar la comprobación funcional del Soft Delete

```
dotnet run --project AceriaData.Console.csproj --configuration Release --no-build
```

Resultado esperado:

```
2.11 OK | Tras borrar visibles: 0 | Totales: 1 | Restaurada: True
```

La ejecución demuestra que el cambio de esquema es funcional: el filtro oculta la entidad marcada, `IgnoreQueryFilters()` permite recuperarla y la restauración vuelve a hacerla visible.

Errores comunes

Error	Causa	Solución
Migración vacía	El modelo y el snapshot ya representan el mismo estado	Generarla durante la transición real 2.10 → 2.11

Error	Causa	Solución
Columna no anulable incompatible con datos existentes	No existe valor válido para las filas previas	Diseñar default, backfill o transición nullable
<code>database update</code> no cambia nada	La migración ya figura en <code>__EFMigrationsHistory</code>	Comprobar el historial y la base objetivo
La reversión pierde información	<code>Down</code> elimina columnas o tablas	Revisar impacto de datos y disponer de backup
<code>migrations remove</code> no es apropiado	La migración ya se compartió o desplegó	Crear una migración correctiva en vez de reescribir la historia
Script idempotente incompleto	Se está usando otro contexto/proyecto	Verificar proyecto, contexto y cadena

Retorno resuelto

Generar únicamente el SQL del delta 2.10 → 2.11 y localizar en el script las cuatro parejas `DeletedAt / IsDeleted`. Después comprobar que el script idempotente consulta `__EFMigrationsHistory` antes de ejecutar los bloques. La solución es usar el rango explícito para el primer script y `--idempotent` para el segundo.

Analogía final

El modelo es el plano actual de una línea de producción; el snapshot es el plano archivado después de la última reforma; `M2_2_11` es la orden de obra que transforma 2.10 en 2.11; y `__EFMigrationsHistory` es el libro de reformas realmente ejecutadas. Soft Delete es la reforma concreta con la que se estudia todo el ciclo.

Resultado esperado

Al terminar el punto se sabe generar y revisar una migración incremental, comprobar `Up` y `Down`, leer el snapshot, aplicar y revertir hasta un checkpoint, entender `migrations remove`, producir SQL e idempotentes y validar funcionalmente el Soft Delete.

Conexión con el siguiente punto

2.12 reorganiza el mismo sistema en Domain, Application, Infrastructure y Console. Las migraciones pasan a Infrastructure y el contexto de diseño se resuelve mediante `IDesignTimeDbContextFactory`, pero el historial y el esquema continúan siendo los validados en 2.11.

Código acumulativo completo del estado 2.11

El siguiente archivo es el `Program.cs` real del estado ejecutable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

[Table("OrdenesFabricacion")]
public class OrdenFabricacion
{
    [Key]
    public int Id { get; set; }
    [Required]
```

```

    [MaxLength(50)]
    public string NumeroOrden { get; set; } = string.Empty;
    [Required]
    [MaxLength(200)]
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    [Required]
    [MaxLength(50)]
    public string Estado { get; set; } = "Pendiente";
    [MaxLength(500)]
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
    public DetalleOrden? Detalle { get; set; }
    public CertificadoCalidad? Certificado { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
    public bool IsDeleted { get; set; }
    public DateTime? DeletedAt { get; set; }
}

[Table("PlanchasAcero")]
public class PlanchaAcero
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    [Precision(18, 3)]
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
    public bool IsDeleted { get; set; }
    public DateTime? DeletedAt { get; set; }
}

[Table("Aleaciones")]
public class Aleacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(100)]
    public string Nombre { get; set; } = string.Empty;
    [Required]
    [MaxLength(20)]
    public string Codigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    [MaxLength(500)]
    public string? Descripcion { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
    public bool IsDeleted { get; set; }
    public DateTime? DeletedAt { get; set; }
}

[Table("EstadosOrden")]
public class EstadoOrden
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string Nombre { get; set; } = string.Empty;
    [MaxLength(250)]
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
    public bool IsDeleted { get; set; }
    public DateTime? DeletedAt { get; set; }
}

```

```

[Table("DetallesOrden")]
public class DetalleOrden
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(200)]
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    [MaxLength(500)]
    public string? Notas { get; set; }
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("CertificadosCalidad")]
public class CertificadoCalidad
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroCertificado { get; set; } = string.Empty;
    public DateTime FechaEmision { get; set; }
    [Required]
    [MaxLength(100)]
    public string OrganismoCertificador { get; set; } = string.Empty;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]
[Table("OrdenesAleaciones")]
public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }
    public DateTime FechaAsignacion { get; set; } = DateTime.Now;
    [Precision(18, 3)]
    public decimal CantidadUtilizada { get; set; }
    [MaxLength(20)]
    public string EstadoRelacion { get; set; } = "Activa";
    public OrdenFabricacion Orden { get; set; } = null!;
    public Aleacion Aleacion { get; set; } = null!;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
    public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();
    public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();
    public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<OrdenFabricacion>(entity =>
        {
            entity.ToTable("OrdenesFabricacion");
            entity.HasKey(o => o.Id);
            entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);
            entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);
            entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");
            entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");
            entity.Property(o => o.Observaciones).HasMaxLength(500);
        });
    }
}

```

```

        entity.HasAlternateKey(o => o.NumeroOrden).HasName("AK_OrdenesFabricacion_NumeroOrden");
        entity.HasIndex(o => o.Cliente).HasDatabaseName("IX_OrdenesFabricacion_Cliente");
        entity.HasIndex(o => new { o.Cliente,
o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Cliente_FechaCreacion");
        entity.HasIndex(o => o.FechaEntrega).HasFilter("[Estado] =
'Pendiente'").HasDatabaseName("IX_OrdenesFabricacion_FechaEntrega_Pendientes");
        entity.HasIndex(o => o.Estado).IncludeProperties(o => new { o.NumeroOrden, o.Cliente,
o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Estado_Incluye");
        entity.HasQueryFilter(o => !o.IsDeleted && o.Estado != "Cancelada");
    });
    modelBuilder.Entity<PlanchaAcero>(entity =>
    {
        entity.ToTable("PlanchasAcero", t =>
        {
            t.HasCheckConstraint("CK_PlanchasAcero_Espesor", "[Espesor] > 0");
            t.HasCheckConstraint("CK_PlanchasAcero_Ancho", "[Ancho] > 0");
            t.HasCheckConstraint("CK_PlanchasAcero_Largo", "[Largo] > 0");
            t.HasCheckConstraint("CK_PlanchasAcero_Peso", "[Peso] > 0");
        });
        entity.HasKey(x => x.Id);
        entity.Property(x => x.Peso).HasPrecision(18, 3);
        entity.Property(x => x.Activa).HasDefaultValue(true);
        entity.HasOne(x => x.Orden)
            .WithMany(o => o.Planchas)
            .HasForeignKey(x => x.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
        entity.HasIndex(x => new { x.OrdenId,
x.Activa }).HasDatabaseName("IX_PlanchasAcero_OrdenId_Activa");
        entity.HasIndex(x => x.Espesor).HasFilter("[Activa] =
1").HasDatabaseName("IX_PlanchasAcero_Espesor_Activas");
        entity.HasQueryFilter(x => !x.IsDeleted && x.Activa);
    });
    modelBuilder.Entity<Aleacion>(entity =>
    {
        entity.ToTable("Aleaciones", t =>
        {
            t.HasCheckConstraint("CK_Aleaciones_PorcentajeCarbono", "[PorcentajeCarbono] >= 0 AND
[PorcentajeCarbono] <= 2");
            t.HasCheckConstraint("CK_Aleaciones_PorcentajeManganeso", "[PorcentajeManganeso] >= 0
AND [PorcentajeManganeso] <= 5");
        });
        entity.HasKey(a => a.Id);
        entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
        entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
        entity.Property(a => a.Descripcion).HasMaxLength(500);
        entity.HasAlternateKey(a => a.Codigo).HasName("AK_Aleaciones_Codigo");
        entity.HasIndex(a => a.Nombre).HasDatabaseName("IX_Aleaciones_Nombre");
        entity.HasIndex(a => new { a.PorcentajeCarbono,
a.PorcentajeManganeso }).HasDatabaseName("IX_Aleaciones_Porcentajes");
        entity.HasQueryFilter(a => !a.IsDeleted);
    });
    modelBuilder.Entity<EstadoOrden>(entity =>
    {
        entity.ToTable("EstadosOrden");
        entity.HasKey(e => e.Id);
        entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
        entity.Property(e => e.Descripcion).HasMaxLength(250);
        entity.Property(e => e.Activo).HasDefaultValue(true);
        entity.HasQueryFilter(e => !e.IsDeleted && e.Activo);
    });
    modelBuilder.Entity<DetalleOrden>(entity =>
    {
        entity.ToTable("DetallesOrden");
        entity.HasKey(d => d.Id);
        entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);
        entity.Property(d => d.Notas).HasMaxLength(500);
        entity.HasOne(d => d.Orden)
            .WithOne(o => o.Detalle)
            .HasForeignKey<DetalleOrden>(d => d.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
        entity.HasQueryFilter(d => !d.Orden.IsDeleted && d.Orden.Estado != "Cancelada");
    });

```

```

});

modelBuilder.Entity<CertificadoCalidad>(entity =>
{
    entity.ToTable("CertificadosCalidad");
    entity.HasKey(c => c.Id);
    entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);
    entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);
    entity.HasAlternateKey(c =>
c.NumeroCertificado).HasName("AK_CertificadosCalidad_NumeroCertificado");
    entity.HasIndex(c => c.FechaEmision).HasDatabaseName("IX_CertificadosCalidad_FechaEmision");
    entity.HasOne(c => c.Orden)
        .WithOne(o => o.Certificado)
        .HasForeignKey<CertificadoCalidad>(c => c.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
    entity.HasQueryFilter(c => !c.Orden.IsDeleted && c.Orden.Estado != "Cancelada");
});
modelBuilder.Entity<OrdenAleacion>(entity =>
{
    entity.ToTable("OrdenesAleaciones");
    entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });
    entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");
    entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);
    entity.Property(x =>
x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");
    entity.HasOne(x => x.Orden)
        .WithMany(o => o.OrdenesAleaciones)
        .HasForeignKey(x => x.OrdenFabricacionId)
        .OnDelete(DeleteBehavior.Cascade);
    entity.HasOne(x => x.Aleacion)
        .WithMany(a => a.OrdenesAleaciones)
        .HasForeignKey(x => x.AleacionId)
        .OnDelete(DeleteBehavior.Restrict);
    entity.HasIndex(x => x.AleacionId).HasDatabaseName("IX_OrdenesAleaciones_AleacionId");
    entity.HasIndex(x => x.EstadoRelacion).HasFilter("[EstadoRelacion] =
'Activa'").HasDatabaseName("IX_OrdenesAleaciones_EstadoRelacion_Activas");
    entity.HasQueryFilter(x => x.EstadoRelacion == "Activa" && !x.Orden.IsDeleted && !
x.Aleacion.IsDeleted);
});
}
}

public interface IOordenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    void EliminarLogicamente(int id);
    void Restaurar(int id);

    int Guardar();
}

public sealed class OrdenRepositorio : IOordenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o =>
o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) =>
_context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public void EliminarLogicamente(int id)
    {
        var orden = _context.OrdenesFabricacion.FirstOrDefault(o => o.Id == id);
        if (orden is null) return;
        orden.IsDeleted = true;
    }
}

```

```

        orden.DeletedAt = DateTime.UtcNow;
    }

    public void Restaurar(int id)
    {
        var orden = _context.OrdenesFabricacion.IgnoreQueryFilters().FirstOrDefault(o => o.Id == id);
        if (orden is null) return;
        orden.IsDeleted = false;
        orden.DeletedAt = null;
    }

    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOOrdenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOOrdenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();
        services.AddDbContext<AceriaDbContext>(options =>
            options
                .UseSqlServer(connectionString, sql =>
                    {
                        sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                        sql.CommandTimeout(60);
                    })
                .LogTo(global::System.Console.WriteLine, new[] {
                    "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)
                .EnableDetailedErrors());

        services.AddScoped<IOOrdenRepositorio, OrdenRepositorio>();
        services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

        using var provider = services.BuildServiceProvider(new ServiceProviderOptions
        {
            ValidateScopes = true,
            ValidateOnBuild = true
        });

        using var scope = provider.CreateScope();
    }
}

```

```

var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();

var orden = new OrdenFabricacion
{
    NumeroOrden = "OF-M2-0001",
    Cliente = "Constructora del Norte",
    FechaCreacion = DateTime.UtcNow,
    FechaEntrega = DateTime.Today.AddDays(14),
    Estado = "Pendiente",
    Observaciones = "Orden de validación M2",
    IsDeleted = false,
    Planchas =
    {
        new PlanchaAcero
        {
            Espesor = 10.5,
            Ancho = 1500,
            Largo = 3000,
            Peso = 371.250m,
            Activa = true,
        }
    },
    Detalle = new DetalleOrden
    {
        ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
        TemperaturaColada = 1550.5,
        Notas = "Colada principal"
    },
    Certificado = new CertificadoCalidad
    {
        NumeroCertificado = "CERT-0001",
        FechaEmision = DateTime.Today,
        OrganismoCertificador = "Laboratorio Aceria"
    },
};

var aleacion = new Aleacion
{
    Nombre = "AISI 1045",
   Codigo = "A1045",
    PorcentajeCarbono = 0.45,
    PorcentajeManganeso = 0.75,
    Descripcion = "Acero medio en carbono"
};
orden OrdenesAleaciones.Add(new OrdenAleacion
{
    Aleacion = aleacion,
    CantidadUtilizada = 1500.500m,
    EstadoRelacion = "Activa"
});
context OrdenesFabricacion.Add(orden);
context EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa",
Activo = true, IsDeleted = false });
context.SaveChanges();

var id = orden.Id;
orden.IsDeleted = true;
orden.DeletedAt = DateTime.UtcNow;
context.SaveChanges();
var visibles = context OrdenesFabricacion.Count();
var todas = context OrdenesFabricacion.IgnoreQueryFilters().Count();
var restaurable = context OrdenesFabricacion.IgnoreQueryFilters().Single(o => o.Id == id);
restaurable.IsDeleted = false;
restaurable.DeletedAt = null;
context.SaveChanges();

global::System.Console.WriteLine($"2.11 OK | Tras borrar visibles: {visibles} | Totales: {todas} |
Restaurada: {context OrdenesFabricacion.Any(o => o.Id == id)}");
}
}

```

Explicación línea a línea del código acumulativo 2.11

Las referencias usan la numeración real de `M02/PROYECTO/2.11/Program.cs`. Se explican todas las líneas no vacías del archivo que compila.

Línea 1: `using System.ComponentModel.DataAnnotations;` → importa el espacio de nombres necesario para este archivo.

Línea 2: `using System.ComponentModel.DataAnnotations.Schema;` → importa el espacio de nombres necesario para este archivo.

Línea 3: `using Microsoft.EntityFrameworkCore;` → importa el espacio de nombres necesario para este archivo.

Línea 4: `using Microsoft.Extensions.Configuration;` → importa el espacio de nombres necesario para este archivo.

Línea 5: `using Microsoft.Extensions.DependencyInjection;` → importa el espacio de nombres necesario para este archivo.

Línea 6: `using Microsoft.Extensions.Logging;` → importa el espacio de nombres necesario para este archivo.

Línea 8: `namespace AceriaData.ConsoleApp;` → declara el espacio de nombres del estado acumulativo.

Línea 10: `[Table("OrdenesFabricacion")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 11: `public class OrdenFabricacion` → declara la clase `OrdenFabricacion` que forma parte del estado acumulativo.

Línea 12: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 13: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 14: `public int Id { get; set; }` → declara la propiedad `Id` de tipo `int` usada por el modelo o la lógica de negocio.

Línea 15: `[Required]` → marca la propiedad siguiente como requerida.

Línea 16: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 17: `public string NumeroOrden { get; set; } = string.Empty;` → declara la propiedad `NumeroOrden` de tipo `string` usada por el modelo o la lógica de negocio.

Línea 18: `[Required]` → marca la propiedad siguiente como requerida.

Línea 19: `[MaxLength(200)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 20: `public string Cliente { get; set; } = string.Empty;` → declara la propiedad `Cliente` de tipo `string` usada por el modelo o la lógica de negocio.

Línea 21: `public DateTime FechaCreacion { get; set; }` → declara la propiedad `FechaCreacion` de tipo `DateTime` usada por el modelo o la lógica de negocio.

Línea 22: `public DateTime? FechaEntrega { get; set; }` → declara la propiedad `FechaEntrega` de tipo `DateTime?` usada por el modelo o la lógica de negocio.

Línea 23: `[Required]` → marca la propiedad siguiente como requerida.

Línea 24: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 25: `public string Estado { get; set; } = "Pendiente";` → declara la propiedad `Estado` de tipo `string` usada por el modelo o la lógica de negocio.

Línea 26: `[MaxLength(500)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 27: `public string? Observaciones { get; set; }` → declara la propiedad Observaciones de tipo `string?` usada por el modelo o la lógica de negocio.

Línea 28: `public List<PlanchaAcero> Planchas { get; set; } = new();` → declara la propiedad Planchas de tipo `List` usada por el modelo o la lógica de negocio.

Línea 29: `public DetalleOrden? Detalle { get; set; }` → declara la propiedad Detalle de tipo `DetalleOrden?` usada por el modelo o la lógica de negocio.

Línea 30: `public CertificadoCalidad? Certificado { get; set; }` → declara la propiedad Certificado de tipo `CertificadoCalidad?` usada por el modelo o la lógica de negocio.

Línea 31: `public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();` → declara la propiedad OrdenesAleaciones de tipo `List` usada por el modelo o la lógica de negocio.

Línea 32: `public bool IsDeleted { get; set; }` → declara la propiedad IsDeleted de tipo `bool` usada por el modelo o la lógica de negocio.

Línea 33: `public DateTime? DeletedAt { get; set; }` → declara la propiedad DeletedAt de tipo `DateTime?` usada por el modelo o la lógica de negocio.

Línea 34: `}` → cierra el bloque de código actual.

Línea 36: `[Table("PlanchasAcero")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 37: `public class PlanchaAcero` → declara la clase PlanchaAcero que forma parte del estado acumulativo.

Línea 38: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 39: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 40: `public int Id { get; set; }` → declara la propiedad Id de tipo `int` usada por el modelo o la lógica de negocio.

Línea 41: `public int OrdenId { get; set; }` → declara la propiedad OrdenId de tipo `int` usada por el modelo o la lógica de negocio.

Línea 42: `public double Espesor { get; set; }` → declara la propiedad Espesor de tipo `double` usada por el modelo o la lógica de negocio.

Línea 43: `public double Ancho { get; set; }` → declara la propiedad Ancho de tipo `double` usada por el modelo o la lógica de negocio.

Línea 44: `public double Largo { get; set; }` → declara la propiedad Largo de tipo `double` usada por el modelo o la lógica de negocio.

Línea 45: `[Precision(18, 3)]` → configura precisión y escala para la propiedad numérica siguiente.

Línea 46: `public decimal Peso { get; set; }` → declara la propiedad Peso de tipo `decimal` usada por el modelo o la lógica de negocio.

Línea 47: `public bool Activa { get; set; } = true;` → declara la propiedad Activa de tipo `bool` usada por el modelo o la lógica de negocio.

Línea 48: `[ForeignKey(nameof(OrdenId))]` → vincula la navegación siguiente con su clave foránea.

Línea 49: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo `OrdenFabricacion` usada por el modelo o la lógica de negocio.

Línea 50: `public bool IsDeleted { get; set; }` → declara la propiedad IsDeleted de tipo `bool` usada por el modelo o la lógica de negocio.

Línea 51: `public DateTime? DeletedAt { get; set; }` → declara la propiedad DeletedAt de tipo `DateTime?` usada por el modelo o la lógica de negocio.

Línea 52: `}` → cierra el bloque de código actual.

Línea 54: `[Table("Aleaciones")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 55: `public class Aleacion` → declara la clase Aleacion que forma parte del estado acumulativo.

Línea 56: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 57: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 58: `public int Id { get; set; }` → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.

Línea 59: `[Required]` → marca la propiedad siguiente como requerida.

Línea 60: `[MaxLength(100)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 61: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad Nombre de tipo string usada por el modelo o la lógica de negocio.

Línea 62: `[Required]` → marca la propiedad siguiente como requerida.

Línea 63: `[MaxLength(20)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 64: `public string Codigo { get; set; } = string.Empty;` → declara la propiedad Codigo de tipo string usada por el modelo o la lógica de negocio.

Línea 65: `public double PorcentajeCarbono { get; set; }` → declara la propiedad PorcentajeCarbono de tipo double usada por el modelo o la lógica de negocio.

Línea 66: `public double PorcentajeManganeso { get; set; }` → declara la propiedad PorcentajeManganeso de tipo double usada por el modelo o la lógica de negocio.

Línea 67: `[MaxLength(500)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 68: `public string? Descripcion { get; set; }` → declara la propiedad Descripcion de tipo string? usada por el modelo o la lógica de negocio.

Línea 69: `public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();` → declara la propiedad OrdenesAleaciones de tipo List usada por el modelo o la lógica de negocio.

Línea 70: `public bool IsDeleted { get; set; }` → declara la propiedad IsDeleted de tipo bool usada por el modelo o la lógica de negocio.

Línea 71: `public DateTime? DeletedAt { get; set; }` → declara la propiedad DeletedAt de tipo DateTime? usada por el modelo o la lógica de negocio.

Línea 72: `}` → cierra el bloque de código actual.

Línea 74: `[Table("EstadosOrden")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 75: `public class EstadoOrden` → declara la clase EstadoOrden que forma parte del estado acumulativo.

Línea 76: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 77: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 78: `public int Id { get; set; }` → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.

Línea 79: `[Required]` → marca la propiedad siguiente como requerida.

Línea 80: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 81: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad Nombre de tipo string usada por el modelo o la lógica de negocio.

Línea 82: `[MaxLength(250)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 83: `public string Descripcion { get; set; } = string.Empty;` → declara la propiedad Descripcion de tipo string usada por el modelo o la lógica de negocio.

Línea 84: `public bool Activo { get; set; } = true;` → declara la propiedad Activo de tipo bool usada por el modelo o la lógica de negocio.

Línea 85: `public bool IsDeleted { get; set; }` → declara la propiedad IsDeleted de tipo bool usada por el modelo o la lógica de negocio.

Línea 86: `public DateTime? DeletedAt { get; set; }` → declara la propiedad DeletedAt de tipo DateTime? usada por el modelo o la lógica de negocio.

Línea 87: `}` → cierra el bloque de código actual.

Línea 89: `[Table("DetallesOrden")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 90: `public class DetalleOrden` → declara la clase DetalleOrden que forma parte del estado acumulativo.

Línea 91: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 92: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 93: `public int Id { get; set; }` → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.

Línea 94: `public int OrdenId { get; set; }` → declara la propiedad OrdenId de tipo int usada por el modelo o la lógica de negocio.

Línea 95: `[Required]` → marca la propiedad siguiente como requerida.

Línea 96: `[MaxLength(200)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 97: `public string ComposicionQuimica { get; set; } = string.Empty;` → declara la propiedad ComposicionQuimica de tipo string usada por el modelo o la lógica de negocio.

Línea 98: `public double TemperaturaColada { get; set; }` → declara la propiedad TemperaturaColada de tipo double usada por el modelo o la lógica de negocio.

Línea 99: `[MaxLength(500)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 100: `public string? Notas { get; set; }` → declara la propiedad Notas de tipo string? usada por el modelo o la lógica de negocio.

Línea 101: `[ForeignKey(nameof(OrdenId))]` → vincula la navegación siguiente con su clave foránea.

Línea 102: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo OrdenFabricacion usada por el modelo o la lógica de negocio.

Línea 103: `}` → cierra el bloque de código actual.

Línea 105: `[Table("CertificadosCalidad")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 106: `public class CertificadoCalidad` → declara la clase CertificadoCalidad que forma parte del estado acumulativo.

Línea 107: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 108: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 109: `public int Id { get; set; }` → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.

Línea 110: `public int OrdenId { get; set; }` → declara la propiedad `OrdenId` de tipo `int` usada por el modelo o la lógica de negocio.

Línea 111: `[Required]` → marca la propiedad siguiente como requerida.

Línea 112: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 113: `public string NumeroCertificado { get; set; } = string.Empty;` → declara la propiedad `NumeroCertificado` de tipo `string` usada por el modelo o la lógica de negocio.

Línea 114: `public DateTime FechaEmision { get; set; }` → declara la propiedad `FechaEmision` de tipo `DateTime` usada por el modelo o la lógica de negocio.

Línea 115: `[Required]` → marca la propiedad siguiente como requerida.

Línea 116: `[MaxLength(100)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 117: `public string OrganismoCertificador { get; set; } = string.Empty;` → declara la propiedad `OrganismoCertificador` de tipo `string` usada por el modelo o la lógica de negocio.

Línea 118: `[ForeignKey(nameof(OrdenId))]` → vincula la navegación siguiente con su clave foránea.

Línea 119: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad `Orden` de tipo `OrdenFabricacion` usada por el modelo o la lógica de negocio.

Línea 120: `}` → cierra el bloque de código actual.

Línea 122: `[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]` → declara una clave primaria compuesta.

Línea 123: `[Table("OrdenesAleaciones")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 124: `public class OrdenAleacion` → declara la clase `OrdenAleacion` que forma parte del estado acumulativo.

Línea 125: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 126: `public int OrdenFabricacionId { get; set; }` → declara la propiedad `OrdenFabricacionId` de tipo `int` usada por el modelo o la lógica de negocio.

Línea 127: `public int AleacionId { get; set; }` → declara la propiedad `AleacionId` de tipo `int` usada por el modelo o la lógica de negocio.

Línea 128: `public DateTime FechaAsignacion { get; set; } = DateTime.Now;` → declara la propiedad `FechaAsignacion` de tipo `DateTime` usada por el modelo o la lógica de negocio.

Línea 129: `[Precision(18, 3)]` → configura precisión y escala para la propiedad numérica siguiente.

Línea 130: `public decimal CantidadUtilizada { get; set; }` → declara la propiedad `CantidadUtilizada` de tipo `decimal` usada por el modelo o la lógica de negocio.

Línea 131: `[MaxLength(20)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 132: `public string EstadoRelacion { get; set; } = "Activa";` → declara la propiedad `EstadoRelacion` de tipo `string` usada por el modelo o la lógica de negocio.

Línea 133: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad `Orden` de tipo `OrdenFabricacion` usada por el modelo o la lógica de negocio.

Línea 134: `public Aleacion Aleacion { get; set; } = null!;` → declara la propiedad `Aleacion` de tipo `Aleacion` usada por el modelo o la lógica de negocio.

Línea 135: `}` → cierra el bloque de código actual.

Línea 137: `public class AceriaDbContext : DbContext` → declara la clase `AceriaDbContext` que forma parte del estado acumulativo.

Línea 138: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 139:

`public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }` → recibe las opciones del DbContext y las pasa a la clase base.

Línea 141: `public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 142: `public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 143: `public DbSet<Aleacion> Aleaciones => Set<Aleacion>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 144: `public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 145: `public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 146: `public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 147: `public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 150: `protected override void OnModelCreating(ModelBuilder modelBuilder)` → entra en la configuración explícita del modelo.

Línea 151: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 152: `base.OnModelCreating(modelBuilder);` → entra en la configuración explícita del modelo.

Línea 153: `modelBuilder.Entity<OrdenFabricacion>(entity =>` → selecciona una entidad para configurarla con Fluent API.

Línea 154: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 155: `entity.ToTable("OrdenesFabricacion");` → asigna explícitamente la tabla usada por la entidad.

Línea 156: `entity.HasKey(o => o.Id);` → define la clave primaria.

Línea 157: `entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 158: `entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);` → selecciona una propiedad escalar para continuar su configuración.

Línea 159: `entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");` → selecciona una propiedad escalar para continuar su configuración.

Línea 160: `entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");` → selecciona una propiedad escalar para continuar su configuración.

Línea 161: `entity.Property(o => o.Observaciones).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 162: `entity.HasAlternateKey(o => o.NumeroOrden).HasName("AK_OrdenesFabricacion_NumeroOrden");` → define una clave alternativa única.

Línea 163: `entity.HasIndex(o => o.Cliente).HasDatabaseName("IX_OrdenesFabricacion_Cliente");` → define un índice sobre las propiedades indicadas.

Línea 164: `entity.HasIndex(o => new { o.Cliente, o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Cliente_FechaCreacion");` → define un índice sobre las propiedades indicadas.

Línea 165: `entity.HasIndex(o => o.FechaEntrega).HasFilter("[Estado] = 'Pendiente')").HasDatabaseName("IX_OrdenesFabricacion_FechaEntrega_Pendientes");` → define un índice sobre las propiedades indicadas.

Línea 166: `entity.HasIndex(o => o.Estado).IncludeProperties(o => new { o.NumeroOrden, o.Cliente, o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Estado_Incluye");` → define un índice sobre las propiedades indicadas.

Línea 167: `entity.HasQueryFilter(o => !o.IsDeleted && o.Estado != "Cancelada");` → aplica un filtro global de consulta.

Línea 168: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 169: `modelBuilder.Entity<PlanchaAcero>(entity =>` → selecciona una entidad para configurarla con Fluent API.

Línea 170: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 171: `entity.ToTable("PlanchasAcero", t =>` → asigna explícitamente la tabla usada por la entidad.

Línea 172: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 173: `t.HasCheckConstraint("CK_PlanchasAcero_Espesor", "[Espesor] > 0");` → añade una restricción CHECK en SQL Server.

Línea 174: `t.HasCheckConstraint("CK_PlanchasAcero_Ancho", "[Ancho] > 0");` → añade una restricción CHECK en SQL Server.

Línea 175: `t.HasCheckConstraint("CK_PlanchasAcero_Largo", "[Largo] > 0");` → añade una restricción CHECK en SQL Server.

Línea 176: `t.HasCheckConstraint("CK_PlanchasAcero_Peso", "[Peso] > 0");` → añade una restricción CHECK en SQL Server.

Línea 177: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 178: `entity.HasKey(x => x.Id);` → define la clave primaria.

Línea 179: `entity.Property(x => x.Peso).HasPrecision(18, 3);` → selecciona una propiedad escalar para continuar su configuración.

Línea 180: `entity.Property(x => x.Activa).HasDefaultValue(true);` → selecciona una propiedad escalar para continuar su configuración.

Línea 181: `entity.HasOne(x => x.Orden)` → inicia la configuración de una navegación de referencia.

Línea 182: `.WithMany(o => o.Planchas)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 183: `.HasForeignKey(x => x.OrdenId)` → declara explícitamente la clave foránea.

Línea 184: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 185: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 186: `entity.HasIndex(x => new { x.OrdenId, x.Activa }).HasDatabaseName("IX_PlanchasAcero_OrdenId_Activa");` → define un índice sobre las propiedades indicadas.

Línea 187: `entity.HasIndex(x => x.Espesor).HasFilter("[Activa] = 1").HasDatabaseName("IX_PlanchasAcero_Espesor_Activas");` → define un índice sobre las propiedades indicadas.

Línea 188: `entity.HasQueryFilter(x => !x.IsDeleted && x.Activa);` → aplica un filtro global de consulta.

Línea 189: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 190: `modelBuilder.Entity<Aleacion>(entity =>` → selecciona una entidad para configurarla con Fluent API.

Línea 191: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 192: `entity.ToTable("Aleaciones", t =>` → asigna explícitamente la tabla usada por la entidad.

Línea 193: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 194: `t.HasCheckConstraint("CK_Aleaciones_PorcentajeCarbono", "[PorcentajeCarbono] >= 0 AND [PorcentajeCarbono] <= 2");` → añade una restricción CHECK en SQL Server.

Línea 195: `t.HasCheckConstraint("CK_Aleaciones_PorcentajeManganeso", "[PorcentajeManganeso] >= 0 AND [PorcentajeManganeso] <= 5");` → añade una restricción CHECK en SQL Server.

Línea 196: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 197: `entity.HasKey(a => a.Id);` → define la clave primaria.

Línea 198: `entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);` → selecciona una propiedad escalar para continuar su configuración.

Línea 199: `entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);` → selecciona una propiedad escalar para continuar su configuración.

Línea 200: `entity.Property(a => a.Descripcion).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 201: `entity.HasAlternateKey(a => a.Codigo).HasName("AK_Aleaciones_Codigo");` → define una clave alternativa única.

Línea 202: `entity.HasIndex(a => a.Nombre).HasDatabaseName("IX_Aleaciones_Nombre");` → define un índice sobre las propiedades indicadas.

Línea 203: `entity.HasIndex(a => new { a.PorcentajeCarbono, a.PorcentajeManganeso }).HasDatabaseName("IX_Aleaciones_Porcentajes");` → define un índice sobre las propiedades indicadas.

Línea 204: `entity.HasQueryFilter(a => !a.IsDeleted);` → aplica un filtro global de consulta.

Línea 205: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 206: `modelBuilder.Entity<EstadoOrden>(entity =>` → selecciona una entidad para configurarla con Fluent API.

Línea 207: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 208: `entity.ToTable("EstadosOrden");` → asigna explícitamente la tabla usada por la entidad.

Línea 209: `entity.HasKey(e => e.Id);` → define la clave primaria.

Línea 210: `entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 211: `entity.Property(e => e.Descripcion).HasMaxLength(250);` → selecciona una propiedad escalar para continuar su configuración.

Línea 212: `entity.Property(e => e.Activo).HasDefaultValue(true);` → selecciona una propiedad escalar para continuar su configuración.

Línea 213: `entity.HasQueryFilter(e => !e.IsDeleted && e.Activo);` → aplica un filtro global de consulta.

Línea 214: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 215: `modelBuilder.Entity<DetalleOrden>(entity =>` → selecciona una entidad para configurarla con Fluent API.

Línea 216: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 217: `entity.ToTable("DetallesOrden");` → asigna explícitamente la tabla usada por la entidad.

Línea 218: `entity.HasKey(d => d.Id);` → define la clave primaria.

Línea 219: `entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);` → selecciona una propiedad escalar para continuar su configuración.

Línea 220: `entity.Property(d => d.Notas).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 221: `entity.HasOne(d => d.Orden)` → inicia la configuración de una navegación de referencia.

Línea 222: `.WithOne(o => o.Detalle)` → establece el extremo de referencia de una relación uno-a-uno.

Línea 223: `.HasForeignKey<DetalleOrden>(d => d.OrdenId)` → declara explícitamente la clave foránea.

Línea 224: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 225: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 226: `entity.HasQueryFilter(d => !d.Orden.IsDeleted && d.Orden.Estado != "Cancelada");` → aplica un filtro global de consulta.

Línea 227: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 229: `modelBuilder.Entity<CertificadoCalidad>(entity =>` → selecciona una entidad para configurarla con Fluent API.

Línea 230: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 231: `entity.ToTable("CertificadosCalidad");` → asigna explícitamente la tabla usada por la entidad.

Línea 232: `entity.HasKey(c => c.Id);` → define la clave primaria.

Línea 233: `entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 234: `entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);` → selecciona una propiedad escalar para continuar su configuración.

Línea 235: `entity.HasAlternateKey(c => c.NumeroCertificado).HasName("AK_CertificadosCalidad_NumeroCertificado");` → define una clave alternativa única.

Línea 236: `entity.HasIndex(c => c.FechaEmision).HasDatabaseName("IX_CertificadosCalidad_FechaEmision");` → define un índice sobre las propiedades indicadas.

Línea 237: `entity.HasOne(c => c.Orden)` → inicia la configuración de una navegación de referencia.

Línea 238: `.WithOne(o => o.Certificado)` → establece el extremo de referencia de una relación uno-a-uno.

Línea 239: `.HasForeignKey<CertificadoCalidad>(c => c.OrdenId)` → declara explícitamente la clave foránea.

Línea 240: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 241: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 242: `entity.HasQueryFilter(c => !c.Orden.IsDeleted && c.Orden.Estado != "Cancelada");` → aplica un filtro global de consulta.

Línea 243: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 244: `modelBuilder.Entity<OrdenAleacion>(entity =>` → selecciona una entidad para configurarla con Fluent API.

Línea 245: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 246: `entity.ToTable("OrdenesAleaciones");` → asigna explícitamente la tabla usada por la entidad.

Línea 247: `entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });` → define la clave primaria.

Línea 248: `entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");` → selecciona una propiedad escalar para continuar su configuración.

Línea 249: `entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);` → selecciona una propiedad escalar para continuar su configuración.

Línea 250: `entity.Property(x => x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");` → selecciona una propiedad escalar para continuar su configuración.

Línea 251: `entity.HasOne(x => x.Orden)` → inicia la configuración de una navegación de referencia.

Línea 252: `.WithMany(o => o.OrdenesAleaciones)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 253: `.HasForeignKey(x => x.OrdenFabricacionId)` → declara explícitamente la clave foránea.

Línea 254: `.OnDelete(DeleteBehavior.Cascade);` → define el comportamiento de eliminación de la relación.

Línea 255: `entity.HasOne(x => x.Aleacion)` → inicia la configuración de una navegación de referencia.

Línea 256: `.WithMany(a => a.OrdenesAleaciones)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 257: `.HasForeignKey(x => x.AleacionId)` → declara explícitamente la clave foránea.

Línea 258: `.OnDelete(DeleteBehavior.Restrict);` → define el comportamiento de eliminación de la relación.

Línea 259: `entity.HasIndex(x => x.AleacionId).HasDatabaseName("IX_OrdenesAleaciones_AleacionId");` → define un índice sobre las propiedades indicadas.

Línea 260: `entity.HasIndex(x => x.EstadoRelacion).HasFilter("[EstadoRelacion] = 'Activa'").HasDatabaseName("IX_OrdenesAleaciones_EstadoRelacion_Activas");` → define un índice sobre las propiedades indicadas.

Línea 261: `entity.HasQueryFilter(x => x.EstadoRelacion == "Activa" && !x.Orden.IsDeleted && !x.Aleacion.IsDeleted);` → aplica un filtro global de consulta.

Línea 262: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 263: `}` → cierra el bloque de código actual.

Línea 264: `}` → cierra el bloque de código actual.

Línea 266: `public interface IOrdenRepositorio` → declara el contrato `IOrdenRepositorio` usado por la aplicación.

Línea 267: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 268: `List<OrdenFabricacion> ObtenerTodas();` → declara en el contrato la operación que debe devolver todas las órdenes visibles para el repositorio.

Línea 269: `OrdenFabricacion? ObtenerPorId(int id);` → declara en el contrato la búsqueda de una orden por su clave primaria y permite devolver null si no existe.

Línea 270: `OrdenFabricacion? ObtenerPorNumero(string numeroOrden);` → declara en el contrato la búsqueda por el identificador de negocio `NumeroOrden`.

Línea 271: `void Agregar(OrdenFabricacion orden);` → declara la operación que incorpora una orden al repositorio para que EF Core la siga como nueva.

Línea 272: `void Eliminar(OrdenFabricacion orden);` → declara la operación que marca una orden para eliminación física cuando el escenario lo requiere.

Línea 273: `void EliminarLogicamente(int id);` → declara en el contrato del repositorio la operación de Soft Delete por identificador.

Línea 274: `void Restaurar(int id);` → declara en el contrato del repositorio la operación que revierte un Soft Delete.

Línea 276: `int Guardar();` → declara la operación que confirma los cambios pendientes y devuelve el número de entradas afectadas.

Línea 277: `}` → cierra el bloque de código actual.

Línea 279: `public sealed class OrdenRepositorio : IOrdenRepositorio` → declara la clase `OrdenRepositorio` que forma parte del estado acumulativo.

Línea 280: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 281: `private readonly AceriaDbContext _context;` → declara el campo privado `_context` de tipo `AceriaDbContext`.

Línea 282: `public OrdenRepositorio(AceriaDbContext context) => _context = context;` → inyecta `AceriaDbContext` en el repositorio y lo conserva como dependencia de persistencia.

Línea 284: `public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();` → ordena el resultado antes de materializarlo.

Línea 285: `public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);` → implementa la búsqueda por clave primaria mediante `Find`, aprovechando primero el `ChangeTracker` y después la base de datos.

Línea 286: `public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);` → recupera la primera coincidencia o null.

Línea 287: `public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);` → registra una entidad nueva para insertarla al guardar.

Línea 288: `public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);` → marca una entidad para borrado al guardar.

Línea 289: `public void EliminarLogicamente(int id)` → implementa la operación que localiza una orden y cambia sus campos de borrado lógico.

Línea 290: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 291: `var orden = _context.OrdenesFabricacion.FirstOrDefault(o => o.Id == id);` → recupera la primera coincidencia o null.

Línea 292: `if (orden is null) return;` → evalúa una condición antes de ejecutar el bloque asociado.

Línea 293: `orden.IsDeleted = true;` → marca la orden como eliminada lógicamente sin borrar su fila física.

Línea 294: `orden.DeletedAt = DateTime.UtcNow;` → registra en UTC el instante en que se realizó el borrado lógico.

Línea 295: `}` → cierra el bloque de código actual.

Línea 297: `public void Restaurar(int id)` → implementa la restauración cargando también entidades ocultas por el filtro global.

Línea 298: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 299: `var orden = _context.OrdenesFabricacion.IgnoreQueryFilters().FirstOrDefault(o => o.Id == id);` → omite de forma explícita los filtros globales para esta consulta.

Línea 300: `if (orden is null) return;` → evalúa una condición antes de ejecutar el bloque asociado.

Línea 301: `orden.IsDeleted = false;` → desactiva la marca de borrado lógico para volver a hacer visible la orden.

Línea 302: `orden.DeletedAt = null;` → elimina la fecha de borrado porque la orden vuelve al estado activo.

Línea 303: `}` → cierra el bloque de código actual.

Línea 305: `public int Guardar() => _context.SaveChanges();` → confirma en SQL Server los cambios del ChangeTracker.

Línea 306: `}` → cierra el bloque de código actual.

Línea 308: `public interface IServicioOrdenes` → declara el contrato IServicioOrdenes usado por la aplicación.

Línea 309: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 310: `string ObtenerResumen(int ordenId);` → declara el servicio que construye un resumen de una orden a partir de su identificador.

Línea 311: `}` → cierra el bloque de código actual.

Línea 313: `public sealed class ServicioOrdenes : IServicioOrdenes` → declara la clase ServicioOrdenes que forma parte del estado acumulativo.

Línea 314: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 315: `private readonly IOrdenRepositorio _repositorio;` → declara el campo privado _repositorio de tipo IOrdenRepositorio.

Línea 316: `private readonly AceriaDbContext _context;` → declara el campo privado _context de tipo AceriaDbContext.

Línea 317: `public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)` → declara el constructor que recibe por inyección el repositorio de órdenes y el DbContext.

Línea 318: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 319: `_repositorio = repositorio;` → guarda en el servicio el repositorio inyectado para reutilizarlo en sus operaciones.

Línea 320: `_context = context;` → guarda el DbContext inyectado para consultar los datos necesarios durante el servicio.

Línea 321: `}` → cierra el bloque de código actual.

Línea 323: `public string ObtenerResumen(int ordenId)` → declara el método que obtiene la orden y compone el resumen solicitado por el servicio.

Línea 324: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 325: `var orden = _repositorio.ObtenerPorId(ordenId);` → declara una variable local y almacena el resultado de la expresión.

Línea 326: `if (orden is null) return $"Orden {ordenId} no encontrada";` → evalúa una condición antes de ejecutar el bloque asociado.

Línea 327: `var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);` → cuenta las filas que cumplen la consulta.

Línea 328: `return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";` → devuelve el resultado al código llamador.

Línea 329: `}` → cierra el bloque de código actual.

Línea 330: `}` → cierra el bloque de código actual.

Línea 332: `public static class Program` → declara la clase Program que forma parte del estado acumulativo.

Línea 333: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 334: `public static void Main()` → declara el punto de entrada ejecutable de la aplicación de consola.

Línea 335: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 336: `var configuration = new ConfigurationBuilder()` → inicia la construcción de la configuración de la aplicación.

Línea 337: `.SetBasePath(AppContext.BaseDirectory)` → establece la ruta base para los archivos de configuración.

Línea 338: `.AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)` → incorpora appsettings.json.

Línea 339: `.AddEnvironmentVariables()` → incorpora variables de entorno.

Línea 340: `.Build();` → materializa el objeto construido por la cadena anterior.

Línea 342: `var connectionString = configuration.GetConnectionString("AceriaDB")` → obtiene la cadena AceriaDB desde configuración.

Línea 343: `?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");` → interrumpe el arranque con un error explícito si la configuración no contiene ConnectionStrings:AceriaDB.

Línea 345: `var services = new ServiceCollection();` → crea la colección de servicios.

Línea 346: `services.AddDbContext<AceriaDbContext>(options =>` → registra el DbContext en inyección de dependencias.

Línea 347: `options` → continúa la configuración del DbContext sobre el parámetro options recibido por la lambda de AddDbContext.

Línea 348: `.UseSqlServer(connectionString, sql =>` → selecciona SQL Server como proveedor de EF Core.

Línea 349: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 350: `sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);` → habilita reintentos ante fallos transitorios.

Línea 351: `sql.CommandTimeout(60);` → establece el timeout de los comandos SQL.

Línea 352: `})` → cierra la lambda de configuración actual y devuelve el control a la llamada encadenada que la contiene.

Línea 353: `.LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)` → configura el logging de EF Core.

Línea 354: `.EnableDetailedErrors();` → activa errores detallados.

Línea 356: `services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();` → registra un servicio con ciclo de vida scoped.

Línea 357: `services.AddScoped<IServicioOrdenes, ServicioOrdenes>();` → registra un servicio con ciclo de vida scoped.

Línea 359: `using var provider = services.BuildServiceProvider(new ServiceProviderOptions` → construye el ServiceProvider con validación de servicios y garantiza su liberación al salir del ámbito mediante using var.

Línea 360: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 361: `ValidateScopes = true,` → activa la validación de ámbitos para detectar dependencias scoped usadas desde ámbitos incorrectos.

Línea 362: `ValidateOnBuild = true` → obliga al contenedor a validar las resoluciones registradas al construir el ServiceProvider.

Línea 363: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 365: `using var scope = provider.CreateScope();` → crea un ámbito de DI para resolver servicios scoped y lo libera automáticamente al terminar el bloque.

Línea 366: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → resuelve un servicio obligatorio del contenedor.

Línea 367: `context.Database.EnsureDeleted();` → reinicia la base del laboratorio para una ejecución reproducible.

Línea 368: `context.Database.Migrate();` → aplica el historial de migraciones pendiente.

Línea 370: `var orden = new OrdenFabricacion` → declara una variable local y almacena el resultado de la expresión.

Línea 371: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 372: `NumeroOrden = "OF-M2-0001",` → asigna el identificador de negocio usado por la orden de validación acumulativa del módulo.

Línea 373: `Cliente = "Constructora del Norte",` → asigna el cliente de la orden utilizada en el escenario práctico del punto.

Línea 374: `FechaCreacion = DateTime.UtcNow,` → asigna a la orden la fecha de creación en UTC para el escenario reproducible del punto.

Línea 375: `FechaEntrega = DateTime.Today.AddDays(14),` → establece una fecha de entrega prevista catorce días después de la fecha local actual.

Línea 376: `Estado = "Pendiente",` → inicializa la orden con el estado Pendiente, coherente con el flujo del laboratorio.

Línea 377: `Observaciones = "Orden de validación M2",` → añade texto opcional para comprobar el mapeo de Observaciones.

Línea 378: `IsDeleted = false,` → inicializa explícitamente la orden del escenario como no eliminada.

Línea 379: `Planchas =` → inicia el inicializador de la colección de planchas asociadas a la orden.

Línea 380: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 381: `new PlanchaAcero` → crea una nueva plancha dentro del inicializador de la colección Planchas de la orden.

Línea 382: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 383: `Espesor = 10.5,` → asigna a la plancha un espesor positivo para satisfacer las reglas del modelo.

Línea 384: `Ancho = 1500,` → asigna el ancho de la plancha usada en el escenario de persistencia.

Línea 385: `Largo = 3000,` → asigna el largo de la plancha usada en el escenario de persistencia.

Línea 386: `Peso = 371.250m,` → asigna un peso decimal con tres posiciones para validar la precisión configurada.

Línea 387: `Activa = true,` → marca la plancha como activa para que sea visible en los estados que aplican ese criterio.

Línea 388: `}` → cierra el bloque de código actual.

Línea 389: `},` → continúa el inicializador o la llamada del bloque actual.

Línea 390: `Detalle = new DetalleOrden` → crea el detalle uno-a-uno que se persistirá junto con la orden principal.

Línea 391: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 392: `ComposicionQuimica = "C: 0.45%, Mn: 0.75%",` → asigna la composición química utilizada para verificar la entidad DetalleOrden.

Línea 393: `TemperaturaColada = 1550.5,` → asigna la temperatura de colada del detalle de fabricación.

Línea 394: `Notas = "Colada principal"` → asigna las notas opcionales del detalle asociado a la orden.

Línea 395: `},` → continúa el inicializador o la llamada del bloque actual.

Línea 396: `Certificado = new CertificadoCalidad` → crea el certificado uno-a-uno asociado a la orden del escenario.

Línea 397: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 398: `NumeroCertificado = "CERT-0001",` → asigna el identificador natural del certificado de calidad.

Línea 399: `FechaEmision = DateTime.Today,` → asigna la fecha de emisión del certificado usado en la prueba.

Línea 400: `OrganismoCertificador = "Laboratorio Aceria"` → asigna el organismo que emite el certificado de calidad.

Línea 401: `},` → continúa el inicializador o la llamada del bloque actual.

Línea 402: `};` → cierra el inicializador del objeto y finaliza la instrucción.

Línea 404: `var aleacion = new Aleacion` → declara una variable local y almacena el resultado de la expresión.

Línea 405: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 406: `Nombre = "AISI 1045",` → asigna el nombre comercial de la aleación utilizada en el escenario.

Línea 407: `Codigo = "A1045",` → asigna el código natural único de la aleación.

Línea 408: `PorcentajeCarbono = 0.45,` → asigna el porcentaje de carbono dentro del rango permitido por la restricción del modelo.

Línea 409: `PorcentajeManganeso = 0.75`, → asigna el porcentaje de manganeso dentro del rango permitido por la restricción del modelo.

Línea 410: `Descripcion = "Acero medio en carbono"` → asigna una descripción opcional a la aleación.

Línea 411: `};` → cierra el inicializador del objeto y finaliza la instrucción.

Línea 412: `orden.OrdenesAleaciones.Add(new OrdenAleacion` → registra una entidad nueva para insertarla al guardar.

Línea 413: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 414: `Aleacion = aleacion`, → conecta la entidad intermedia `OrdenAleacion` con la aleación creada para el escenario.

Línea 415: `CantidadUtilizada = 1500.500m`, → registra la cantidad utilizada con tres decimales, coherente con `HasPrecision(18, 3)`.

Línea 416: `EstadoRelacion = "Activa"` → marca como activa la relación entre la orden y la aleación.

Línea 417: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 418: `context.OrdenesFabricacion.Add(orden);` → registra una entidad nueva para insertarla al guardar.

Línea 419: `context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo = true, IsDeleted = false });` → registra una entidad nueva para insertarla al guardar.

Línea 420: `context.SaveChanges();` → confirma en SQL Server los cambios del `ChangeTracker`.

Línea 422: `var id = orden.Id;` → declara una variable local y almacena el resultado de la expresión.

Línea 423: `orden.IsDeleted = true;` → marca la orden como eliminada lógicamente sin borrar su fila física.

Línea 424: `orden.DeletedAt = DateTime.UtcNow;` → registra en UTC el instante en que se realizó el borrado lógico.

Línea 425: `context.SaveChanges();` → confirma en SQL Server los cambios del `ChangeTracker`.

Línea 426: `var visibles = context.OrdenesFabricacion.Count();` → cuenta las filas que cumplen la consulta.

Línea 427: `var todas = context.OrdenesFabricacion.IgnoreQueryFilters().Count();` → omite de forma explícita los filtros globales para esta consulta.

Línea 428: `var restaurable = context.OrdenesFabricacion.IgnoreQueryFilters().Single(o => o.Id == id);` → omite de forma explícita los filtros globales para esta consulta.

Línea 429: `restaurable.IsDeleted = false;` → restaura la entidad cargada ignorando filtros al desactivar su marca `IsDeleted`.

Línea 430: `restaurable.DeletedAt = null;` → limpia la fecha de eliminación de la entidad restaurada.

Línea 431: `context.SaveChanges();` → confirma en SQL Server los cambios del `ChangeTracker`.

Línea 432: `global::System.Console.WriteLine($"2.11 OK | Tras borrar visibles: {visibles} | Totales: {todas} | Restaurada: {context.OrdenesFabricacion.Any(o => o.Id == id)}");` → escribe la evidencia de ejecución usada para validar este estado.

Línea 433: `}` → cierra el bloque de código actual.

Línea 434: `}` → cierra el bloque de código actual.

Punto 2.12 - Clean Architecture y Arquitectura Hexagonal

Audiencia: Desarrolladores que ya han completado el modelado acumulativo 2.1-2.11.

Proyecto: 2.12 parte de 2.11 y refactoriza el mismo sistema en Domain, Application, Infrastructure y Console. El refactor cambia la organización del código, no el dominio ni el objetivo de persistencia.

Objetivos de aprendizaje

- Separar dominio y persistencia.
- Mantener EF Core fuera de Domain y Application.
- Definir puertos de repositorio y unidad de trabajo.
- Implementar adaptadores en Infrastructure.
- Registrar dependencias desde una extensión de IServiceCollection.
- Conservar el historial de migraciones y el comportamiento del modelo.

Paso 1: Abrir la solución multiproyecto

```
cd M02/PROYECTO/2.12
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

La solución contiene cuatro proyectos:

```
src/
├─ AceriaData.Domain
├─ AceriaData.Application
├─ AceriaData.Infrastructure
└─ AceriaData.Console
```

Paso 2: Mantener el dominio independiente de EF Core

```
namespace AceriaData.Domain.Entities;

public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    public string Estado { get; set; } = "Pendiente";
    public string? Observaciones { get; set; }
    public bool IsDeleted { get; set; }
    public DateTime? DeletedAt { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
    public DetalleOrden? Detalle { get; set; }
    public CertificadoCalidad? Certificado { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
```

```

    public bool IsDeleted { get; set; }
    public DateTime? DeletedAt { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class Aleacion
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public stringCodigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    public string? Descripcion { get; set; }
    public bool IsDeleted { get; set; }
    public DateTime? DeletedAt { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

public class EstadoOrden
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
    public bool IsDeleted { get; set; }
    public DateTime? DeletedAt { get; set; }
}

public class DetalleOrden
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    public string? Notas { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class CertificadoCalidad
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public string NumeroCertificado { get; set; } = string.Empty;
    public DateTime FechaEmision { get; set; }
    public string OrganismoCertificador { get; set; } = string.Empty;
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }
    public DateTime FechaAsignacion { get; set; } = DateTime.Now;
    public decimal CantidadUtilizada { get; set; }
    public string EstadoRelacion { get; set; } = "Activa";
    public OrdenFabricacion Orden { get; set; } = null!;
    public Aleacion Aleacion { get; set; } = null!;
}

```

Las entidades están en Domain y no necesitan `DbContext`, `DbSet`, `IEntityTypeConfiguration` ni referencias a paquetes de EF Core.

Paso 3: Definir los puertos de aplicación

```
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    List<OrdenFabricacion> ObtenerTodas();
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}

public interface IUnidadDeTrabajo : IDisposable
{
    IOrdenRepositorio Ordenes { get; }
    int Guardar();
}
```

Línea 1: `IOrdenRepositorio` → expresa operaciones que necesita la aplicación sin exponer EF Core.

Línea 2: `IUnidadDeTrabajo` → coordina repositorios y el guardado como abstracción de aplicación.

Paso 4: Crear un caso de uso

```
using AceriaData.Application.Interfaces;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.UseCases;

public sealed class CrearOrdenUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    public CrearOrdenUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;

    public int Ejecutar(string numeroOrden, string cliente)
    {
        _unidad Ordenes.Agregar(new OrdenFabricacion
        {
            NumeroOrden = numeroOrden,
            Cliente = cliente,
            FechaCreacion = DateTime.UtcNow,
            Estado = "Pendiente"
        });
        return _unidad.Guardar();
    }
}
```

El caso de uso depende de interfaces de Application y de entidades de Domain. No referencia `Microsoft.EntityFrameworkCore`.

Paso 5: Colocar EF Core y toda la configuración acumulada en Infrastructure

El `AceriaDbContext` real de 2.12 contiene los `DbSet` y descubre las configuraciones de Infrastructure mediante `ApplyConfigurationsFromAssembly`:

```

using AceriaData.Domain.Entities;
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Infrastructure.Persistence;

public sealed class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
    public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();
    public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();
    public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.ApplyConfigurationsFromAssembly(typeof(AceriaDbContext).Assembly);
    }
}

```

Línea 1: `AceriaDbContext : DbContext` → mantiene EF Core exclusivamente en Infrastructure.

Línea 2: `DbSet<...>` → expone las siete entidades acumuladas del modelo 2.1-2.11.

Línea 3: `OnModelCreating` → centraliza la construcción del modelo relacional.

Línea 4: `ApplyConfigurationsFromAssembly(typeof(AceriaDbContext).Assembly)` → descubre automáticamente todas las clases `IEntityTypeConfiguration<T>` del ensamblado.

Línea 5: `base.OnModelCreating(modelBuilder)` → conserva el comportamiento base antes de aplicar la configuración del dominio.

Paso 5.1: Conservar las configuraciones Fluent API acumuladas

Mover el DbContext no basta: para que 2.12 represente realmente el mismo modelo que 2.11 hay que conservar relaciones, claves alternativas, índices, restricciones, valores por defecto y filtros globales. Estos son los archivos reales del estado 2.12.

OrdenFabricacionConfiguration.cs

```

using AceriaData.Domain.Entities;
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;

namespace AceriaData.Infrastructure.Persistence.Configurations;

public sealed class OrdenFabricacionConfiguration : IEntityTypeConfiguration<OrdenFabricacion>
{
    public void Configure(EntityTypeBuilder<OrdenFabricacion> b)
    {
        b.ToTable("OrdenesFabricacion");
        b.HasKey(x => x.Id);
        b.HasAlternateKey(x => x.NumeroOrden).HasName("AK_OrdenesFabricacion_NumeroOrden");
        b.Property(x => x.NumeroOrden).IsRequired().HasMaxLength(50);
        b.Property(x => x.Cliente).IsRequired().HasMaxLength(200);
        b.Property(x => x.FechaCreacion).HasDefaultValueSql("GETDATE()");
        b.Property(x => x.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");
        b.Property(x => x.Observaciones).HasMaxLength(500);
        b.HasQueryFilter(x => !x.IsDeleted && x.Estado != "Cancelada");
        b.HasIndex(x => x.Cliente).HasDatabaseName("IX_OrdenesFabricacion_Cliente");
        b.HasIndex(x => new { x.Cliente,
x.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Cliente_FechaCreacion");
        b.HasIndex(x => x.FechaEntrega).HasFilter("[Estado] =
'Pendiente'").HasDatabaseName("IX_OrdenesFabricacion_FechaEntrega_Pendientes");
        b.HasIndex(x => x.Estado).IncludeProperties(x => new { x.NumeroOrden, x.Cliente,
x.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Estado_Incluye");
    }
}

```

PlanchaAceroConfiguration.cs

```

using AceriaData.Domain.Entities;
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;

namespace AceriaData.Infrastructure.Persistence.Configurations;

public sealed class PlanchaAceroConfiguration : IEntityTypeConfiguration<PlanchaAcero>
{
    public void Configure(EntityTypeBuilder<PlanchaAcero> b)
    {
        b.ToTable("PlanchasAcero", t =>
        {
            t.HasCheckConstraint("CK_PlanchasAcero_Espesor", "[Espesor] > 0");
            t.HasCheckConstraint("CK_PlanchasAcero_Ancho", "[Ancho] > 0");
            t.HasCheckConstraint("CK_PlanchasAcero_Largo", "[Largo] > 0");
            t.HasCheckConstraint("CK_PlanchasAcero_Peso", "[Peso] > 0");
        });
        b.HasKey(x => x.Id);
        b.Property(x => x.Peso).HasPrecision(18, 3);
        b.Property(x => x.Activa).HasDefaultValue(true);
        b.HasOne(x => x.Orden).WithMany(x => x.Planchas).HasForeignKey(x =>
x.OrdenId).OnDelete(DeleteBehavior.Cascade).IsRequired();
        b.HasIndex(x => new { x.OrdenId, x.Activa }).HasDatabaseName("IX_PlanchasAcero_OrdenId_Activa");
        b.HasIndex(x => x.Espesor).HasFilter("[Activa] =
1").HasDatabaseName("IX_PlanchasAcero_Espesor_Activas");
        b.HasQueryFilter(x => !x.IsDeleted && x.Activa);
    }
}

```

ModeloConfiguration.cs

```

using AceriaData.Domain.Entities;
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;

namespace AceriaData.Infrastructure.Persistence.Configurations;

public sealed class AleacionConfiguration : IEntityTypeConfiguration<Aleacion>
{

```

```

    public void Configure(EntityTypeBuilder<Aleacion> b)
    {
        b.ToTable("Aleaciones", t =>
        {
            t.HasCheckConstraint("CK_Aleaciones_PorcentajeCarbono", "[PorcentajeCarbono] >= 0 AND [PorcentajeCarbono] <= 2");
            t.HasCheckConstraint("CK_Aleaciones_PorcentajeManganeso", "[PorcentajeManganeso] >= 0 AND [PorcentajeManganeso] <= 5");
        });
        b.HasKey(x => x.Id);
        b.HasAlternateKey(x => x.Codigo).HasName("AK_Aleaciones_Codigo");
        b.Property(x => x.Nombre).IsRequired().HasMaxLength(100);
        b.Property(x => x.Codigo).IsRequired().HasMaxLength(20);
        b.Property(x => x.Descripcion).HasMaxLength(500);
        b.HasIndex(x => x.Nombre).HasDatabaseName("IX_Aleaciones_Nombre");
        b.HasIndex(x => new { x.PorcentajeCarbono, x.PorcentajeManganeso }).HasDatabaseName("IX_Aleaciones_Porcentajes");
        b.HasQueryFilter(x => !x.IsDeleted);
    }
}

public sealed class EstadoOrdenConfiguration : IEntityTypeConfiguration<EstadoOrden>
{
    public void Configure(EntityTypeBuilder<EstadoOrden> b)
    {
        b.ToTable("EstadosOrden");
        b.HasKey(x => x.Id);
        b.Property(x => x.Nombre).IsRequired().HasMaxLength(50);
        b.Property(x => x.Descripcion).HasMaxLength(250);
        b.Property(x => x.Activo).HasDefaultValue(true);
        b.HasQueryFilter(x => !x.IsDeleted && x.Activo);
    }
}

public sealed class DetalleOrdenConfiguration : IEntityTypeConfiguration<DetalleOrden>
{
    public void Configure(EntityTypeBuilder<DetalleOrden> b)
    {
        b.ToTable("DetallesOrden");
        b.HasKey(x => x.Id);
        b.Property(x => x.ComposicionQuimica).IsRequired().HasMaxLength(200);
        b.Property(x => x.Notas).HasMaxLength(500);
        b.HasOne(x => x.Orden).WithOne(x => x.Detalle).HasForeignKey<DetalleOrden>(x => x.OrdenId).OnDelete(DeleteBehavior.Cascade).IsRequired();
        b.HasQueryFilter(x => !x.Orden.IsDeleted && x.Orden.Estado != "Cancelada");
    }
}

public sealed class CertificadoCalidadConfiguration : IEntityTypeConfiguration<CertificadoCalidad>
{
    public void Configure(EntityTypeBuilder<CertificadoCalidad> b)
    {
        b.ToTable("CertificadosCalidad");
        b.HasKey(x => x.Id);
        b.HasAlternateKey(x => x.NumeroCertificado).HasName("AK_CertificadosCalidad_NumeroCertificado");
        b.Property(x => x.NumeroCertificado).IsRequired().HasMaxLength(50);
        b.Property(x => x.OrganismoCertificador).IsRequired().HasMaxLength(100);
        b.HasIndex(x => x.FechaEmision).HasDatabaseName("IX_CertificadosCalidad_FechaEmision");
        b.HasOne(x => x.Orden).WithOne(x => x.Certificado).HasForeignKey<CertificadoCalidad>(x => x.OrdenId).OnDelete(DeleteBehavior.Cascade).IsRequired();
        b.HasQueryFilter(x => !x.Orden.IsDeleted && x.Orden.Estado != "Cancelada");
    }
}

public sealed class OrdenAleacionConfiguration : IEntityTypeConfiguration<OrdenAleacion>
{
    public void Configure(EntityTypeBuilder<OrdenAleacion> b)
    {
        b.ToTable("OrdenesAleaciones");
        b.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });
        b.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");
        b.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);
        b.Property(x => x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");
    }
}

```

```

        b.HasOne(x => x.Orden).WithMany(x => x.OrdenesAleaciones).HasForeignKey(x =>
x.OrdenFabricacionId).OnDelete(DeleteBehavior.Cascade);
        b.HasOne(x => x.Aleacion).WithMany(x => x.OrdenesAleaciones).HasForeignKey(x =>
x.AleacionId).OnDelete(DeleteBehavior.Restrict);
        b.HasIndex(x => x.AleacionId).HasDatabaseName("IX_OrdenesAleaciones_AleacionId");
        b.HasIndex(x => x.EstadoRelacion).HasFilter("[EstadoRelacion] =
'Activa'").HasDatabaseName("IX_OrdenesAleaciones_EstadoRelacion_Activas");
        b.HasQueryFilter(x => x.EstadoRelacion == "Activa" && !x.Orden.IsDeleted && !
x.Aleacion.IsDeleted);
    }
}

```

Línea 1: `HasAlternateKey` → conserva los identificadores naturales configurados en 2.8.

Línea 2: `HasIndex` / `HasFilter` / `IncludeProperties` → conserva los índices simples, compuestos, filtrados y con columnas incluidas introducidos en 2.9.

Línea 3: `HasCheckConstraint` → mantiene las reglas de integridad que deben cumplirse también fuera de la aplicación.

Línea 4: `HasQueryFilter` → conserva los filtros globales y el Soft Delete de 2.10-2.11.

Línea 5: `HasOne` / `WithMany` / `WithOne` → conserva las relaciones uno-a-muchos, uno-a-uno y muchos-a-muchos acumuladas.

| Paso 6: Implementar y registrar los adaptadores de Infrastructure

Antes de registrar los puertos hay que implementar sus adaptadores. El archivo real `Repositories.cs` contiene tanto `OrdenRepositorio` como `UnidadDeTrabajo`:

```

using AceriaData.Application.Interfaces;
using AceriaData.Domain.Entities;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Infrastructure.Repositories;

public sealed class OrdenRepositorio : IOrdenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) =>
_context.OrdenesFabricacion.FirstOrDefault(x => x.NumeroOrden == numeroOrden);
    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(x =>
x.Id).ToList();
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
}

public sealed class UnidadDeTrabajo : IUnidadDeTrabajo
{
    private readonly AceriaDbContext _context;
    private IOrdenRepositorio? _ordenes;
    public UnidadDeTrabajo(AceriaDbContext context) => _context = context;
    public IOrdenRepositorio Ordenes => _ordenes ??= new OrdenRepositorio(_context);
    public int Guardar() => _context.SaveChanges();
    public void Dispose() => _context.Dispose();
}

```

Línea 1: `OrdenRepositorio : IOrdenRepositorio` → implementa en Infrastructure el puerto definido por Application.

Línea 2: `ObtenerPorNumero` → traduce una operación de aplicación a una consulta de EF Core.

Línea 3: `Agregar` / `Eliminar` → modifican el estado del DbContext sin llamar por sí mismos a `SaveChanges`.

Línea 4: `UnidadDeTrabajo : IUnidadDeTrabajo` → coordina el repositorio y el `DbContext` compartido.

Línea 5: `Guardar() => _context.SaveChanges()` → concentra la confirmación de cambios en la unidad de trabajo.

Después se registran SQL Server y los adaptadores con la extensión real `AddAceriaInfrastructure`:

```
using AceriaData.Application.Interfaces;
using AceriaData.Infrastructure.Persistence;
using AceriaData.Infrastructure.Repositories;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;

namespace AceriaData.Infrastructure;

public static class DependencyInjection
{
    public static IServiceCollection AddAceriaInfrastructure(this IServiceCollection services, string
connectionString)
    {
        services.AddDbContext<AceriaDbContext>(o => o.UseSqlServer(connectionString));
        services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
        services.AddScoped<IUnidadDeTrabajo, UnidadDeTrabajo>();
        return services;
    }
}
```

Línea 1: `AddDbContext<AceriaDbContext>` → registra el contexto con el proveedor de SQL Server.

Línea 2: `AddScoped<IOrdenRepositorio, OrdenRepositorio>()` → enlaza el puerto de Application con su adaptador de Infrastructure.

Línea 3: `AddScoped<IUnidadDeTrabajo, UnidadDeTrabajo>()` → registra la unidad de trabajo con el mismo ciclo de vida scoped.

Paso 7: Ejecutar desde la capa de entrada

```
using AceriaData.Application.UseCases;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddEnvironmentVariables()
    .Build();

var cs = configuration.GetConnectionString("AceriaDB")
    ?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");

var services = new ServiceCollection();
services.AddAceriaInfrastructure(cs);
services.AddScoped<CrearOrdenUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions { ValidateOnBuild = true,
ValidateScopes = true });
using var scope = provider.CreateScope();
var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();

var crear = scope.ServiceProvider.GetRequiredService<CrearOrdenUseCase>();
crear.Ejecutar("OF-M2-HEX-0001", "Cliente Arquitectura");
var orden = context.OrdenesFabricacion.Single();
Console.WriteLine($"2.12 OK | {orden.NumeroOrden} | {orden.Cliente}");
```

La consola construye el contenedor, resuelve el caso de uso y ejecuta la aplicación. El resultado esperado contiene `2.12 OK`.

Paso 8: Validar migraciones desde la nueva ubicación

Para que las herramientas de EF Core puedan crear el contexto de forma reproducible en tiempo de diseño, Infrastructure contiene la factory real:

```
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Design;

namespace AceriaData.Infrastructure.Persistence;

public sealed class AceriaDesignTimeDbContextFactory : IDesignTimeDbContextFactory<AceriaDbContext>
{
    public AceriaDbContext CreateDbContext(string[] args)
    {
        var options = new DbContextOptionsBuilder<AceriaDbContext>()
            .UseSqlServer("Server=(localdb)\
\MSSQLLocalDB;Database=AceriaDB;Trusted_Connection=True;TrustServerCertificate=True;")
            .Options;
        return new AceriaDbContext(options);
    }
}
```

Línea 1: `IDesignTimeDbContextFactory<AceriaDbContext>` → ofrece a `dotnet ef` una forma explícita de construir el contexto.

Línea 2: `UseSqlServer(...)` → usa la misma instancia LocalDB y la misma base AceriaDB del curso.

Línea 3: `return new AceriaDbContext(options)` → devuelve el contexto configurado sin depender del flujo interactivo de Console.

Con la factory y las migraciones trasladadas a Infrastructure, se valida el historial y se ejecuta el estado final:

```
dotnet ef migrations list --project src/AceriaData.Infrastructure --startup-project src/
AceriaData.Console
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

El historial de migraciones de 2.1-2.11 se conserva y se añade el estado de arquitectura 2.12. El segundo comando debe terminar mostrando el marcador `2.12 OK`.

Explicación línea a línea del estado 2.12 por capas

A diferencia de 2.1-2.11, el estado 2.12 está repartido en varios proyectos y archivos. Las líneas siguientes explican **cada línea no vacía** de los once archivos fuente que la práctica muestra y que el repositorio ejecutable utiliza.

src/AceriaData.Domain/Entities.cs

Línea 1: `namespace AceriaData.Domain.Entities;` → declara el espacio de nombres `AceriaData.Domain.Entities` y sitúa el archivo en su capa arquitectónica.

Línea 3: `public class OrdenFabricacion` → declara la entidad raíz de la orden de fabricación dentro de `Domain`.

Línea 4: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 5: `public int Id { get; set; }` → define la clave técnica de la entidad; su configuración concreta se mantiene en Infrastructure.

Línea 6: `public string NumeroOrden { get; set; } = string.Empty;` → define el identificador de negocio de la orden.

Línea 7: `public string Cliente { get; set; } = string.Empty;` → almacena el cliente asociado a la orden.

Línea 8: `public DateTime FechaCreacion { get; set; }` → almacena la fecha de creación de la orden.

Línea 9: `public DateTime? FechaEntrega { get; set; }` → almacena una fecha de entrega opcional.

Línea 10: `public string Estado { get; set; } = "Pendiente";` → almacena el estado de la orden y parte con el valor de dominio Pendiente.

Línea 11: `public string? Observaciones { get; set; }` → almacena observaciones opcionales.

Línea 12: `public bool IsDeleted { get; set; }` → mantiene la marca utilizada por Soft Delete.

Línea 13: `public DateTime? DeletedAt { get; set; }` → registra opcionalmente cuándo se produjo el borrado lógico.

Línea 14: `public List<PlanchaAcero> Planchas { get; set; } = new();` → expone la navegación de colección hacia las planchas de la orden.

Línea 15: `public DetalleOrden? Detalle { get; set; }` → expone la navegación opcional uno-a-uno hacia DetalleOrden.

Línea 16: `public CertificadoCalidad? Certificado { get; set; }` → expone la navegación opcional uno-a-uno hacia CertificadoCalidad.

Línea 17: `public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();` → expone la colección de filas de unión de la relación muchos-a-muchos.

Línea 18: `}` → cierra el bloque de código actual.

Línea 20: `public class PlanchaAcero` → declara la entidad de plancha relacionada con una orden.

Línea 21: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 22: `public int Id { get; set; }` → define la clave técnica de la entidad; su configuración concreta se mantiene en Infrastructure.

Línea 23: `public int OrdenId { get; set; }` → almacena la clave foránea hacia OrdenFabricacion.

Línea 24: `public double Espesor { get; set; }` → almacena el espesor de la plancha.

Línea 25: `public double Ancho { get; set; }` → almacena el ancho de la plancha.

Línea 26: `public double Largo { get; set; }` → almacena el largo de la plancha.

Línea 27: `public decimal Peso { get; set; }` → almacena el peso con precisión configurada posteriormente mediante Fluent API.

Línea 28: `public bool Activa { get; set; } = true;` → indica si la plancha está activa.

Línea 29: `public bool IsDeleted { get; set; }` → mantiene la marca utilizada por Soft Delete.

Línea 30: `public DateTime? DeletedAt { get; set; }` → registra opcionalmente cuándo se produjo el borrado lógico.

Línea 31: `public OrdenFabricacion Orden { get; set; } = null!;` → expone la navegación hacia la orden principal.

Línea 32: `}` → cierra el bloque de código actual.

Línea 34: `public class Aleacion` → declara la entidad de aleación utilizada por la relación muchos-a-muchos explícita.

Línea 35: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 36: `public int Id { get; set; }` → define la clave técnica de la entidad; su configuración concreta se mantiene en Infrastructure.

Línea 37: `public string Nombre { get; set; } = string.Empty;` → almacena el nombre funcional de la entidad.

Línea 38: `public string Codigo { get; set; } = string.Empty;` → almacena el código natural de la aleación.

Línea 39: `public double PorcentajeCarbono { get; set; }` → almacena el porcentaje de carbono validado por una restricción CHECK.

Línea 40: `public double PorcentajeManganeso { get; set; }` → almacena el porcentaje de manganeso validado por una restricción CHECK.

Línea 41: `public string? Descripcion { get; set; }` → almacena una descripción de negocio.

Línea 42: `public bool IsDeleted { get; set; }` → mantiene la marca utilizada por Soft Delete.

Línea 43: `public DateTime? DeletedAt { get; set; }` → registra opcionalmente cuándo se produjo el borrado lógico.

Línea 44: `public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();` → expone la colección de filas de unión de la relación muchos-a-muchos.

Línea 45: `}` → cierra el bloque de código actual.

Línea 47: `public class EstadoOrden` → declara la entidad que representa un estado de negocio de la orden.

Línea 48: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 49: `public int Id { get; set; }` → define la clave técnica de la entidad; su configuración concreta se mantiene en Infrastructure.

Línea 50: `public string Nombre { get; set; } = string.Empty;` → almacena el nombre funcional de la entidad.

Línea 51: `public string Descripcion { get; set; } = string.Empty;` → almacena una descripción de negocio.

Línea 52: `public bool Activo { get; set; } = true;` → indica si el estado está activo.

Línea 53: `public bool IsDeleted { get; set; }` → mantiene la marca utilizada por Soft Delete.

Línea 54: `public DateTime? DeletedAt { get; set; }` → registra opcionalmente cuándo se produjo el borrado lógico.

Línea 55: `}` → cierra el bloque de código actual.

Línea 57: `public class DetalleOrden` → declara la entidad dependiente de la relación uno-a-uno con OrdenFabricacion.

Línea 58: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 59: `public int Id { get; set; }` → define la clave técnica de la entidad; su configuración concreta se mantiene en Infrastructure.

Línea 60: `public int OrdenId { get; set; }` → almacena la clave foránea hacia OrdenFabricacion.

Línea 61: `public string ComposicionQuimica { get; set; } = string.Empty;` → almacena la composición química del detalle.

Línea 62: `public double TemperaturaColada { get; set; }` → almacena la temperatura de colada.

Línea 63: `public string? Notas { get; set; }` → almacena notas opcionales del detalle.

Línea 64: `public OrdenFabricacion Orden { get; set; } = null!;` → expone la navegación hacia la orden principal.

Línea 65: `}` → cierra el bloque de código actual.

Línea 67: `public class CertificadoCalidad` → declara la entidad dependiente que representa el certificado de calidad de una orden.

Línea 68: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 69: `public int Id { get; set; }` → define la clave técnica de la entidad; su configuración concreta se mantiene en Infrastructure.

Línea 70: `public int OrdenId { get; set; }` → almacena la clave foránea hacia OrdenFabricacion.

Línea 71: `public string NumeroCertificado { get; set; } = string.Empty;` → almacena el identificador natural del certificado.

Línea 72: `public DateTime FechaEmision { get; set; }` → almacena la fecha de emisión del certificado.

Línea 73: `public string OrganismoCertificador { get; set; } = string.Empty;` → almacena el organismo que emitió el certificado.

Línea 74: `public OrdenFabricacion Orden { get; set; } = null!;` → expone la navegación hacia la orden principal.

Línea 75: `}` → cierra el bloque de código actual.

Línea 77: `public class OrdenAleacion` → declara la entidad de unión explícita entre OrdenFabricacion y Aleacion.

Línea 78: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 79: `public int OrdenFabricacionId { get; set; }` → almacena la parte de la clave compuesta que referencia a OrdenFabricacion.

Línea 80: `public int AleacionId { get; set; }` → almacena la parte de la clave compuesta que referencia a Aleacion.

Línea 81: `public DateTime FechaAsignacion { get; set; } = DateTime.Now;` → almacena cuándo se creó la relación orden-aleación.

Línea 82: `public decimal CantidadUtilizada { get; set; }` → almacena la cantidad de aleación utilizada.

Línea 83: `public string EstadoRelacion { get; set; } = "Activa";` → almacena el estado lógico de la relación orden-aleación.

Línea 84: `public OrdenFabricacion Orden { get; set; } = null!;` → expone la navegación hacia la orden principal.

Línea 85: `public Aleacion Aleacion { get; set; } = null!;` → expone la navegación hacia Aleacion.

Línea 86: `}` → cierra el bloque de código actual.

src/AceriaData.Application/Interfaces.cs

Línea 1: `using AceriaData.Domain.Entities;` → importa las entidades del dominio sin introducir dependencias de EF Core en Domain.

Línea 3: `namespace AceriaData.Application.Interfaces;` → declara el espacio de nombres AceriaData.Application.Interfaces y sitúa el archivo en su capa arquitectónica.

Línea 5: `public interface IOrdenRepositorio` → declara el puerto de persistencia de órdenes que Application necesita y que Infrastructure implementará.

Línea 6: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 7: `OrdenFabricacion? ObtenerPorId(int id);` → declara la operación para recuperar una orden por su clave técnica.

Línea 8: `OrdenFabricacion? ObtenerPorNumero(string numeroOrden);` → declara la operación para recuperar una orden por su número de negocio.

Línea 9: `List<OrdenFabricacion> ObtenerTodas();` → declara la operación que devuelve las órdenes visibles para el repositorio.

Línea 10: `void Agregar(OrdenFabricacion orden);` → declara la operación que registra una nueva orden.

Línea 11: `void Eliminar(OrdenFabricacion orden);` → declara la operación que marca una orden para eliminación física cuando el caso de uso lo requiera.

Línea 12: `}` → cierra el bloque de código actual.

Línea 14: `public interface IUnidadDeTrabajo : IDisposable` → declara el puerto de unidad de trabajo que coordina el repositorio y la confirmación de cambios.

Línea 15: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 16: `IOrdenRepositorio Ordenes { get; }` → expone el repositorio de órdenes a través del puerto de unidad de trabajo.

Línea 17: `int Guardar();` → declara la operación que confirma de forma coordinada los cambios pendientes.

Línea 18: `}` → cierra el bloque de código actual.

src/AceriaData.Application/CrearOrdenUseCase.cs

Línea 1: `using AceriaData.Application.Interfaces;` → importa los puertos definidos por Application para que Infrastructure pueda implementarlos.

Línea 2: `using AceriaData.Domain.Entities;` → importa las entidades del dominio sin introducir dependencias de EF Core en Domain.

Línea 4: `namespace AceriaData.Application.UseCases;` → declara el espacio de nombres `AceriaData.Application.UseCases` y sitúa el archivo en su capa arquitectónica.

Línea 6: `public sealed class CrearOrdenUseCase` → declara el caso de uso de Application encargado de crear una orden sin depender de EF Core.

Línea 7: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 8: `private readonly IUnidadDeTrabajo _unidad;` → guarda la dependencia del caso de uso contra la abstracción `IUnidadDeTrabajo`.

Línea 9: `public CrearOrdenUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;` → inyecta `IUnidadDeTrabajo` en el caso de uso sin acoplar Application a Infrastructure.

Línea 11: `public int Ejecutar(string numeroOrden, string cliente)` → define la operación de aplicación que crea una orden y devuelve el resultado del guardado.

Línea 12: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 13: `_unidad.Ordenes.Agregar(new OrdenFabricacion` → crea una orden a través del puerto `IOrdenRepositorio` expuesto por la unidad de trabajo.

Línea 14: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 15: `NumeroOrden = numeroOrden,` → copia al dominio el número de orden recibido por el caso de uso.

Línea 16: `Cliente = cliente,` → copia al dominio el cliente recibido por el caso de uso.

Línea 17: `FechaCreacion = DateTime.UtcNow,` → asigna una fecha de creación UTC desde la aplicación.

Línea 18: `Estado = "Pendiente"` → inicializa la nueva orden con el estado `Pendiente`.

Línea 19: `});` → cierra el inicializador o la llamada iniciada en las líneas anteriores.

Línea 20: `return _unidad.Guardar();` → confirma los cambios una única vez mediante la unidad de trabajo y devuelve las filas afectadas.

Línea 21: `}` → cierra el bloque de código actual.

Línea 22: `}` → cierra el bloque de código actual.

src/AceriaData.Infrastructure/Persistence/AceriaDbContext.cs

Línea 1: `using AceriaData.Domain.Entities;` → importa las entidades del dominio sin introducir dependencias de EF Core en Domain.

Línea 2: `using Microsoft.EntityFrameworkCore;` → importa las APIs de Entity Framework Core necesarias en Infrastructure o en el composition root.

Línea 4: `namespace AceriaData.Infrastructure.Persistence;` → declara el espacio de nombres `AceriaData.Infrastructure.Persistence` y sitúa el archivo en su capa arquitectónica.

Línea 6: `public sealed class AceriaDbContext : DbContext` → declara el `DbContext` de Infrastructure y concentra el modelo EF Core.

Línea 7: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 8: `public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }` → recibe `DbContextOptions` por inyección y los pasa al constructor base de `DbContext`.

Línea 10: `public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();` → expone el `DbSet` `OrdenesFabricacion` para consultar y persistir esa entidad.

Línea 11: `public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();` → expone el `DbSet` `PlanchasAcero` para consultar y persistir esa entidad.

Línea 12: `public DbSet<Aleacion> Aleaciones => Set<Aleacion>();` → expone el `DbSet` `Aleaciones` para consultar y persistir esa entidad.

Línea 13: `public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();` → expone el `DbSet` `EstadosOrden` para consultar y persistir esa entidad.

Línea 14: `public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();` → expone el `DbSet` `DetallesOrden` para consultar y persistir esa entidad.

Línea 15: `public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();` → expone el `DbSet` `CertificadosCalidad` para consultar y persistir esa entidad.

Línea 16: `public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();` → expone el `DbSet` `OrdenesAleaciones` para consultar y persistir esa entidad.

Línea 18: `protected override void OnModelCreating(ModelBuilder modelBuilder)` → sobrescribe `OnModelCreating` para aplicar la configuración relacional del modelo.

Línea 19: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 20: `base.OnModelCreating(modelBuilder);` → ejecuta primero la configuración base de `DbContext`.

Línea 21: `modelBuilder.ApplyConfigurationsFromAssembly(typeof(AceriaDbContext).Assembly);` → descubre y aplica automáticamente todas las implementaciones `IEntityTypeConfiguration` del ensamblado de Infrastructure.

Línea 22: `}` → cierra el bloque de código actual.

Línea 23: `}` → cierra el bloque de código actual.

src/AceriaData.Infrastructure/Persistence/Configurations/OrdenFabricacionConfiguration.cs

Línea 1: `using AceriaData.Domain.Entities;` → importa las entidades del dominio sin introducir dependencias de EF Core en Domain.

Línea 2: `using Microsoft.EntityFrameworkCore;` → importa las APIs de Entity Framework Core necesarias en Infrastructure o en el composition root.

Línea 3: `using Microsoft.EntityFrameworkCore.Metadata.Builders;` → importa las APIs de Entity Framework Core necesarias en Infrastructure o en el composition root.

Línea 5: `namespace AceriaData.Infrastructure.Persistence.Configurations;` → declara el espacio de nombres `AceriaData.Infrastructure.Persistence.Configurations` y sitúa el archivo en su capa arquitectónica.

Línea 7: `public sealed class OrdenFabricacionConfiguration : IEntityTypeConfiguration<OrdenFabricacion>` → declara la configuración Fluent API de `OrdenFabricacion`.

Línea 8: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 9: `public void Configure(EntityTypeBuilder<OrdenFabricacion> b)` → implementa el método `Configure` de `IEntityTypeConfiguration` para describir el mapeo de la entidad.

Línea 10: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 11: `b.ToTable("OrdenesFabricacion");` → asigna la entidad a la tabla SQL indicada; cuando incluye lambda, configura además restricciones de tabla.

Línea 12: `b.HasKey(x => x.Id);` → define explícitamente la clave primaria de la entidad.

Línea 13: `b.HasAlternateKey(x => x.NumeroOrden).HasName("AK_OrdenesFabricacion_NumeroOrden");` → define una clave alternativa para el identificador natural y permite referenciarlo como principal.

Línea 14: `b.Property(x => x.NumeroOrden).IsRequired().HasMaxLength(50);` → marca la propiedad como requerida y limita su longitud en el esquema.

Línea 15: `b.Property(x => x.Cliente).IsRequired().HasMaxLength(200);` → marca la propiedad como requerida y limita su longitud en el esquema.

Línea 16: `b.Property(x => x.FechaCreacion).HasDefaultValueSql("GETDATE()");` → configura una propiedad cuyo valor por defecto se obtiene mediante una expresión SQL del servidor.

Línea 17: `b.Property(x => x.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");` → configura un valor por defecto de base de datos para la propiedad.

Línea 18: `b.Property(x => x.Observaciones).HasMaxLength(500);` → limita la longitud máxima de la columna correspondiente.

Línea 19: `b.HasQueryFilter(x => !x.IsDeleted && x.Estado != "Cancelada");` → aplica el filtro global que oculta registros eliminados o inactivos en las consultas normales.

Línea 20: `b.HasIndex(x => x.Cliente).HasDatabaseName("IX_OrdenesFabricacion_Cliente");` → crea y nombra un índice sobre la propiedad indicada.

Línea 21: `b.HasIndex(x => new { x.Cliente, x.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Cliente_FechaCreacion");` → crea un índice compuesto sobre las propiedades indicadas.

Línea 22: `b.HasIndex(x => x.FechaEntrega).HasFilter("[Estado] = 'Pendiente'").HasDatabaseName("IX_OrdenesFabricacion_FechaEntrega_Pendientes");` → crea un índice filtrado para el subconjunto de filas indicado.

Línea 23: `b.HasIndex(x => x.Estado).IncludeProperties(x => new { x.NumeroOrden, x.Cliente, x.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Estado_Incluye");` → crea un índice con columnas incluidas para cubrir consultas frecuentes.

Línea 24: `}` → cierra el bloque de código actual.

Línea 25: `}` → cierra el bloque de código actual.

src/AceriaData.Infrastructure/Persistence/Configurations/PlanchaAceroConfiguration.cs

Línea 1: `using AceriaData.Domain.Entities;` → importa las entidades del dominio sin introducir dependencias de EF Core en Domain.

Línea 2: `using Microsoft.EntityFrameworkCore;` → importa las APIs de Entity Framework Core necesarias en Infrastructure o en el composition root.

Línea 3: `using Microsoft.EntityFrameworkCore.Metadata.Builders;` → importa las APIs de Entity Framework Core necesarias en Infrastructure o en el composition root.

Línea 5: `namespace AceriaData.Infrastructure.Persistence.Configurations;` → declara el espacio de nombres `AceriaData.Infrastructure.Persistence.Configurations` y sitúa el archivo en su capa arquitectónica.

Línea 7: `public sealed class PlanchaAceroConfiguration : IEntityTypeConfiguration<PlanchaAcero>` → declara la configuración Fluent API de `PlanchaAcero`.

Línea 8: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 9: `public void Configure(EntityTypeBuilder<PlanchaAcero> b)` → implementa el método `Configure` de `IEntityTypeConfiguration` para describir el mapeo de la entidad.

Línea 10: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 11: `b.ToTable("PlanchasAcero", t =>` → asigna la entidad a la tabla SQL indicada; cuando incluye lambda, configura además restricciones de tabla.

Línea 12: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 13: `t.HasCheckConstraint("CK_PlanchasAcero_Espesor", "[Espesor] > 0");` → añade una restricción CHECK en SQL Server para reforzar una regla de integridad en la base de datos.

Línea 14: `t.HasCheckConstraint("CK_PlanchasAcero_Ancho", "[Ancho] > 0");` → añade una restricción CHECK en SQL Server para reforzar una regla de integridad en la base de datos.

Línea 15: `t.HasCheckConstraint("CK_PlanchasAcero_Largo", "[Largo] > 0");` → añade una restricción CHECK en SQL Server para reforzar una regla de integridad en la base de datos.

Línea 16: `t.HasCheckConstraint("CK_PlanchasAcero_Peso", "[Peso] > 0");` → añade una restricción CHECK en SQL Server para reforzar una regla de integridad en la base de datos.

Línea 17: `});` → cierra el inicializador o la llamada iniciada en las líneas anteriores.

Línea 18: `b.HasKey(x => x.Id);` → define explícitamente la clave primaria de la entidad.

Línea 19: `b.Property(x => x.Peso).HasPrecision(18, 3);` → configura precisión y escala para el valor decimal.

Línea 20: `b.Property(x => x.Activa).HasDefaultValue(true);` → configura un valor por defecto de base de datos para la propiedad.

Línea 21: `b.HasOne(x => x.Orden).WithMany(x => x.Planchas).HasForeignKey(x => x.OrdenId).OnDelete(DeleteBehavior.Cascade).IsRequired();` → configura una relación uno-a-muchos y encadena la clave foránea y el comportamiento de borrado.

Línea 22: `b.HasIndex(x => new { x.OrdenId, x.Activa }).HasDatabaseName("IX_PlanchasAcero_OrdenId_Activa");` → crea un índice compuesto sobre las propiedades indicadas.

Línea 23: `b.HasIndex(x => x.Espesor).HasFilter("[Activa] = 1").HasDatabaseName("IX_PlanchasAcero_Espesor_Activas");` → crea un índice filtrado para el subconjunto de filas indicado.

Línea 24: `b.HasQueryFilter(x => !x.IsDeleted && x.Activa);` → aplica el filtro global que oculta registros eliminados o inactivos en las consultas normales.

Línea 25: `}` → cierra el bloque de código actual.

Línea 26: `}` → cierra el bloque de código actual.

src/AceriaData.Infrastructure/Persistence/Configurations/ModeloConfiguration.cs

Línea 1: `using AceriaData.Domain.Entities;` → importa las entidades del dominio sin introducir dependencias de EF Core en Domain.

Línea 2: `using Microsoft.EntityFrameworkCore;` → importa las APIs de Entity Framework Core necesarias en Infrastructure o en el composition root.

Línea 3: `using Microsoft.EntityFrameworkCore.Metadata.Builders;` → importa las APIs de Entity Framework Core necesarias en Infrastructure o en el composition root.

Línea 5: `namespace AceriaData.Infrastructure.Persistence.Configurations;` → declara el espacio de nombres `AceriaData.Infrastructure.Persistence.Configurations` y sitúa el archivo en su capa arquitectónica.

Línea 7: `public sealed class AleacionConfiguration : IEntityTypeConfiguration<Aleacion>` → declara la configuración Fluent API de Aleacion.

Línea 8: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 9: `public void Configure(EntityTypeBuilder<Aleacion> b)` → implementa el método `Configure` de `IEntityTypeConfiguration` para describir el mapeo de la entidad.

Línea 10: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 11: `b.ToTable("Aleaciones", t =>` → asigna la entidad a la tabla SQL indicada; cuando incluye lambda, configura además restricciones de tabla.

Línea 12: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 13: `t.HasCheckConstraint("CK_Aleaciones_PorcentajeCarbono", "[PorcentajeCarbono] >= 0 AND [PorcentajeCarbono] <= 2");` → añade una restricción CHECK en SQL Server para reforzar una regla de integridad en la base de datos.

Línea 14: `t.HasCheckConstraint("CK_Aleaciones_PorcentajeManganeso", "[PorcentajeManganeso] >= 0 AND [PorcentajeManganeso] <= 5");` → añade una restricción CHECK en SQL Server para reforzar una regla de integridad en la base de datos.

Línea 15: `});` → cierra el inicializador o la llamada iniciada en las líneas anteriores.

Línea 16: `b.HasKey(x => x.Id);` → define explícitamente la clave primaria de la entidad.

Línea 17: `b.HasAlternateKey(x => x.Codigo).HasName("AK_Aleaciones_Codigo");` → define una clave alternativa para el identificador natural y permite referenciarlo como principal.

Línea 18: `b.Property(x => x.Nombre).IsRequired().HasMaxLength(100);` → marca la propiedad como requerida y limita su longitud en el esquema.

Línea 19: `b.Property(x => x.Codigo).IsRequired().HasMaxLength(20);` → marca la propiedad como requerida y limita su longitud en el esquema.

Línea 20: `b.Property(x => x.Descripcion).HasMaxLength(500);` → limita la longitud máxima de la columna correspondiente.

Línea 21: `b.HasIndex(x => x.Nombre).HasDatabaseName("IX_Aleaciones_Nombre");` → crea y nombra un índice sobre la propiedad indicada.

Línea 22: `b.HasIndex(x => new { x.PorcentajeCarbono, x.PorcentajeManganeso }).HasDatabaseName("IX_Aleaciones_Porcentajes");` → crea un índice compuesto sobre las propiedades indicadas.

Línea 23: `b.HasQueryFilter(x => !x.IsDeleted);` → aplica el filtro global que oculta registros eliminados o inactivos en las consultas normales.

Línea 24: } → cierra el bloque de código actual.

Línea 25: } → cierra el bloque de código actual.

Línea 27: `public sealed class EstadoOrdenConfiguration : IEntityTypeConfiguration<EstadoOrden>`
→ declara la configuración Fluent API de EstadoOrden.

Línea 28: { → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 29: `public void Configure(EntityTypeBuilder<EstadoOrden> b)` → implementa el método Configure de IEntityTypeConfiguration para describir el mapeo de la entidad.

Línea 30: { → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 31: `b.ToTable("EstadosOrden");` → asigna la entidad a la tabla SQL indicada; cuando incluye lambda, configura además restricciones de tabla.

Línea 32: `b.HasKey(x => x.Id);` → define explícitamente la clave primaria de la entidad.

Línea 33: `b.Property(x => x.Nombre).IsRequired().HasMaxLength(50);` → marca la propiedad como requerida y limita su longitud en el esquema.

Línea 34: `b.Property(x => x.Descripcion).HasMaxLength(250);` → limita la longitud máxima de la columna correspondiente.

Línea 35: `b.Property(x => x.Activo).HasDefaultValue(true);` → configura un valor por defecto de base de datos para la propiedad.

Línea 36: `b.HasQueryFilter(x => !x.IsDeleted && x.Activo);` → aplica el filtro global que oculta registros eliminados o inactivos en las consultas normales.

Línea 37: } → cierra el bloque de código actual.

Línea 38: } → cierra el bloque de código actual.

Línea 40: `public sealed class DetalleOrdenConfiguration : IEntityTypeConfiguration<DetalleOrden>`
→ declara la configuración Fluent API de DetalleOrden.

Línea 41: { → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 42: `public void Configure(EntityTypeBuilder<DetalleOrden> b)` → implementa el método Configure de IEntityTypeConfiguration para describir el mapeo de la entidad.

Línea 43: { → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 44: `b.ToTable("DetallesOrden");` → asigna la entidad a la tabla SQL indicada; cuando incluye lambda, configura además restricciones de tabla.

Línea 45: `b.HasKey(x => x.Id);` → define explícitamente la clave primaria de la entidad.

Línea 46: `b.Property(x => x.ComposicionQuimica).IsRequired().HasMaxLength(200);` → marca la propiedad como requerida y limita su longitud en el esquema.

Línea 47: `b.Property(x => x.Notas).HasMaxLength(500);` → limita la longitud máxima de la columna correspondiente.

Línea 48: `b.HasOne(x => x.Orden).WithOne(x => x.Detalle).HasForeignKey<DetalleOrden>(x => x.OrdenId).OnDelete(DeleteBehavior.Cascade).IsRequired();` → configura una relación uno-a-uno y encadena su clave foránea y comportamiento de borrado.

Línea 49: `b.HasQueryFilter(x => !x.Orden.IsDeleted && x.Orden.Estado != "Cancelada");` → aplica el filtro global que oculta registros eliminados o inactivos en las consultas normales.

Línea 50: } → cierra el bloque de código actual.

Línea 51: } → cierra el bloque de código actual.

Línea 53: `public sealed class CertificadoCalidadConfiguration :`

`IEntityTypeConfiguration<CertificadoCalidad>` → declara la configuración Fluent API de `CertificadoCalidad`.

Línea 54: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 55: `public void Configure(EntityTypeBuilder<CertificadoCalidad> b)` → implementa el método `Configure` de `IEntityTypeConfiguration` para describir el mapeo de la entidad.

Línea 56: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 57: `b.ToTable("CertificadosCalidad");` → asigna la entidad a la tabla SQL indicada; cuando incluye `lambda`, configura además restricciones de tabla.

Línea 58: `b.HasKey(x => x.Id);` → define explícitamente la clave primaria de la entidad.

Línea 59: `b.HasAlternateKey(x =>`

`x.NumeroCertificado).HasName("AK_CertificadosCalidad_NumeroCertificado");` → define una clave alternativa para el identificador natural y permite referenciarlo como principal.

Línea 60: `b.Property(x => x.NumeroCertificado).IsRequired().HasMaxLength(50);` → marca la propiedad como requerida y limita su longitud en el esquema.

Línea 61: `b.Property(x => x.OrganismoCertificador).IsRequired().HasMaxLength(100);` → marca la propiedad como requerida y limita su longitud en el esquema.

Línea 62: `b.HasIndex(x =>`

`x.FechaEmision).HasDatabaseName("IX_CertificadosCalidad_FechaEmision");` → crea y nombra un índice sobre la propiedad indicada.

Línea 63: `b.HasOne(x => x.Orden).WithOne(x => x.Certificado).HasForeignKey<CertificadoCalidad>(x => x.OrdenId).OnDelete(DeleteBehavior.Cascade).IsRequired();` → configura una relación uno-a-uno y encadena su clave foránea y comportamiento de borrado.

Línea 64: `b.HasQueryFilter(x => !x.Orden.IsDeleted && x.Orden.Estado != "Cancelada");` → aplica el filtro global que oculta registros eliminados o inactivos en las consultas normales.

Línea 65: `}` → cierra el bloque de código actual.

Línea 66: `}` → cierra el bloque de código actual.

Línea 68: `public sealed class OrdenAleacionConfiguration :`

`IEntityTypeConfiguration<OrdenAleacion>` → declara la configuración Fluent API de `OrdenAleacion`.

Línea 69: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 70: `public void Configure(EntityTypeBuilder<OrdenAleacion> b)` → implementa el método `Configure` de `IEntityTypeConfiguration` para describir el mapeo de la entidad.

Línea 71: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 72: `b.ToTable("OrdenesAleaciones");` → asigna la entidad a la tabla SQL indicada; cuando incluye `lambda`, configura además restricciones de tabla.

Línea 73: `b.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });` → define la clave primaria compuesta de la entidad de unión con ambas claves foráneas.

Línea 74: `b.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");` → configura una propiedad cuyo valor por defecto se obtiene mediante una expresión SQL del servidor.

Línea 75: `b.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);` → configura precisión y escala para el valor decimal.

Línea 76: `b.Property(x =>`

`x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");` → configura un valor por defecto de base de datos para la propiedad.

Línea 77: `b.HasOne(x => x.Orden).WithMany(x => x.OrdenesAleaciones).HasForeignKey(x => x.OrdenFabricacionId).OnDelete(DeleteBehavior.Cascade);` → configura una relación uno-a-muchos y encadena la clave foránea y el comportamiento de borrado.

Línea 78: `b.HasOne(x => x.Aleacion).WithMany(x => x.OrdenesAleaciones).HasForeignKey(x => x.AleacionId).OnDelete(DeleteBehavior.Restrict);` → configura una relación uno-a-muchos y encadena la clave foránea y el comportamiento de borrado.

Línea 79: `b.HasIndex(x => x.AleacionId).HasDatabaseName("IX_OrdenesAleaciones_AleacionId");` → crea y nombra un índice sobre la propiedad indicada.

Línea 80: `b.HasIndex(x => x.EstadoRelacion).HasFilter("[EstadoRelacion] = 'Activa']").HasDatabaseName("IX_OrdenesAleaciones_EstadoRelacion_Activas");` → crea un índice filtrado para el subconjunto de filas indicado.

Línea 81: `b.HasQueryFilter(x => x.EstadoRelacion == "Activa" && !x.Orden.IsDeleted && !x.Aleacion.IsDeleted);` → aplica el filtro global que oculta registros eliminados o inactivos en las consultas normales.

Línea 82: `}` → cierra el bloque de código actual.

Línea 83: `}` → cierra el bloque de código actual.

src/AceriaData.Infrastructure/Repositories/Repositories.cs

Línea 1: `using AceriaData.Application.Interfaces;` → importa los puertos definidos por Application para que Infrastructure pueda implementarlos.

Línea 2: `using AceriaData.Domain.Entities;` → importa las entidades del dominio sin introducir dependencias de EF Core en Domain.

Línea 3: `using AceriaData.Infrastructure.Persistence;` → importa el DbContext y los componentes de persistencia de Infrastructure.

Línea 4: `using Microsoft.EntityFrameworkCore;` → importa las APIs de Entity Framework Core necesarias en Infrastructure o en el composition root.

Línea 6: `namespace AceriaData.Infrastructure.Repositories;` → declara el espacio de nombres AceriaData.Infrastructure.Repositories y sitúa el archivo en su capa arquitectónica.

Línea 8: `public sealed class OrdenRepository : IOrdenRepository` → declara el adaptador de Infrastructure que implementa IOrdenRepository.

Línea 9: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 10: `private readonly AceriaDbContext _context;` → mantiene la referencia al DbContext compartido por el adaptador dentro del ámbito DI.

Línea 11: `public OrdenRepository(AceriaDbContext context) => _context = context;` → inyecta AceriaDbContext en el adaptador de repositorio.

Línea 12: `public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);` → implementa la búsqueda por clave mediante DbSet.Find.

Línea 13: `public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(x => x.NumeroOrden == numeroOrden);` → implementa la búsqueda por número de orden mediante una consulta LINQ traducida a SQL.

Línea 14: `public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(x => x.Id).ToList();` → materializa las órdenes ordenadas por Id.

Línea 15: `public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);` → registra la orden como Added en el ChangeTracker sin ejecutar todavía SaveChanges.

Línea 16: `public void Eliminar(OrdenFabricacion orden) =>`

`_context.OrdenesFabricacion.Remove(orden);` → marca la orden como Deleted en el ChangeTracker; la escritura se confirma al guardar.

Línea 17: `}` → cierra el bloque de código actual.

Línea 19: `public sealed class UnidadDeTrabajo : IUnidadDeTrabajo` → declara el adaptador que implementa IUnidadDeTrabajo y concentra SaveChanges.

Línea 20: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 21: `private readonly AceriaDbContext _context;` → mantiene la referencia al DbContext compartido por el adaptador dentro del ámbito DI.

Línea 22: `private IOrdenRepositorio? _ordenes;` → mantiene la instancia perezosa del repositorio de órdenes.

Línea 23: `public UnidadDeTrabajo(AceriaDbContext context) => _context = context;` → inyecta el mismo AceriaDbContext en la unidad de trabajo.

Línea 24: `public IOrdenRepositorio Ordenes => _ordenes ??= new OrdenRepositorio(_context);` → crea perezosamente OrdenRepositorio y reutiliza el DbContext de la unidad de trabajo.

Línea 25: `public int Guardar() => _context.SaveChanges();` → confirma en SQL Server todos los cambios pendientes del DbContext.

Línea 26: `public void Dispose() => _context.Dispose();` → libera el DbContext cuando termina la unidad de trabajo.

Línea 27: `}` → cierra el bloque de código actual.

src/AceriaData.Infrastructure/DependencyInjection.cs

Línea 1: `using AceriaData.Application.Interfaces;` → importa los puertos definidos por Application para que Infrastructure pueda implementarlos.

Línea 2: `using AceriaData.Infrastructure.Persistence;` → importa el DbContext y los componentes de persistencia de Infrastructure.

Línea 3: `using AceriaData.Infrastructure.Repositories;` → importa los adaptadores concretos que implementan los puertos de Application.

Línea 4: `using Microsoft.EntityFrameworkCore;` → importa las APIs de Entity Framework Core necesarias en Infrastructure o en el composition root.

Línea 5: `using Microsoft.Extensions.DependencyInjection;` → importa el contenedor de dependencias y sus métodos de registro/resolución.

Línea 7: `namespace AceriaData.Infrastructure;` → declara el espacio de nombres AceriaData.Infrastructure y sitúa el archivo en su capa arquitectónica.

Línea 9: `public static class DependencyInjection` → declara una clase estática usada como contenedor de extensiones de registro.

Línea 10: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 11: `public static IServiceCollection AddAceriaInfrastructure(this IServiceCollection services, string connectionString)` → define el método de extensión que encapsula todos los registros de Infrastructure.

Línea 12: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 13: `services.AddDbContext<AceriaDbContext>(o => o.UseSqlServer(connectionString));` → registra AceriaDbContext con SQL Server usando la cadena recibida por el composition root.

Línea 14: `services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();` → vincula IOrdenRepositorio con OrdenRepositorio mediante ciclo de vida scoped.

Línea 15: `services.AddScoped<IUnidadDeTrabajo, UnidadDeTrabajo>();` → vincula IUnidadDeTrabajo con UnidadDeTrabajo mediante ciclo de vida scoped.

Línea 16: `return services;` → devuelve la colección para permitir encadenar el registro de servicios.

Línea 17: `}` → cierra el bloque de código actual.

Línea 18: `}` → cierra el bloque de código actual.

src/AceriaData.Console/Program.cs

Línea 1: `using AceriaData.Application.UseCases;` → importa el caso de uso que la capa de entrada va a resolver y ejecutar.

Línea 2: `using AceriaData.Infrastructure;` → importa el espacio de nombres requerido por los tipos usados en este archivo.

Línea 3: `using AceriaData.Infrastructure.Persistence;` → importa el DbContext y los componentes de persistencia de Infrastructure.

Línea 4: `using Microsoft.EntityFrameworkCore;` → importa las APIs de Entity Framework Core necesarias en Infrastructure o en el composition root.

Línea 5: `using Microsoft.Extensions.Configuration;` → importa la API de configuración usada para leer appsettings.json y variables de entorno.

Línea 6: `using Microsoft.Extensions.DependencyInjection;` → importa el contenedor de dependencias y sus métodos de registro/resolución.

Línea 8: `var configuration = new ConfigurationBuilder()` → inicia la configuración de la aplicación de consola.

Línea 9: `.SetBasePath(AppContext.BaseDirectory)` → establece la carpeta base desde la que se localizará appsettings.json.

Línea 10: `.AddJsonFile("appsettings.json", optional: false)` → carga appsettings.json como fuente obligatoria de configuración.

Línea 11: `.AddEnvironmentVariables()` → añade variables de entorno, que pueden complementar o sobrescribir la configuración.

Línea 12: `.Build();` → construye el objeto IConfiguration definitivo.

Línea 14: `var cs = configuration.GetConnectionString("AceriaDB")` → obtiene la cadena AceriaDB desde la configuración.

Línea 15: `?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");` → falla de forma explícita si la cadena de conexión no está configurada.

Línea 17: `var services = new ServiceCollection();` → crea la colección de servicios del composition root.

Línea 18: `services.AddAceriaInfrastructure(cs);` → registra en Console todos los adaptadores de Infrastructure mediante su extensión.

Línea 19: `services.AddScoped<CrearOrdenUseCase>();` → registra el caso de uso para resolverlo dentro del mismo ámbito.

Línea 21: `using var provider = services.BuildServiceProvider(new ServiceProviderOptions { ValidateOnBuild = true, ValidateScopes = true });` → construye el proveedor de inyección de dependencias con validación y garantiza su liberación automática al terminar.

Línea 22: `using var scope = provider.CreateScope();` → crea un ámbito scoped para resolver DbContext, repositorios y casos de uso, y lo libera automáticamente al terminar.

Línea 23: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → resuelve el DbContext registrado en Infrastructure.

Línea 24: `context.Database.EnsureDeleted();` → elimina la base del laboratorio para que la ejecución del laboratorio sea reproducible; no sustituye a Migrations.

Línea 25: `context.Database.Migrate();` → aplica el historial real de migraciones hasta el esquema 2.12.

Línea 27: `var crear = scope.ServiceProvider.GetRequiredService<CrearOrdenUseCase>();` → resuelve el caso de uso desde el contenedor.

Línea 28: `crear.Ejecutar("0F-M2-HEX-0001", "Cliente Arquitectura");` → ejecuta el caso de uso y persiste la orden a través de los puertos de Application.

Línea 29: `var orden = context.OrdenesFabricacion.Single();` → consulta la única orden creada para verificar el resultado del escenario práctico.

Línea 30: `Console.WriteLine($"2.12 OK | {orden.NumeroOrden} | {orden.Cliente}");` → imprime el marcador 2.12 OK y los datos persistidos que confirma el resultado esperado del punto.

src/AceriaData.Infrastructure/Persistence/AceriaDesignTimeDbContextFactory.cs

Línea 1: `using Microsoft.EntityFrameworkCore;` → importa las APIs de Entity Framework Core necesarias en Infrastructure o en el composition root.

Línea 2: `using Microsoft.EntityFrameworkCore.Design;` → importa el contrato de factory de diseño utilizado por las herramientas dotnet ef.

Línea 4: `namespace AceriaData.Infrastructure.Persistence;` → declara el espacio de nombres `AceriaData.Infrastructure.Persistence` y sitúa el archivo en su capa arquitectónica.

Línea 6: `public sealed class AceriaDesignTimeDbContextFactory : IDesignTimeDbContextFactory<AceriaDbContext>` → declara la factory de diseño que permite a dotnet ef construir `AceriaDbContext` sin arrancar Console.

Línea 7: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 8: `public AceriaDbContext CreateDbContext(string[] args)` → implementa el contrato de diseño que usan las herramientas EF Core.

Línea 9: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 10: `var options = new DbContextOptionsBuilder<AceriaDbContext>()` → inicia la construcción de `DbContextOptions` para el contexto de diseño.

Línea 11: `.UseSqlServer("Server=(localdb)\MSSQLLocalDB;Database=AceriaDB;Trusted_Connection=True;TrustServerCertificate=True;")` → configura SQL Server LocalDB y la base `AceriaDB` para las operaciones de diseño.

Línea 12: `.Options;` → extrae las opciones ya configuradas del builder.

Línea 13: `return new AceriaDbContext(options);` → crea y devuelve `AceriaDbContext` con esas opciones para dotnet ef.

Línea 14: `}` → cierra el bloque de código actual.

Línea 15: `}` → cierra el bloque de código actual.

Errores comunes

Error	Causa	Solución
Domain referencia EF Core	Se movieron entidades junto con tipos de persistencia	Mantener Domain como clases de negocio sin paquete EF Core.
Application usa DbContext	Se filtró una dependencia de infraestructura	Depender de <code>IOrdenRepositorio/ IUnidadDeTrabajo</code> .

Error	Causa	Solución
EF Tools no encuentra el contexto	Startup o project incorrectos	Indicar <code>--project Infrastructure --startup-project Console</code> .
La consola no resuelve servicios	Falta AddAceriaInfrastructure	Registrar la infraestructura antes de BuildServiceProvider.

Retorno

Retorno: Crear una orden desde CrearOrdenUseCase y comprobar que Application no referencia Microsoft.EntityFrameworkCore.

Solución: revisar los `.csproj` : Domain y Application no contienen referencias a `Microsoft.EntityFrameworkCore`; Infrastructure sí contiene `Microsoft.EntityFrameworkCore.SqlServer` y `Design`. Ejecutar después la solución y comprobar el marcador `2.12 OK`.

Analogía final

Clean Architecture separa el conocimiento del negocio de la tecnología de persistencia: EF Core pasa a ser un adaptador reemplazable alrededor del núcleo.

Resultado esperado

M2 termina con una solución multiproyecto compilable y ejecutable, el mismo modelo acumulado, EF Core aislado en Infrastructure y una aplicación que accede a persistencia mediante puertos definidos fuera de la infraestructura.

Conexión con el siguiente módulo

M03/PROYECTO0/3.1 deberá partir de este estado completo de 2.12.